เอกสารประกอบรายวิชา 188 322
สถาปัตยกรรมคอมพิวเตอร์
Computer Architecture

ธัชพงศ์ กตัญญูกุล

April 5, 2014

Contents

1 ภาพรวม 5
1.1 ระดับชั้นสาระสำคัญและเทคโนโลยีของคอมพิวเตอร์ 5
1.2 เปิดฝาครอบ 9
1.3 สมถนะ (Performance) 12
1.4 กำแพงกำลังงาน (The Power Wall) และการเปลี่ยนจาก uniprocessors ไปสู่ multiprocessors 20
1.5 กระบวนการผลิตไมโครโปรเซสเซอร์ชิป 21
1.6 แหล่งของภาพประกอบ 23

2 ชุดคำสั่ง: ภาษาของคอมพิวเตอร์ 25
2.1 การคำนวณพีชคณิต 25
2.2 Operands สำหรับฮาร์ดแวร์ 26
2.2.1 Memory Operands 27
2.2.2 Constant or Immediate Operands 29
2.3 Signed and Unsigned Numbers 29
2.4 Representing Instructions in the Computer 30
2.4.1 Logical Operations 34
2.5 Instructions for Making Decisions 35
2.5.1 Loops 36
2.5.2 Case/Switch Statement 38
2.6 Supporting Procedures in Computer Hardware 39
2.6.1 Spilling Registers 40
2.6.2 Nested Procedures 41
2.6.3 Allocating Space for New Data on the Stack 44
2.6.4 Allocating Space for New Data on the Heap 45
2.7 Communicating with People 46
2.7.1 Characters and Strings in Java 48
2.8 MIPS Addressing for 32-bit Immediate and Addresses 48
2.8.1 32-bit Immediate Operands 48
2.8.2 Addressing in Branches and Jumps 49
2.8.3 Decoding Machine Language. 52
2.9 Translating and Starting Program 52

3 Arithmetic for Computers 63
3.1 Signed and Unsigned Numbers 63
3.2 Addition and Subtraction 65
3.2.1 Subtraction 67
3.3 Logical Operations 69

3.4 Contructing ALU 69
3.4.1 A 32-bit ALU 72
3.5 set on less than 72
3.6 Multiplication 78
3.7 Division 83
3.8 Floating Point 83

4 The Processor 93
4.1 Building a Datapath 94
4.1.1 R-type 95
4.1.2 คำสั่งย้ายข้อมูลกับหน่วยความจำ 98
4.1.3 คำสั่ง branch 100
4.1.4 คำสั่ง Jump 102
4.1.5 วงจรควบคุม 106
4.2 Multi-Cycle Machines 110
4.2.1 Exceptions 112
4.2.2 ตัวอย่าง Multicycle Machine 115

5 Pipelining 117
5.1 Pipeline Hazards 121
5.1.1 Structural Hazards 121
5.1.2 Data Hazards 121
5.1.3 Control Hazards 126
5.2 Parallelism and Advanced Instruction-Level Parallelism 130
5.2.1 Power Efficiency and Advanced Pipelining 130

6 Large and Fast: Exploiting Memory Hierarchy 131
6.1 แคช: พื้นฐาน 134
6.1.1 Multiword Cache Block 138
6.1.2 Handling Cache Misses 141
6.1.3 Handling Writes 141
6.2 Reducing Cache Misses by More Flexible Placement of Blocks 142
6.3 Virtual Memory 142

Chapter 1

ภาพรวม

คอมพิวเตอร์มีส่วนประกอบหลักๆ คือ หน่วยประมวลผล (processor), หน่วยความจำ (memory), อินพุต (input) และ เอาต์พุต (output). เพื่อที่จะสามารถจัดการกับความซับซ้อนของระบบคอมพิวเตอร์ได้ วิศวกร จะมอง ฮาร์ดแวร์ (hardware) และซอฟต์แวร์ (software) ออกเป็นหลายๆระดับชั้น โดยที่ ระดับที่อยู่ล่างกว่าจะมีรายละเอียด ที่ระดับบนกว่าไม่มี. หลักการนี้ เรียกว่า หลักการกำหนดสาระสำคัญ (principle of abstraction). สิ่งสำคัญที่เป็นตัวประสานระหว่างฮาร์ดแวร์และซอฟต์แวร์ระดับล่าง คือ สถาปัตยกรรมชุดคำสั่ง (instruction set architecture). การที่ใช้ตัวประสานระหว่าง ระดับชั้นของฮาร์ดแวร์ และ ระดับชั้นของซอฟต์แวร์ นี้ช่วยให้ การออกแบบสร้างฮาร์ดแวร์ เป็นอิสระจากซอฟต์แวร์ระดับล่าง: ฮาร์ดแวร์ที่ราคาและสมรรถนะต่างกัน สามารถใช้งานซอฟต์แวร์เดียวกันได้.

1.1 ระดับชั้นสาระสำคัญและเทคโนโลยีของคอมพิวเตอร์

> "If I have seen further it is by standing on the shoulders of giants." – Isaac Newton

คอมพิวเตอร์ สร้างจากวงจรดิจิตอลอิเล็กทรอนิกส์. เทคโนโลยีดิจิตอลพัฒนาจากพื้นฐานอนาล็อกอิเล็กทรอนิกส์. อิเล็กทรอนิกส์เป็นผลผลิตมาจากพื้นฐาน ฟิสิกส์, เคมี และ เทคโนโลยีทางวัสดุ. รูป 1.1 ก. จอห์น บาร์ดีน (John Bardeen), วิลเลี่ยม ชอคลีย์ (William Shockley) และ วอลเตอร์ แบรตเทน (Walter Brattain), กลุ่มผู้ประดิษฐ์ทรานซิสเตอร์ตัวแรกของโลก. รูป 1.1 ข-ฉ แสดงลำดับขั้นสาระสำคัญ (hierarchy of abstraction): รูป ข. แบบจำลองแสดงชั้นสารกึ่งตัวนำที่ประกอบเป็นทรานซิสเตอร์; รูป ค. ทรานซิสเตอร์แบบ point-contact ตัวแรกของโลก; รูป ง. วงจรไฟฟ้าอย่างง่ายของ TTL NAND Gate ที่ใช้ทรานซิสเตอร์เป็นส่วนประกอบ; รูป จ. วงจรดิจิตอลอย่างง่ายของหน่วย Arithmetic Logic Unit (ALU) ที่ใช้ logic gates เป็นส่วนประกอบ; รูป ฉ. วงจรดิจิตอลระดับรีจีสเตอร์ของหน่วยประมวลผลคอมพิวเตอร์ ที่ใช้รีจีสเตอร์และหน่วยดิจิตอลต่างๆรวมทั้ง ALU เป็นส่วนประกอบ.

เมื่อคอมพิวเตอร์ คือ วงจรดิจิตอลอิเล็กทรอนิกส์ อินพุตของวงจรดิจิตอลอิเล็กทรอนิกส์แต่ละขา มีค่าอยู่แค่ สองแบบ: 1 (on) หรือ 0 (off). เพื่อจะให้คอมพิวเตอร์ทำงาน เราต้องใส่อินพุตให้วงจรดิจิตอลอิเล็กทรอนิกส์เป็นค่า 1 หรือ 0 ให้ครบขาอินพุตของมัน ค่าแต่ละค่านี้ จะเรียกเป็นบิต (bit) จำนวนขาของวงจรดิจิตอลอิเล็กทรอนิกส์ จะบอกจำนวนบิต เช่น วงจรมีอินพุต 8 ขา ก็รับอินพุต 8 บิต. โดยเพื่อความสะดวก วิศวกรจะอ่าน บิตที่เรียงกันนี้ เป็นเลขฐานสอง (binary number). คอมพิวเตอร์จะทำงานตามคำสั่งที่เราใส่เข้าไป ซึ่งคำสั่งแต่ละคำสั่งจะเรียกว่า คำสั่ง

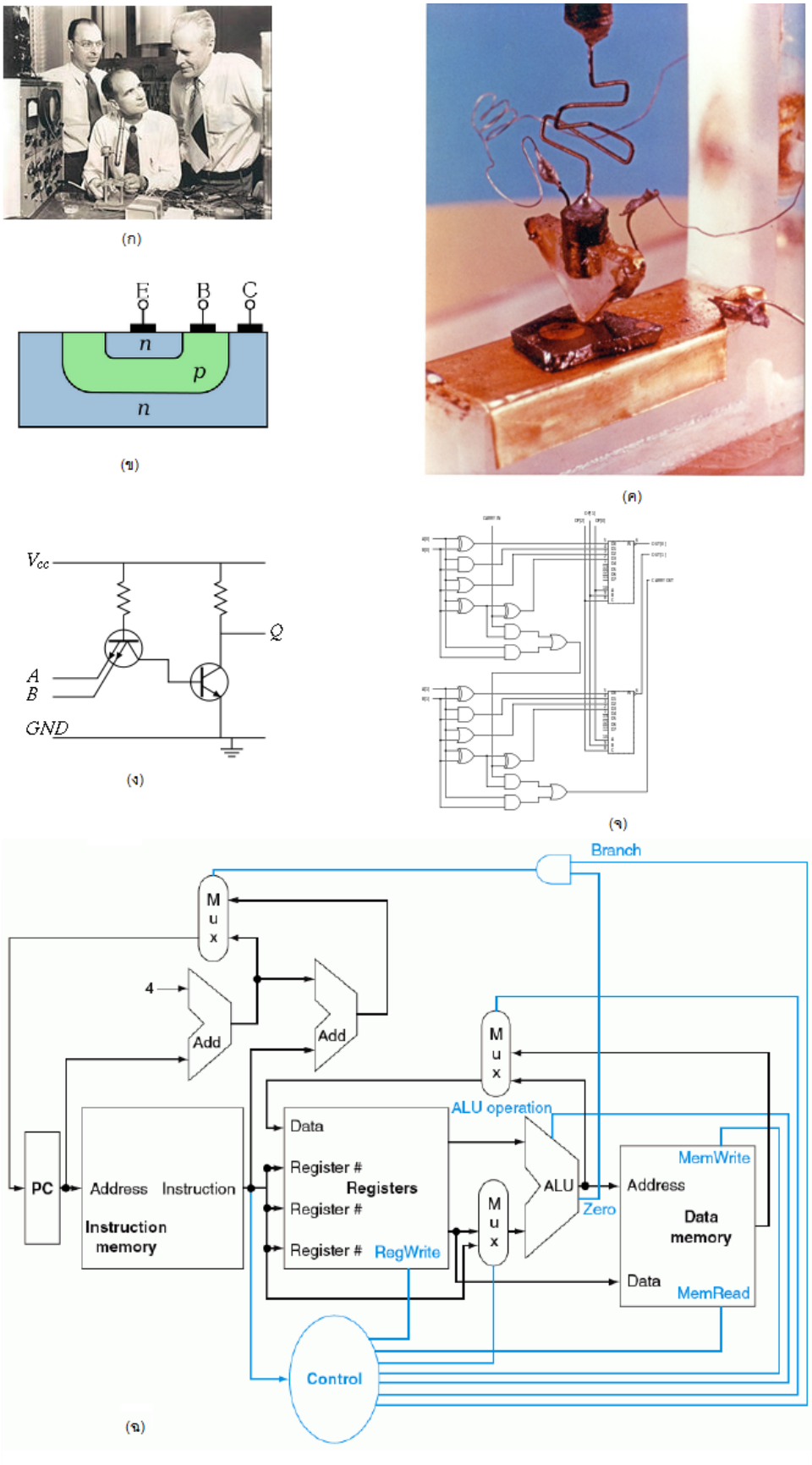


Figure 1.1: (ก) จอห์น บาร์ดีน, วิลเลี่ยม ชอคลีย์ และ วอลเตอร์ แบรตเทน, กลุ่มผู้ประดิษฐ์ทรานซิสเตอร์ตัวแรกของโลก ในปี ค.ศ. 1948; (ข) แบบจำลองแสดงชั้นสารกึ่งตัวนำของทรานซิสเตอร์; (ค) ทรานซิสเตอร์ ของ บาร์ดีน และ แบรตเทน; (ง) รูปวงจรไฟฟ้าอย่างง่ายของ TTL NAND Gate; (จ) รูปวงจรดิจิตอลอย่างง่ายของหน่วย Arithmetic Logic Unit (ALU); (ฉ) รูปวงจรการเชื่อมต่อหน่วยประกอบต่างๆของหน่วยประมวลผลคอมพิวเตอร์

เครื่อง (instruction) ซึ่ง คำสั่งเครื่องก็จะอยู่ในรูปเลขฐานสองเหมือนกัน (คอมพิวเตอร์ เป็น วงจรดิจิตอลอิเล็กทรอนิกส์).

โปรแกรมเมอร์ยุคบุกเบิกสั่งคอมพิวเตอร์โดยตรงด้วยคำสั่งเลขฐาน 2 ซึ่งน่าเบื่อมาก. ทำให้ไม่นานนัก โปรแกรมเมอร์ก็คิดภาษาแอสเซมบลี (assembly language) ขึ้นมา. ซึ่งแรกๆ พวกเขาก็จะแปลด้วยมือเป็นเลขฐานสองเอง. ต่อมาก็มีคนเขียนเป็นโปรแกรม (ด้วยคำสั่งเลขฐานสอง) ให้คอมพิวเตอร์ แปลงจากคำสั่งในภาษาแอสเซมบลีไปเป็นเลขฐานสอง. โปรแกรมที่ใช้แปลงภาษาแอสเซมบลีไปเป็นคำสั่งเลขฐานสอง (อินพุตโดยตรงสำหรับ คอมพิวเตอร์) ถูกเรียกว่า แอสเซมบลอร์ (assembler). ตัวอย่างเช่น คำสั่ง `add A, B` (ในภาษาแอสเซมบลี) ถูกแปลงเป็น `1000110010100000`. ถึงแม้การเขียนภาษาแอสเซมบลีจะง่ายว่าเขียนคำสั่งในรูปเลขฐานสอง แต่การเขียนภาษาแอสเซมบลีก็เป็นงานที่ยาก น่าเบื่อ. และก็ยังบังคับให้โปรแกรมเมอร์ต้องคิดราวกับว่าพวกเขาเป็นเครื่องจักรเสียเอง. ภาษาระดับสูง (high-level programming language) พร้อมกับคอมไพเลอร์ (compiler) จึงได้ถูกพัฒนาขึ้น เพื่อให้ (1) โปรแกรมเมอร์สามารถเขียนโปรแกรมโดยคิดได้อย่างเป็นธรรมชาติมากขึ้น (2) ภาษาระดับสูง ช่วยเพิ่มผลผลิต ทำให้โปรแกรมเขียนโปรแกรมออกมาได้มากขึ้น เพราะ การเขียนโปรแกรมด้วยภาษาระดับสูง มักจะใช้คำสั่งน้อยบรรทัดกว่า ความกระชับของภาษาระดับสูงนี้ สำคัญมากสำหรับผลผลิตของโปรแกรมเมอร์ (3) เนื่องจากภาษาแอสเซมบลี เป็นคำสั่งเฉพาะของเครื่องแต่ละแบบ การแยกภาษาระดับสูง ออกจากภาษาแอสเซมบลี ยังทำให้โปรแกรมที่ได้จากภาษาสูง สามารถถูกคอมไพเลอร์แปลง เป็นคำสั่งเครื่องสำหรับเครื่องแต่ละแบบ ได้เลย โดยไม่ต้องเขียนโปรแกรมขึ้นใหม่.

รูปที่ 1.2 แสดงคำสั่งภาษาซีที่ถูกแปลงไปเป็นคำสั่งภาษาเครื่องสำหรับเครื่องสถาปัตยกรรม MIPS.

เมื่อกิจกรรมการเขียนโปรแกรมขยายกว้างขวางขึ้น โปรแกรมเมอร์เห็นว่า การรวบรวมโปรแกรมที่ใช้บ่อยๆไว้ด้วยกัน สำหรับให้โปรแกรมอื่น มาเรียกใช้จะมีประโยชน์มากกว่า ที่แต่ละโปรแกรมจะต้องถูกเขียนทุกอย่างขึ้นมาใหม่. โปรแกรมที่ถูกใช้บ่อยๆที่เอามารวมกันนี้จะเรียกว่า ซับรูทีนไลบรารี (subroutine library). ตัวอย่างของ ซับรูทีนไลบรารี มาตรฐานของภาษาซี (C standard library) ได้แก่ float.h (สำหรับทำงานกับตัวเลขทศนิยมแบบ floating point), math.h (สำหรับฟังชั่นการคำนวณทางคณิตศาสตร์), stdio.h (สำหรับการควบคุมอินพุตและเอาต์พุต), string.h (สำหรับการทำงานกับตัวอักษร), และ time.h (สำหรับการทำงานกับเวลาและวันที่) เป็นต้น.

ราวๆทศวรรษที่ 50 นั้นคอมพิวเตอร์ทำงานได้เร็วกว่าเดิมมาก และแม้จะมีคอมไพเลอร์และไลบรารีช่วยให้การเขียนโปรแกรมง่ายขึ้น แต่คอมพิวเตอร์จะรันโปรแกรมได้ทีละหนึ่งโปรแกรม โดยที่ ผู้ใช้แต่ละคนใช้เครื่องคอมพิวเตอร์ตามเวลาที่จองไว้. เขาจะมาที่ตัวเครื่องคอมพิวเตอร์พร้อมกับโปรแกรมและข้อมูล ในรูปของกระดาษเจาะรู (punched paper cards). เขาจะโหลดโปรแกรม (โดยการใส่กระดาษเจาะรู) เข้าไปในเครื่องคอมพิวเตอร์. คอมพิวเตอร์จะทำงานตามโปรแกรมจนจบ (หรือจนกระทั่ง เครื่องแฮงค์). การที่แต่ละโปรแกรมต้องโหลดเข้าไปด้วยมือแบบนี้ ทำให้มีช่วงเวลาที่คอมพิวเตอร์ว่างอยู่มากเมื่อเทียบกับเวลาที่ใช้งานของมัน. เพื่อให้สามารถใช้งานคอมพิวเตอร์ได้มีประสิทธิภาพมากขึ้น วิศวกรจึงประดิษฐ์เพื่อคอยโหลดโปรแกรมต่างๆเข้าไปให้กับคอมพิวเตอร์แทนการโหลดด้วยมือ. โปรแกรมนี้จะถูกโหลดเข้าไปในเครื่องคอมพิวเตอร์ก่อนโปรแกรมอื่นๆ และโปรแกรมนี้จะคอย โหลดโปรแกรมอื่นๆเข้าไปให้คอมพิวเตอร์รัน โดยไม่ปล่อยให้คอมพิวเตอร์มีเวลาว่างเสียเปล่า. ภายหลังมีการรวม ไลบรารีอื่นๆเช่น ไลบรารีการจัดการอินพุตเอาต์พุต การจัดการหน่วยความจำ รวมไปจนถึง เปลือกระบบ (user shell) เพื่อความสะดวกสำหรับผู้ใช้ เข้าไปกับโปรแกรมฝังตัวนี้ และโปรแกรมนี้ถูกเรียกว่า ระบบปฏิบัติการ (operating system). ระบบปฏิบัติการ เป็นระบบที่ช่วยจัดการทรัพยากรต่างๆ (เช่น การใช้งาน

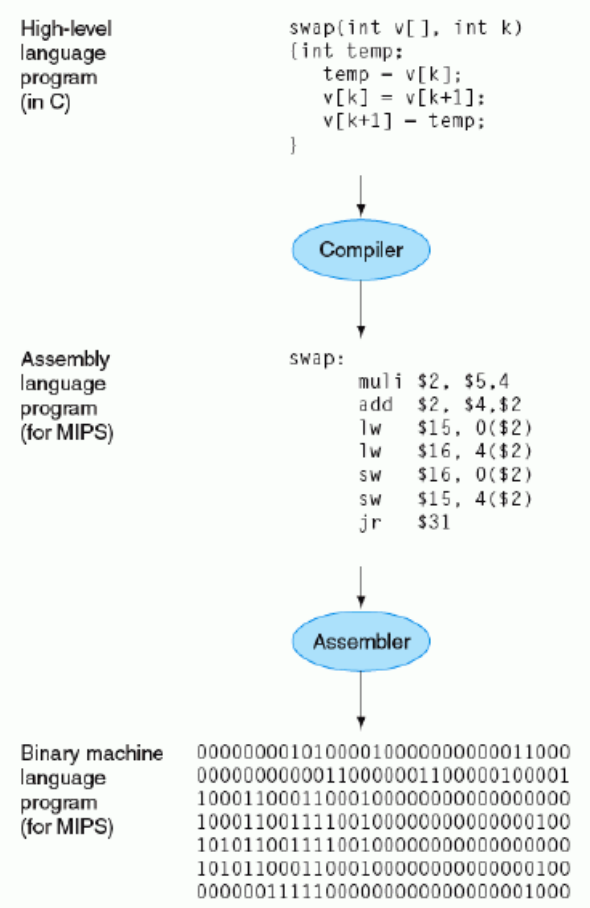


Figure 1.2: คำสั่งภาษาซีแปลงไปเป็นคำสั่งของเครื่อง

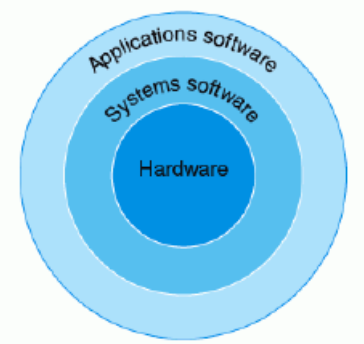


Figure 1.3: รูปอย่างง่ายแสดงลำดับชั้นของฮาร์ดแวร์และซอฟแวร์

หน่วยประมวลผล การใช้งานหน่วยความจำ การใช้อินพุตเอาต์พุต) ของเครื่องคอมพิวเตอร์ เพื่อที่โปรแกรมสามารถที่จะรันบนเครื่องได้อย่างมีประสิทธิภาพ.

เมื่อโปรแกรมหรือเรียกว่าซอฟต์แวร์ มีหลายๆชนิด วิศวกรแบ่งซอฟต์แวร์ตามลักษณะการใช้งานออกเป็น ซอฟต์แวร์ระบบ (system software) และ แอพพลิเคชั่น (application software หรือย่อๆว่า application). ซอฟต์แวร์ระบบ เป็นซอฟต์แวร์ที่ซอฟต์แวร์อื่นๆมักเรียกใช้ เช่น ระบบปฏิบัติการ คอมไพเลอร์ แอสเซมเบลอร์ โดยมากผู้ที่ใช้งานซอฟต์แวร์ระบบคือ วิศวกร หรือโปรแกรมเมอร์. แอพพลิเคชั่น เป็นซอฟต์แวร์ สำหรับผู้ใช้ทั่วๆไป เช่น โปรแกรมตารางจัดการ (spreadsheet, e.g., MS Excel, OpenOffice Calc), โปรแกรมประมวลคำ (word processor, e.g., MS Word, OpenOffice Writer).

รูป 1.3 แสดงระดับชั้น โดยมีฮาร์ดแวร์อยู่ระดับล่างสุด ต่อมาด้วยซอฟต์แวร์ระบบ และ บนสุดเป็นแอพพลิเคชั่น. วิศวกรฮาร์ดแวร์ รู้เรื่องรายละเอียดของวงจร เพื่อให้ฮาร์ดแวร์สามารถทำงานได้ตามชุดคำสั่งที่กำหนด. วิศวกรซอฟต์แวร์ระบบ รู้เรื่องรายละเอียดของการควบคุมจัดการและบริหารการใช้งานฮาร์ดแวร์ ให้มีประสิทธิภาพ โดยผ่านชุดคำสั่งที่กำหนด. วิศวกรซอฟต์แวร์ ไม่จำเป็นต้องรู้รายละเอียดของฮาร์ดแวร์ เช่น ลักษณะการจัดวางวงจร. โปรแกรมเมอร์รู้เรื่องรายละเอียดของแอพพลิเคชั่น และ การเรียกใช้งานซอฟต์แวร์ระบบที่จำเป็น. แอพพลิเคชั่นโปรแกรมเมอร์ไม่จำเป็นต้องรู้รายละเอียดของวิธีการบริหารทรัพยากรของซอฟต์แวร์ระบบ เช่น วิธีการจัดการหน่วยความจำ.

ว่ากันเรื่องคอมไพเลอร์ สำหรับคอมไพเลอร์ส่วนใหญ่ (เช่น MS Visual Basic, gcc, Turbo Pascal) ใช้แปลงโปรแกรม ที่อาจเป็น ซอฟต์แวร์ระบบ หรือ แอพพลิเคชั่น ก็ได้. แอพพลิเคชั่นที่ได้ หลังจากแปลงเสร็จแล้ว ก็ไม่จำเป็นต้องเรียกใช้คอมไพเลอร์อีก ดังนั้น เมื่อพูดถึงคอมไพเลอร์แล้วรูป 1.3 อาจให้ภาพที่ไม่ชัดเจนนัก. รูป 1.4 แสดงลำดับชั้นของซอฟต์แวร์ โดยแบ่งตามประเภท.

หมายเหตุ คอมไพเลอร์ของภาษาจาวา (Java compiler) เป็นกรณีที่แตกต่าง ซึ่ง จาวาคอมไพเลอร์ จะแปลงโปรแกรมเป็นไบคโค้ด (bytecode). ซึ่ง ไบคโค้ด ก็เหมือนกับ ภาษาแอสเซมบลีของจาวาเอง และ ใช้รันผ่านเจวีเอม (JVM หรือ Java Virtual Machine) ซึ่งเวลารันโปรแกรม เจวีเอมจะแปลงไบค์โค้ด เป็นคำสั่งฐานสองให้กับเครื่อง. จาวาคอมไพเลอร์บางตัว (เช่น gcj) สามารถที่จะเลือกแปลงโปรแกรมเป็นไบคโค้ด เพื่อรันผ่านเจวีเอม หรือ เป็นภาษาเครื่องโดยตรงได้.

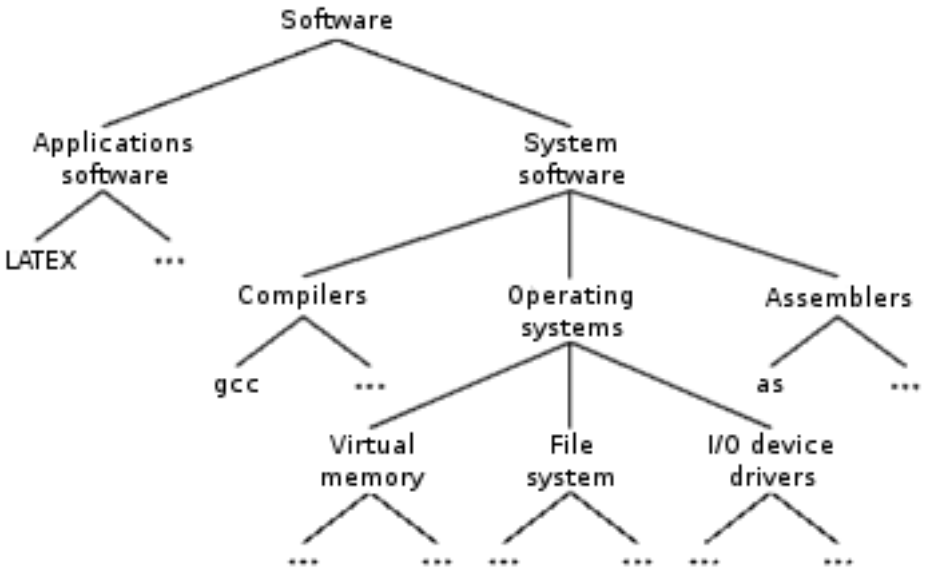


Figure 1.4: รูปอย่างง่ายแสดงลำดับชั้นของซอฟต์แวร์ โดยแบ่งตามประเภท

1.2 เปิดฝาครอบ

> I got the idea for the mouse while attending a talk at a computer conference. The speaker was so boring that I started daydreaming and hit upon the idea. – Doug Engelbart

รูป 1.7 แสดงคอมพิวเตอร์ตั้งโต๊ะ ประกอบด้วยอุปกรณ์เอาท์พุต ได้แก่ จอแสดงผล (monitor) ซึ่งในรูปเป็นจอแบบ liquid crystal display หรือ LCD, อุปกรณ์อินพุต ได้แก่ คีย์บอร์ด (keyboard) และ เมาส์ (mouse), และ กล่องเครื่อง (computer case).

รูป 1.8 แสดงภายในกล่องเครื่อง ซึ่งมี เมนบอร์ด (motherboard) ต่อกับ หน่วยจ่ายไฟ (power supply) และ อุปกรณ์อื่นๆ เช่น เครื่องอ่านเขียนดีวีดี (DVD drive), ฮาร์ดดิส (hard drive), และ ซิปไดร์ฟ (ZIP drive). โดยบนเมนบอร์ดจะมีซีพียู หรือ หน่วยประมวลผลกลาง (Central Processing Unit, ย่อเป็น CPU) ซึ่งจะต่อกับซิงค์ระบายความร้อน (heat zinc) และ พัดลมและฝาครอบ (fan with cover ดังแสดงในรูป).

เมนบอร์ด คือ วงจรที่ใช้เชื่อมต่อ ส่วนประกอบที่สำคัญของคอมพิวเตอร์เข้าด้วยกัน ได้แก่ ซีพียู (ใต้ซิงค์ระบายความร้อน มุมซ้ายบน), หน่วยความจำหลัก (memory) ซึ่งเป็นแผงเสียบอยู่ใกล้ๆ ซีพียู, และ อุปกรณ์ที่ใช้ควบคุมอินพุต เอาท์พุต (input/output units, ย่อๆ I/O units) รวมถึง ช่องต่อสายสัญญาณอินพุตหรือเอาท์พุต และ ช่องต่อสายไฟเข้ามาจากหน่วยจ่ายไฟ. รูป 1.9 แสดงเมนบอร์ดหลังจากถอดออกมาจากกล่องเครื่อง (รูปซ้าย) และ แผนผังส่วนต่างๆ (รูปขวา).

ซีพียู เป็นส่วนที่ทำการคำนวณ ย้าย จัดการข้อมูล สั่งอินพุตเอาท์พุต. ส่วนประกอบที่สำคัญของซีพียู คือ เส้นทางข้อมูลและส่วนควบคุม. หน่วยความจำ เป็นที่ ที่เก็บโปรแกรมที่รันอยู่ และเก็บผลการคำนวณต่างๆจากการรัน รวมถึงข้อมูลที่ใช้ในการรันโปรแกรมด้วย. หน่วยความจำสร้างจากชิป DRAM (Dynamic Random Access Memory chips). ในรูป หน่วยความจำที่ใช้เป็นแผงแบบ DIMMs (Dual Inline Memory Modules) ซึ่งจะมี DRAM chips อยู่ทั้งสองด้านของแผง ต่างจาก SIMMs ที่มีอยู่เพียงด้านเดียว.

รูป 1.11 แสดงภายในตัวซีพียู Pentium 4 (รูปซ้าย) และ แผนผังคร่าวๆของหน้าที่ของส่วนต่างๆภายในซีพียู เช่น ส่วนควบคุม (Control), เส้นทางของข้อมูล (datapath), แคชคำสั่ง (Instruction cache), แคชข้อมูล (Data cache), ส่วนต่อเชื่อมแคชลำดับสองและหน่วยความจำ (Secondary cache and memory interface), ส่วนต่อเชื่อมอินพุตเอาท์พุต (I/O interface), เป็นต้น.

รูป 1.12 แสดงภายใน AMD Barcelona และ แผนผังคร่าวๆของหน้าที่ของส่วนต่างๆภายใน. ชิป AMD Barcelona มี หน่วยประมวลผล อยู่ 4 ชุด. หน่วยประมวลผล หลายชุด ที่บรรจุอยู่ในชิปตัวเดียวกัน เรียกว่า แก่น (core).

สังเกตุว่า ภายในซีพียูก็มีหน่วยความจำอยู่ เรียกว่า แคช (cache memory) ซึ่งเป็นหน่วยความจำ ขนาดเล็ก แต่ทำงานรวดเร็ว และทำหน้าที่เป็นเหมือน ที่พักข้อมูล (buffer) ของหน่วยความจำหลัก. แคช ถูกสร้างขึ้นจากเทคโนโลยีที่ต่างจากหน่วยความจำหลัก. ขณะที่หน่วยความจำหลักถูกสร้างจาก DRAM, แคชถูกสร้างจาก SRAM (Static Random Access Memory). SRAM เร็วกว่า แต่หนาแน่นน้อยกว่า DRAM ดังนั้นราคาจึงแพงกว่า DRAM.

อีกครั้ง ถ้าสังเกตุ เราเริ่มดูลงไประดับลึกเรื่อยๆ และ เห็นรายละเอียดมากขึ้นมากระดับ. สาระสำคัญ (abstraction) คือ โมเดลที่ซ่อนรายละเอียดระดับล่างชั่วคราว เพื่อให้เราสะดวกที่เข้าใจหรือออกแบบระบบที่ซับซ้อนได้.

การเชื่อมต่อระหว่างซอฟแวร์กับฮาร์ดแวร์ นับว่าเป็น abstraction ที่สำคัญที่สุดอย่างหนึ่งของคอมพิวเตอร์. รายละเอียดของฮาร์แวร์โครงสร้างทางวงจรดิจิตอล เทคโนโลยีอิเลคทรอนิคส์ เทคโนโลยีการผลิต lithography ที่ใช้ จะถูกซ่อนจากมุมมองของซอฟแวร์. โดยสิ่งที่ซอฟแวร์ มองเห็นและติดต่อกับฮาร์ดแวร์ได้ โดยผ่าน สถาปัตยกรรมชุดคำสั่ง (instruction set architecture, ย่อๆ ISA) หรือ เรียกง่ายๆว่า architecture ของคอมพิวเตอร์.

สถาปัตยกรรมชุดคำสั่ง หรือ เรียกง่ายๆ ชุดคำสั่ง จะบอกทุกอย่างที่ โปรแกรมเมอร์ต้องรู้ เพื่อจะสร้าง โปรแกรมภาษาเครื่องให้ทำงานได้อย่างถูกต้อง ไม่ว่าจะเป็น คำสั่งทำการคำนวณ ย้ายข้อมูล หรือจัดการอุปกรณ์ I/O. โดยทั่วไปแล้ว OS จะช่วยเรื่องการจัดการ I/O, บริหารหน่วยความจำ, หรือ จัดการงานระดับล่างอื่นๆ ดังนั้น แอพพลิเคชั่นโปรแกรมเมอร์ไม่ต้องห่วงเรื่องพวกนี้. เราจะเรียก ชุดคำสั่งพื้นฐาน กับ ชุดคำสั่งของโอเอส(operating system interface) รวมๆกันว่า application binary interface (ABI).

นักออกแบบคอมพิวเตอร์ จะแยก สถาปัตยกรรม ออกจาก implementation อย่างชัดเจน: implementation (หรือ เรียกว่า microarchitecture) คือ ฮาร์ดแวร์ที่จะทำงานตามที่สถาปัตยกรรมชุดคำสั่งกำหนด. เช่น โปรแกรมที่เขียนสำหรับ สถาปัตยกรรม IA32 (หรือ เรียกว่า i386) สามารถนำไปใช้ได้กับ คอมพิวเตอร์ Intel 80386 (ที่ออกมาตั้งแต่ ค.ศ. 1985) จนถึง Intel Atom (ที่ออกมา ค.ศ. 2008) หรือแม้แต่ AMD Athlon ที่แม้จะมี implementation ต่างกันแต่รองรับชุดคำสั่งของ IA32 เหมือนกันได้.

ที่เก็บข้อมูล หน่วยความจำ ที่เราพูดถึงไม่ว่าจะเป็น DRAM (ที่ใช้เป็นหน่วยความจำหลัก) หรือ SRAM (ที่ใช้เป็นแคช) เมื่อตัดพลังงานออก, เช่น ปิดเครื่อง, ข้อมูลก็จะหายไป. เราจะเรียกหน่วยเก็บข้อมูลพวกนี้ว่าเป็น volatile memory.

หน่วยเก็บข้อมูล เช่น ฮาร์ดดิส หรือ DVD แม้ว่าจะปิดเครื่องไป ข้อมูลก็จะยังคงอยู่. พวกนี้จัดเป็น nonvolatile memory. เราจะใช้ volatile memory ในการเก็บข้อมูล หรือ โปรแกรม เวลาที่เรารันโปรแกรม แต่ใช้ nonvolatile memory เก็บข้อมูลหลังจากนั้น. ดังนั้นเราเรียกพวกแรกว่า main memory หรือ primary memory และ พวกที่สองว่า secondary memory. ฮาร์ด

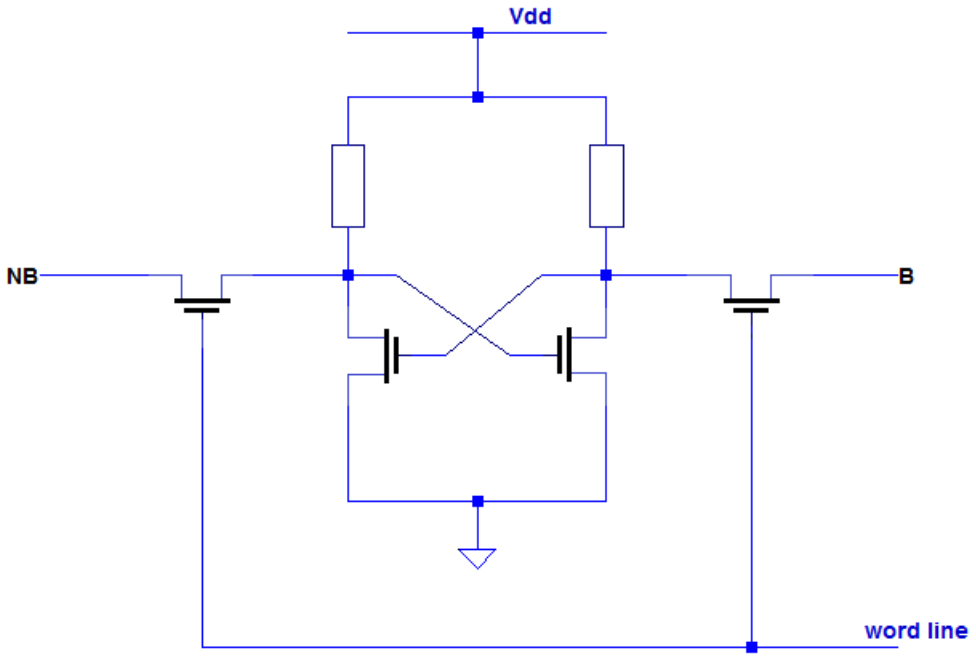


Figure 1.5: SRAM 1 cell

ดิส เป็น nonvolatile magnetic disks ที่ใช้คุณสมบัติทางแม่เหล็กไฟฟ้า กับจานเคลือบสารแม่เหล็กในการเก็บข้อมูล. Flash memory เป็น nonvolatile semiconductors ที่ราคาถูก แต่ว่าทำงานช้ากว่า DRAM (แต่ Flash memory ก็ราคาแพงและเร็วกว่า magnetic disks). แต่ flash memory bits จะเสื่อมหลังจากถูกเขียนไปราวๆ 100,000 ถึง 1 ล้านครั้ง ดังนั้น file system ต้องเก็บสถิติการเขียนเอาไว้ และ จะต้องมีวิธีจัดการ เช่นการย้ายข้อมูลที่ใช้บ่อยๆเป็นต้น.

DRAM access time ประมาณ 50-70 nsec. ฮาร์ดดิสค์ ประมาณ 5-20 msec. ปี ค.ศ. 2008, ราคาต่อ gigabyte ฮาร์ดดิสค์ ถูกกว่า DRAM ราวๆ 30-100 เท่า.

รูป 1.5 แสดงโครงสร้างวงจรสำหรับ SRAM หนึ่งบิต

1.3 สมถนะ (Performance)

response time (หรือ execution time) เวลารวมที่คอมพิวเตอร์ใช้ในการทำงานจนเสร็จ (รวมเวลา ติดต่อหน่วยความจำ, ติดต่อฮาร์ดดิสค์, ติดต่อ I/O, OS overhead, CPU execution time, เป็นต้น).

throughput (หรือ bandwidth) คือ จำนวนงานที่ทำเสร็จต่อหน่วยเวลา.

เราจะใช้ response time เป็นตัววัดหลักสำหรับ computer performance. เพื่อความสะดวก เรานิยาม สมถนะของเครื่อง X จาก

$$\text{Performance}_X = \frac{1}{\text{Execution time}_X} \tag{1.1}$$

ถ้าเราเปรียบเทียบคอมพิวเตอร์ 2 เครื่อง X กับ Y เราพูดได้ว่า ``X is $n$ times faster than Y'' หรือ ``X is n times as fast as Y'' เมื่อ n คือ

$$\frac{\text{Performance}_X}{\text{Performance}_Y} = \frac{\text{Execution time}_Y}{\text{Execution time}_X} = n \tag{1.2}$$

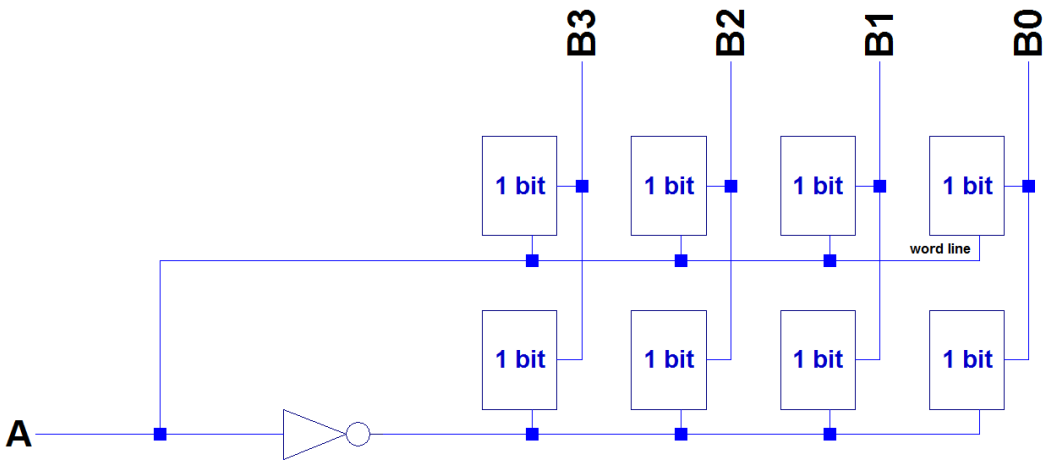


Figure 1.6: 4-bit word SRAM อ้างได้ 2 แอดเดรส (A = 0 กับ A = 1)



Figure 1.7: คอมพิวเตอร์ตั้งโต๊ะ Dell Optiplex GX260

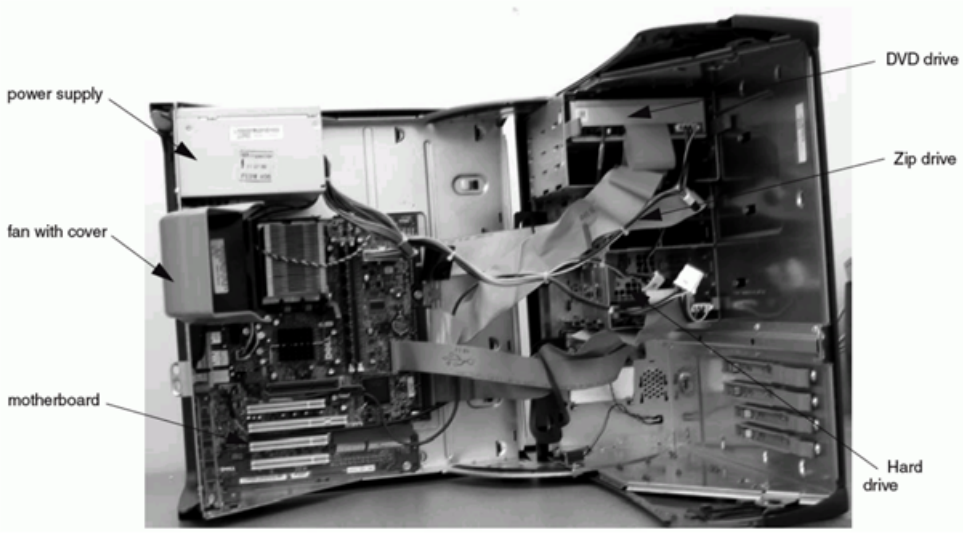


Figure 1.8: ภายในกล่องเครื่องของคอมพิวเตอร์

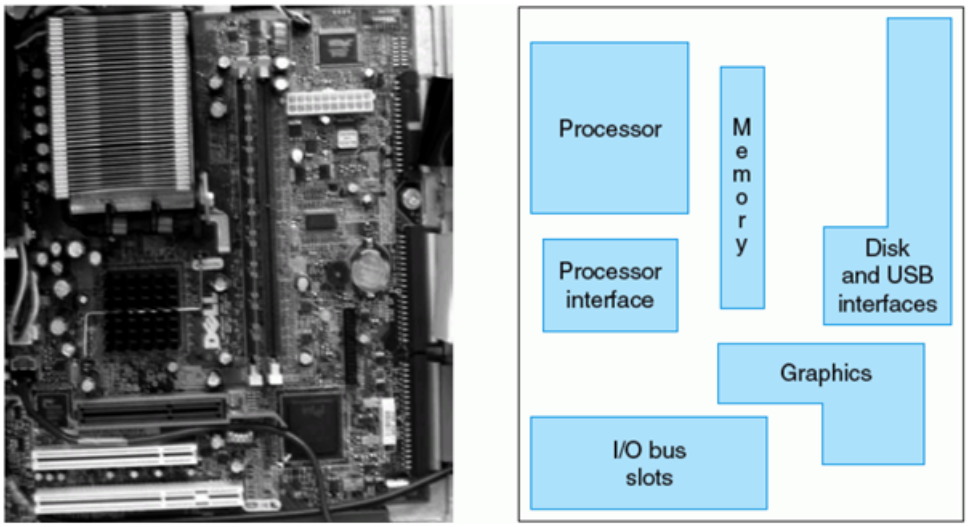


Figure 1.9: เมนบอร์ดของคอมพิวเตอร์

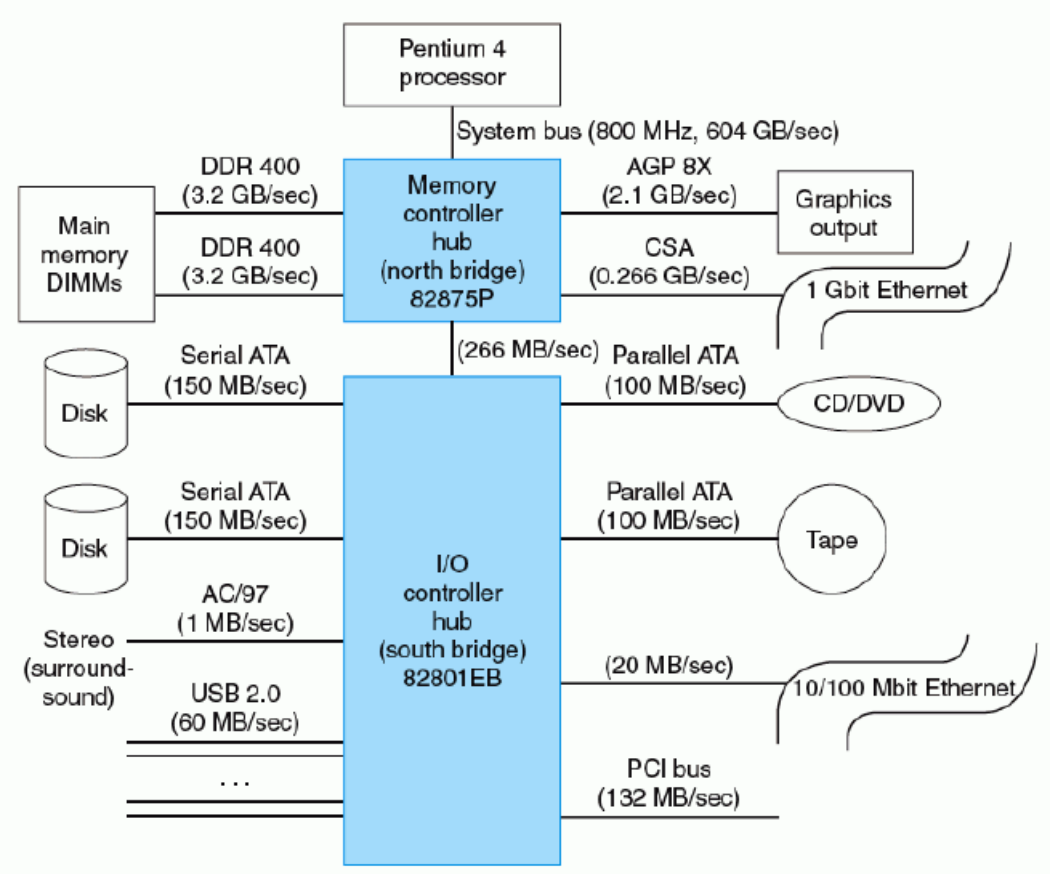


Figure 1.10: โครงสร้าง I/O ของคอมพิวเตอร์ที่ใช้ Pentium 4 กับ Intel 875 chipset

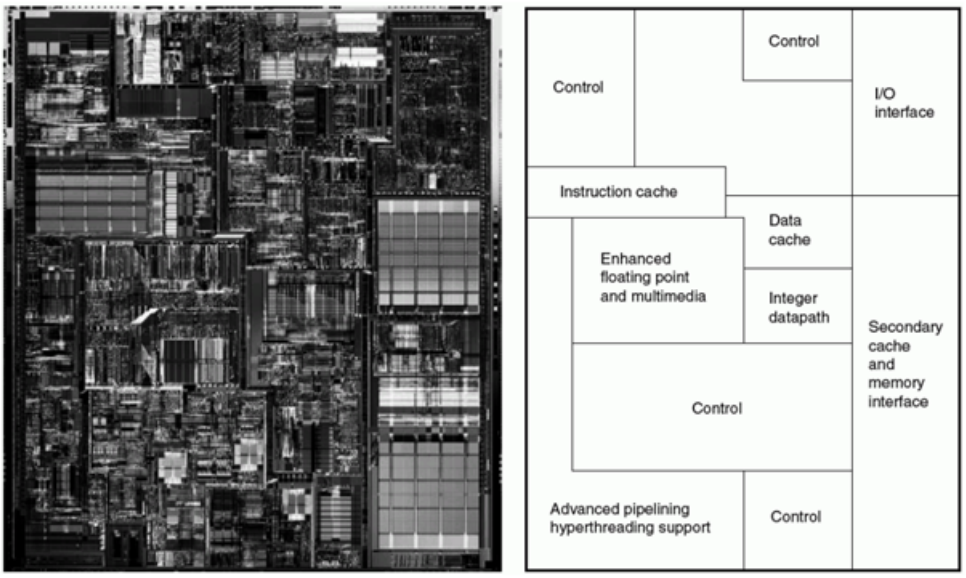


Figure 1.11: ภายในซีพียู Pentium 4

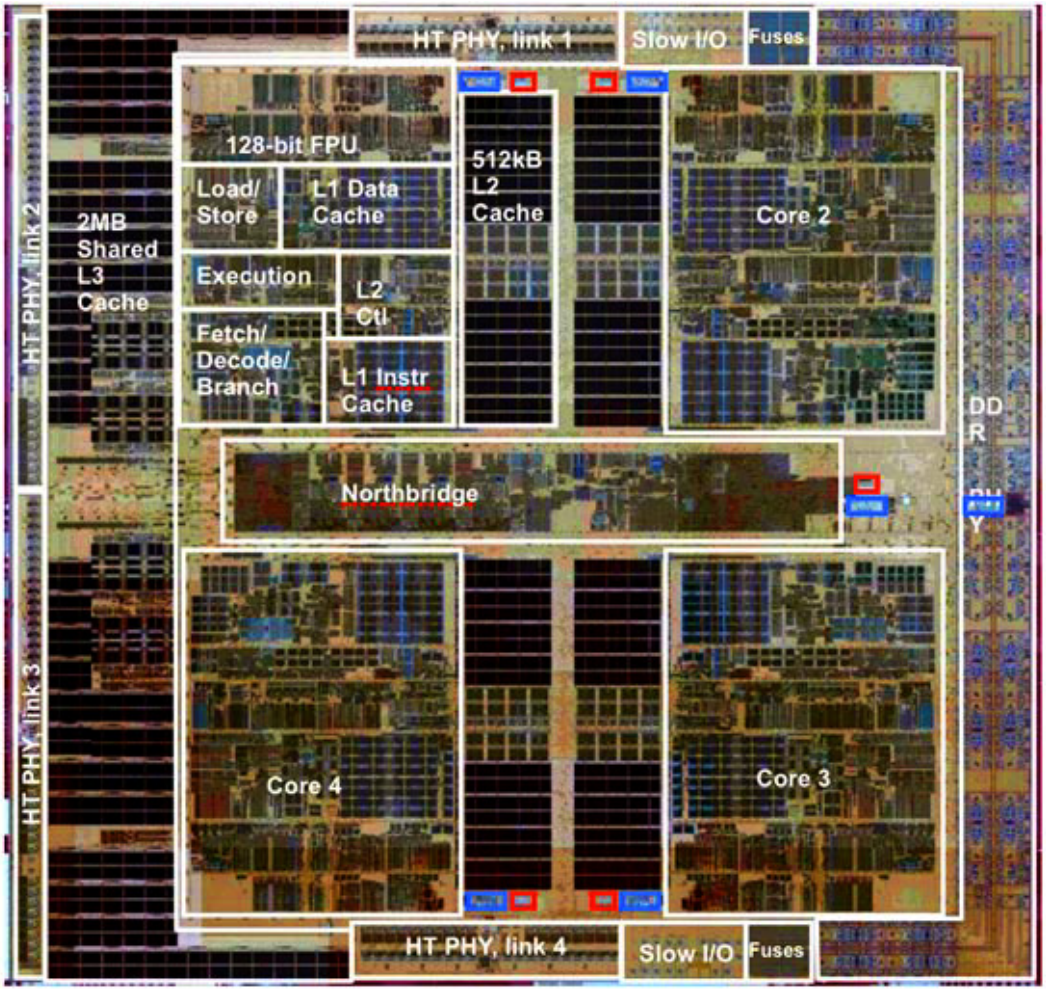


Figure 1.12: ภายใน AMD Barcelona และแผนผังหน้าที่ของแต่ละส่วน

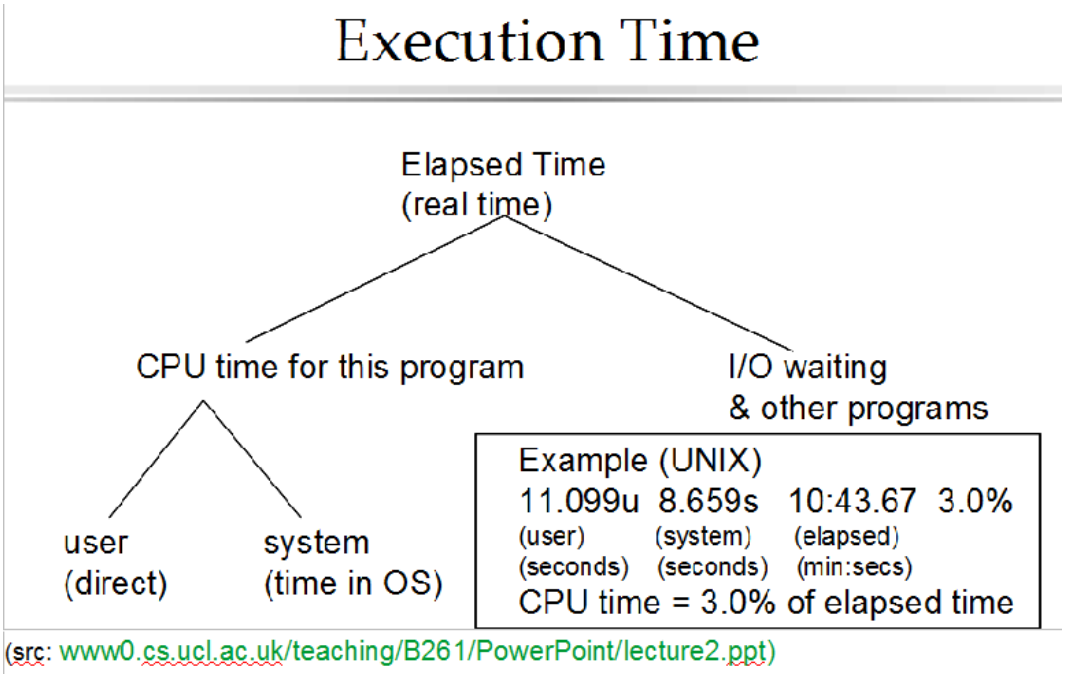


Figure 1.13: Execution Time

เวลาของซีพียู:

$$\text{CPU execution time for a program} = \text{CPU clock cycles for a program} \times \text{Clock cycle time} \tag{1.3}$$

Design Trade-off เทคนิคหลายๆอย่างที่ใช้ลดจำนวน clock cycles มักจะมีผลไปเพิ่ม clock cycle time.

ตัวอย่าง คอมพิวเตอร์ A มี clock rate 400 MHz, ใช้เวลา 10 วินาที เพื่อรันโปรแกรม ก เสร็จ. นักออกแบบคอมพิวเตอร์กำลังออกแบบ คอมพิวเตอร์ B ตั้งเป้าไว้ว่าให้ทำโปรแกรม ก เสร็จในเวลา 6 วินาที โดยจะใช้เทคนิคในการเพิ่ม clock rate. แต่เทคนิคนี้มีผลข้างเคียงคือทำให้ เครื่อง B ใช้ clock cycles เป็น 1.2 เท่าของเครื่อง A. เช่นนี้แล้ว เครื่อง B ควรมี clock rate เท่าไร?

$$\begin{aligned}
\text{CPU time}_A &= \text{clock cycles}_A/\text{clock rate}_A \\
\text{clock cycles}_A &= 10\text{sec} \times 400 \times 10^6\text{Hz} = 4\text{G cycles} \\
\text{CPU time}_B &= 1.2 \times \text{clock cycles}_A/\text{clock rate}_B \\
\text{clock rate}_B &= 1.2 \times 4\text{G cycles}/6\text{sec} = 0.8\text{GHz} = 800\text{Hz}
\end{aligned}$$

จากสมการ 1.3, จำนวน CPU clock cycles เราสามารถหาได้จาก:

$$\text{CPU clock cycles (for finishing a program)} = \text{Instruction for a program} \times \text{average clock cycles per instructi} \tag{1.4}$$

จำนวนสัญญาณนาฬิกาเฉลี่ยต่อคำสั่ง (average clock cycles per instruction) เรียกย่อๆ เป็น CPI. CPI ช่วยเป็นช่องทางในการเปรียบเทียบ implementations ที่ต่างกันสำหรับ instruction set architecture เดียวกัน, เนื่องจาก จำนวน instruction (instruction count) ของโปรแกรมจะเท่ากัน.

ตัวอย่าง สำหรับ ISA เดียวกัน, เปรียบเทียบ 2 implementations: คอมพิวเตอร์ A มี clock cycle time of 1ns และ CPI of 2.0 สำหรับ โปรแกรม ก และ คอมพิวเตอร์ B มี clock cycle time of 2ns และ CPI of 1.2 สำหรับ โปรแกรม ก.

$$\begin{aligned}
\text{CPU time}_A &= \text{clock cycles for a program} \times \text{cycle time} = (I \times 2.0) \times 1ns \\
\text{CPU time}_B &= (I \times 1.2) \times 2ns
\end{aligned} \tag{1.5}$$

เมื่อ I เป็นจำนวน instructions ของโปรแกรม. ค่า I จะเท่ากันทั้ง 2 implementations, เพราะว่า ใช้ ISA เดียวกัน.

ดังนั้น, เราจะได้

$$\frac{\text{Performance}_A}{\text{Performance}_B} = \frac{\text{Execution Time}_B}{\text{Execution Time}_A} = (2.4 \times I)/(2.0 \times I) = 1.2. \tag{1.6}$$

เรียกได้ว่า คอมพิวเตอร์ A เร็วเป็น 1.2 เท่าของเครื่อง B สำหรับโปรแกรม ก.

ทบทวน

- CPU time = Instruction count x CPI x Clock cycle time.
- CPU time = Instruction count x CPI / Clock rate.

How can we determine the value of these factors in the performance equation?

- We can measure the CPU execution time by running the program.
- The clock cycle time is usually published as part of the documentation for a machine.
- The instruction count and CPI can be more difficult to obtain.
- Once we know execution time and cycle time, we need only one of the instruction count or the CPI to determine each other.
- We can measure the instruction count by using software tools that profile the execution or by using a simulator of the architecture.
- Alternatively, we can use hardware counters, included on some processors, to record a variety of measurements, including the number of instructions executed.
- The instruction count depends on ISA (as well as a compiler used), but not on implementation.
- The CPI depends on a wide variety of design details, including the memory system, the processor structure, the mix of instruction types executed in a program.

ตัวอย่าง Comparing code segments นักออกแบบคอมไพเลอร์ กำลังเลือก code segment สำหรับคอมพิวเตอร์ X โดยคอมพิวเตอร์ X มี ISA ที่มี 3 กลุ่มคำสั่ง:

Instruction class	CPI for this instruction class
A	1
B	2
C	3

สำหรับคำสั่ง ก ในภาษาสูง, นักออกแบบคอมไพเลอร์ กำลังเลือกระหว่าง 2 code sequences ที่ใช้คำสั่งดังนี้:

	Instruction counts for instruction class			
Code sequence	A	B	C	remark
SEQ1	2	1	2	
SEQ2	4	1	1	

Code sequence ไหนใช้คำสั่งมากกว่า

- Code SEQ1 ใช้ 2 + 1 + 2 = 5 instructions.
- Code SEQ2 ใช้ 4 + 1 + 1 = 6 instructions.

Code SEQ1 ใช้คำสั่งน้อยกว่า.

Code sequence ไหนเร็วกว่า จาก CPU execution time = CPU clock cycles per program x clock cycle time,

- Code SEQ1: CPU clock cycles = $\sum_i CPI_i \cdot C_i = (1 \times 2) + (2 \times 1) + (3 \times 2)$ = 10 cycles. CPU exe time = $10 \times$ cycle time.
- Code SEQ2: CPU clock cycles = $(1 \times 4) + (2 \times 1) + (3 \times 1)$ = 9 cycles (for the same statement). CPU exe time = $9 \times$ cycle time.

Code SEQ2 เร็วกว่า.

CPI ของแต่ละ code sequence เป็นเท่าไร?

- Code SEQ1: CPI = Clock cycles / instruction count = 10 / 5 = 2.0.
- Code SEQ2: CPI = 9 / 6 = 1.5

Code SEQ2 has a lower CPI.

ตัวอย่างนี้แสดงให้เห็นถึงการใช้ instruction count หรือ CPI ในการวัดสมถนะคอมพิวเตอร์.

ความเร็วของการรันโปรแกรม ขึ้นกับ algorithm, ภาษา, คอมไพเลอร์, สถาปัตยกรรม, และตัวฮาร์ดแวร์.

- Algorithm มีผลกับ Instruction count และ อาจจะ CPI ด้วย. อัลกอริทึ่ม จะบอกว่าจำนวนคำสั่งในโปรแกรมภาษาสูงมีเท่าไร และจำนวนคำสั่งในภาษาสูงก็สัมพันธ์กับจำนวนคำสั่งของเครื่อง. อัลกอริทึ่ม อาจจะมีผลกับ CPI โดยการเลือกใช้คำสั่งที่ช้ากว่า หรือ เร็วกว่า.
- Programming language มีผลกับ Instruction count และ CPI. คำสั่งในภาษาสูงจะแปลงไปเป็นคำสั่งของเครื่อง. ภาษามีผลกับ CPI ด้วยเช่น การใช้ data abstraction เยอะๆ เช่น Java จะใช้ indirect calls มาก ซึ่งมี CPI สูง.
- Compiler มีผลกับ Instruction count และ CPI.
- Instruction Set Archetecture มีผลกับ Instruction count, clock rate, และ CPI. ISA มีผลกับทุกส่วนประกอบของสมการ CPU Performance เพราะมันระบุคำสั่งที่ใช้ได้, และความซับซ้อนของมันมีผลต่อการสร้างฮาร์ดแวร์ ที่มีผลต่อ CPI และ clock rate.

ด้วย pipelining, โปรเซสเซอร์บางตัวสามารถ fetch และ execute หลายๆคำสั่งต่อ 1 clock cycle ทำให้ CPI < 1. บางครั้ง นักออกแบบอาจจะใช้ IPC (instructions per clock cycle) แทน CPI เช่น IPC = 2 คือ CPI = 0.5.

ตัวอย่าง ถ้าแอพพลิเคชั่นที่เขียนด้วยจาวา ใช้เวลารัน 15 วินาที, พอมีจาวาคอมไพเลอร์ตัวใหม่ออกมาแอพพลิเคชั่นนี้จะใช้คำสั่งแค่ 0.6 เท่าของจำนวนคำสั่งเดิม แต่คำสั่งชุดใหม่นี้มี CPI เป็น 1.1 เท่าของ CPI เดิม. แอพพลิเคชั่นนี้จะใช้เวลารันเท่าไร ถ้าคอมไพล์ใหม่.

$$\begin{aligned}
\text{จาก exec time} &= \text{instructions} \times \text{CPI/clock rate}, \\
\text{exec time เดิม} &= 15 = n \times m/r. \\
\text{exec time ใหม่} &= x = 0.6n \times 1.1m/r. \\
\text{ดังนั้น exec time ใหม่} &= 15 \times 0.6 \times 1.1 = 9.9sec.
\end{aligned}$$

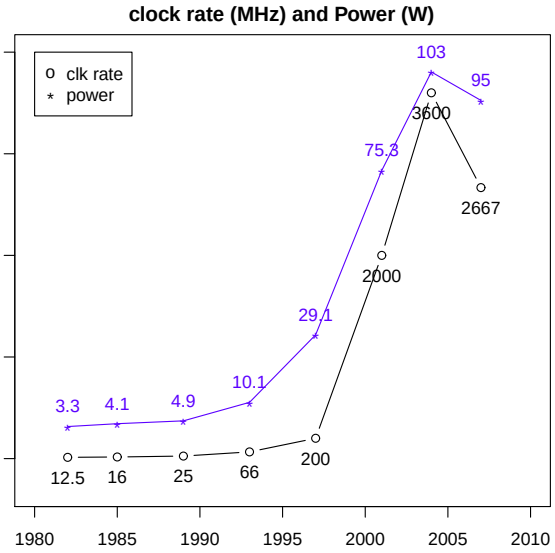


Figure 1.14: อัตราสัญญาณนาฬิกาและกำลังงานของไมโครโปรเซสเซอร์ Intel x86 ในช่วงเวลา 25 ปี: 80286, 80386, 80486, Pentium, Pentium Pro, P4 Willamette, P4 Prescott, และ Core 2 Kentsfield

1.4 กำแพงกำลังงาน (The Power Wall) และการเปลี่ยนจาก uniprocessors ไปสู่ multiprocessors

รูป 1.14 แสดงความสัมพันธ์ของอัตราสัญญาณนาฬิกากับกำลังงานของซีพียู ซึ่ง Intel Pentium 4 Prescott ใช้ถึง 103 วัตต์ ประเด็นนี้ทำให้นักออกแบบตระหนักว่า การปรับปรุงประสิทธิภาพของซีพียู ต่อไปจะต้องมองหาวิธีอื่นมากกว่า แค่เพิ่มความเร็วสัญญาณนาฬิกา.

> Up to now, most software has been like music written for a solo performer; with the current generation of chips we're getting a little experience with duets and quartets and other small ensembles; but scoring a work for large orchestra and chorus is a different kind of challenge. - Brian Hayes, Computing in a Parallel Universe, 2007.

เพื่อลดปัญหาเรื่องกำลังงาน, ซีพียู ตั้งแต่ปี ค.ศ. 2006 ที่ประกอบไปด้วยหลายหน่วยประมวลผล (processor) และ เพื่อกันการสับสนระหว่าง processor และ microprocessor. หน่วยประมวลผลที่ประจุอยู่ใน package เดียวกันจะเรียกว่า core แทน. ดังนั้น quadcore microprocessor คือ ชิปที่มีหน่วยประมวลผล 4 หน่วยอยู่ภายใน.

ด้วยการที่เพิ่มหน่วยประมวลผล แทนที่การเพิ่มความเร็วของหน่วยประมวลผลแต่ละหน่วย ทำให้ โปรแกรมเมอร์ ต้องเขียนโปรแกรมใหม่ หากต้องการใช้ประโยชน์จาก multicore technology.

การเขียนโปรแกรมเชิงขนาน (parallel programming) ยาก. หนึ่ง เพราะว่า นอกจากโปรแกรมต้องทำงานได้ถูกต้อง ตามต้องการแล้ว และ มี interface ที่ดีกับผู้ใช้ แล้ว โปรแกรมยังต้องเร็วด้วย. สอง เพราะว่า โปรแกรมเมอร์ต้องกระจายงานให้กับหน่วยประมวลผลแต่ละตัว และ ลด overhead ของ scheduling และ coordinating ไปด้วย.

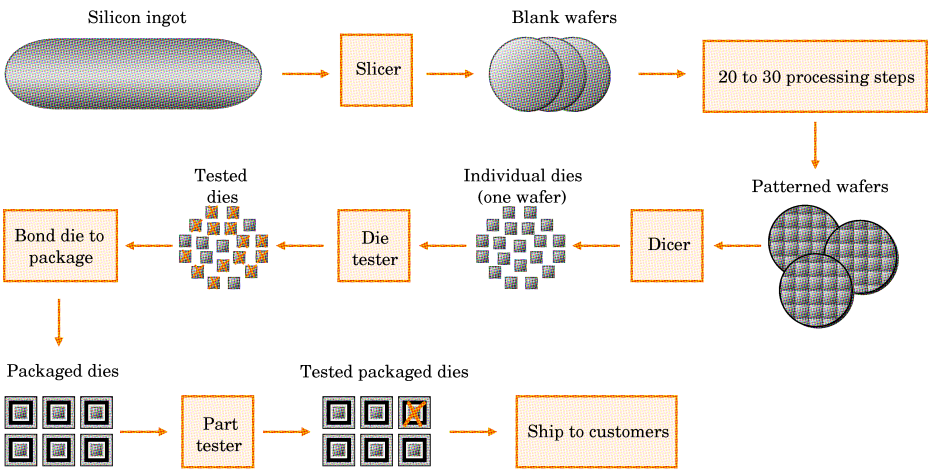


Figure 1.15: กระบวนการผลิตไมโครโปรเซสเซอร์

1.5 กระบวนการผลิตไมโครโปรเซสเซอร์ชิป

แผ่นซิลิกอนเวเฟอร์ (Silicon wafers) ทำมาจากทราย. จากซิลิกอนเวเฟอร์ ก็จะผ่านกระบวนการทางเคมีต่างๆรวมถึง การใส่ impurity (N-type: donors/group V เช่น phosphorus, arsenic หรือ P-type: acceptors/group III เช่น boron, aluminium), เพื่อที่จะให้คุณสมบัติทางไฟฟ้าที่ต้องการ.

ซึ่งเมื่อเปลี่ยนคุณสมบัติทางไฟฟ้าของส่วนต่างๆตามต้องการแล้ว ก็จะได้วงจรไฟฟ้าที่ต้องการบนแผ่นเวเฟอร์ ซึ่งเวเฟอร์ที่ผ่านกระบวนการจนวงจรเสร็จสมบูรณ์แล้วเราจะเรียกว่า ``fabricated wafers'' และกระบวนการที่ทำ จะเรียกว่า ``fabrication''.

แผ่น fabricated wafer หนึ่งแผ่นจะมีวงจรอยู่หลายวงจร, [กระบวนการ dicing] วงจรแต่ละวงจรจะถูกตัดแยกออกมาเป็นชิ้น เรียกว่า ``die''. จากนั้น die ก็จะถูกนำไปติดกับ package. กระบวนการทั้งหมดหลังจาก fabrication จะเรียกรวมๆว่า ``assembly''. รูป 1.15 แสดงแผนผังคร่าวของกระบวนการผลิตไมโครโปรเซสเซอร์.

ค่าใช้จ่ายในการผลิตวงจรรวม คำนวณได้จาก

$$\begin{aligned}
\text{cost per die} &= \frac{\text{cost per wafer}}{\text{dies per wafer} \times \text{yield}} \\
\text{dies per wafer} &\approx \frac{\text{wafer area}}{\text{die area}} \\
\text{yield} &= \frac{1}{(1 + \text{defects per area} \times \text{die area}/2)^2}
\end{aligned}$$

สมการบนสุดตรงไปตรงมา. สมการที่สองเป็นการประมาณ. ส่วนสมการที่สามได้จากการทดลอง/ปฏิบัติ.

ตัวอย่าง (PH4e ex1.11) The following table shows manufacturing data for various processors.

	Wafer Diameter	Dies per Wafer	Defects per Unit Area	Cost per Wafer
a.	15cm	84	0.020 defects/cm^2	\$12
b.	20cm	100	0.031 defects/cm^2	\$15

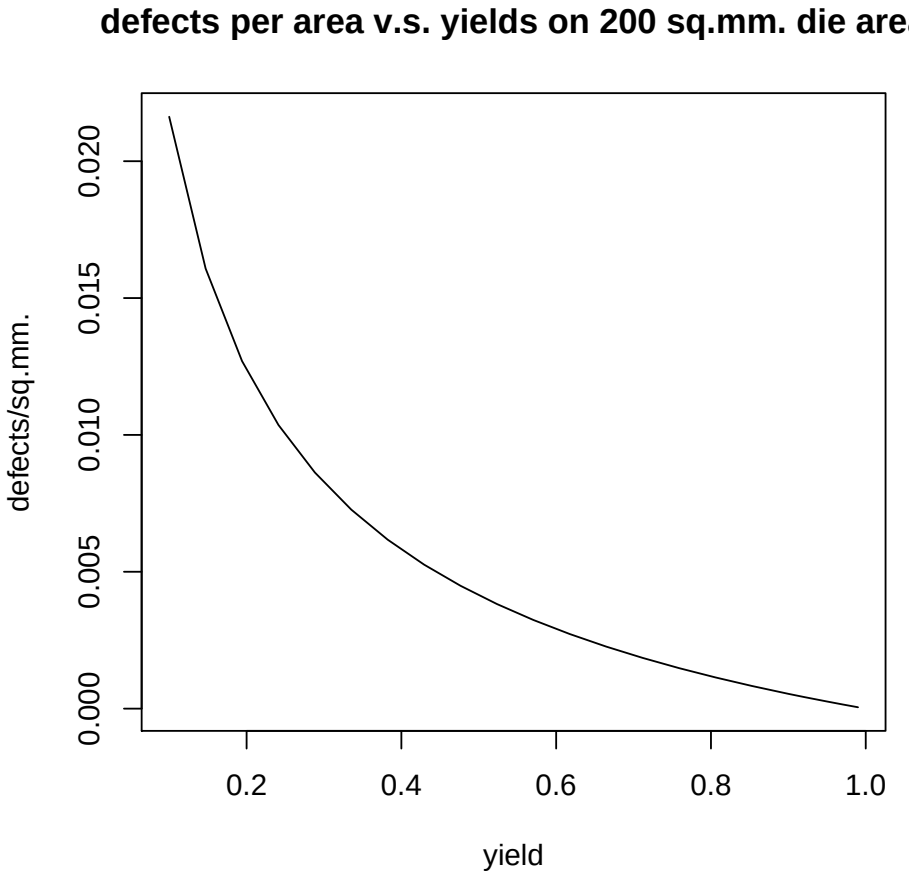


Figure 1.16: ความสัมพันธ์ระหว่าง defects per area กับ yield

เราจะได้ die area เป็น $\frac{\pi 15^2}{84} = 8.41$ และ $\frac{\pi 20^2}{100} = 12.57$ sq. mm. สำหรับ processor a และ b ตามลำดับ.

ได้ yield เป็น $\frac{1}{(1+0.020 \cdot 8.41/2)^2} = 0.85$ และ $\frac{1}{(1+0.031 \cdot 12.57/2)^2} = 0.70$ สำหรับ processor a และ b ตามลำดับ.

และจะได้ cost per die เป็น $12/(84 \cdot 0.85) = 0.168$ และ $15/(100 \cdot 0.70) = 0.214$ สำหรับ processor a และ b ตามลำดับ.

รูป 1.16 แสดงความสัมพันธ์ระหว่าง defects per area กับ yield. (กราฟวาดจาก R Code ข้างล่าง)

```
defects.per.area <- function(yield, die.area){
  d <- 2/die.area * (1/sqrt(yield) - 1)
return(d)
}

yields <- c(seq(0.1,0.9, len=18), 0.92, 0.95, 0.97, 0.99)

plot(yields, defects.per.area(yields, die.area=200), type='l',
  main='defects per area v.s. yields on 200 sq.mm. die area',
  xlab='yield', ylab='defects/sq.mm.')
```

1.6 แหล่งของภาพประกอบ

รูป 1.1: ก. http://en.wikipedia.org/wiki/File:Bardeen_Shockley_Brattain_1948.JPG. ข. http://en.wikipedia.org/wiki/File:NPN_BJT_(Planar)_Cross-section.svg. ค. http://www.computerhistory.org/semiconductor/timeline/1947-invention.html. ง. http://en.wikipedia.org/wiki/File:TTL_npn_nand.svg. จ. http://en.wikipedia.org/wiki/File:2-bit_ALU.svg. ฉ. P&H 3e, Fig 5.2.

รูป 1.2 P&H 3e, Fig 1.4.

รูป 1.3 P&H 2e, Fig 1.2.

รูป 1.4 P&H 2e, Fig 1.3.

รูป 1.7 P&H 3e, Fig 1.6.

รูป 1.8 P&H 3e, Fig 1.8.

รูป 1.9 P&H 3e, Fig 1.10.

รูป 1.11 P&H 3e, Fig 1.9.

รูป 1.12 http://hothardware.com.

รูป 1.15 P&H 2e Fig 1.15 (f0115.eps).

Chapter 2

ชุดคำสั่ง: ภาษาของคอมพิวเตอร์

เราสั่งงานคอมพิวเตอร์ได้โดย ใช้คำสั่งภาษาคอมพิวเตอร์ (instruction).

``It is easy to see by formal-logical methods that there exist certain [instruction sets]" that are in abstract adequate to control and cause the execution of any sequence of operations .... The really decisive considerations from the present point of view, in selecting an [instruction set], are more of a practical nature: simplicity of the equipment demanded by the [instruction set], and the clarity of its application to the actually important problems together with the speed of its handling of those problems." - Burks, Goldstine, and von Neumann, 1947.

นักออกแบบคอมพิวเตอร์มีเป้าหมายเหมือนกัน คือ: สร้างชุดคำสั่งที่ง่ายที่จะสร้างฮาร์ดแวร์และคอมไพเลอร์ ขณะเดียวกันก็ทำให้การทำงานของคอมพิวเตอร์มีสมถนะสูงสุด ในค่าใช้จ่ายที่ต่ำที่สุด.

2.1 การคำนวณพีชคณิต

คอมพิวเตอร์ต้องสามารถคำนวณพีชคณิต(บวก-ลบ)ได้. รูปแบบแอสเซมบลีของ MIPS คือ

```
add a, b, c
```

สำหรับ สั่งให้คอมพิวเตอร์ บวก ตัวแปร `b` และ `c` แล้วเก็บผลใน `a`.

หากเราต้องการบวกตัวแปร `b`, `c`, `d`, และ `e` เราอาจทำ

```
add a, b, c
add a, a, d
add a, a, e
```

สำหรับทำนิพจน์ $a = b + c + d + e$. MIPS กำหนดจำนวน operands สำหรับคำสั่ง `add` ไว้คงที่ 3 ตัว (สองตัวหลังบวกกันแล้วเก็บไว้ในตัวแรก). การที่บังคับให้ทุกๆคำสั่งมี 3 operands เท่ากันทำให้ฮาร์ดแวร์เรียบง่าย ซึ่งตรงกับ หลักการออกแบบข้อที่หนึ่ง Design Principle 1: Simplicity favors regularity.

ตัวอย่าง: คอมไพล์คำสั่งภาษาซี เป็น MIPS คำสั่งภาษาซี:

```
a = b + c;
d = a - e;
```

จะแปลงไปเป็น MIPS ได้:

```
add a, b, c
sub d, a, e
```

ตัวอย่าง: คอมไพล์คำสั่งซับซ้อนภาษาซี เป็น MIPS คำสั่งภาษาซี:

```
f = (g + h) - (i + j);
```

จะถูกคอมไพเลอร์ภาษาซีแปลงเป็นคำสั่ง MIPS หลายๆคำสั่ง

```
add t0, g, h # temporary t0
add t1, i, j # temporary t1
sub f, t0, t1
```

2.2 Operands สำหรับฮาร์ดแวร์

Operands สำหรับคำสั่งเครื่อง มีจำกัด: มันต้องจำกัด เพราะว่า ตัว Operands คือ registers ที่จะถูกสร้างเป็นฮาร์ดแวร์. รีจิสเตอร์ (register) คือ วงจรดิจิตอลที่ใช้เก็บข้อมูลหนึ่ง word. ขนาดของรีจิสเตอร์ของสถาปัตยกรรม MIPS คือ 32 bits.

ข้อแตกต่างที่สำคัญระหว่าง ตัวแปรในภาษาสูง กับ รีจิสเตอร์ คือ รีจิสเตอร์มีจำนวนจำกัด. MIPS มีรีจิสเตอร์ 32 ตัว. Operand ของคำสั่ง จะต้องมาจากหนึ่งในรีจิสเตอร์ 32 ตัวนี้.

จุดประสงค์ที่จำกัดจำนวนรีจิสเตอร์ไว้แค่ 32 ตัว เป็นไปตามหลักการออกแบบที่สอง Design Principle 2: Smaller is faster.

ถ้ามีจำนวนรีจิสเตอร์มากๆ อาจจะทำให้สัญญาณเดินทางช้าลง (clock period ต้องยาวขึ้น) เพราะสัญญาณต้องเดินทางไกลขึ้น.

เราอาจอ้างถึงรีจิสเตอร์ตามดัชนี 0 ถึง 31 แต่ MIPS ตั้งชื่อรีจิสเตอร์ไว้ โดยใช้ `$' นำหน้าชื่อ และนิยมใช้ `$s0`, `$s1`, ... สำหรับ ตัวแปรในภาษาสูง และ `$t0`, `$t1`, ... สำหรับรีจิสเตอร์ชั่วคราว.

ตัวอย่าง: คอมไพล์ภาษาซี โดยใช้รีจิสเตอร์. จากภาษาซี

```
f = (g + h) - (i + j);
```

สมมติให้ ตัวแปร `f`, `g`, `h`, `i`, และ `j` ถูกหมายเป็น รีจิสเตอร์ `$s0`, `$s1`, `$s2`, `$s3`, `$s4` ตามลำดับ. คอมไพเลอร์จะให้ MIPS โค้ดเป็น:

```
add $t0, $s1, $s2
add $t1, $s3, $s4
sub $s0, $t0, $t1
```

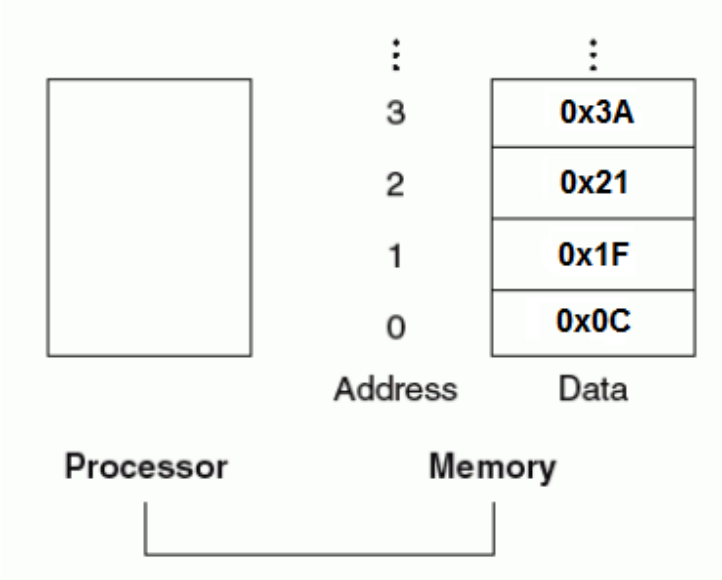


Figure 2.1: แอดเดรสและข้อมูลในหน่วยความจำ

2.2.1 Memory Operands

นอกจากตัวแปรพื้นฐานแล้ว ภาษาสูงยังมีโครงสร้างข้อมูลที่ซับซ้อนขึ้น เช่น array และ structure. โครงสร้างข้อมูลที่ซับซ้อนเหล่านี้ (อาร์เรย์ และ สตรัค) จะถูกเก็บในหน่วยความจำ. ดังนั้น MIPS มีคำสั่งที่ใช้ย้ายข้อมูลหน่วยความจำและรีจิสเตอร์ คำสั่งเหล่านี้เรียกว่า data transfer instructions. เพื่อจะเข้าถึงข้อมูลในหน่วยความจำ เราจะต้องใส่ แอดเดรส (address). ให้คิดถึงหน่วยความจำ เป็นเหมือนกับ อาร์เรย์หนึ่งมิติขนาดใหญ่ ที่แอดเดรส ทำหน้าที่เหมือน ดัชนีของอาร์เรย์ โดย แอดเดรส เริ่มที่ 0. รูป 2.1 แสดงตัวอย่างหน่วยความจำ แอดเดรส และ ข้อมูลที่ประจุอยู่. ที่แอดเดรส 2, ข้อมูล `Memory[2]` คือ `0x21` (หรือ 21_{16} เท่ากับ 33 ฐานสิบ).

คำสั่งที่ใช้ย้ายข้อมูลจากหน่วยความจำไปรีจิสเตอร์จะเรียกว่าโหลด (load). รูปแบบคำสั่งของ MIPS คือ `lw RD, off(RB)`.

ตัวอย่าง: โหลดค่าจากหน่วยความจำ ภาษาซี

```
g = h + A[8];
```

สมมติคอมไพล์ใช้ `$s1` และ `$s2` สำหรับ `g` กับ `h` ตามลำดับ และ ใช้ `$s3` เป็นค่าแอดเดรสเริ่มต้น (เรียกว่า base address) ของอาร์เรย์ `A`, MIPS code จะเป็น:

```
lw $t0, 8($s3)
add $s1, $s2, $t0
```

ค่า 8 เรียกว่า offset และ `$s3` เรียกว่า base register. หมายเหตุ ค่า 8 ที่ใช้ยังถูกต้อง ดังที่จะอธิบายต่อไปนี้.

เนื่องจาก ข้อมูลขนาดละ 8 bits (หรือ เรียก 1 byte) มีประโยชน์ในหลายๆโปรแกรม สถาปัตยกรรมส่วนใหญ่จะอ้างหน่วยความจำเป็น byte (1 แอดเดรส อ้างถึง 1 byte). ดังนั้นใน MIPS-32 แอดเดรสของ word จะต้องใช้หนึ่งในสี่แอดเดรสของ byte ที่ประกอบเป็น word. และแอดเดรสของ word ที่ติดกันจะต่างกัน 4. การที่แอดเดรสของแต่ละ word เป็น multiple of 4 นี้จะ

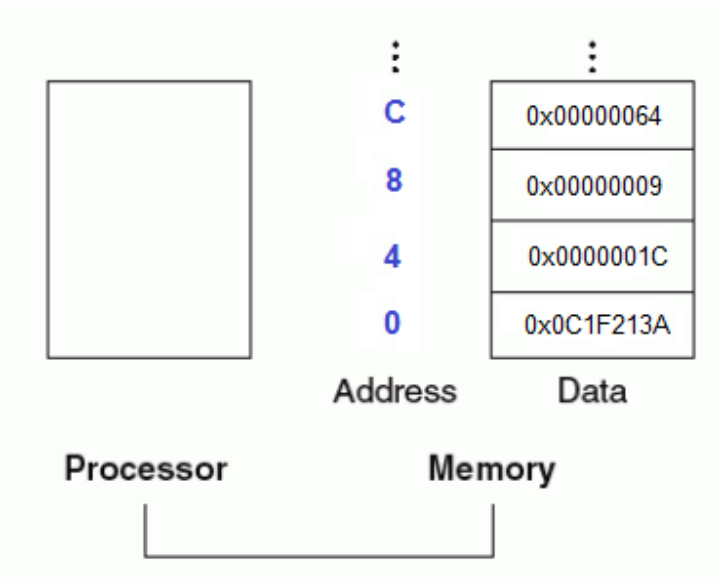


Figure 2.2: แอดเดรสของหน่วยความจำของคอมพิวเตอร์ 32 bit

เรียกว่า alignment restriction. ตัวอย่างเช่น เราโหลด word ที่ `address 0`, `address 4`, `address 8`, ... ได้ แต่เราไม่สามารถโหลด word ที่ `address 1`, หรือ `address 2`, หรือ `address 3` ได้.

แอดเดรสของ byte ที่เลือกมาเป็น แอดเดรสของ word จะมี 2 แบบ: big end ใช้ แอดเดรสของ byte ซ้ายสุดของ word (byte ที่มีค่าใหญ่ที่สุด) เรียกเป็น big endian กับ little end ใช้แอดเดรสของ byte ขวาสุดของ word (byte ที่มีค่าเล็กที่สุด) เรียกเป็น little endian. ตัวอย่างเช่น จากรูป 2.1 ถ้าคอมพิวเตอร์เป็น big endian ค่า word ที่ `address 0` จะเป็น `0x0C1F213A`. แต่ถ้าใช้ little end, ค่า word ที่ `address 0` จะเป็น `0x3A211F0C`. Byte addressing มีผลกับ array index. เพื่อจะอ้างถึงแอดเดรสอย่างถูกต้อง ค่า offset ที่จะไปบวกกับรีจิสเตอร์ `$s3` จะต้องคูณ 4 เข้าไปด้วย ($4 \times 8 = 32$) ดังนั้น ที่ถูกต้อง มันต้องเป็น `lw $t0, 32($s3)` เพื่อโหลดค่า `A[8]`. รูป 2.2 แสดง word address สำหรับ คอมพิวเตอร์ 32-bit ที่เป็น big endian.

คำสั่งโหลดใช้ย้ายข้อมูลจากหน่วยความจำเข้ามาในรีจิสเตอร์ ส่วนคำสั่งที่ใช้เก็บข้อมูลจากรีจิสเตอร์เข้าไปในหน่วยความจำคือ store. รูปแบบ คือ `sw RD, off(RB)`.

ตัวอย่าง load/store. ภาษาซี

```
A[12] = h + A[8];
```

สมมติคอมไพเลอร์ ใช้ `$s2` กับ `$s3` เก็บตัวแปร `h` และ base address ของอาร์เรย์ `A` ตามลำดับ, จะได้ MIPS assembly เป็น:

```
lw $t0, 32($s3)
add $t0, $s2, $t0
sw $t0, 48($s3)
```

สำหรับบางโปรแกรมที่ใช้ตัวแปร มากกว่า จำนวนรีจิสเตอร์ที่มีอยู่ คอมไพเลอร์จะเลือกตัวแปรที่ใช้บ่อยๆมากกว่าให้อยู่ในรีจิสเตอร์ และ เก็บตัวแปรที่เกินไว้ในหน่วยความจำ โดยใช้คำสั่ง load กับ store. สิ่งที่คอมไพเลอร์เกินตัวแปรที่เกินไว้ในหน่วยความจำนี้ เรียกว่า ``spilling registers".

2.2.2 Constant or Immediate Operands

คำสั่งที่ใช้บวกกับค่าคงที่ เรียกว่า add immediate หรือ `addi`, เช่น `addi $s3, $s3, 4` จะบวก 4 เข้ากับค่าในรีจิสเตอร์ `$s3` และ เก็บกลับเข้าไปใน `$s3` (เทียบเท่ากับ เพิ่มค่า `$s3` ขึ้น 4).

นอกจากนี้ เพื่อความสะดวก MIPS ให้ รีจิสเตอร์ 0 เป็น `$zero` สำหรับ เก็บค่าคงที่ 0. ทำเช่นนี้ การย้ายข้อมูลจากรีจิสเตอร์ตัวหนึ่งไปอีกตัวก็เทียบเท่ากับบวกด้วย 0 เช่น `add $s2, $s1, $zero` เทียบเท่าการสำเนาข้อมูลจาก `$s1` ไป `$s2`.

หมายเหตุ MIPS supports negative constants, ดังนั้นจึงไม่มีคำสั่งลบค่าคงที่ใน MIPS.

2.3 Signed and Unsigned Numbers

ขณะที่มนุษย์เราจะใช้ตัวเลขในฐานสิบ คอมพิวเตอร์ฮาร์ดแวร์จะเก็บข้อมูลเป็นเลขฐานสอง. ตัวเลขในฐานสิบจะเรียก ``decimal number'' ส่วนตัวเลขฐานสองจะเรียก ``binary number''. ตัวเลขแต่ละหลัก (digit) ในเลขฐานสองของข้อมูล จะเรียกว่า บิต (bit). แต่ละบิต จะมีได้หนึ่งในสองค่า: 0 หรือ 1 (on หรือ off, high หรือ low, true หรือ false).

ถ้าเขียนเป็นสูตรค่าของเลขในฐานต่างๆโดยทั่วไป, $d_n \cdots d_3 d_2 d_1 d_0$ ในฐาน B จะมีค่า $d_n B^n + \cdots + d_3 B^3 + d_2 B^2 + d_1 B^1 + d_0 B^0$. ตัวอย่างเช่น 1011_{two} แทนค่า

$$\begin{aligned}&(1 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (1 \times 2^0)\\ &= (1 \times 8) + 0 + (1 \times 2) + (1 \times 1) = 11\end{aligned}$$

ในฐานสิบ.

เราจะเรียกบิตที่มีค่ามากที่สุดว่า most significant bit (msb) เช่น บิตที่ 31 ของ MIPS และบิตที่มีค่าน้อยที่สุดว่า least significant bit (lsb) เช่น บิตที่ 0 ของ MIPS.

MIPS word มี 32 บิต ดังนั้นสามารถ แทนข้อมูล ได้ 2^{32} แบบ. คิดง่ายๆ ถ้าค่า word นี้ แทนเลขฐานสิบก็จะได้

$$\begin{aligned}00000000000000000000000000000000_{two} &= 0_{ten}\\ 00000000000000000000000000000001_{two} &= 1_{ten}\\ 00000000000000000000000000000010_{two} &= 2_{ten}\\ &\cdots\\ 11111111111111111111111111111101_{two} &= 4,294,967,293_{ten}\\ 11111111111111111111111111111110_{two} &= 4,294,967,294_{ten}\\ 11111111111111111111111111111111_{two} &= 4,294,967,295_{ten}\end{aligned}$$

เลขที่ฮาร์ดแวร์แทนได้มีจำกัด แต่ ตัวเลขในโลกจริงๆไม่ได้จำกัด. ฮาร์ดแวร์ สามารถบวก ลบ คูณ หาร ข้อมูลในรูปฐานสองเหล่านี้ได้. ถ้า เราทำการบวก ลบ คูณ หาร ตัวเลข แล้ว คำตอบที่ถูกต้องออกมา เกินช่วงที่ฮาร์ดแวร์สามารถแสดงได้ เราจะเรียกว่า เกิด overflow ขึ้น. เมื่อเกิด overflow ขึ้น ก็ขึ้นกับ programming language, OS, หรือ โปรแกรมที่เราเขียนว่าจะจัดการกับ overflow ยังไง.

โปรแกรมคอมพิวเตอร์คำนวณได้ทั้งเลขบวก และ เลขลบ เราต้องการรูปแบบแสดง (representation) ที่แยก เลขบวก ออกจาก เลขลบ ได้. วิธีง่ายๆคือ two's complement representation. ค่า two's complement ของ x (ที่มี n บิต: $x_{n-1}x_{n-2}\cdots x_0$) เท่ากับ $2^n - x$ (= 0 กับหลักยืม $-x$) ตัวอย่างเช่น $x = 1011_{two} = -5_{ten}$ แล้วค่า 2's complement = $\underline{1}0000 - 1011 =$

$0101 = 5_{ten}$. หรือ $x = 0011_{two} = 3_{ten}$ ค่า 2's complement = $\underline{1}0000 - 0011 = 1101 = -3_{ten}$.

$$
\begin{aligned}
00000000000000000000000000000000_{two} &= 0_{ten} \\
00000000000000000000000000000001_{two} &= 1_{ten} \\
00000000000000000000000000000010_{two} &= 2_{ten} \\
&\dots \\
01111111111111111111111111111101_{two} &= 2,147,483,645_{ten} \\
01111111111111111111111111111110_{two} &= 2,147,483,646_{ten} \\
01111111111111111111111111111111_{two} &= 2,147,483,647_{ten} \\
10000000000000000000000000000000_{two} &= -2,147,483,648_{ten} \\
10000000000000000000000000000001_{two} &= -2,147,483,647_{ten} \\
10000000000000000000000000000010_{two} &= -2,147,483,646_{ten} \\
&\dots \\
11111111111111111111111111111101_{two} &= -3_{ten} \\
11111111111111111111111111111110_{two} &= -2_{ten} \\
11111111111111111111111111111111_{two} &= -1_{ten}
\end{aligned}
$$

ข้อดีของ two's complement representation คือ (1) $x - y = x + (-y)$ ทำให้สะดวกในการสร้างฮาร์ดแวร์; (2) เลขลบจะมี '1' ที่ msb. ดังนั้นฮาร์ดแวร์ แค่ตรวจ msb ก็จะรู้ได้ว่าเลขนี้เป็นเลขลบหรือไม่. ถ้า msb = 0, เลขอาจเป็น 0 หรือ เลขบวกก็ได้. ดังนั้น msb สำหรับ signed number representation มักถูกเรียกเป็น sign bit. เราอาจเขียนสูตรค่าของ signed number เป็น:

$$(x_{31} \times -2^{31}) + (x_{30} \times 2^30) + (x_{29} \times 2^{29}) + \ldots + (x_1 \times 2^1) + (x_0 \times 2^0) \tag{2.1}$$

เพื่อความสะดวก ดูตัวอย่างของ 4-bit signed number,

$$
\begin{aligned}
0011_{two} &= (0 \times -2^3) + (0 \times 2^2) + (1 \times 2^1) + (1 \times 2^0) = 3 \\
1011_{two} &= (1 \times -2^3) + (0 \times 2^2) + (1 \times 2^1) + (1 \times 2^0) = -5
\end{aligned}
$$

สังเกตุ. ถ้าให้ $\bar{x}$ แทน x ที่กลับบิต แล้ว $x + \bar{x} = 111\ldots111_{two}$ เสมอ. และเมื่อ $111\ldots111_{two} = -1_{ten}$, ดังนั้น $x + \bar{x} = -1_{ten}$ และ $x + \bar{x} + 1 = 0$. เราจึงได้ $\bar{x} + 1 = -x$ ซึ่งคือ วิธีลัดในการทำ 2's complement. ตัวอย่างเช่น $x = 1011_{two}$, $\bar{x} = 0100_{two}$, และ 2's complement = $0100 + 1 = 0101_{two}$.

``Two's complement gets its name from the rule that the unsigned sum of an n-bit number and its negative is 2^n; hence, the complement or negation of a two's complement number x is $2^n - x$."

2.4 Representing Instructions in the Computer

คำสั่งถูกเก็บในคอมพิวเตอร์ ในรูปลำดับของบิต และอาจมองเป็นตัวเลขได้. รีจิสเตอร์ก็จะถูกอ้างถึงโดยตัวเลข เช่น `$s0` ถึง `$s7` จะถูกแปลงเป็นเลข 16 ถึง 23. ส่วน `$t0` ถึง `$t7` จะถูกแปลงเป็น 8 ถึง 15.

ตัวอย่าง: แปลงคำสั่ง MIPS แอสเซมบลีเป็นคำสั่งภาษาเครื่อง. คำสั่งแอสเซมบลี:

```
add $t0, $s1, $s2
```

นี้จะแปลงเป็นเลข ในฐานสิบคือ

0	17	18	8	0	32

โดยแต่ละส่วนของคำสั่ง เรียกว่า ``ฟิลด์" (field). ฟิลด์แรกสุด และท้ายสุด (ตัวอย่างนี้คือ 0 กับ 32) จะบอกว่า คำสั่งนี้คือ คำสั่งบวก. ฟิลด์ที่สอง บอกรีจิสเตอร์ที่เป็น source operand ของการบวก: 17 คือ รีจิสเตอร์ `$s1`. ฟิลด์ที่สาม บอกรีจิสเตอร์ที่เป็น source operand อีกตัว: 18 คือ รีจิสเตอร์ `$s2`. ฟิลด์ที่สี่ บอกรีจิสเตอร์ที่จะรับผลการบวก: 8 คือ รีจิสเตอร์ `$t0`. ฟิลด์ที่ห้า (ที่เป็น 0) ไม่ได้ใช้ในคำสั่งนี้. คำสั่งนี้ถ้าแสดงเป็นฐานสอง ก็คือ

000000	10001	10010	01000	00000	10000
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

เลย์เอาท์ของคำสั่ง แบบที่เห็นนี้เรียกว่า instruction format. คำสั่งของ MIPS จะมี 32 บิต (ขนาดเท่ากับ data word) เรื่องนี้ก็เข้ากับ design principle: simplicity favors regularity, ทุกคำสั่งของ MIPS ยาว 32 บิต.

เราจะเรียก คำสั่งของเครื่อง ในรูปของตัวเลข แบบนี้ว่า คำสั่งภาษาเครื่อง machine language และเรียก ลำดับของคำสั่งภาษาเครื่อง ว่า machine code. เพื่อความสะดวก แทนที่จะเขียนเลขฐานสองยาวๆ เราอาจเขียน เลขฐานสิบหก (hexadecimal) แทนได้:

- $0000_{two} = 0_{hex}$,
- $0001_{two} = 1_{hex}$,
- $0010_{two} = 2_{hex}$,
- $0011_{two} = 3_{hex}$,
- $0100_{two} = 4_{hex}$,
- $0101_{two} = 5_{hex}$,
- $0110_{two} = 6_{hex}$,
- $0111_{two} = 7_{hex}$,
- $1000_{two} = 8_{hex}$,
- $1001_{two} = 9_{hex}$,
- $1010_{two} = A_{hex}$,
- $1011_{two} = B_{hex}$,
- $1100_{two} = C_{hex}$,
- $1101_{two} = D_{hex}$,
- $1110_{two} = E_{hex}$,
- $1111_{two} = F_{hex}$,

หมายเหตุ `C` กับ `Java` ใช้ `0x` นำหน้าตัวเลขฐานสิบหก เช่น `0x031A` แทน $031A_{hex}$.

MIPS Fields

เพื่อความสะดวก ฟิลด์ของ MIPS มีชื่อ

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

โดย

- `op`: basic operation of the instruction, ``opcode".
- `rs`: the first register operand.
- `rt`: the second register operand.
- `rd`: the register destination operand.
- `shamt`: shift amount.
- `funct`: function, or ``function code", selects the specific variant of the operation in the op field.

บางคำสั่ง เช่น load word หรือ store word ต้องการฟิลด์ที่ยาวกว่า 5 บิต สำหรับ ค่า offset (ค่าคงที่ ที่จะใช้บวกกับ base address) เพราะว่า 5 บิต ใช้อ้างได้แค่ 2^5 หรือ 32 แอดเดรส. ดังนั้น ถ้าเราต้องการฟิลด์ที่ยาวกว่า 5 บิต เราอาจจะต้องยอมยืดหยุ่น design principle บ้าง ดังนั้นเราจึงมีหลักการออกแบบข้อสุดท้าย:

Design Principle 4: Good design demands good compromises. ความยืดหยุ่น ที่นักออกแบบ MIPS ที่เลือกที่จะ ยังคงรักษาให้ทุกคำสั่งยาวเท่ากันหมด และ ผ่อนปรนให้คำสั่งมีได้หลายรูปแบบขึ้น. ดังเช่น

op	rs	rt	rd	shamt	funct

เรียก ``R-type"/R-format (R for register). รูปแบบที่สอง คือ ``I-type"/I-format (I for immediate):

op	rs	rt	constant or address
6 bits	5 bits	5 bits	16 bits

ซึ่ง การใช้ 16-bit address offset ทำให้ load word หรือ store word สามารถย้ายข้อมูลกับหน่วยความจำในช่วง $\pm 2^{13}$ หรือ 8192 words จาก base address (`rs`): 1 bit is for sign 2 bits makes 4-byte address; therefore it is $\pm 2^{13}$, not 2^{16}. ถ้า ใช้ 16-bit นี้เป็นค่าคงที่ มันจะแทนค่าได้ $\pm 2^{15}$ ค่า: $-32,768$ ถึง $32,768$.

Instruction	Format	6 bits (op)	5 bits	5 bits	5 bits	5 bits	6 bits
`add`	R	000000_{two}	rs	rt	rd	0	10000_{two}
`sub`	R	000000_{two}	rs	rt	rd	0	10100_{two}
`addi`	I	001000_{two}	rs	rt	constant		
`lw`	I	100011_{two}	rs	rt	address		
`sw`	I	101011_{two}	rs	rt	address		

ค่าของ opcode (ฟิลด์แรก) จะเป็นตัวบอกว่า format จะเป็นแบบไหน.

Pseudo instruction เพื่อความสะดวกในการเขียนแอสเซมบลี,

```
nop
```

เทียบเท่ากับการทำ

```
sll $0, $0, 0
```

หรือการทำงานกับค่าคงที่ 32 บิต อย่างคำสั่ง

```
li rd, imm32
```

เทียบเท่ากับคำสั่ง

```
lui rd, const16
ori rd, rs, const16
```

เช่น `li $v0,0x48abcdef` พอโหลดเข้าไปใน QtSpim, เราจะเห็น

```
lui $1, 18603     # m/c = 3c0148ab.
                  # $1 has 0x48ab0000.
ori $16, $1, -12817 # m/c = 3430cdef.
                    # then, $16 = 0x48abcdef.
```

ตัวอย่าง: แปลคำสั่ง เป็น คำสั่งภาษาเครื่อง. คำสั่ง `A[300] = h + A[300];` จะถูกคอมไพล์เป็น

```
lw $t0, 1200($t1)
add $t0, $s2, $t0
sw $t0, 1200($t1)
```

เมื่อ `$t1` เก็บ base ของอาร์เรย์ `A` และ `$s2` เชื่อมกับ ตัวแปร `h`.

ซึ่งคือ machine code:

op	rs	rt	rd	shamt	funct
100011	01001	01000	0000 0100 1011 0000		
000000	10010	01000	01000	00000	100000
101011	01001	01000	0000 0100 1011 0000		

สังเกตุ machine code ของคำสั่ง `lw` กับ `sw` ต่างกันที่ opcode และต่างกันบิตเดียว.

ภาพใหญ่. คอมพิวเตอร์ปัจจุบันสร้างบนหลักการใหญ่ๆ 2 อย่าง:

- 1. คำสั่ง เก็บเป็นตัวเลข (แบบเดียวกับข้อมูล).
- 2. โปรแกรม เก็บในหน่วยความจำ และ จะถูกอ่าน/เขียน, เหมือนกับตัวเลข.

หลักการนี้ นำไปสู่ stored-program concept: หน่วยความจำ เก็บทั้ง source code ของโปรแกรมคำนวณภาษี, machine code ที่คอมไพล์มาจาก source code, ข้อมูลภาษี สำหรับโปรแกรมคำนวณภาษี และแม้แต่ machine code ของ compiler ที่ใช้.

ผลจากการเก็บคำสั่งในรูปตัวเลขนี้ ทำให้โปรแกรมสามารถบรรจุอยู่ในรูปไฟล์ของตัวเลข(ฐานสอง)ได้. ผลทางการค้าคือคอมพิวเตอร์สามารถใช้ซอฟต์แวร์ที่สร้างมาแล้วได้ ถ้าคอมพิวเตอร์ compatible กับชุดคำสั่ง(ที่ซอฟต์แวร์ใช้). ดังนั้นทำให้อุตสาหกรรมคอมพิวเตอร์จึงใช้สถาปัตยกรรมชุดคำสั่ง อยู่ไม่กี่แบบ.

การที่ไม่ว่าโปรแกรมหรือข้อมูลก็ถูกเก็บในรูปตัวเลข ทำให้การใช้งานหน่วยความจำง่ายขึ้นมาก เช่น หน่วยความจำจะใช้เก็บข้อมูลหรือโปรแกรมก็ได้ (ไม่ต้องทำแยกเฉพาะ).

2.4.1 Logical Operations

ถึงแม้ คอมพิวเตอร์จะทำงานเป็น word แต่หลายๆสถานการณ์ การทำงานกับเฉพาะบางส่วนของ word หรือแม้แต่ทำงานในระดับบิต ก็มีประโยชน์มากๆ. ดังนั้นทั้ง programming languages และ instrction set architectures ก็มีคำสั่งเกี่ยวกับการจัดการบิตใน word ให้ด้วย.

Logical operations	C operators	Java operators	MIPS instructions
Shift left	<<	<<	`sll`
Shift right	>>	>>	`srl`
Bit-by-bit AND	&	&	`and, andi`
Bit-by-bit OR	|	|	`or, ori`
Bit-by-bit NOT	~	~	`nor`

หมายเหตุ MIPS ใช้ `nor` กับ operand ที่เป็น 0 ทำ bitwise NOT.

ตัวอย่าง shift. สมมติ รีจิสเตอร์ `$s0` มีค่า $0000000000000000000000000001001_{two} = 9_{ten}$ อยู่. และ ถ้า เรา ทำคำสั่ง `sll $t2, $s0, 4` หรือ shift left ไป 4 บิต ค่าใน รีจิสเตอร์ `$t2` จะได้เป็น: $00000000000000000000000010010000_{two} = 144_{ten}$.

หมายเหตุ คำสั่ง shift จะเติมที่ว่างด้วย 0.

กลับมาที่คำสั่งใน R-format, ฟิลด์ shamt ย่อมาจาก shift amount, ดังนั้น machine code ของ `sll $t2, $s0, 4` คือ

op	rs	rt	rd	shamt	funct
0	0	16	10	4	0

opcode และ funct ของคำสั่ง `sll` เป็น 0 ทั้งคู่. รีจิสเตอร์ตัวที่ 10 คือ `$t2` และ ตัวที่ 16 คือ `$s0`. ฟิลด์ rs ไม่ได้ใช้ จึงให้เป็น 0.

การทำ shift left ไป i บิต ให้ผลเทียบเท่ากับการคูณด้วย 2^i (คล้ายกับการ shift เลขฐานสิบไป หนึ่งหลัก ก็เหมือนการคูณ สิบ) ดังนั้นการทำ shift left ไป 4 บิต เท่ากับการคูณ $2^4 = 16_{ten}$ และจากตัวอย่าง การ shift left 9 ไป 4 บิต จึงได้ผลลัพธ์เท่ากับค่า $9 \times 16 = 144_{ten}$.

Operation อีกอย่างที่มีประโยชน์กับการทำงานเป็นบิต คือ AND. ปฏิบัติการ AND ทำงานเป็นบิต โดยผลลัพธ์จะมีค่าบิตเป็น 1 ก็ต่อเมื่อบิตของ operands ทั้งสองตัวเป็น 1.

ตัวอย่าง การทำปฏิบัติการ AND. ถ้าเราทำคำสั่ง `and $t0, $t1, $t2` และสมมติค่าในรีจิสเตอร์ `$t2` และ `$t1` ดังข้างล่าง

$$\begin{aligned} & 00000000000000000000110111000000_{two} && \text{ค่า \$t1} \\ AND\ & 00000000000000000001111000000000_{two} && \text{ค่า \$t2} \\ = & 00000000000000000000110000000000_{two} && \text{ค่า \$t0 หลังคำสั่ง AND.} \end{aligned}$$

จากตัวอย่างข้างต้น จะเห็นว่าเราสามารถใช้ AND ในการเลือกบิตออกมาได้ โดยนำค่า word ไป AND กับ bit pattern (ซึ่ง ค่า 0 ใน bit pattern จะบังคับให้ผลลัพธ์ออกมาเป็น 0 ในขณะที่ 1 ใน bit pattern จะให้ค่าที่อยู่ใน word ต้นฉบับออกมา). การใช้ bit pattern ร่วมกับปฏิบัติการ AND เพื่อเลือกค่าบางส่วนของ word ออกมานี้ มักถูกเรียกว่า การทำ masking เพราะ bit pattern ทำหน้าที่เหมือน mask ที่ปิดบังบางส่วนของ word เอาไว้.

ปฏิบัติการ OR. คู่กับ AND คือ OR. ปฏิบัติการ OR ทำงานเป็นบิต และจะให้ค่าผลลัพธ์ของบิตเป็น 1 เมื่อ ค่าบิตของ operand ตัวใดตัวหนึ่งเป็น 1.

ตัวอย่าง การทำปฏิบัติการ OR. ถ้าเราทำคำสั่ง `or $t0, $t1, $t2` และ สมมติค่าในรีจิสเตอร์ `$t2` และ `$t1` ดังข้างล่าง

$$\begin{array}{rll} & 00000000000000000000011011000000_{two} & \text{ค่า \$t1} \\ OR & 00000000000000000001111000000000_{two} & \text{ค่า \$t2} \\ = & 00000000000000000001111011000000_{two} & \text{ค่า \$t0 หลังคำสั่ง OR.} \end{array}$$

ปฏิบัติการ NOT. ปฏิบัติการ NOT ทำงานเป็นบิต รับ operand ตัวเดียว และจะให้ค่าผลลัพธ์ของบิตเป็น 1 เมื่อ ค่าบิตของ operand เป็น 0 และให้ค่าผลลัพธ์เป็น 0 เมื่อบิตของ operand เป็น 1. เพื่อรักษารูปแบบ 3 operands ของ MIPS, MIPS ใช้ NOR (NOT OR) แทน, โดย ถ้าให้ค่า ของ operand ตัวหนึ่งเป็น 0 หมด, NOR ก็จะใช้งานได้เหมือน NOT: `A NOR 0 = NOT (A OR 0) = NOT A`.

ตัวอย่าง การทำปฏิบัติการ NOT. ถ้าเราทำคำสั่ง `nor $t0, $t1, $zero`, เราจะได้

$$\begin{array}{rll} & 00000000000000000000011011000000_{two} & \text{ค่า \$t1} \\ NOR & 00000000000000000000000000000000_{two} & \text{ค่ารีจิสเตอร์ \$zero เป็นศูนย์เสมอ} \\ = & 11111111111111111111001000111111_{two} & \text{ค่า \$t0 หลังทำคำสั่ง NOR.} \end{array}$$

2.5 Instructions for Making Decisions

สิ่งที่ทำให้คอมพิวเตอร์ต่างจากเครื่องคิดเลขธรรมดาๆ ก็คือ ความสามารถในการตัดสินใจ (เปลี่ยนลำดับคำสั่งได้). คอมพิวเตอร์สามารถเลือกทำคำสั่งตามข้อมูล หรือ ค่าที่ได้มาจากการคำนวณได้. การตัดสินใจมีอยู่ทั่วไปใน programming languages: พวก if statement. ภาษาแอสเซมบลีของ MIPS มีคำสั่งตัดสินใจอยู่ คือ:

- หนึ่ง `beq reg1, reg2, L1`
- และ สอง `bne reg1, reg2, L1`.

คำสั่งที่หนึ่ง (mnemonic `beq` ย่อจาก branch if equal) จะเปรียบเทียบค่าในรีจิสเตอร์ reg1 กับ reg2 และถ้าค่าในรีจิสเตอร์ทั้งสองเท่ากัน คำสั่งต่อไปที่คอมพิวเตอร์จะรัน คือ คำสั่งที่มีฉลาก L1 อยู่ แต่ถ้าค่าทั้งสองไม่เท่ากัน คอมพิวเตอร์จะไม่มีการเปลี่ยนลำดับการรันคำสั่ง (ดังนั้นคำสั่งต่อไปที่คอมพิวเตอร์จะรัน ก็คือคำสั่งที่ถัดจาก `beq`).

คำสั่งที่สอง (mnemonic `bne` ย่อจาก branch if not equal) ถ้าค่าในรีจิสเตอร์ทั้งสองไม่เท่ากัน ถึงจะแยกออกไปทำคำสั่งที่มีฉลาก L1 อยู่. ทั้งสองคำสั่งนี้ จะถูกเรียกรวมๆว่า conditional branches.

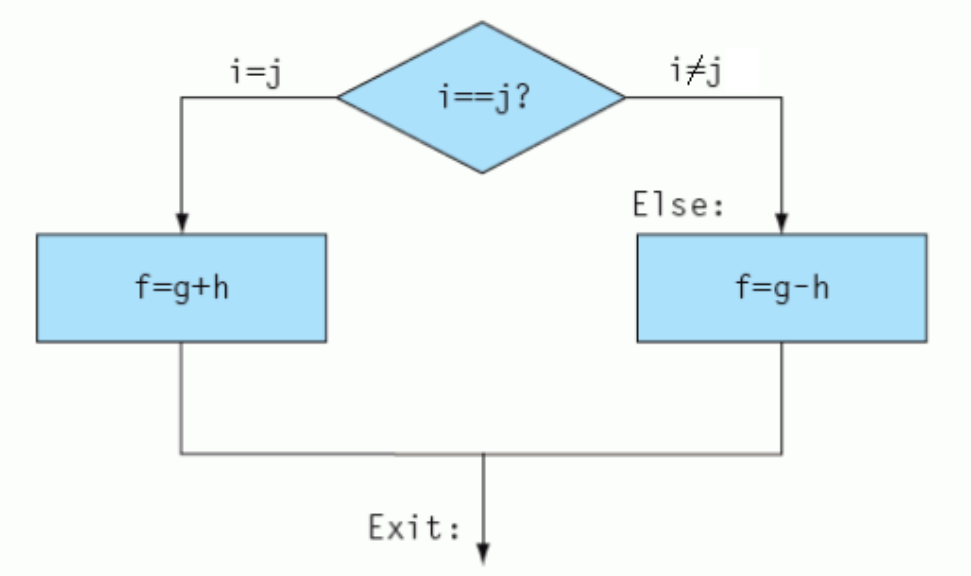


Figure 2.3: ตัวอย่าง flowchart ของ if statement

ตัวอย่าง การแปล if-then-else เป็น conditional branches. สมมติรีจิสเตอร์ `$s0` ถึง `$s4` แทนค่าตัวแปร `f, g, h, i, j` ตามลำดับ. คำสั่งภาษาซี:

```
if(i == j) f = g + h; else f = g - h;
```

ซึ่งอธิบายลำดับการทำคำสั่งดังแสดงในรูป 2.3. ดูเผิ่นๆ อาจเหมือนกับว่าเราต้องการกระโดด (หรือ branch) เมื่อ ค่าที่เปรียบเทียบเท่ากัน (ใช้ `beq`). แต่โดยทั่วๆไปแล้วในทางปฏิบัติโปรแกรมจะมีประสิทธิภาพมากกว่า ถ้าเราทำทดสอบตรงกันข้ามแล้วให้กระโดดออกไปที่ `else` ดังนี้:

```
          bne $s3, $s4, Else
          add $s0, $s1, $s2
          j Exit
Else:     sub $s0, $s1, $s2
Exit:
```

หมายเหตุ `j L1` เป็น unconditional branch (j ย่อจาก jump), โดยคอมพิวเตอร์จะเปลี่ยนลำดับการทำงานไปที่ตำแหน่งคำสั่งตามฉลากที่ระบุในคำสั่ง jump.
คอมไพเลอร์จะช่วยสร้าง branches และ labels ให้ แม้เราจะไม่เห็นมันเวลาเขียนโปรแกรมในภาษาสูง.
สังเกตุอย่างหนึ่งว่า แอสเซมเบลอร์ ช่วยโปรแกรมเมอร์ภาษาแอสเซมบลี ให้สามารถใช้ label ได้ (เช่น Else และ Exit) โดยที่โปรแกรมเมอร์ ไม่ต้องคำนวณค่าแอสเดรสโดยตรง.

2.5.1 Loops

การตัดสินใจสำคัญสำหรับทั้งการทำ if statement และ การทำ loop. เราสามารถใช้ conditional/unconditional branches ทำได้ทั้งสองอย่าง.

ตัวอย่าง การแปล while loop ของภาษาซี. ภาษาซี:

```
while (save[i] == k)
   i += 1;
```

สมมติ i กับ k รับผิดชอบด้วย `$s3` กับ `$s5` ตามลำดับ, และค่า base address ของอาร์เรย์ `save` เก็บใน `$s6`, แล้ว MIPS assembly code สำหรับโค้ดภาษาซีนี้คือ:

```
Loop:  sll $t1, $s3, 2     ## $t1 = i * 4
       add $t1, $t1, $s6   ## $t1 = address of save[i]
       lw $t0, 0($t1)      ## $t0 = save[i]
       bne $t0, $s5, Exit
       addi $s3, $s3, 1
       j Loop
Exit:
```

Hardware/Software Interface. basic block คือ คำสั่งต่างๆเรียงกันอยู่ โดยไม่มี branch (หรือ jump) แทรก (ยกเว้นคำสั่งท้ายสุด อาจมีได้) และก็ไม่ได้มี label ให้คำสั่ง branch อื่นกระโดดมา (ไม่ใช่ branch target ยกเว้นคำสั่งแรกสุด อาจมีได้). การคอมไพล์มักจะเริ่มจากการแบ่งโปรแกรมออกเป็น basic blocks.

ถึงแม้การตรงตรวจสอบเปรียบเทียบเท่ากับหรือไม่เท่ากับ จะใช้บ่อยมาก แต่หลายครั้งการเปรียบเทียบมากกว่าน้อยกว่าก็ใช้บ่อยเหมือนกัน เช่น การทำ for loop ที่ทำจนกว่าค่าอินเด็กซ์จะเป็นศูนย์. MIPS มีคำสั่งเปรียบเทียบค่าในรีจิสเตอร์สองตัว โดยจะให้ผลลัพธ์เป็น 1 เมื่อค่าของรีจิสเตอร์ตัวที่หนึ่งน้อยกว่าค่าของตัวที่สอง หรือถ้าไม่อย่างนั้นจะให้ผลลัพธ์เป็น 0. คำสั่งนี้คือ `slt` (set on less than), ดังตัวอย่าง:

```
slt $t0, $s3, $s4
```

จะเปรียบเทียบค่าใน `$s3` กับ `$s4`, ถ้า `$s3 < $s4`, ค่าของ `$t0 = 1`. แต่ถ้า `$s3` $\geq$ `$s4`, ค่าของ `$t0 = 0`.

การเปรียบเทียบค่าตัวแปรกับค่าคงที่ก็ใช้กันมาก ดังนั้น MIPS มี `slti` สำหรับเปรียบเทียบค่า register กับค่าคงที่ เช่น

```
slti $t0, $s2, 10
```

จะเปรียบเทียบค่าใน `$s2` กับ 10_{two}, ถ้า `$s2` < 10, ค่าของ `$t0 = 1`. แต่ถ้า `$s2` ≥ 10, ค่าของ `$t0 = 0`.

Hardware/Software Interface. MIPS คอมไพเลอร์ใช้ `slt, slti, beq, bne` และ ค่า 0 (จากรีจิสเตอร์ `$zero`) เพื่อทำ เปรียบเทียบ เท่ากับ ไม่เท่ากับ น้อยกว่า น้อยกว่าหรือเท่ากับ มากกว่า และ มากกว่าหรือเท่ากับ.

นักออกแบบ MIPS ทำตามคำเตือนของ von Neumann เรื่อง ความเรียบง่ายของฮาร์ดแวร์ MIPS ไม่มีคำสั่ง branch on less than เพราะว่า คำสั่งนี้จะซับซ้อนเกินไป อาจมีผลให้ ต้องยืด clock cycle time ยาวออกไป หรือ อาจต้องเพิ่มจำนวน clock ต่อคำสั่งเข้ามาก. นักออกแบบ MIPS เชื่อว่า การทำ 2 คำสั่งได้เร็วๆมีประโยชน์มากกว่า.

เนื่องจากตัวเลขอาจแสดงในรูปแบบ signed หรือ unsigned number, MIPS ใช้ `slt` กับ `slti` สำหรับ signed numbers และ `sltu` กับ `sltiu` สำหรับ unsigned numbers.

ตัวอย่าง Signed v.s. Unsigned Comparison. สมมติ ให้ `$s0` มีค่า $11111111111111111111111111111111_{two}$ และ `$s1` มีค่า $00000000000000000000000000000001_{two}$ ค่า ของ รี จิ ส เตอร์ `$t0` กับ `$t1` จะเป็นเท่าไร หลังทำคำสั่ง:

```
slt $t0, $s0, $s1
sltu $t1, $s0, $s1
```

ค่าของ `$s0` จะเป็น -1_{ten} ถ้าเป็น integer หรือ เป็น $4,294,967,295_{ten}$ ถ้าเป็น unsigned integer. ค่าของ `$s1` จะเป็น 1_{ten} ไม่ว่าจะเป็น integer หรือ unsigned integer.
ดังนั้นค่าของ `$t0` จะเป็น 1 (เพราะ $-1 < 1$) และ ค่าของ `$t1` จะเป็น 0 (เพราะ $4,294,967,295 > 1$).

เรื่องการเปรียบเทียบโดยใช้ unsigned comparison ยังมีประโยชน์อีกอย่างคือ การใช้ตรวจสอบ index out of bound เช่น ถ้าเรากำหนด index ของอาร์เรย์ให้มีค่าตั้งแต่ 0 ถึง $N-1$, และเราเก็บค่า N ในรีจิสเตอร์ `$t2` และเราต้องการตรวจสอบว่า index ที่เก็บในรีจิสเตอร์ `$s1` มีค่าอยู่ภายใน 0 ถึง $N-1$ หรือไม่เราสามารถทำได้โดย

```
sltu $t0, $s1, $t2
beq $t0, $zero, IndexOutOfBounds
```

ถ้าค่าของ `$t0` ออกมาเท่ากับ 0, ก็แปลว่า `$s1` $\geq$ `$t2`:

- ค่าของ `$s1` $=$ `$t2` แปลว่า `$s1` $= N$ ``out of bound"
- ค่าของ `$s1` $>$ `$t2` แปลว่า (กรณี 1) `$s1` $> N$ ``out of bound" หรือ (กรณี 2) `$s1` $< 0$ ``out of bound"

แต่ถ้าค่า `$t0` ออกมาเท่ากับ 1 แปลว่า `$s1` มีค่าอยู่ภายใน 0 ถึง $N-1$.

ตัวอย่าง. สมมติเราต้องการตรวจว่า ค่า i อยู่ในช่วง $0,\ldots,9$ ดังนั้น $N = 10$ ให้ `$s1` เก็บค่า i และ `$t2` เก็บค่า 10 ไว้,

ถ้า `$s1` เป็น $-2,147,483,648_{ten} = 10000000000000000000000000000000_{two}$ (ซึ่ง unsigned value = $2,147,483,648_{ten}$) ดังนั้น `sltu $t0, $s1, $t2` จะให้ `$t0` เป็น 0 (เพราะ $2,147,483,648 > 10$).

ถ้า `$s1` เป็น $-1_{ten} = 11111111111111111111111111111111_{two}$ (ซึ่ง unsigned value = $4,294,967,295_{ten}$) ดังนั้น `sltu $t0, $s1, $t2` จะให้ `$t0` เป็น 0.

ถ้า `$s1` เป็น $0_{ten} = 00000000000000000000000000000000_{two}$ (ซึ่ง unsigned value = 0_{ten}) ดังนั้น `sltu $t0, $s1, $t2` จะให้ `$t0` เป็น 1 (เพราะ $0 < 10$).

ถ้า `$s1` เป็น $9_{ten} = 00000000000000000000000000001001_{two}$ (ซึ่ง unsigned value = 9_{ten}) ดังนั้น `sltu $t0, $s1, $t2` จะให้ `$t0` เป็น 1.

ถ้า `$s1` เป็น $10_{ten} = 00000000000000000000000000001010_{two}$ (ซึ่ง unsigned value = 10_{ten}) ดังนั้น `sltu $t0, $s1, $t2` จะให้ `$t0` เป็น 0 (เพราะ $10 = 10$).

เราสามารถใช้ตรวจสอบ out of bound ได้โดยการเปรียบเทียบทีเดียว.

2.5.2 Case/Switch Statement

หลายๆภาษาสูงจะมี case หรือ switch statement ให้โปรแกรมเมอร์เลือกใช้ เมื่อมีทางเลือกหลายๆทาง สำหรับ ค่าตัวแปร ที่แตกต่างกัน. วิธีง่ายที่สุดในการทำ switch ก็คือการใช้ if-then-else หลายๆอันร่วมกัน.

หรือ อาจจะทำได้โดยใช้ ตารางแอสเดรสของทางเลือก (jump address table หรือเรียก jump table) และ โปรแกรมแค่ใช้ดัชนีเข้าไปที่ตารางและกระโดดไปตามแอดเดรสที่ระบุ.

MIPS มีคำสั่ง `jr` (jump register) ช่วยให้กระบวนการนี้สะดวกขึ้น โดยหลังจาก โหลดค่าแอดเดรสลงรีจิสเตอร์ แล้วสามารถเรียกคำสั่งนี้ได้เลย (ดูหัวข้อ 2.6 เพิ่มเติม).

2.6 Supporting Procedures in Computer Hardware

โพรซีเดอร์ (procedure) หรือ ฟังชั่น (function) คือ โปรแกรมย่อยๆที่ทำหน้าที่เฉพาะอย่างขึ้นอยู่กับค่าพารามิเตอร์ที่ โพรซีเดอร์ หรือ ฟังชั่น ได้รับ. พารามิเตอร์และค่าที่ตอบกลับ (returned variable) ทำหน้าที่เป็นจุดเชื่อมต่อ (interface) ระหว่าง โพรซีเดอร์ กับ โปรแกรมที่เรียกใช้มัน. การใช้งานโพรซีเดอร์, โปรแกรมทำดังนี้

- นำค่าพารามิเตอร์ไปไว้ให้โพรซีเดอร์.
- ย้ายอำนาจควบคุมไปให้กับโพรซีเดอร์
- จองพื้นที่หน่วยความจำที่โพรซีเดอร์ต้องการ
- ทำคำสั่งคำสั่งในโพรซีเดอร์
- นำผลจากโพรซีเดอร์ไปเก็บให้กับโปรแกรมที่เรียกโพรซีเดอร์
- ย้ายอำนาจควบคุมกลับไปให้กับโปรแกรมที่เรียกโพรซีเดอร์

ตามความนิยมแล้ว MIPS จะใช้

- `$a0`--`$a3` (argument registers) ใช้สำหรับส่งค่าพารามิเตอร์ให้กับโพรซีเดอร์.
- `$v0`--`$v1` (value registers) ให้สำหรับส่งค่าผลลัพธ์จากโพรซีเดอร์
- `$ra` (return address register) ให้เก็บค่าแอดเดรสในโปรแกรมหลักที่จะกลับมาหลังจากทำโพรซีเดอร์เสร็จ.

เพื่ออำนวยความสะดวกในการใช้โพรซีเดอร์ MIPS มีคำสั่งที่ใช้กระโดดไปทำงานที่แอดเดรสของคำสั่งโพรซีเดอร์ และ เก็บค่าแอดเดรสของคำสั่งถัดไปในโปรแกรมหลัก (คำสั่งที่จะต้องกลับมาทำหลังจบโพรซีเดอร์). คำสั่งนั้นคือ `jal` (jump-and-link) โดย ค่าแอดเดรสของคำสั่งถัด จะถูกเก็บไว้ในรีจิสเตอร์ `$ra`. เช่น หากโปรแกรมหลักจะเรียกใช้โพรซีเดอร์ (โปรแกรมที่เรียกโพรซีเดอร์เรียกว่า ``caller'') โปรแกรมหลัก จะใส่ค่าพารามิเตอร์ไว้ในรีจิสเตอร์ `$a0`--`$a3` แล้ว กระโดดไปโพรซีเดอร์ (เรียกว่า ``callee'') พร้อมเก็บค่า return address ไว้ด้วยคำสั่ง

```
jal ProcedureAddress
```

จะเก็บค่าแอดเดรสของคำสั่งถัดใน `$ra` และย้ายไปทำแอดเดรสของโพรซีเดอร์. แอดเดรสที่เก็บใน `$ra` จะเรียกว่า return address.

ภายในโพรซีเดอร์หลังจากที่โพรซีเดอร์ทำงานเสร็จ จะเก็บผลลัพธ์ไว้ใน `$v0` และ `$v1` แล้วย้ายอำนาจควบคุมกลับไปที่ caller ได้โดย

```
jr $ra
```

ซึ่ง caller สามารถไปดูผลลัพธ์ได้ที่ `$v0` และ `$v1` และทำงานต่อไปได้.

รีจิสเตอร์พิเศษที่ใช้เก็บแอดเดรสของคำสั่งปัจจุบัน (บอกว่า ซีพียูกำลังทำคำสั่งที่แอดเดรสไหนอยู่) เรียกว่า ``program counter'', ย่อๆว่า PC (หรือ อาจเรียก instruction address register). ดังนั้นการเก็บค่าแอดเดรสของคำสั่งถัดไป ก็เท่ากับการเก็บค่า `PC` $+4$ ไว้ใน `$ra`.

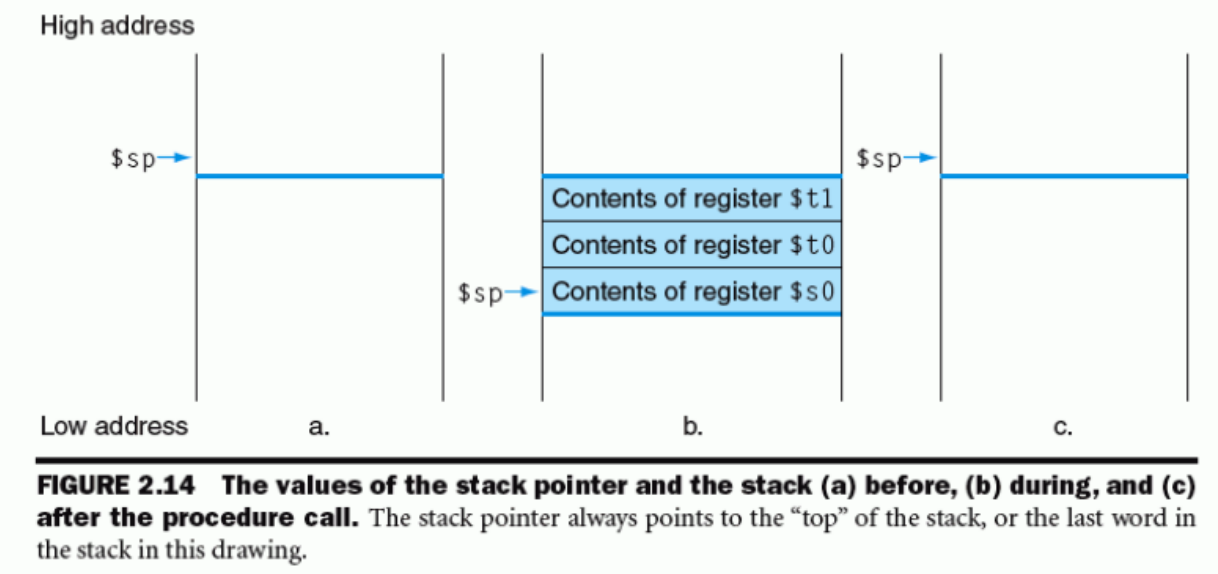


FIGURE 2.14 The values of the stack pointer and the stack (a) before, (b) during, and (c) after the procedure call. The stack pointer always points to the "top" of the stack, or the last word in the stack in this drawing.

Figure 2.4: Stack

2.6.1 Spilling Registers

ถ้าเราต้องการใช้รีจิสเตอร์ภายในโพรซีเดอร์ นอกจาก (`$a0`--`$a3` กับ `$v0` และ `$v1`), เราจะใช้ stack--โครงสร้างข้อมูล (ในหน่วยความจำ) ที่จัดการข้อมูลแบบ last-in-first-out. MIPS ใช้ stack pointer ซึ่งเป็น register (**`$sp`**) ให้การเก็บตำแหน่งล่าสุด ของ stack (ดูรูป 2.4. Stack ถูกใช้บ่อยมากจนมีชื่อเรียกเฉพาะสำหรับการนำข้อมูลไปเก็บใน stack เรียกว่า `push' และการนำข้อมูลออกจาก stack เรียกว่า `pop'.

จากประวัติของ stack ความนิยมคือ stack เริ่มจากแอดเดรสมากๆ แล้วเมื่อ stack ใหญ่ขึ้น แอดเดรสจะลดลง.

ตัวอย่าง. แปล ฟังชั่นภาษาซี

```
int myfunc (int g, int h, int i, int j)
{
   int f;

   f = (g + h) - (i + j);
   return f;
}
```

สมมติรีจิสเตอร์ `$a0`, `$a1`, `$a2`, `$a3` ใช้สำหรับตัวแปร `g`, `h`, `i`, `j` และ `$s0` ใช้สำหรับตัวแปร `f`. แอสเซมบลี จะเป็น:

```
myfunc: addi $sp, $sp, -12   ## make room for 3 items
        sw $t1, 8($sp)       ## save value of caller scope
        sw $t0, 4($sp)       ## save value of caller scope
        sw $s0, 0($sp)       ## save value of caller scope

        add $t0, $a0, $a1    ## $t0 = g + h
        add $t1, $a2, $a3    ## $t1 = i + j
```

```
        sub $s0, $t0, $t1    ## f = $t0 + $t1

        add $v0, $s0, $zero  ## save return value $v0 = $s0

        lw $s0, 0($sp)       ## restore value for caller
        lw $t0, 4($sp)
        lw $t1, 8($sp)
        addi $sp, $sp, 12

        jr $ra
```

โดย 4 บรรทัดแรกเป็นการเก็บค่าเดิมของรีจิสเตอร์ที่เราจะใช้ไว้ (เผื่อ caller ใช้มันทำอะไรสักอย่างอยู่). ต่อมา 3 บรรทัดทำ `f = (g+h) - (i+j);`. บรรทัดต่อมาเก็บผลลัพธ์ไว้ที่ `$v0`. และ 4 บรรทัดต่อมาคืนค่าเดิมของรีจิสเตอร์. บรรทัดสุดท้าย กระโดดกลับไปที่ caller.

เพื่อป้องกันการเก็บค่ารีจิสเตอร์ที่ caller ไม่ได้ใช้มากเกินไป, ธรรมเนียมของ MIPS คือ แบ่งรีจิสเตอร์ออกเป็น 2 กลุ่ม

- `$t0`--`$t7` และ `$t8` กับ `$t9` เป็นรีจิสเตอร์ชั่วคราว, callee จะไม่รักษาค่าของมันไว้.
- `$s0`--`$s7` ถ้า callee จะใช้จะต้องเก็บและคืนค่าเก่าของมันหลังใช้เสร็จ.

ดังนั้นโค้ดข้างต้นจริงๆ จะมีแค่

```
myfunc: addi $sp, $sp, -4   ## make room for 1 item
        sw $s0, 0($sp)      ## save value of caller scope

        add $t0, $a0, $a1    ## $t0 = g + h
        add $t1, $a2, $a3    ## $t1 = i + j
        sub $s0, $t0, $t1    ## f = $t0 + $t1

        add $v0, $s0, $zero  ## save return value $v0 = $s0

        lw $s0, 0($sp)       ## restore value for caller
        addi $sp, $sp, 4

        jr $ra
```

2.6.2 Nested Procedures

โพรซีเดอร์ที่ไม่ได้เรียกโพรซีเดอร์อื่นต่อ เราจะเรียกว่า leaf procedures.

ตัวอย่าง. recursive procedure ข้างล่างใช้คำนวณค่า factorial,

```
int fact (int n)
{
   if (n < 1) return (1);
   else return (n * fact( n - 1 ));
}
```

แอสเซมบลีจะเป็น

```
fact:
    addi $sp, $sp, -8
    sw   $ra, 4($sp)      ##  save the return address
    sw   $a0, 0($sp)      ##  save the argument n

    slti $t0, $a0, 1
    beq  $t0, $zero, L1

    addi $v0, $zero, 1    ## n < 1, return 1
addi $sp, $sp, 8
jr   $ra

 L1:
    addi $a0, $a0, -1     ## else, prepare to call fact(n-1)
    jal  fact             ## call fact(n-1)

    lw   $a0, 0($sp)
    lw   $ra, 4($sp)
    addi $sp, $sp, 8

    mul  $v0, $a0, $v0    ## n * the result from fact(n-1)

    jr   $ra
```

โดย 3 บรรทัดแรกเก็บค่า return address กับ ค่าอาร์กูเมนต์ไว้ก่อน. ต่อมา 2 บรรทัดเปรียบเทียบค่า `n` กับ 1. โดยถ้า $n < 1$ จะทำ 3 บรรทัดต่อมา ซึ่งประกอบด้วย เก็บค่า 1 ไว้ที่ `$v0` (เทียบเท่ากับ `return(1)`, ทำความสะอาด stack (ค่า `$ra` กับ `$a0` ที่เซฟไว้ ไม่ต้องใช้ เพราะเป็น leaf procedure ค่าเหล่านี้ไม่เปลี่ยน), และ กระโดดกลับไปหา caller. ต่อมาที่ฉลาก `L1` คือกรณี $n \geq 1$, 2 บรรทัดเป็นการเรียกโพรซีเดอร์ `fact` อีกครั้ง (สังเกตุ `$a0` ตอนนี้ใช้ผ่านค่า `n - 1` สำหรับการเรียกโพรซีเดอร์นี้). บรรทัดต่อมาจะถูกทำ ก็ต่อเมื่อ `fact(n-1)` ที่ถูกเรียกไปให้ผลลัพธ์กลับมาแล้ว (อยู่ที่ `$v0`) แต่ค่า `$a0` เปลี่ยนไปแล้ว เราจะต้องเรียกค่าเดิมกลับมาก่อน พร้อมกับขยับ stack กลับ แล้วจึงจำค่า `$a0` ไปคูณ กับผลลัพธ์จากการเรียกโพรซีเดอร์ซ้อน เก็บผลลัพธ์ไว้ที่ `$v0` และกระโดดกลับไปที่ caller.

ตาราง 2.1 แสดง ว่าอะไรบ้างที่ภายในโพรซีเดอร์จะต้องรักษาค่าเดิมเอาไว้ และอะไรบ้างที่ไม่ต้องทำ

Table 2.1: สิ่งที่โพรซีเดอร์จะรักษาไว้ให้และสิ่งที่โพรซีเดอร์สามารถใช้ได้เลย ไม่จำเป็นต้องรักษาค่าเก่าไว้

Preserved	Not preserved
Saved registers: `$s0`--`$s7`	Temporary registers: `$t0`--`$t9`
Stack pointer register: `$sp`	Argument register: `$a0`--`$a3`
Return address register: `$ra`	Return value registers: `$v0`--`$v1`
Stack above the stack pointer	Stack below the stack pointer

Hardware/Software Interface. ตัวแปรในภาษาซี จะมีทั้งที่เป็น automatic กับ static. Automatic variable จะมี scope อยู่แค่ภายในโพรซีเดอร์ และจะถูกยกเลิกหลังจากออกจากโพรซีเดอร์. ส่วน static variable จะมีการจองพื้นที่หน่วยความจำไว้เลย ดังนั้นจะคงอยู่ตลอดโปรแกรม เช่น `static void main(void)` ฟังชั่นหลักนี้จะถูกจองพื้นที่ไว้เลย ตัวแปรของภาษาซีที่ประกาศอยู่นอกโพรซีเดอร์ จะเป็น static ทั้งหมด (โดยอัตโนมัติ) รวมถึง ตัวแปร ที่ประกาศพร้อมด้วย คำสำคัญ static. เพื่อความสะดวกในการจัดการข้อมูลที่เป็น static, ซอฟแวร์ของ MIPS เตรียมรีจิสเตอร์ ชื่อ `$gp` ในการชี้ไปที่ตำแหน่งของข้อมูลที่เป็น static.

ตัวอย่าง static variable. ลองรันโค้ดภาษาซีข้างล่างแล้ว ดูผล

```
#include <stdio.h>

int a;
static int b;

void doInc(){
    int i;
    static int j;

    a += 1;
    b += 1;
    i += 1;
    j += 1;
    printf("doInc: i = %d, j = %d\n", i, j);
    printf("doInc: a = %d, b = %d\n", a, b);
}

void main(){
   int i = 0;
   static int j = 100;

   printf("main: a = %d, b = %d\n", a, b);
   printf("main: i = %d; j = %d\n", i, j);

   doInc();

   printf("main: a = %d, b = %d\n", a, b);
   printf("main: i = %d; j = %d\n", i, j);

   doInc();

}
```

ผล[1]

```
main: a = 0, b = 0
main: i = 0; j = 100
```

[1]ผลที่ได้นี้ คอมไพล์ด้วย GNU gcc 4.7.1.

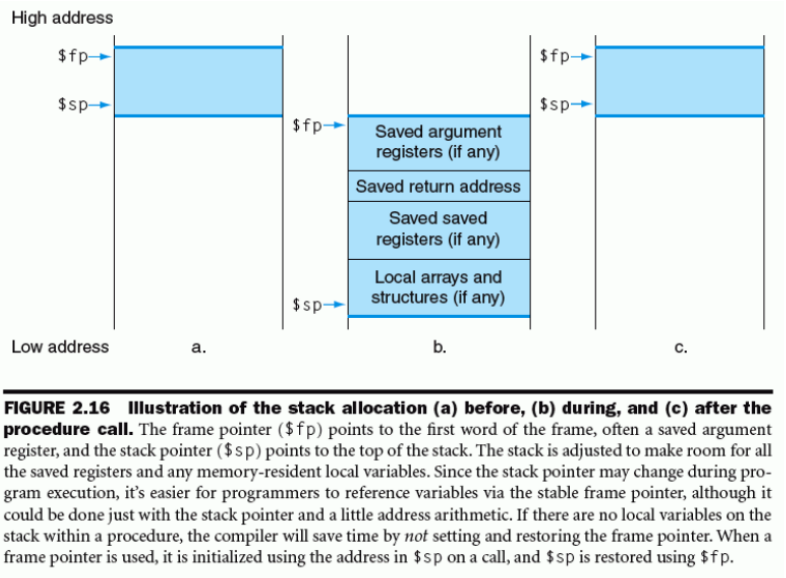


FIGURE 2.16 Illustration of the stack allocation (a) before, (b) during, and (c) after the procedure call. The frame pointer (`$fp`) points to the first word of the frame, often a saved argument register, and the stack pointer (`$sp`) points to the top of the stack. The stack is adjusted to make room for all the saved registers and any memory-resident local variables. Since the stack pointer may change during program execution, it's easier for programmers to reference variables via the stable frame pointer, although it could be done just with the stack pointer and a little address arithmetic. If there are no local variables on the stack within a procedure, the compiler will save time by *not* setting and restoring the frame pointer. When a frame pointer is used, it is initialized using the address in `$sp` on a call, and `$sp` is restored using `$fp`.

Figure 2.5: Procedure Frame

```
doInc: i = 1994624214, j = 1
doInc: a = 1, b = 1
main: a = 1, b = 1
main: i = 0; j = 100
doInc: i = 1994624214, j = 2
doInc: a = 2, b = 2
```

สังเกตุว่า ค่าของตัวแปร `a` กับ `b` ทั้งใน `main` และ `doInc` คือตัวเดียวกัน (หลังจาก `doInc` เปลี่ยนค่าไปแล้ว, `main` ก็เห็นเช่นเดียว). แต่ ตัวแปร `i` กับ `j` ของ `main` กับ `doInc` เป็นคนละชุดกัน. สังเกตุอีกอย่าง ทุกครั้งที่เรียก `doInc`, ค่า `i` ถูกจองใหม่ ค่าจะเริ่มใหม่ แต่ค่า `j` ของฟังชั่น `doInc` จะถูกจองไว้ตลอดโปรแกรม ไม่ว่าเรียก `doInc` ค่าของ `j` จะอ้างถึงตัวเดิม.

เสริม. เช่นเดียวกับที่เราสามารถเรียกใช้ method ของ static class ของจาวาได้โดยตรง (ไม่ต้องสร้างเป็น object ขึ้นมาก่อน) ซึ่งต่างจาก dynamic class.

2.6.3 Allocating Space for New Data on the Stack

นอกจากการใช้ stack เก็บรักษาค่าเก่าของรีจิสเตอร์ต่างๆแล้ว, เราอาจใช้มันค่าตัวแปรที่ใช้เฉพาะภายในโพรซีเดอร์ได้ (ตัวแปรท้องถิ่น local variables) เช่นพวก array และ structure เป็นต้น. ส่วนของ stack ที่ใช้เก็บรักษาค่าของรีจิสเตอร์และตัวแปรท้องถิ่น จะเรียกว่า procedure frame หรือ activation record.

ซอฟต์แวร์ของ MIPS บางอย่างใช้ frame pointer (`$fp`) ในการชี้ไปที่ word แรกของ procedure frame. การใช้ frame pointer ช่วยให้ เราสามารถ stack pointer ได้สะดวกขึ้น เช่น ใช้ `$fp` เป็นจุดอ้างอิงที่คงที่ตลอดโพรซีเดอร์ และ `$sp` อาจขยับภายในโพรซีเดอร์ได้. รูป 2.5 แสดง procedure frame ก่อน, ระหว่าง, และหลังการเรียกใช้โพรซีเดอร์.

Elaboration. ถ้าฟังชั่นมีพารามิเตอร์ มากกว่า 4 ตัว, ธรรมเนียมปฏิบัติก็คือ ให้พารามิเตอร์ตัวที่เกินมาไปเก็บไว้ใน stack โดย วางไว้เหนือ `$fp` และ ภายในโพรซีเดอร์สามารถ ดึงค่าพารามิเตอร์

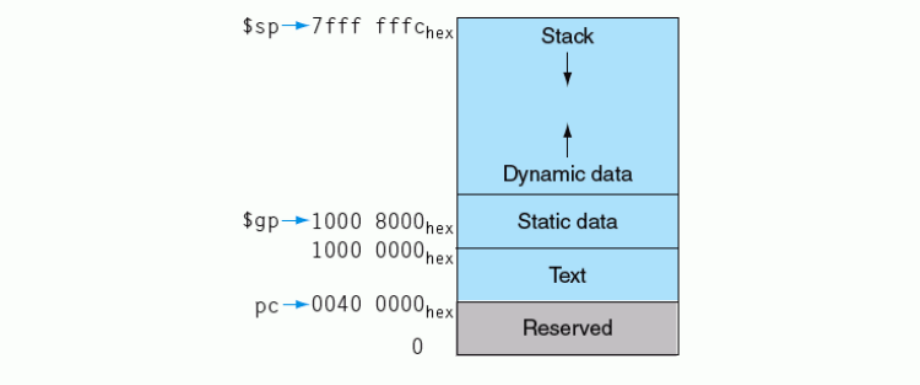


FIGURE 2.17 The MIPS memory allocation for program and data. These addresses are only a software convention, and not part of the MIPS architecture. Starting top down, the stack pointer is initialized to $7fff\ fffc_{hex}$ and grows down toward the data segment. At the other end, the program code ("text") starts at $0040\ 0000_{hex}$. The static data starts at $1000\ 0000_{hex}$. Dynamic data, allocated by `malloc` in C and via `new` in Java, is next and grows up toward the stack in an area called the heap. The global pointer, `$gp`, is set to an address to make it easy to access data. It is initialized to $1000\ 8000_{hex}$ so that it can access from $1000\ 0000_{hex}$ to $1000\ ffff_{hex}$ using the positive and negative 16-bit offsets from `$gp`

Figure 2.6: MIPS memory allocation

เหล่านั้นมาได้โดยอ้างจาก `$fp`.

อีกเรื่องเกี่ยวกับ `$fp`, บางคอมไพเลอร์มองว่า `$fp` ไม่จำเป็นเช่น ซีคอมไพเลอร์จากบริษัท MIPS เองไม่ใช้ `$fp` แต่ใช้ register 30 เหมือนกับเป็น `$s8`, แต่ GNU MIPS C เห็นว่ามีประโยชน์และเลือกที่จะใช้ `$fp`.

2.6.4 Allocating Space for New Data on the Heap

นอกจากตัวแปรท้องถิ่นภายในโพรซีเดอร์แล้ว, โปรแกรมเมอร์ภาษาซี ยังต้องการพื้นที่หน่วยความจำ สำหรับ static variables และ dynamic data structures (เช่น linked list). รูป 2.6 แสดงธรรมเนียมปฏิบัติของ MIPS ในการแบ่งปันสัดส่วนหน่วยความจำ.

ส่วนที่ใช้เป็น stack จะโตจากแอดเดรสมากลงมาสู่แอดเดรสน้อย. ส่วนแรกของแอดเดรสน้อยๆ (`0x0000 0000` ไปจนถึง `0x003F FFFF`) เป็นส่วนที่สงวนเอาไว้. แอดเดรสจาก `0x0040 0000` เป็นต้นไปจนถึง `0x0FFF FFFF` เป็นส่วนสำหรับโปรแกรม เรียกว่า text segment. จากแอดเดรส `0x1000 0000` เป็นส่วนของข้อมูล โดยช่วงแรกจะใช้สำหรับ static data (แอดเดรส `0x1000 0000` ถึง `0x1000 FFFF`) ช่วงที่สองจะใช้สำหรับ dynamic data เช่น ข้อมูลที่เปลี่ยนขนาดได้ อาจจะเป็น linked list เป็นต้น. ช่วงที่จะใช้สำหรับ dynamic data นี้บางครั้งจะเรียก heap. สังเกตุว่า ช่วง dynamic data และ ส่วนสำหรับ stack โตเข้าหากัน. วิธีการจัดการทำให้เราสามารถใช้หน่วยความจำได้อย่างมีประสิทธิภาพ เนื่องจาก ขนาดของข้อมูลทั้งสองชนิดสามารถเพิ่มขึ้นลดลงได้ตลอด.

ในภาษาซี จอง พื้นที่หน่วยความจำใน heap ด้วยฟังชั่น `malloc()` และจะได้ค่า pointer ที่ชี้ไปที่พื้นที่นั้นมา. เมื่อใช้พื้นที่เสร็จแล้ว สามารถคืนพื้นที่นั้นได้โดย ฟังชั่น `free()`. ภาษาจาวาจะจองพื้นที่ด้วยคำสั่ง `new` และเพื่อป้องกันการลืมคืนหน่วยความจำและปัญหาที่เกี่ยวข้อง เช่นในภาษาซี, จาวามีระบบ automatic memory allocation และ garbage collection ไว้จัดการคืน heap และทำความสะอาดเรื่องที่เกี่ยวข้องให้.

ตาราง 2.2 สรุป ธรรมเนียมปฏิบัติ ของ MIPS เรื่องการรักษาค่าของรีจิสเตอร์ต่างๆ.

Table 2.2: สรุปธรรมเนียมปฏิบัติ ของ MIPS เรื่องการรักษาค่าของรีจิสเตอร์: register 1 (`$at`) สงวนไว้ให้ assembler และ registers 26--27 (`$k0` กับ `$k0`) สงวนไว้ให้ Operating System

Name	Register	Usage	Preserved on call?
`$zero`	0	The constant value 0	n.a.
`$v0--$v1`	2--3	Values for results and expression evaluation	no
`$a0--$a3`	4--7	Arguments	no
`$t0--$t7`	8--15	Temporaries	no
`$s0--$s7`	16--23	Saved	yes
`$t8--$t9`	24--25	More temporaries	no
`$gp`	28	Global pointer	yes
`$sp`	29	Stack pointer	yes
`$fp`	30	Frame pointer	yes
`$ra`	31	Return address	yes

2.7 Communicating with People

เราใช้รหัส ASCII (American Standard Code for Information Interchange) ในการแทนอักขระ (characters). ตาราง 2.7

สถาปัตยกรรม MIPS มีคำสั่งที่ใช้ย้ายข้อมูลเป็นไบต์คือ

- `lb` (load byte) สำหรับโหลดค่าจากหน่วยความจำเข้ามาเก็บในรีจิสเตอร์ โดยจะใส่ไว้ใน 8 บิตขวาสุดของรีจิสเตอร์.
- `sb` (store byte) สำหรับนำ 8 บิตขวาสุดของรีจิสเตอร์ ไปใส่ไว้ในหน่วยความจำ

เช่น ถ้าเราต้องการย้ายข้อมูลที่แอดเดรส `0x10000004` ไปเก็บที่ แอดเดรส `0x1000003C` สามารถทำได้โดย

```
li $s0, 0x10000004
lb $t0, 0($s0)
sb $t0, 0x38($s0)
```

Hardware/Software Interface. เมื่อโหลดไบต์ ข้อมูล 8 บิต ตามแอดเดรสที่กำหนด จะถูกสำเนาจากหน่วยความจำ มาเก็บไว้ที่ 8 บิตขวาสุดของรีจิสเตอร์ แล้ว MIPS ให้ทางเลือก 2 แบบ คือ แบบ signed load กับ unsigned load. แบบ signed load (`lb`) จะถือว่าบิตหน้าสุดของไบต์ เป็น signed bit และจะใช้ค่าของ signed บิตในการเติมรีจิสเตอร์ให้เต็ม เรียกว่าทำ sign extension. ส่วนแบบ unsigned load (`lbu`) จะใช้ 0 เติมรีจิสเตอร์จนเต็ม.

นอกจากทำงานกับ อักขระเป็นตัวเดี่ยวๆ แล้วบ่อยครั้งที่เราทำงานกับข้อมูลที่เป็น string (สายอักขระ), ซึ่งก็ชุดของอักขระเรียงกันหลายๆตัว. มีหลายวิธีที่จะใช้ในการเก็บค่าของ string เช่น อาจจะ สงวนตำแหน่งแรก (ไบต์แรก) ของ string ไว้เก็บความยาวของ string หรือ ใช้อักขระพิเศษในการปิดท้ายของ string เช่น ภาษาซีจะใช้ค่า `0x00` (ค่า null ใน ASCII) ในการปิดท้าย string ดังนั้น `Cal` ในภาษาซีจะใช้ 4 ไบต์ คือ `0x43616C00`.

ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter
32	space	48	0	64	@	80	P	96	`	112	p
33	!	49	1	65	A	81	Q	97	a	113	q
34	"	50	2	66	B	82	R	98	b	114	r
35	#	51	3	67	C	83	S	99	c	115	s
36	$	52	4	68	D	84	T	100	d	116	t
37	%	53	5	69	E	85	U	101	e	117	u
38	&	54	6	70	F	86	V	102	f	118	v
39	'	55	7	71	G	87	W	103	g	119	w
40	(	56	8	72	H	88	X	104	h	120	x
41	)	57	9	73	I	89	Y	105	i	121	y
42	*	58	:	74	J	90	Z	106	j	122	z
43	+	59	;	75	K	91	[	107	k	123	{
44	,	60	<	76	L	92	\	108	l	124	|
45	-	61	=	77	M	93	]	109	m	125	}
46	.	62	>	78	N	94	^	110	n	126	~
47	/	63	?	79	O	95	_	111	o	127	DEL

FIGURE 2.21 ASCII representation of characters. Note that upper- and lowercase letters differ by exactly 32; this observation can lead to shortcuts in checking or changing upper- and lowercase. Values not shown include formatting characters. For example, 8 represents backspace, 9 represents a tab character, and 13 a carriage return. Another useful value is 0 for null, the value the programming language C uses to mark the end of a string.

Figure 2.7: ตารางอักขระแบบ ASCII

ตัวอย่าง: คอมไพล์ โพรซีเดอร์ string copy. โพรซีเดอร์ `strcpy` คัดลอกสายอักขระ `y` ไปที่สายอักขระ `x` โดยใช้ `null` เป็น termination (ซึ่งเป็นไปตามธรรมเนียมปฏิบัติของภาษาซี)

```
void strcpy (char x[], char y[])
{
   int i;

   i = 0;

   /* do until found termination */
   while( (x[i] = y[i]) != '\0')
     i += 1;
}
```

ให้ base addresses ของ `x` กับ `y` ส่งผ่านมากับรีจิสเตอร์ `$a0` และ `$a1`, และใช้รีจิสเตอร์ `$s0` ตัวแปร `i`, อาจแปลงเป็น MIPS แอสเซมบลีได้ดังนี้

```
strcpy:  addi $sp, $sp, -4     ## adjust stack for 1 item
         sw   $s0, 0($sp)      ## save $s0

         add  $s0, $zero, $zero    ## i = 0;

   L1:   add  $t1, $s0, $a1        ## address of y[i]
         lbu  $t2, 0($t1)          ## $t2 = y[i]

         add  $t3, $s0, $a0        ## address of x[i]
         sb   $t2, 0($t3)          ## x[i] = y[i]

         beq  $t2, $zero, L2       ## if y[i] == 0, go to L2
         addi $s0, $s0, 1          ## i = i + 1
         j    L1                   ## go to L1
```

```
L2:  lw   $s0, 0($sp)          ## restore $s0 its old value
     addi $sp, $sp, 4
     jr   $ra
```

สังเกตุว่า เราไม่ต้องคูณ 4 กับค่า `i` ดังที่ทำกับข้อมูลที่เป็น `integer` เพราะว่า `char` มีขนาดแค่ 1 ไบต์ แต่ `integer` มีขนาด 4 ไบต์.

เนื่องจากโพรซีเดอร์ `strcpy` เป็น leaf procedure, คอมไพเลอร์อาจใช้ temporary register เช่น `$t4` สำหรับค่า `i` แทนที่จะใช้ `$s0`. ซึ่งถ้าใช้ `$t4` แทน, โพรซีเดอร์ก็ไม่จำเป็นต้องรักษาค่าเก่าไว้ (เพราะ ธรรมเนียมของ MIPS ไม่จำเป็นต้องรักษาค่าเก่าของ `$t4`). ดังนั้นในทางปฏิบัติ สำหรับภายในโพรซีเดอร์ คอมไพเลอร์จะใช้ temporary registers จนหมดก่อนจึงค่อยไปใช้ save registers.

2.7.1 Characters and Strings in Java

Unicode เป็น universal encoding สำหรับอักขระของภาษาต่างๆของคน. จาวา ใช้ unicode สำหรับ `char` ดังนั้นตัวแปร `char` ของจาวาจะมีขนาด 16 บิตโดยดีฟอลต์ (UTF-16). สถาปัตยกรรม MIPS มีคำสั่งเฉพาะให้สำหรับการ load หรือ store ข้อมูล 16 บิต, โดยเรียกมันว่า halfword ได้แก่คำสั่ง

- `lh` โหลดค่าขนาด 16 บิตในหน่วยความจำ มาใส่ไว้ที่ 16 บิตทางขวามือของรีจิสเตอร์ โดยเติมบิตที่เหลือตาม signed bit.
- `lhu` โหลดค่าขนาด 16 บิตในหน่วยความจำ มาใส่ไว้ที่ 16 บิตทางขวามือของรีจิสเตอร์ โดยเติมบิตที่เหลือด้วย 0.
- `sh` ใส่ค่าจาก 16 บิตทางขวามือของรีจิสเตอร์ ในหน่วยความจำ.

2.8 MIPS Addressing for 32-bit Immediate and Addresses

บางครั้งเรามีความจำเป็นต้องใช้ ค่าคงที่ขนาด 32 บิต หรือต้องการใส่ค่าแอดเดรสขนาด 32 บิต เข้าไปโดยตรง, สถาปัตยกรรม MIPS ทำโดยวิธีดังต่อไปนี้.

2.8.1 32-bit Immediate Operands

สถาปัตยกรรม MIPS มีคำสั่ง load upper immediate (`lui`) โดยจะใส่ค่าคงที่ขนาด 16 บิตไว้ที่ 16 บิตบน ของรีจิสเตอร์และเติม 0 สำหรับบิตที่เหลือในรีจิสเตอร์.

เช่น หากเราต้องการโหลดค่า $0000000000111101000010010000000_{two}$ เข้าไปในรีจิสเตอร์ `$s0` เราทำได้โดย

```
lui $s0, 0x003D
ori $s0, $s0, 0x0900
```

ที่เราไม่ใช้ `addi` เพราะ `addi` จะเติม 16-bit immediate ให้เป็น 32 บิตด้วย sign extension ก่อนจะบวก (มีโอกาสที่ sign bit จะเป็น 1 และทำให้ 16 บิตบนเพี้ยนจากค่าที่ได้จาก `lui`). แต่คำสั่ง `ori` จะเติม 16-bit immediate ให้เป็น 32 บิตด้วยการเติม 0 ที่ 16 บิตบน ทำให้ ไม่มีผลกับค่า 16 บิตที่ได้จาก `lui`.

2.8.2 Addressing in Branches and Jumps

คำสั่งในการกระโดดของสถาปัตยกรรม MIPS ใช้รูปแบบ J-type, ซึ่งใช้ 6 บิตสำหรับ opcode และที่เหลือ (รวม 26 บิต) สำหรับแอดเดรส. ตัวอย่าง เช่น

```
j 10000
```

คือให้กระโดดไปที่แอดเดรส 10000_{two}. และคำสั่งเครื่องจะเป็น

2	10000_{ten}
000010_{two}	$00000000000010011100010000_{two}$

ต่างจากคำสั่ง jump, conditional branch ต้องการ 2 operands สำหรับระบุรีจิสเตอร์ 2 ตัวที่จะใช้เปรียบเทียบกัน ดังนั้น 6 บิตสำหรับ opcode, 5 บิตสำหรับรีจิสเตอร์แต่ละตัว ใช้ 2 ตัว ก็ต้องใช้ 10 บิต, ที่เหลือ 16 บิตสำหรับ branch address. ตัวอย่างเช่น

```
bne $s0, $s1, Exit
```

จะมีคำสั่งเครื่องเป็น

5	16	17	Exit
000101_{two}	10000_{two}	10001_{two}	16 bits for address labelled Exit

ปัญหาคือว่า ถ้าเราใช้ 16 บิตในการระบุแอดเดรสโดยตรงแปลว่า แอดเดรสจะมีค่าในช่วงได้แค่ $2^{16} = 65,536$ ค่า หรือ มีคำสั่งในภาษาเครื่องได้แค่ 65,536 คำสั่งต่อโปรแกรม ซึ่งน้อยเกินไปเทียบกับโปรแกรมในปัจจุบัน.

จากการสังเกตุ conditional branch ใช้กับ loops หรือ if statements และ พวกนี้มักจะ branch ไปคำสั่งที่อยู่ใกล้ๆ, ดังนั้นคำตอบก็คือใช้ ค่า 16 บิตนี้ไปรวมกับค่าในรีจิสเตอร์ PC เพื่อเป็นค่าที่จะ branch ไป:

$$\text{Program counter} = \text{Program counter} + \text{Branch address} \tag{2.2}$$

ดังนั้นเราก็สามารถกระโดดไปได้ช่วง $PC \pm 2^{15}$ ซึ่งเพียงพอสำหรับ loops หรือ if statements ส่วนใหญ่.

วิธีการใช้ค่า address offset ไปบวกกับ PC แบบนี้จะเรียกว่า PC-relative addressing. เนื่องจากในการทำฮาร์ดแวร์ (บทที่ 4) เราจะทำให้ PC เพิ่มไป 4 เพื่อเตรียมไปทำคำสั่งต่อไป ดังนั้นสถาปัตยกรรม MIPS จะทำ PC-relative addressing กับ ค่า PC + 4 (ค่าแอดเดรสของคำสั่งต่อไป) แทนที่จะเป็น PC (ค่าแอดเดรสของคำสั่งปัจจุบัน).

สถาปัตยกรรม MIPS จะทำ conditional branch ด้วย PC-relative addressing เพราะ conditional branch มักจะอ้างไปถึงคำสั่งในระแวกใกล้ๆ. แต่สำหรับ คำสั่ง jump-and-link ที่ใช้เรียกโพรซีเดอร์ สถานการณ์ต่างไป เพราะ คำสั่งของโพรซีเดอร์ที่จะกระโดดไปทำ มักไม่ได้อยู่ใกล้ๆกับ คำสั่งที่เรียก. ดังนั้น MIPS จึงใช้วิธี immediate addressing และรูปแบบ J-type สำหรับทั้ง jump (`j`) และ jump and link (`jal`).

เนื่องจากคำสั่ง MIPS ทุกคำสั่งยาว 4 ไบต์เหมือนกันทั้งหมด. ในการทำ PC-relative addressing, MIPS ค่า offset จะอ้างถึง offset ของ word เทียบกับ คำสั่งถัดไป (PC + 4) แทนที่จะเป็นไบต์. ดังนั้นจาก 16 บิตทำให้สามารถ branch ไปได้ในช่วง 2^{16} คำสั่ง (ไม่ใช่แค่ 2^{16} ไบต์ $= 2^{14}$ คำสั่ง) เช่นกัน คำสั่ง jump, ก็ใช้เป็น word address ทำให้ 26 บิต อ้างไปถึง 2^{26} คำสั่ง.

ตัวอย่าง jump และ branch. จากตัวอย่างในหัวข้อ 2.5.1,

```
Loop:  sll $t1, $s3, 2
       add $t1, $t1, $s6
       lw $t0, 0($t1)
       bne $t0, $s5, Exit
       addi $s3, $s3, 1
       j Loop
Exit:
```

สมมติว่า loop เริ่มที่แอดเดรส 0x004C0008 ในหน่วยความจำ, ภาษาเครื่องของ MIPS จะเป็น:

0x004C0008	0	0	19	9	2	0	sll ...
0x004C000C	0	9	22	9	0	32	add ...
0x004C0010	35	9	8	0			lw ...
0x004C0014	5	8	21	0x0002			bne ...
0x004C0018	8	19	19	1			addi ...
0x004C001C	2	0x0130002					j ...
0x004C0020	...						

ภาษาเครื่องข้างต้น ตัวเป็นเขียนเป็นฐานสอง ยกเว้นตัวเลขที่เขียนด้วย 0x นำหน้า. คำสั่ง sll และ add เป็น R-format ดังนั้นแต่ละคำสั่งจะแบ่งเป็น 6 ฟิลด์. คำสั่ง lw, bne, และ addi เป็น I-format, ฟิลด์สุดท้ายขนาด 16 บิต. ส่วนคำสั่ง j เป็น J-format.

ก่อนจะอธิบายต่อ, คุยเรื่อง PC ก่อนหน่อยหนึ่ง, ณ ตอนนี้เราอาจจะมองเหมือนกับว่าค่า PC จะคงที่ตลอดการรันคำสั่งที่ PC ชี้อยู่และจะคงค่านั้นจนกว่าจะทำคำสั่งเสร็จ. แต่ในความเป็นจริง เพื่อสร้างฮาร์ดแวร์ให้มีประสิทธิภาพ หลังจาก fetch คำสั่งเข้าไปให้ซีพียูเรียบร้อยแล้ว แทนที่จะรอจนทำคำสั่งเสร็จ แล้วค่อยขยับค่าของ PC ไปที่คำสั่งถัดไป, คอมพิวเตอร์ในปัจจุบันจะไม่รอ แต่จะขยับค่า PC ไปรอที่คำสั่งถัดไปเลย (PC $\leftarrow$ PC $+ 4$). (ดังที่เราจะคุยเรื่องนี้กันในบทที่ 4.)

กลับมาตัวอย่างภาษาเครื่องข้างต้น, สังเกตุค่าที่ฟิลด์สุดท้ายของคำสั่ง bne (opcode เป็น 5 $= 000101_{two}$) จะมีค่า 0x0002 อยู่. ค่า 0x0002 $= 2_{ten}$ นี้เป็น signed number หมายถึง 2 คำสั่งข้างหน้า หรือ $2 \times 4 = 8$ แอดเดรสข้างหน้า.

ดังนั้น จากตัวอย่างข้างต้น ขณะจะทำคำสั่ง bne, PC จะมีค่า PC มีค่า 0x004C0014 อยู่ และหลังจาก fetch คำสั่งเรียบร้อยแล้ว ค่า PC จะขยับไปเป็น 0x004C0018 โดย เมื่อทำการเปรียบเทียบเสร็จแล้ว

- หากไม่ทำ branch, คอมพิวเตอร์ก็จะทำคำสั่งถัดไป (ที่ 0x004C0018) ได้ทันที.
- แต่หากจะทำ branch ค่า PC จะขยับไปตามค่าที่ระบุ: PC $+ 8 =$ 0x004C0018 $+8 =$ 0x004C0020. สังเกตุว่า นี่คือแอดเดรสของฉลาก Exit.

รูป 2.8 แสดง MIPS addressing modes แบบต่างๆ: immediate mode (เช่น addi), register mode (เช่น add), base addressing mode (เช่น lw), PC-relative addressing mode (เช่น bne), และ pseudodirect addressing mode (เช่น j).

ตอนนี้ลองเปรียบเทียบ PC-relative addressing mode ข้างต้นกับ pseudodirect addressing ของคำสั่ง j, ค่าในฟิลขนาด 26 บิต ของคำสั่ง j คือ 0x0130002. สำหรับ pseudodirect addressing, ค่าแอดเดรสนี้จะใส่เข้าไปที่ PC โดย(เกือบ)ตรง:

- (1) 26-bit word address, 0x0130002 จะเป็น 28-bit byte address: 0x0130002 $\times 4 =$ 0x04C0008.
- (2) ค่าขนาด 28 บิตนี้จะเซตค่าของ PC ใน 28 บิตล่าง โดยคงค่าเดิมของ PC ใน 4 บิตบนไว้.

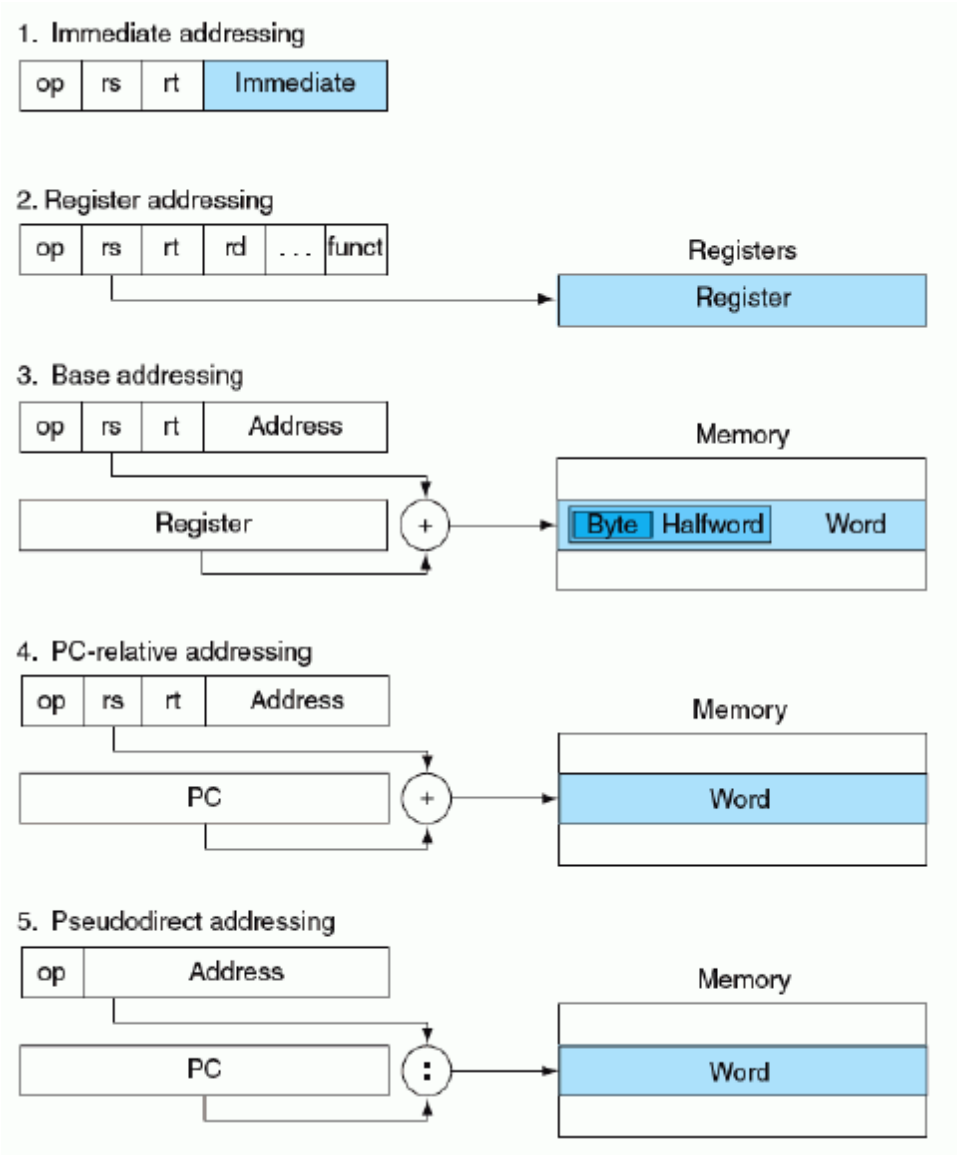


Figure 2.8: MIPS addressing modes.

ที่คำสั่ง `j` ก่อนที่จะ fetch คำสั่ง, PC มีค่า `0x004C001C`. หลังจากทำคำสั่ง `j`, ค่าของ PC จะถูกเซ็ตเป็น `0x004C0008`. ที่ขีดเส้นใต้คือค่าที่ถูกเซ็ตและ 4 บิตแรก (เท่ากับหลักแรกในฐาน 16) คือค่าเดิมของ PC. สังเกตุว่า นี่คือแอดเดรสของฉลาก `Loop`.

Elaboration. สำหรับ immediate addressing mode, ที่ใช้ค่า 26 บิต อ้างถึงคำสั่ง (เทียบเท่ากับ 28-bit byte address), เรายังต้องการอีก 4 บิต เพื่อเติม PC ให้เต็ม 32 บิต. สถาปัตยกรรม MIPS จะไม่เปลี่ยนค่า 4 บิตบน ของ PC (ใช้ค่าเดิมของ PC 4 บิตบน), ดังนั้น loader and linker (หัวข้อ 2.9) จะต้องระวังไม่วางส่วนของโพรซีเดอร์เกินขอบเขตของแอดเดรสของ `jal` (ซึ่งคือ $2^{28} = 256$ MB หรือ ราว 64 ล้านคำสั่ง). หรือถ้าหลีกเลี่ยงไม่ได้ แทนที่จะใช้คำสั่ง jump, ก็อาจเปลี่ยนไปใช้ คำสั่ง jump register (`jr`) แทน โดย มีคำสั่งโหลดค่าแอดเดรสไว้ในรีจิสเตอร์ก่อน.

Hardware/Software Interface. ส่วนใหญ่ conditional branch จะกระโดดไปแอดเดรสใกล้ๆ. แต่ในกรณีที่เกิดต้องกระโดดไปไกลเกินกว่า ที่ 16 บิตของคำสั่ง branch จะไปได้, แอสเซม

เบลอจะใช้ unconditional jump ช่วย.

ตัวอย่าง Branching Far Away. คำสั่ง

```
beq $s0, $s1, L1
```

อาจจะแปลงไปเป็น

```
    bne $s0, $s1, L2
    j L1
L2:
```

ซึ่งสามารถกระโดดไปได้ไกลขึ้น.

2.8.3 Decoding Machine Language.

บางครั้งเราอาจจะต้องทำ reverse-engineer คือ แปลงจาก machine language กลับมาเป็นภาษาแอสเซมบลี เช่น ``core dump''.

ตัวอย่าง Decoding Machine Code. จากคำสั่งภาษาเครื่อง `0x00af8020` แปลงไปเป็นภาษาแอสเซมบลีของ MIPS,

- (1) แปลงเป็นฐานสอง และ หา op code
  $0000\,0000\,1010\,1111\,1000\,0000\,0010\,0000_{two}$
- (2) 6 บิตบนคือ op code ซึ่งคำสั่งนี้คือ 000000_{two} ซึ่งดูตารางในรูป 2.9ประกอบ จะพบว่านี่คือคำสั่ง R-format ที่ตารางบนสุด แถว `0(000)` (บิต 31-29) กับ หลัก `0(000)` (บิต 28-26).
- (3) สำหรับ R-format ดูตารางล่าง ซึ่งจาก 6 บิตท้าย (funct field) ซึ่งคือ 100000_{two} จะบอกได้ว่าเป็นคำสั่ง `add`: แถว `4(100)` (บิต 5-3) กับ หลัก `0(000)` (บิต 2-0).
- (4) เราถอดรหัสค่าที่เหลือตามฟิลด์ (ดูรูป 2.10): $0000\,00, 00\,101, 0\,1111, \; 1000\,0, 000\,00, 10\,0000_{two}$
  - rs $= 00101$ คือ รีจิสเตอร์ `$a1`
  - rt $= 01111$ คือ รีจิสเตอร์ `$t7`
  - rd $= 10000$ คือ รีจิสเตอร์ `$s0`
  - shamt $= 00000$ คำสั่ง `add` ไม่ได้ใช้ฟิลด์นี้
- ดังนั้นนี่คือคำสั่ง `add $s0, $a1, $t7`.

2.9 Translating and Starting Program

จากโปรแกรมในภาษาซี มีขั้นตอนใหญ่อยู่ 4 ขั้นตอนในการแปลจาก text file จนเป็น executable file บนฮาร์ดดิสค์ และโหลดเข้าไปในหน่วยความจำและรันโปรแกรม. รูป 2.9 แสดงลำดับขั้นตอนในการแปลโปรแกรมภาษาซี ไปเป็น executable file จนถึงโหลดเข้าหน่วยความจำ.

op(31:26)								
28–26 31–29	0(000)	1(001)	2(010)	3(011)	4(100)	5(101)	6(110)	7(111)
0(000)	R-format	Bltz/gez	jump	jump & link	branch eq	branch ne	blez	bgtz
1(001)	add immediate	addiu	set less than imm.	sltiu	andi	ori	xori	load upper imm
2(010)	TLB	FlPt						
3(011)								
4(100)	load byte	load half	lwl	load word	lbu	lhu	lwr	
5(101)	store byte	store half	swl	store word			swr	
6(110)	lwc0	lwc1						
7(111)	swc0	swc1						

op(31:26)=010000 (TLB), rs(25:21)								
23–21 25–24	0(000)	1(001)	2(010)	3(011)	4(100)	5(101)	6(110)	7(111)
0(00)	mfc0		cfc0		mtc0		ctc0	
1(01)								
2(10)								
3(11)								

op(31:26)=000000 (R-format), funct(5:0)								
2–0 5–3	0(000)	1(001)	2(010)	3(011)	4(100)	5(101)	6(110)	7(111)
0(000)	shift left logical		shift right logical	sra	sllv		srlv	srav
1(001)	jump reg.	jalr			syscall	break		
2(010)	mfhi	mthi	mflo	mtlo				
3(011)	mult	multu	div	divu				
4(100)	add	addu	subtract	subu	and	or	xor	not or (nor)
5(101)			set l.t.	sltu				
6(110)								
7(111)								

Figure 2.9: MIPS instruction encoding.

Name	Fields						Comments
Field size	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	All MIPS instructions 32 bits
R-format	op	rs	rt	rd	shamt	funct	Arithmetic instruction format
I-format	op	rs	rt	address/immediate			Transfer, branch, imm. format
J-format	op	target address					Jump instruction format

Figure 2.10: MIPS instruction format.

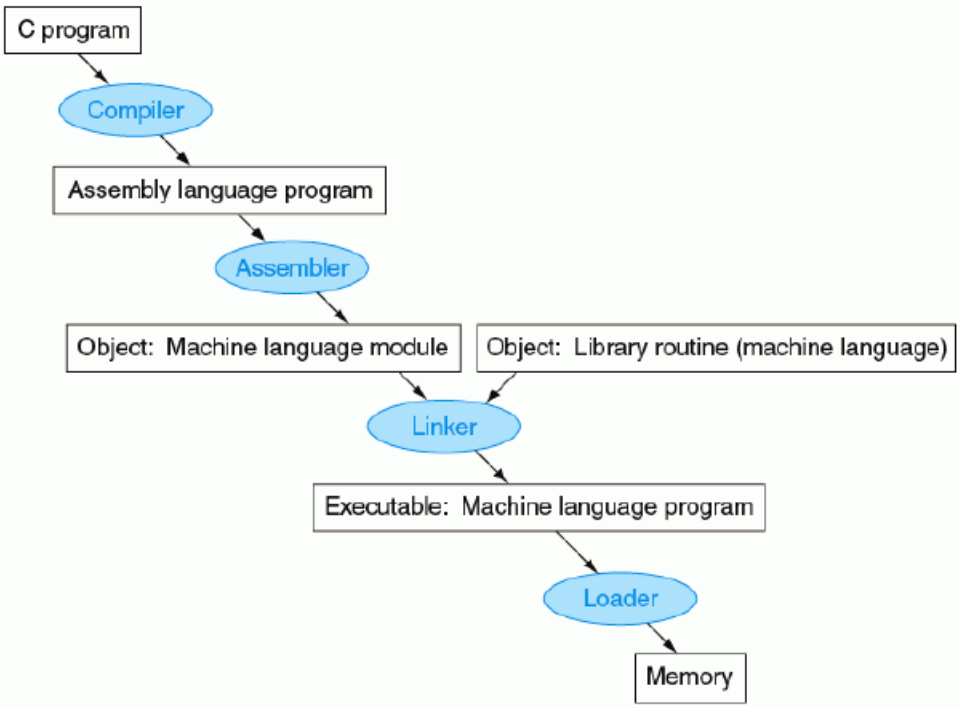


FIGURE 2.28 A translation hierarchy for C. A high-level-language program is first compiled into an assembly language program and then assembled into an object module in machine language. The linker combines multiple modules with library routines to resolve all references. The loader then places the machine code into the proper memory locations for execution by the processor. To speed up the translation process, some steps are skipped or combined together. Some compilers produce object modules directly, and some systems use linking loaders that perform the last two steps. To identify the type of file, UNIX follows a suffix convention for files: C source files are named `x.c`, assembly files are `x.s`, object files are named `x.o`, statically linked library routines are `x.a`, dynamically linked library routes are `x.so`, and executable files by default are called `a.out`. MS-DOS uses the suffixes `.C`, `.ASM`, `.OBJ`, `.LIB`, `.DLL`, and `.EXE` to the same effect.

Figure 2.11: ลำดับการแปลโปรแกรมภาษาซี.

คอมไพเลอร์ (Compiler) คอมไพเลอร์แปลโปรแกรมภาษาซี เป็น โปรแกรมภาษาแอสเซมบลี (assembly language program)

คอมพิวเตอร์ในยุคแรกๆ, พวกระบบปฏิบัติการและแอสเซมเบลอร์หลายตัวๆช่วงปี ค.ศ. 1975 ถูกเขียนขึ้นโดยตรงด้วยภาษาแอสเซมบลี เพราะว่าหน่วยความจำตอนนั้นมีขนาดเล็กมาก และคอมไพเลอร์ในยุคนั้นก็ยังไม่มีประสิทธิภาพพอ. แต่ในปัจจุบัน ทั้งขนาดหน่วยความจำที่ใหญ่ขึ้นมาก (เป็นหลายแสนเท่าของยุคนั้น) รวมถึง ประสิทธิภาพของคอมไพเลอร์ก็ดีขึ้นมาก ทำให้ คอมไพเลอร์สามารถแปลโปรแกรมเป็น โปรแกรมภาษาแอสเซมบลีได้ดีพอๆกับผู้เชี่ยวชาญภาษาแอสเซมบลี และบางครั้ง โดยเฉพาะกับโปรแกรมใหญ่แล้ว บางทีดีกว่าด้วย.

แอสเซมเบลอร์ (Assembler) เนื่องจากภาษาแอสเซมบลีเป็นการเชื่อมต่อ (interface) กับภาษาสูง, เพื่อความเข้าใจง่าย คำสั่งกับค่าเฉพาะบางอย่าง หรือ กลุ่มของคำสั่ง ในภาษาแอสเซมบลี อาจจะมองเป็น คำสั่งต่างหากเองได้เช่น คำสั่งเราอาจต้องการคำสั่งในการย้ายข้อมูลระหว่างรีจิสเตอร์ เช่น อยากย้ายข้อมูลจากรีจิสเตอร์ `$s0` ไปที่ `$s1` เราต้องการคำสั่ง `move $s1, $s0` แต่สำหรับฮาร์ดแวร์ เราไม่จำเป็นต้องสร้างอะไรพิเศษสำหรับคำสั่ง `move` เพราะ เราสามารถใช้คำสั่ง `add $s1, $zero, $s0` ซึ่งจะให้ผลที่ต้องการเหมือนกับมีคำสั่ง `move` อยู่จริงๆได้. หรือเราอาจต้องการคำสั่งที่ไม่ต้องทำอะไรเลย (no operation), `nop` แต่ฮาร์ดแวร์ไม่จำเป็นต้องสร้างอะไรเพิ่มเพื่อจะมีคำสั่ง `nop`, เราอาจจะใช้คำสั่ง `sll $t0, $t0, 0x0` ซึ่งคือให้ shift ค่าในรีจิสเตอร์ `$t0` ไป 0 บิต ซึ่งก็คือค่าเดิม ไม่ได้ทำอะไรเลย ก็จะให้ผลเหมือนกับ `nop` ได้. คำสั่งพวกนี้ พวกที่ใช้คำสั่งอื่น ที่ตั้งให้ทำงานได้ตามความหมายที่ต้องการ เราจะเรียกว่าเป็น คำสั่งเทียม (pseudoinstruction). โดย เวลาเราเขียนโปรแกรมภาษาแอสเซมบลี เราสามารถเรียกใช้ คำสั่งเทียม นี้ได้เช่นเดียวกับคำสั่งจริงๆ แล้วแอสเซมเบลอร์จะทำหน้าที่แปลง คำสั่งเทียมนี้ เป็นคำสั่งจริงๆที่ทำงานได้ตามความหมายของมันแทน เช่น หากเราต้องการโหลดค่า 32 บิต เราอาจจะใช้คำสั่ง `li` ได้ เช่น

```
li $s0, 0x80A00030
```

คำสั่ง `li` เป็นคำสั่งเทียม ซึ่งแอสเซมเบลอร์จะแทนเป็นคำสั่ง `lui` กับ `ori` ให้

```
lui $s0, 0x80A0
ori $s0, $s0, 0x0030
```

เช่นเดียวกันคำสั่งเทียมพวก `blt` จะถูกแทนด้วย `slt` กับ `bne` หรือ คำสั่งในการ branch ไปแอดเดรสที่ไกลเกินระยะ แอสเซมเบลอร์ก็ช่วยแทนด้วยคำสั่ง branch และ jump ให้.

สรุปง่ายๆว่า คำสั่งเทียม ช่วยให้ เสมือนว่า สถาปัตยกรรม MIPS มีคำสั่งมากหลากหลายมากขึ้น และ นักเขียนโปรแกรมภาษาแอสเซมบลี ยังสามารถเขียนโปรแกรมที่อ่านได้ง่ายขึ้นด้วย.

หน้าที่หลักของแอสเซมเบลอร์ คือ แปลงโปรแกรมภาษาแอสเซมบลี (ในรูปของ text file) ไปเป็นคำสั่งเครื่อง (machine code) ในรูปของ object file, ซึ่งจะรวมทั้ง ส่วนของ คำสั่งภาษาเครื่อง (machine code), ข้อมูลโปรแกรมใช้ (data), และข้อมูลที่ใช้ประกอบในการโหลดโปรแกรม เข้าไปที่หน่วยความจำ (เช่น ส่วนของ data ควรอยู่ที่ไหน).

ในการแปลงคำสั่งแอสเซมบลีไปเป็นภาษาเครื่อง (ในฐานสอง), แอสเซมเบลอร์ ต้องจัดการคำนวณหาค่าแอดเดรส ของฉลาก (labels) ต่างๆในโปรแกรมออกมาก่อน. แอสเซมเบลอร์ เก็บข้อมูลของ การใช้ฉลากในโปรแกรมไว้ใน ตารางสัญลักษณ์ (symbol table). โดย ในตารางสัญลักษณ์ จะเก็บชื่อฉลาก และ ค่าแอดเดรส ที่มันอ้างถึง.

ในระบบปฏิบัติการ UNIX, object file จะประกอบไปด้วย 6 ส่วน:

- object file header เป็นเหมือนสารบัญ หรือ metadata ที่บอกขนาดและตำแหน่งของส่วนประกอบอื่นๆใน object file.
- text segment เป็นส่วนที่เก็บ คำสั่งภาษาเครื่อง (machine code).
- static data segment เป็นส่วนที่เก็บข้อมูลสำหรับการรันโปรแกรม.
- relocation information เป็นส่วนที่ระบุว่า คำสั่งไหน และ ข้อมูล word ไหน ที่ใช้แอดเดรสแบบสมบูรณ์ (absolute addresses) ที่จะ มีผลกับ ตำแหน่งในหน่วยความจำที่โปรแกรมถูกโหลดเข้าไป.
- symbol table เป็นส่วนที่ใช้เก็บข้อมูลของฉลาก รวมถึง ฉลากของ external references.
- debugging information เป็นส่วนที่บอกว่า modules ต่างๆในโปรแกรมถูกคอมไพล์มาอย่างไร เพื่อให้ debugger สามารถเชื่อมโยง machine code กลับไปเป็น source file ในภาษาซีได้ (แบบที่อ่านได้ง่ายขึ้น).

ลิงค์เกอร์ (Linker) เวลาที่เราคอมไพล์โปรแกรมจริงๆ คอมไพล์เสร็จ ลองรันดู แล้วโปรแกรมทำงานไม่ถูกใจ เราก็แก้โค้ดใหม่ คอมไพล์ใหม่. เวลาที่เราแก้โค้ด บางทีเราแก้แค่ไม่กี่บรรทัด, ถ้าคอมไพลเลอร์ต้องคอมไพล์ทุกอย่างใหม่หมด ทุกครั้ง มันก็จะเสียเวลามาก. โดยเฉพาะอย่างนี้เวลาที่เรา เรียกใช้ standard library routines เพราะว่า เราเขียนโปรแกรมของเรา แก้โปรแกรมของเรา, ส่วนใหญ่แล้ว ไม่มีใครไปแก้ส่วนที่เป็น standard library routines. ดังนั้น แทนที่จะคอมไพล์ทุกอย่างใหม่หมด, คอมไพเลอร์โพรซีเดอร์แต่ละส่วนแยกกัน และเอามันมาประกอบกันทีหลัง: ทำให้การแก้โค้ด แค่หนึ่งบรรทัด สองบรรทัด จะแค่คอมไพล์ใหม่แค่โพรซีเดอร์ที่เกี่ยวข้องอันเดียวแล้วประกอบเข้าไปใหม่. ซึ่งคอมไพเลอร์ จะใช้ link editor หรือ linker ในการประกอบ machine codes ของส่วนต่างๆเข้าด้วยกัน เป็น machine code ที่พร้อมรันได้ (ในรูปของ executable file). ลิงค์เกอร์ทำงาน 3 ขั้นตอน:

- วางคำสั่งและข้อมูลของ modules ต่างๆ ไว้ในหน่วยความจำ
- หาค่าแอดเดรสของฉลากต่างๆทั้งฉลากของข้อมูลและคำสั่ง
- จัดการแทนค่าฉลากเป็นค่าแอดเดรสที่ถูกต้อง

ลิงค์เกอร์ใช้ relocation information กับ symbol table ที่มากับทุกๆ object module ในการหาค่าแอดเดรสของฉลากต่างๆ (ไม่ว่าจะเป็นฉลากจาก branch, jump, หรือ ฉลากระบุตำแหน่งของข้อมูล).

รวมทั้ง, ลิงเกอร์จะเตรียมการสำหรับ เวลาที่โปรแกรมถูกโหลดลงในหน่วยความจำ, แล้วทุกๆที่ในโปรแกรมที่อ้างถึงแอดเดรสแบบสมบูรณ์ (absolute addressing) จะถูกจัดการให้เป็นแอดเดรสที่ถูกต้องตามตำแหน่งที่ถูกโหลดเข้าในหน่วยความจำ.

ลิงค์เกอร์จะสร้าง executable file ออกมา, executable file คือ ไฟล์ที่พร้อมจะรันในคอมพิวเตอร์ได้. ซึ่งโดยทั่วไปแล้ว, executable file ก็อยู่ในรูปแบบเดียวกับ object file เพียงแต่ ทุกๆฉลากและจุดอ้างอิง จะถูกจัดการเรียบร้อย และพร้อมจะรันได้. แต่ก็มีไฟล์ที่เป็น partially linked file เช่นพวก library routines ที่ยังมีจุดอ้างอิงที่ยังไม่ได้จัดการให้เป็นแอดเดรส (unresolved addresses).

ตัวอย่าง Linking Object Files. ตัวอย่างข้างล่างมี 2 object files สำหรับ โพรซีเดอร์ A กับ โพรซีเดอร์ B. เราจะดูตัวอย่างว่า ลิงเกอร์จะจัดการจุดอ้างอิงและฉลากใน object files ทั้งสองนี้อย่างไร.

Object file header		
Name	Procedure A	
Text size	0x100	
Data size	0x20	
Text segment		
Address	Instruction	
0	[1000 11,11 100,0 0100, 0000 0000 0000 0000]	
4	[0000 11, 00 0000 0000 0000 0000 0000 0000]	
...	...	
Data segment		
Address	Data	
0	0x20100A07	
...	...	
Relocation information		
Address	Instruction type	Dependency
0	lw	X
4	jal	B
Symbol table		
Label	Address	
X	-	
B	-	

```
A:  lw $a0, 0($gp)
    jal B
    ...
X:  0x20100A07
```

Object file header		
Name	Procedure B	
Text size	0x200	
Data size	0x30	
Text segment		
Address	Instruction	
0	[1010 11,11 100,0 0101, 0000 0000 0000 0000]	
4	[0000 11, 00 0000 0000 0000 0000 0000 0000]	
...	...	
Data segment		
Address	Data	
0	0x00100007	
...	...	
Relocation information		
Address	Instruction type	Dependency
0	sw	Y
4	jal	A
Symbol table		
Label	Address	
Y	-	
A	-	

```
B:  sw $a1, 0($gp)
    jal A
    ...
Y:  0x00100007
```

โพรซีเดอร์ A ต้องการแก้ค่าแอดเดรสของฉลาก X และ B ในคำสั่งที่แอดเดรส 0 และ 4 ดังระบุใน relocation information.

จากธรรมเนียมการแบ่งส่วนหน่วยความจำของ MIPS (รูป 2.6), ส่วนสำหรับโปรแกรมคำสั่งเริ่มที่แอดเดรส 0x0040 0000 และส่วนสำหรับข้อมูลเริ่มที่แอดเดรส 0x1000 0000

ส่วนคำสั่งของโพรซีเดอร์ A จะถูกใส่ไว้ในหน่วยความจำเริ่มที่แอดเดรส 0x0040 0000 และส่วนของข้อมูลเริ่มที่แอดเดรส 0x1000 0000. ส่วน header ของ A บอกว่าส่วนคำสั่งของ A มีขนาด 0x100 และส่วนข้อมูลมีขนาด 0x20. ดังนั้นส่วนคำสั่งของโพรซีเดอร์ B จะถูกใส่ไว้ในหน่วยความจำเริ่มที่แอดเดรสถัดมา คือ 0x0040 0100 และส่วนข้อมูลของ B เริ่มที่แอดเดรส 0x1000 0020.

คำสั่ง `jal` อ้างถึงแอดเดรสแบบ pseudodirect และ 26 บิตของ address field คือ word addressing ซึ่งมีค่าเท่ากับ 28 บิตของ byte addressing (เทียบเท่าการคูณ 4) หากต้องการกระโดดไปที่แอดเดรส `0x0040 0000`, ค่า address field ของ `jal` จะคือ $400000_{hex}/4 = 100000_{hex}$.

คำสั่ง `lw` และ `sw` อ้างถึงแอดเดรสแบบ base โดยค่า offset จะทำ signed extension และบวกค่าใน base register. ดังนั้นหากต้องการอ้างถึงข้อมูลที่แอดเดรส `0x1000 0000`, ค่าเริ่มต้นของ `$gp` จะอยู่ที่ `0x1000 8000`. ค่า offset คือ:

$$\begin{aligned} \text{offset} + \$\text{gp} &= \text{target address} \\ \text{offset} &= \text{target address} - \$\text{gp} \\ &= (10000000_{hex}) - (10008000_{hex}) = -8000_{hex} \end{aligned}$$

และ $-8000_{hex} = 0x8000$ (ซึ่งเมื่อทำ sign extension เป็น 32 บิตจะคือ `0xFFFF8000`).

ฉลาก	แอดเดรส	คำสั่ง	mode	ค่าที่แทนฉลาก
A	0x00400000	jal	pseudodirect	0x0100000
B	0x00400100	jal	pseudodirect	0x0100040
X	0x10000000	lw	base	$-8000_{hex} = 0x8000$
Y	0x10000020	lw	base	$-7FE0_{hex} = 0x8020$

ตาราง 2.3 แสดงตัวอย่างข้อมูลภายในของไฟล์ที่ได้จากลิงค์เกอร์.

Table 2.3: ตัวอย่างผลที่ได้จากลิงค์เกอร์

Executable file header		
Text size	0x300	
Data size	0x50	
Text segment		
Address	Instruction	
0x0040 0000	[1000 11,11 100,0 0100, 1000 0000 0000 0000]	lw $a0, 0x8000($gp)
0x0040 0004	[0000 11, 00 0001 0000 0000 0000 0100 0000]	jal 0x00400100
...	...	
0x0040 0100	[1010 11,11 100,0 0101, 1000 0000 0010 0000]	sw $a1, 0x8020($gp)
0x0040 0104	[0000 11, 00 0001 0000 0000 0000 0000 0000]	jal 0x00400000
Data segment		
Address	Data	
0x1000 0000	0x20100A07	
...	...	
0x1000 0020	0x00100007	
...	...	

โหลดเดอร์ (Loader) หลังจากผ่านลิงค์เกอร์ ตอนนี้เราได้ executable file อยู่ในฮาร์ดดิสค์พร้อมที่จะรันได้แล้ว. โหลดเดอร์ในระบบปฏิบัติการ UNIX จะอ่าน executable file เข้าไปในหน่วยความจำและเริ่มโปรแกรม ตามขั้นตอน:

- อ่าน header ของไฟล์ เพื่อดูว่าส่วนของคำสั่ง และ ข้อมูล ใหญ่ขนาดไหน.
- จองพื้นที่หน่วยความจำสำหรับ คำสั่ง และ ข้อมูล.

- คัดลอก คำสั่ง และ ข้อมูล จาก executable file ไปใส่ไว้ในหน่วยความจำ.
- ในกรณีที่ `main` โปรแกรมมีพารามิเตอร์, คัดลอกค่าของพารามิเตอร์ ไปไว้ใน stack.
- initialize รีจิสเตอร์ต่างๆ และ ตั้งค่า stack pointer ให้พร้อมใช้งาน.
- กระโดดไปที่ start-up routine ที่จะคัดลอกค่าของพารามิเตอร์ไปไว้ที่ argument registers และ เรียกฟังชั่น `main` ของโปรแกรมทำงาน. เมื่อฟังชั่น `main` ทำงานเสร็จ ก็จะกลับมาที่ start-up routine เพื่อจบโปรแกรมด้วย `exit` system call.

Dynamically Linked Libraries การคอมไพล์ library routines แล้วติดมันไปกับโค้ดของโปรแกรม รวมเป็น executable file เลย มีข้อดีคือ สามารถทำงานได้อย่างรวดเร็ว แต่ก็มีข้อเสียคือ library routines กลายเป็นส่วนหนึ่งของ executable code, ดังนั้นหากมี library เวอร์ชั่นใหม่ออกมา (อาจจะออกมาแก้ bugs), executable file ที่ก็ยังใช้ routines เดิมอยู่ (จนกว่าจะคอมไพล์ใหม่). นอกจากนั้น อาจทำให้ executable file ที่ได้มีขนาดใหญ่มากเกินไป แล้วหลายๆ executable files อาจจะใช้ library เหมือนๆกัน.

ดังนั้นแทนที่จะคอมไพล์ library ไปเป็นส่วนหนึ่งของ executable file, dynamically linked libraries (DLLs) คือ library routines ที่จะลิงค์กับโปรแกรมเฉพาะตอนที่รันโปรแกรม. รูป 2.12 แสดงการลิงค์ library ระหว่างที่รันโปรแกรม. โดยครั้งแรก การเรียกโพรซีเดอร์จาก DLL จะถูกโยงไปยังโพรซีเดอร์ตัวตุ๊กตาก่อน โดยตัวตุ๊กตานี้ ก็จะทำการค้นหา DLL ที่ต้องการ, โหลด DLL ขึ้นมา, และก็จะทำลิงค์ของ DLL นี้ให้ถูกต้อง, เปลี่ยนการโยงให้โยงไปหา DLL โดยตรง, และ เรียก DLL ทำงาน. เมื่อ DLL ทำงานเสร็จก็จะกลับไปทำงานที่โปรแกรม caller และถ้าหากมีเรียก DLL นี้อีก ก็จะโยงไปที่ DLL โดยตรงเลย ไม่ต้องไปที่โพรซีเดอร์ตัวตุ๊กตาอีก. ดังนั้นด้วยวิธี เราก็สามารถโหลด เฉพาะ libraries ที่ต้องการใช้ขึ้นมาได้. การใช้ DLL นิยมมากทั้งใน MS Windows และ UNIX.

Starting a Java Program จุดประสงค์หลายอย่างของภาษาจาวา ต่างจากภาษาซี, จุดประสงค์สำคัญอันหนึ่งก็คือ ให้ โปรแกรมภาษาจาวา สามารถรันได้ กับเครื่องคอมพิวเตอร์เครื่องไหนก็ได้ (แม้ว่าอาจจะทำให้โปรแกรมต้องรันช้าลงไปบ้าง).

โปรแกรมภาษาจาวาจะถูกคอมไพล์เป็นไบนารี่โค้ด ของ Java bytecode, ซึ่งเป็นคำสั่งระดับล่างของจาวา สำหรับรันบน java interpreter. รูป 2.13 แสดงขั้นตอนการแปลภาษาจาวา. เนื่องจากโปรแกรมภาษาจาวาจะถูกคอมไพล์เป็น java bytecode ซึ่งไม่ขึ้นกับเครื่องที่รันอยู่ โปรแกรมภาษาจาวาจึงมี portability สูง (ง่ายที่จะไปรันกับเครื่องต่างๆ)

ในการรันภาษาจาวา, java interpreter หรือ JVM (Java Virtual Machine) จะทำหน้าที่แปลง java bytecode เป็น machine code ของเครื่องที่รันไปพร้อมๆกับการรันโปรแกรม. ดังนั้นโปรแกรมภาษาจาวา จึงทำงานช้ากว่าโปรแกรมที่เขียนด้วยภาษาซีมาก.

เพื่อลดปัญหาประสิทธิภาพของโปรแกรมภาษาจาวานี้, JIT (Just In Time compiler) จะดูว่าเวลารันแล้ว methods ได้บ้างที่ใช้งานเยอะและจะคอมไพล์มักออกมาเป็น machine code ของเครื่องที่รันอยู่. ส่วนของ machine code ที่ได้จะถูกเก็บไว้ สำหรับการรันครั้งต่อไป ดังนั้นโปรแกรมภาษาจาวาจะรันได้เร็วขึ้น เมื่อรันครั้งต่อไป. ทำให้โปรแกรมภาษาจาวาที่ถูกรันบ่อยๆ จะเสียเวลาในการแปลเป็นภาษาเครื่องน้อยมาก (เพราะหลายๆส่วนถูกแปลงเป็น machine code ไปแล้ว). ด้วยความเร็วของคอมพิวเตอร์ บวกกับความสามารถของคอมไพเลอร์ปัจจุบัน ทำให้ความเร็วของโปรแกรมภาษาจาวา กับ ซี หรือ C++ ใกล้กันมากขึ้นเรื่อยๆ.

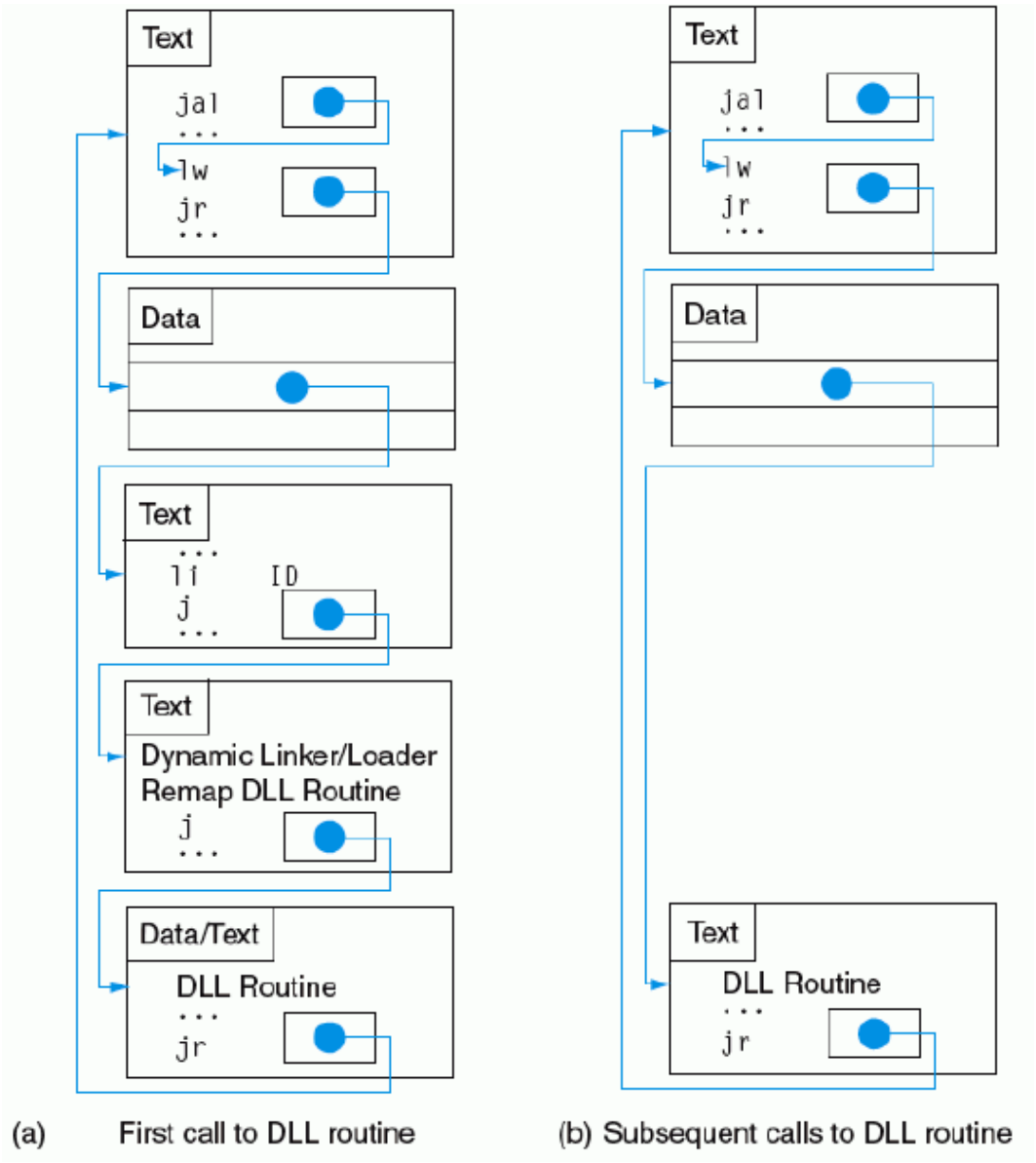


Figure 2.12: Dynamically linked library via lazy procedure linkage. การเรียก DLL ครั้งแรก (a) จะต้องหา DLL ที่ต้องการ, เมื่อเจอแล้วก็จะ map และ link มันไว้ ทำให้ การเรียกครั้งต่อๆมา (b) ทำได้สะดวกขึ้น.

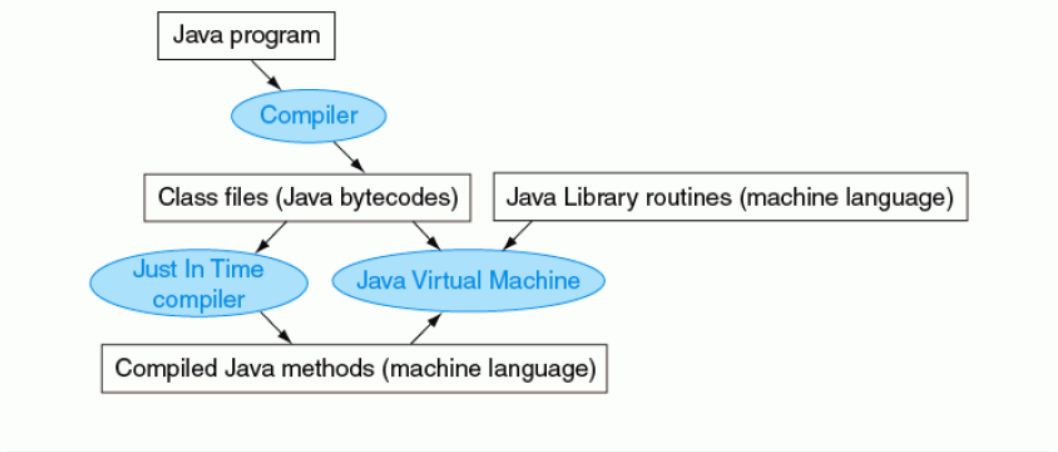


FIGURE 2.30 A translation hierarchy for Java. A Java program is first compiled into a binary version of Java bytecodes, with all addresses defined by the compiler. The Java program is now ready to run on the interpreter, called the Java Virtual Machine (JVM). The JVM links to desired methods in the Java library while the program is running. To achieve greater performance, the JVM can invoke the Just In Time (JIT) compiler, which selectively compiles methods into the native machine language of the machine on which it is running.

Figure 2.13: การแปลภาษาจาวา

Chapter 3

Arithmetic for Computers

บทนี้เราจะคุยกันเรื่อง การใช้ฮาร์ดแวร์แทนตัวเลข (representation of numbers) อัลกอริทึ่ม ที่ใช้คำนวณพีชคณิต (arithmetic algorithms) รวมถึงฮาร์ดแวร์ที่ใช้ทำการคำนวณ และ implementation ของคำสั่งคำนวณต่างๆ.

คำถาม

- ตัวเลขลบจะแสดงอย่างไร
- ตัวเลขที่ใหญ่ที่สุดที่แสดงได้ใน computer word เท่าไร
- จะเกิดอะไรขึ้นถ้าการคำนวณให้ผลลัพธ์เป็นตัวเลขที่ใหญ่เกินกว่าสจะแสดงได้
- แล้วเศษส่วน กับ จำนวนจริงหละ
- แล้วเรื่อง bug ของ Pentium ช่วงปี 1994 หละ
- แล้วเราจะสร้างฮาร์ดแวร์สำหรับ บวก ลบ คูณ และ หาร อย่างไร

3.1 Signed and Unsigned Numbers

ตัวอย่างการแปลง binary เป็น decimal: $\texttt{0b1010 0100} = (1\times 2^7)+(0\times 2^6)+(1\times 2^5)+(0\times 2^4)+(0\times 2^3)+(1\times 2^2)+(0\times 2^1)+(0\times 2^0) = 164_{10}$ หรือ $\texttt{0xA4}$.

Overflow คือ สถานะการณ์ที่ตัวเลขที่ควรจะเป็นคำตอบที่ถูกต้องของการคำนวณ ไม่สามารถแสดงได้ด้วยฮาร์ดแวร์ เช่น หากนำระบบขนาดสี่บิตมาบวก: $4+\texttt{0b0100}+\texttt{0b0101} = \texttt{0b1001} = 9_{10}$ ซึ่งถูกต้อง แต่ หากคำนวณ $8+9$ ซึ่งคำตอบที่ถูกต้องคือ 17_{10} ซึ่ง ระบบขนาดสี่บิต ไม่สามารถแสดงได้ เช่นนี้จะเกิด overflow ขึ้น คือ ผลบวกไหลล้นออกจากความสามารถของฮาร์ดแวร์ที่จะแทนได้.

Negative number representation. เราสามารถเลือกที่จะแทนเลขลบได้หลายแบบ เช่น

- (1) กำหนด sign bit: (sign and magnitude representation). ข้อเสีย (i) จะเลือกให้บิตไหนเป็น sign bit. (ii) ฮาร์ดแวร์ที่จะทำการบวกลบจะต้องการขั้นตอนพิเศษเพื่อทำจัดการกับ sign bit. (iii) การแยก sign bit ออกมา เช่น $\texttt{0b0011} = +3_{10}$ และ $\texttt{0b1011} = -3_{10}$ ทำให้เรามี $\texttt{0b0000} = +0_{10}$ และ $\texttt{0b1000} = -0_{10}$.
- (2) ใช้ระบบการแสดงเลข ที่ทำให้ปฏิบัติการคำนวณด้วยฮาร์ดแวร์ ให้ผลเหมือนกับ การคำนวณพีชคณิตที่ถูกต้อง ซึ่งทางเลือกนี้จะช่วยให้เราได้ฮาร์ดแวร์ที่ง่ายขึ้น.

ลองดูตัวอย่างนี้ของระบบสี่บิต เช่น

```
5 + 5 = 10            : 0101 + 0101 = 1010
5 - 5 = 5 + (-5) = 0 : 0101 + abcd = 0000
```

ถ้าเราเลือก `abcd` ให้เป็น `1011`, เราจะได้ `0101 + 1011` $= 1,\mathbf{0000}$ หรือ สี่บิตเป็นศูนย์ และ มีบิตพิเศษเกินมาทางซ้ายสุด.

ลองใหม่

```
1 + (-1) = 0  : 0001 + abcd = x,0000
```

สมมติให้ `x` แทนบิตพิเศษเกินทางซ้ายสุด ซึ่งตอนนี้เราไม่สนใจ เราจะได้ `abcd = 1111` ที่จะทำให้คำตอบซึ่งแสดงได้ในระบบสี่บิตของเราทำงานได้ถูกต้อง. เพราะว่า `0001 + 1111` $= 1,\mathbf{0000}$.

ลองเพิ่ม

```
4 + (-4) = 0 : 0100 + abcd = x,0000; then abcd = 1100
6 + (-6) = 0 : 0110 + abcd = x,0000; then abcd = 1010
```

ดังนั้น ถ้าให้ 4-bit word: $A = [a_3a_2a_1a_0]$ และ $A + B = [\mathbf{0000}]$. พิจารณา $[a_3a_2a_1a_0] + [\bar{a}_3\bar{a}_2\bar{a}_1\bar{a}_0] = [1111]$, แล้ว

$$[a_3a_2a_1a_0] + [\bar{a}_3\bar{a}_2\bar{a}_1\bar{a}_0] + 1 \quad = \quad 1[\mathbf{0000}]. \tag{3.1}$$

เราจะเห็นว่า $B = \bar{A} + 1$.

Two's complement representation วิธีแสดงตัวเลขแบบ two's complement, มองเลขฐานสอง N บิต:
$[B_{N-1}b_{N-2}b_{N-3}\cdots b_3b_2b_1b_0]$ ว่ามีค่าเป็น $(B_{N-1} \times -2^{N-1}) + (b_{N-2} \times 2^{N-2}) + (b_{N-3} \times 2^{N-3}) + \cdots + (b_2 \times 2^2) + (b_1 \times 2^1) + (b_0 \times 2^0)$.

เช่น ระบบ 5 บิต:

$$\begin{aligned} [01001] &= (0 \times -2^4) + (1 \times 2^3) + (0 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) \\ &= 9 \end{aligned} \tag{3.2}$$

หรือ

$$[10111] = (1 \times -2^4) + (0 \times 2^3) + (1 \times 2^2) + (1 \times 2^1) + (1 \times 2^0) = -9.$$

วิธี two's complement ได้ชื่อมาจากการที่ค่า unsigned sum ของ n-bit number กับค่าลบของมัน จะเป็น 2^n. เช่น 5-bit number `[01001]` (หรือ 9) กับ `[10111]` จะเป็น `1, [00000]` ซึ่ง(เมื่อรวมบิตพิเศษ)เท่ากับ 2^5. ค่า two's complement ของเลขขนาด n บิต x ก็คือ $2^n - x$. หรือ จากสมการ 3.1, เราจะเห็นว่าวิธีลัดในการคำนวณค่า two's complement ของ x ก็คือ (1) ทำ bitwise inversion และ (2) บวก 1:

$$x + \bar{x} + 1 = 2^n \tag{3.3}$$

และ two's complement ของ x คือ $\bar{x} + 1$.

Property of reciprocals. ให้เลขขนาด n บิต $y = \bar{x} + 1$, ค่า two's complement ของ y คือ

$$2^n - y = 2^n - (\bar{x} + 1) \tag{3.4}$$

จากสมการ 3.3,

$$x = 2^n - \bar{x} - 1 \tag{3.5}$$

นำสมการ 3.5 แทนใน 3.4,

$$2^n - y = x. \tag{3.6}$$

หรือ พูดง่ายๆ คือ $x'' = x$, เมื่อ $'$ แทนการทำ two's complement
ข้อสังเกตุเกี่ยวกับการขยายขนาดคำแบบ sign extension,

```
4-bit words
5  = 0b0101
-5 = 0b1011

8-bit words
5  = 0b0000 0101
-5 = 0b1111 1011
```

การจะขยาย word ขนาดเล็กเป็นขนาดใหญ่ขึ้น เราจะนำดู msb (most significant bit) ของ word และ ใช้ msb เติมบิตที่เหลือของ word ที่ใหญ่กว่าดังตัวอย่างข้างต้น.

3.2 Addition and Subtraction

ดูตัวอย่างการคำนวณพีชคณิตสำหรับ words ขนาดสี่บิต ด้วยขนาดสี่บิต, word ในรูปแบบ two's complement สามารถแทนค่าได้

$$\begin{aligned}
0000 &= 0_{dec} \\
0001 &= 1_{dec} \\
0010 &= 2_{dec} \\
0011 &= 3_{dec} \\
0100 &= 4_{dec} \\
0101 &= 5_{dec} \\
0110 &= 6_{dec} \\
0111 &= 7_{dec} \\
1000 &= -8_{dec} \\
1001 &= -7_{dec} \\
1010 &= -6_{dec} \\
1011 &= -5_{dec} \\
1100 &= -4_{dec} \\
1101 &= -3_{dec} \\
1110 &= -2_{dec} \\
1111 &= -1_{dec}.
\end{aligned}$$

สังเกตุ, ค่าบวกมากที่สุดที่ 4-bit word แสดงได้คือ 7 และ ค่าลบมากที่สุดที่แสดงได้คือ -8. ถ้า 5-bit word ค่ามากสุดน้อยสุดจะเป็น $\{-16, 15\}$. ถ้า 6-bit word ค่ามากสุดน้อยสุดจะเป็น $\{-32, 31\}$. ถ้า n-bit word ค่ามากสุดน้อยสุดจะเป็น $\{-2^{n-1}, 2^{n-1}-1\}$.

```
(a) 5 + 2 = 7
    [0101]
    +
    [0010]
    ------
    [0111] = 7

(b) -5 + -2 = -7
     [1011]
     +
     [1110]
     ------
     [1001] = -7

(c) 5 + 3 = 8
     [0101]
     +
     [0011]
     ------
     [1000] = -8
```

สังเกตุ 4-bit word จะแทนค่าตัวเลขได้มากที่สุดแค่ 7 แต่ผลลัพธ์ที่ถูกใหญ่เกินกว่าที่ word จะแสดงได้, คำตอบที่ได้ใน 4-bit word จะผิด

ลองดูตัวอย่างข้างล่างเพิ่มเติม

```
(d) -5 + -3 = -8
     [1011]
     +
     [1101]
     ------
     [1000] = -8
```

อันนี้ไม่มีปัญหา เพราะว่า 4-bit word สามารถแสดงค่า -8 ได้. แต่ถ้าเราลอง

```
(e) -5 + -4 = -9
     [1011]
     +
     [1100]
     ------
     [0111] = 7
```

คำตอบผิด เพราะว่า 4-bit word แสดงค่าลบได้แค่ -8 ถ้าผลลัพธ์ที่ต้องการเกิน ค่าแสดงจะผิด.

เมื่อค่าผลลัพธ์ที่แสดงใน word ผิดจาก คำตอบที่ถูกต้อง เราเรียกว่า overflow เนื่องจากคำตอบที่ถูกต้อง มีค่าเกินกว่าที่ฮาร์ดแวร์จะแสดงได้. เมื่อพิจารณาลักษณะของ overflow สำหรับการบวก, สัญญาณบ่งชี้ว่าเกิด overflow ขึ้นมีสองอย่างคือ (1) การบวก เลขบวกสองตัว แล้วได้

ผลลัพธ์เป็นลบ (เช่น ตัวอย่างการบวก c) และ (2) การบวก เลขลบสองตัว แล้วได้ผลลัพธ์เป็นบวก (เช่น ตัวอย่างการบวก e).

และสำหรับการแสดงเลขแบบ two's complement, เครื่องหมายบวก/ลบ ของตัวเลข ดูได้ง่ายๆจาก msb ของเลข เช่น $[\underline{0}101]$ แทน $+5$ (msb = 0), $[\underline{1}011]$ แทน -5 (msb = 1) เป็นต้น.

ดังนั้น สำหรับ two's complement representation ถ้า $A+B=C$, ตาราง 3.1 แสดงกรณีที่จะเกิด overflow.

Table 3.1: Overflow จากการบวก

MSB_A	MSB_B	MSB_C	Overflow
0	0	0	no
0	0	1	yes
0	1	0	no
0	1	1	no
1	0	0	no
1	0	1	no
1	1	0	yes
1	1	1	no

จากตาราง 3.1, เมื่อวิเคราะห์ตรรกะ เราจะได้

$$ovf = \overline{msb}_A \cdot \overline{msb}_B \cdot msb_C + msb_A \cdot msb_B \cdot \overline{msb}_C$$

3.2.1 Subtraction

``Subtraction: Addition's Tricky Pal'' - No. 10, Top Ten Courses for Athletes at a Football Factor, David Letterman et al., Book of Top Ten Lists, 1990.

เพื่อความสะดวก เราจะทำการลบเลขสองตัว ด้วย การบวกเลขตัวแรก กับ two's complement ของเลขตัวที่สอง: $X-Y=X+(-Y)$.

4-bit word subtraction. การคำนวณ 3.7 ถึง 3.12 แสดงตัวอย่างการทำลบ. สังเกตุการคำนวณ 3.11 กับ 3.12 เกิด overflow ขึ้น. ตาราง 3.2 แสดงเงื่อนไขของ overflow สำหรับการบวกและลบ.

$$5-3 = 5+(-3)=2 \quad (3.7)$$

$$\begin{array}{rcl} 0101 & & 0101 \\ -\ 0011 & = & +\ \underline{1101} \\ . & & 0010 = 2_{dec} \end{array}$$

$$5-(-2) = 5+(2)=7 \quad (3.8)$$

$$\begin{array}{rcl} & & 0101 \\ & & +\ \underline{0010} \\ & & 0111 = 7_{dec} \end{array}$$

$$-5-3 = -5+(-3)=-8 \quad (3.9)$$

$$\begin{array}{rcl} & & 1011 \\ & & +\ \underline{1101} \\ & & 1000 = -8_{dec} \end{array}$$

$$-5-(-2) = -5+(2)=-3 \quad (3.10)$$

$$\begin{array}{rcl} & & 1011 \\ & & +\ \underline{0010} \\ & & 1101 = -3_{dec} \end{array}$$

$$5-(-4) = 5+(4)=9 \text{ (เกินค่าที่แสดงได้)} \quad (3.11)$$

$$\begin{array}{rcl} & & 0101 \\ & & +\ \underline{0100} \\ & & 1001 = -7_{dec} \text{ (overflow)} \end{array}$$

$$-5-4 = -5+(-4)=-9 \text{ (เกินค่าที่แสดงได้)} \quad (3.12)$$

$$\begin{array}{rcl} & & 1011 \\ & & +\ \underline{1100} \\ & & 0111 = 7_{dec} \text{ (overflow)} \end{array}$$

Table 3.2: Overflow conditions for addition and subtraction

Operation	Operand A	Operand B	Result indicating overflow
$A+B$	≥ 0	≥ 0	< 0
$A+B$	< 0	< 0	≥ 0
$A-B$	≥ 0	< 0	< 0
$A-B$	< 0	≥ 0	≥ 0

หมายเหตุ การคำนวณ unsigned integers ยังคงใช้มากสำหรับการคำนวณ memory addresses ซึ่งไม่สนใจ overflow. ดังนั้น MIPS จึงให้ทางเลือกที่ บางคำสั่งจะมี overflow ขณะที่บางคำสั่งไม่มี เช่น `add`, `addi`, `sub` จะมีการทำ overflow exception. ในขณะที่ `addu` (unsigned addition), `addiu` (unsigned immediate addition), และ `subu` (unsigned subtraction) จะไม่ทำ overflow exception.

อีกอย่าง ภาษาซี จะไม่สนใจ overflow, ดังนั้น MIPS C compilers จะเลือกใช้ `addu`, `addiu`, `subu` แทน `add`, `addi`, `sub`.

MIPS จะตรวจจับ overflow จากกลไกของ exception, ที่อาจเรียก interrupt: exception หรือ interrupt เป็น unscheduled procedure call ที่สำคัญ. ดูหัวข้อ 4.2.1 เพิ่มเติมสำหรับเรื่อง exception.

3.3 Logical Operations

SHIFT Logical. ปฏิบัติการ shift จะเคลื่อนบิตทั้งหมดใน word ไปทางซ้าย หรือ ทางขวา และเติมบิตที่ว่างด้วย 0.

```
$s0: 0000 0000 0000 0000 0000 0000 0000 1101

sll $t2, $s0, 8   # reg $t2 = reg $s0 << 8 bits
Machine Code is
[op | rs | rt | rd | shamt | funct ]
[ 0 | 0  | 16 | 10 |   8   |   0   ]

result:
$t2: 0000 0000 0000 0000 0000 1101 0000 0000
```

AND Logical. AND เป็น ปฏิบัติการที่ทำงานเป็นบิต โดยจะให้บิตที่ผลลัพธ์เป็น 1 เมื่อบิตที่ตำแหน่งนั้นของ operands ทั้งสองเป็น 1 ทั้งคู่.

```
$t1: 0000 0000 0000 0000 0000 1101 0000 0000
$t2: 0000 0000 0000 0000 0011 1100 0000 0000

and $t0, $t1, $t2   # reg $t0 = reg $t1 & reg $t2

result:
$t0: 0000 0000 0000 0000 0000 1100 0000 0000
```

OR Logical. OR เป็น ปฏิบัติการที่ทำงานเป็นบิต โดยจะให้บิตที่ผลลัพธ์เป็น 1 เมื่อบิตที่ตำแหน่งนั้นของ operand อย่างน้อยหนึ่งตัว เป็น 1.

```
$t1: 0000 0000 0000 0000 0000 1101 0000 0000
$t2: 0000 0000 0000 0000 0011 1100 0000 0000

or $t0, $t1, $t2   # reg $t0 = reg $t1 & reg $t2

result:
$t0: 0000 0000 0000 0000 0011 1101 0000 0000
```

3.4 Contructing ALU

เราจะนำวงจรคำนวณตรรกะและพีชคณิตมารวมกัน เรียกว่า ALU (Arithmetic Logic Unit). ก่อนเราจะนำวงจรคำนวณต่างๆมารวมกัน เพื่อความสะดวกเราจะใช้อุปกรณ์ Multiplexer หรือย่อๆว่า MUX ช่วย. รูป 3.1 แสดงสัญญลักษณ์แทน 4-to-1 MUX หรือ 4-input MUX (ถึงแม้จริงๆ จะมีอีกอินพุตหนึ่ง S).

อุปกรณ์ MUX จะทำหน้าที่คล้ายสวิตช์ โดย เอาต์พุต C ของ MUX จะเท่ากับ อินพุต A เมื่อ S = `00'; จะเท่ากับ B เมื่อ S = `01'; จะเท่ากับ C เมื่อ S = `10'; และ จะเท่ากับ D เมื่อ S = `11'. รูป 3.2 แสดงการใช้ MUX ในการสร้าง 1-bit logical unit: ใช้สัญญาณ `Operation` ในการเลือก

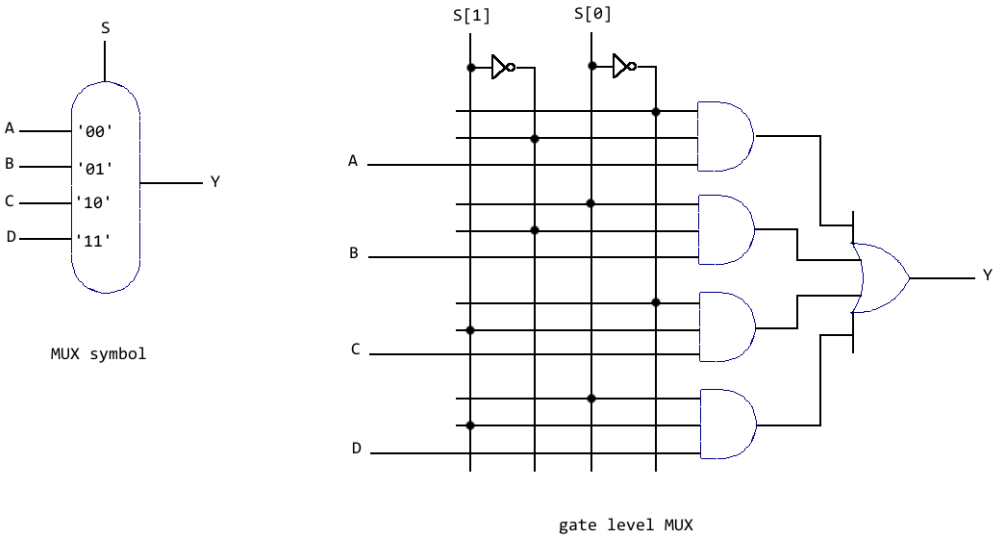


Figure 3.1: อุปกรณ์ MUX

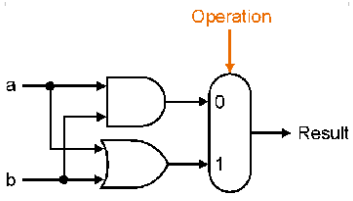


Figure 3.2: The 1-bit logical unit for AND and OR.

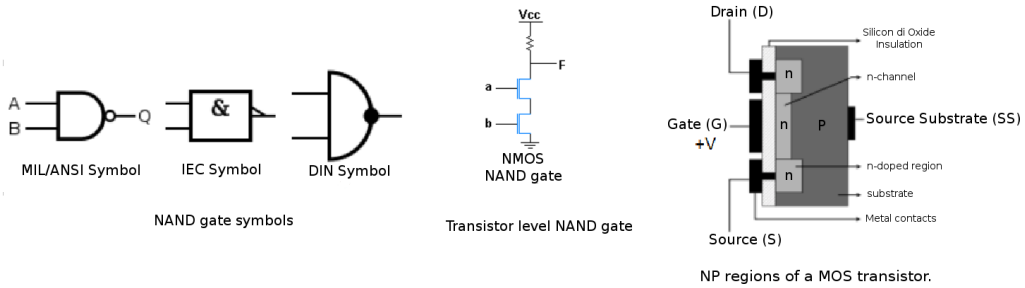


Figure 3.3: NAND gate: symbols, a schematic circuit, และ NP regions of NMOS transistor.

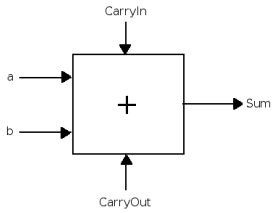


Figure 3.4: A 1-bit adder (full adder).

วงจร. ถ้า `Operation = 0`, ค่า `Result = a AND b`. แต่ถ้า `Operation = 1`, ค่า `Result = a OR b`.

รูป 3.3 แสดงสัญญลักษณ์แทน NAND gates แบบต่างๆ (ภาพแรก), วงจรไฟฟ้าของ NAND gate (ภาพกลาง), และ ภาพอย่างง่ายของโครงสร้างทางกายภาพของการประกอบสารกึ่งตัวตัวชนิด N และ P เป็น ทรานซิสเตอร์ชนิด NMOS. สำหรับทรานซิสเตอร์ชนิด NMOS, เมื่อจ่ายแรงดันเป็นบวกที่เพียงพอไปที่ขา gate จะสร้าง n-channel ขึ้นทำให้กระแสสามารถไหลระหว่างขา source กับ drain ได้. คุณสมบัติของ NAND คือ เอาต์พุต $F = \overline{a+b}$: เมื่อ a กับ b เป็น 1 พร้อมกัน ค่า F จะเป็น 0, ถ้าไม่อย่างนั้น F จะเป็น 1. ทบทวน ตรรกะ, เราสามารถสร้างตรรกะอื่นๆได้จาก NAND:

- logical not: $\overline{A \cdot A} = \bar{A}$.
- logical and: $\overline{\overline{A \cdot B}} = A \cdot B$.
- logical or: $\overline{\bar{A} \cdot \bar{B}} = A + B$ (de Morgan's).

นอกจากการคำนวณตรรกะแบบบิตต่อบิต การคำนวณพีชคณิต บวก ลบ ก็เป็นปฏิบัติการที่ใช้บ่อยๆ. รูป 3.4 แสดงสัญญลักษณ์ของวงจร 1-bit full adder, ซึ่งสำหรับการบวกหนึ่งบิต เราต้องการวงจรที่มีพฤติกรรมดังแสดงในตาราง 3.3. ซึ่งสามารถเขียนเป็นสมการตรรกะได้

$$s = (\bar{a} \cdot \bar{b} \cdot c_i) + (\bar{a} \cdot b \cdot \bar{c_i}) + (a \cdot \bar{b} \cdot \bar{c_i}) + (a \cdot b \cdot c_i) \quad (3.13)$$

$$= a \oplus b \oplus c_i \quad (3.14)$$

$$c_o = b \cdot c_i + a \cdot b + a \cdot c_i \quad (3.15)$$

โดย $\oplus$ แทน ตัวปฏิบัติการ logical exclusive OR.

Table 3.3: ตารางตรรกะของวงจร 1-bit full adder

Inputs			Outputs		Comments
a	b	Carry In (c_i)	Carry Out (c_o)	Sum (s)	(base 2)
0	0	0	0	0	$0+0+0=00$
0	0	1	0	1	$0+0+1=01$
0	1	0	0	1	$0+1+0=01$
0	1	1	1	0	$0+1+1=10$
1	0	0	0	1	$1+0+0=01$
1	0	1	1	0	$1+0+1=10$
1	1	0	1	0	$1+1+0=10$
1	1	1	1	1	$1+1+1=11$

3.4.1 A 32-bit ALU

เราสามารถนำ full adder และ logical units มาประกอบเป็น one-bit ALU ได้ดังแสดงในรูป 3.5. ภาพล่างแสดง one-bit ALU ที่จะใช้กับ MSB (ALU31 ในรูป 3.6) ส่วนภาพบนสำหรับบิตอื่นๆ (เช่น ALU0 - ALU2 ในรูป 3.6). รูป 3.6 แสดงการนำ one-bit ALUs ในรูป 3.5 มาต่อเป็น word ALU.

จากรูป 3.5 และ 3.6 หากเราต้องการทำ logical AND เราก็แค่จ่าย `Operation = 0` และ `Binvert = 0`; หากเราต้องการทำ logical OR เราก็แค่จ่าย `Operation = 1` และ `Binvert = 0`; หากเราต้องการทำ logical NOT เราก็แค่จ่าย `Operation = 1`, `Binvert = 1` และ จ่ายค่ารีจิสเตอร์ `$zero` เข้าเป็น Operand A: $0+\bar{B}=\bar{B}$;

หากเราต้องการทำ $A+B$, เราก็แค่จ่าย `Operation = 2` (หรือ 10_{two}), `Binvert = 0` และ `CarryIn = 0`. สำหรับ $A-B$, เราจะทำ $A+(-B)$ เราก็แค่จ่าย `Operation = 2`, `Binvert = 1` และ `CarryIn = 1`: $A+(\bar{B}+1) \equiv A+(-B)$ เพราะว่า $(\bar{B}+1)$ คือการทำ two's complement ของ B.

ส่วน `Operation = 3` สำหรับการทำคำสั่ง `slt` (set on less than) ซึ่งเราจะดูในหัวข้อ 3.5. อุปกรณ์ `Overflow detection` สามารถสร้างได้ตามตรรกะที่แสดงในตาราง 3.1 ซึ่งจะให้เอาต์พุตเป็นสัญญาณระบุ การเกิด overflow ขึ้น. เราสามารถนำสัญญาณนี้ได้ใช้ เพื่อใช้โปรแกรมหรือ ระบบปฏิบัติการสามารถจัดการกับ overflow ได้อย่างเหมาะสมต่อไป ดูหัวข้อ 4.2.1 สำหรับกลไกและฮาร์ดแวร์สำหรับการจัดการกับ exception รวมถึง overflow exception.

3.5 set on less than

เราสามารถ implement คำสั่ง set on less than (`slt rd, rs, rt`) ได้ง่าย โดย (1) ทำการลบ `rs - rt`, (2) ถ้า `rs < rt`, ผลจะเป็นเลขลบ หรือ MSB ของผลลัพธ์ จะเป็น `1', (3) ให้ `rd` เท่ากับ MSB ของผลลบ ซึ่งเราทำได้โดยต่อวงจรดังรูป 3.6.

เมื่อเราต้องการทำ `slt` เราก็จะทำเหมือนลบ: `Operand A = rs`, `Operand B = rt`, `Binvert = 1` และ `CarryIn = 1`, ผลลัพธ์จาก full adders ในรูป 3.5 จะคือ ผลลบระหว่าง `rs` กับ `rt`. แต่จริงๆแล้วเราไม่ได้ต้องการรู้ว่าผลลบเป็นเท่าไร เราแค่ต้องการรู้ว่า ถ้า `rs < rt`, ผลจะเป็นเลขลบ หรือ MSB ของผลลัพธ์ จะเป็น `1'. ไม่อย่างนั้น MSB ของลัพธ์จะเป็น `0'. และ MSB ของผลลบสามารถดูได้จาก `ALU31` ขา `Set` (ดูภาพล่าง รูป 3.5).

และคำตอบที่อยากเห็นคือ

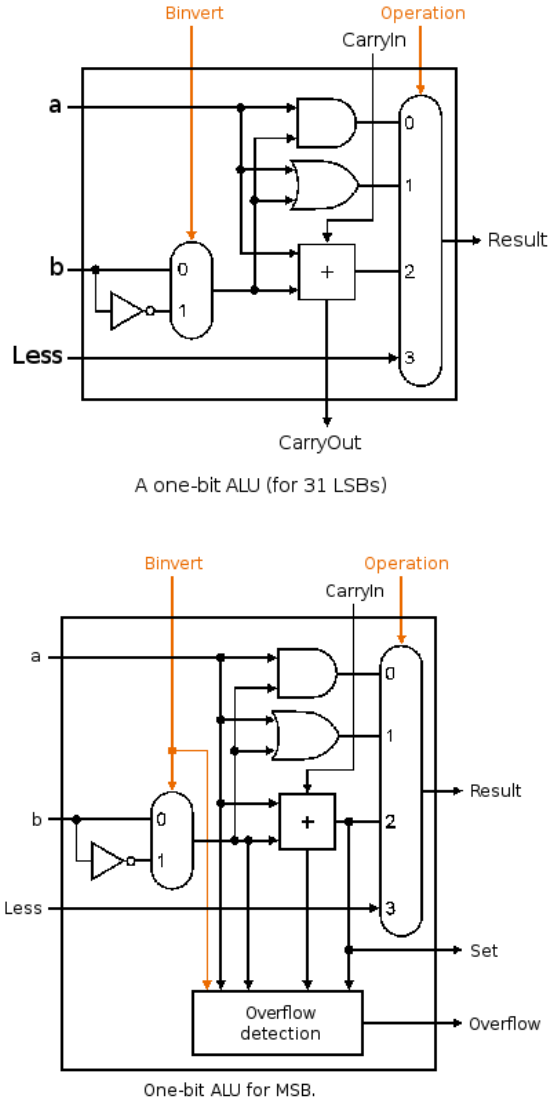


Figure 3.5: One-bit ALUs for 31 LSBs (upper diagram) and for MSB (lower diagram).

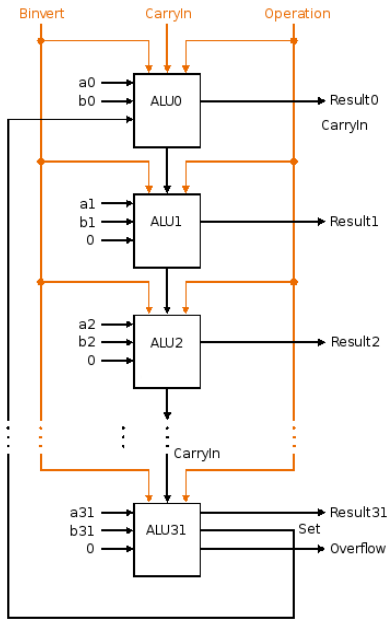


Figure 3.6: 32-bit ALU ที่ประกอบจาก one-bit ALUs.

- ถ้า `Set = 1` ให้เห็นคำตอบเป็น `0000 ... 0001`.
- ถ้า `Set = 0` ให้เห็นคำตอบเป็น `0000 ... 0000`.

สังเกตุว่า คำตอบที่อยากเห็น บิต 31-1 จะเป็น `0' หมด แต่บิต 0 จะขึ้นกับ `Set` เราถึงต่อสายสัญญาณจาก `Set` ของ MSB ไปเป็น บิต 0 ของ LSB ของคำตอบที่จะแสดง: ขา Less ของ ALU0 รับสัญญาณจากขา Set ของ ALU31. ส่วนขา Less ของบิตอื่นๆ (ALU1 ถึง ALU30) จะถูกผูกไว้กับค่า `0'. ดังนั้น เราจะเลือก `Operation = 3` เพื่อให้คำตอบออกมาเป็นแบบที่เราต้องการ[1].

Overflow สามารถป่วนเรื่องได้. รูป 3.6 และคำอธิบายข้างต้น จะสามารถทำ `slt`ได้ถูกต้อง ยกเว้นกรณีที่เกิด overflow ขึ้น. เพราะหากเกิด overflow ขึ้น เครื่องหมายที่ผลการลบ จะไม่ถูกต้อง.

ถ้าเราทำการลบค่าของ `rs` กับ `rt` โอกาสที่จะเกิด overflow ได้มีแค่ `rs` เป็นลบ `rt` เป็นบวก หรือ `rs` เป็นบวก `rt` เป็นลบ เพราะ ถ้าเครื่องหมายเดียวกัน พอไปลบกัน ค่าก็จะหักล้างกัน ไม่มีทางที่ ค่าจะล้นออกมาทำให้เกิด overflow ได้. ตาราง 3.4 วิเคราะห์ผลการเปรียบเทียบ เมื่อพิจารณา overflow ประกอบ.

เมื่อ ไม่เกิด overflow ขึ้น ถ้า `rs < rt` ผลที่ได้จะมี `MSB = 1` แต่ถ้า `rs > rt` ผลที่ได้จะมี `MSB = 0`, ดังแสดงในแถวที่ 1-2 ของตาราง. ตัวอย่างของ 4-bit words เช่น

```
rs = 5 [0101] < rt = 7 [0111],
   rs - rt = -2 [1110]; msb = 1.

rs = 5 [0101] > rt = 3 [0011],
   rs - rt = 2 [0010]; msb = 0.

rs = -5 [1011] < rt = 2 [0010],
```

[1] หมายเหตุ วงจร full adder และ logic gates ต่างๆ ทำงานตลอดเวลา ไม่ว่าเราจะเลือก MUX ให้แสดงค่าอะไรออกมา.

`rs - rt = -7 [1001]; msb = 1.`

`rs = 5 [0101] > rt = -2 [1110],`
`rs - rt = 7 [0111]; msb = 0.`

แต่เมื่อ เกิด overflow, ค่า MSB จากผลลบ กับ ผลการเปรียบเทียบจะไม่เหมือนเดิม เช่น

`rs = -5 [1011] < rt = 4 [0100],`
`rs - rt = [0111]; OVERFLOW! and msb = 0.`

`rs = 5 [0101] > rt = -4 [1100],`
`rs - rt = [1001]; OVERFLOW! and msb = 1.`

พูดง่ายๆ หากเกิด overflow ขึ้น ถ้า `MSB = 0` หมายถึง `rs < rt` (ซึ่งกลับกับตอนที่ ไม่มี overflow). และ ถ้า `MSB = 1` หมายถึง `rs > rt`. ตาราง 3.4 สรุปความสัมพันธ์นี้.

Table 3.4: การเปรียบเทียบค่าสำหรับ `slt` โดยใช้การลบ และ พิจารณา overflow ประกอบ

Overflow	รีจิสเตอร์	สิ่งที่เราเห็น		สิ่งที่เรารู้	สิ่งที่เราต้องการ
		ผลลบ	MSB ของผลลบ		
0	`rs > rt`	`result > 0`	0	`rs > rt`	`rd = 0`
0	`rs < rt`	`result < 0`	1	`rs < rt`	`rd = 1`
1	`rs > 0, rt < 0`	`result < 0`	1	`rs > rt`	`rd = 0`
1	`rs < 0, rt > 0`	`result > 0`	0	`rs < rt`	`rd = 1`

เพื่อสร้างวงจรสำหรับ `slt` เราแค่ map จาก สิ่งที่เราเห็น ไป สู่สิ่งที่เราต้องการ, ตาราง 3.5 และ truth table ที่เราต้องการจะนำไปสร้างวงจร. วิเคราะห์ตรรกะดังตาราง 3.5, เราจะได้

$$LESS_{LSB} = OVF \textbf{ xor } Result_{MSB}. \tag{3.16}$$

ซึ่งเราสามารถนำไปสร้าง ALU ได้ดังแสดงในรูป 3.7.

Table 3.5: ตารางตรรกะสำหรับ `slt`

Overflow (OVF)	MSB of the subtraction result ($SubtractionResult_{MSB}$)	สัญญาณ Less สำหรับ LSB ของ rd ($Less_{LSB}$)
0	0	0
0	1	1
1	0	1
1	1	0

The test on equality. เราใช้ ALU ช่วยในการทำการคำนวณหลายๆอย่าง รวมถึงเปรียบเทียบ เท่ากับ อยู่บ่อยๆ เช่น คำสั่ง `beq rs, rt, offset` ที่เราต้องเปรียบเทียบค่าของรีจิสเตอร์ `rs` กับ ค่าของ `rt`. เราทำได้โดย ทำการลบ: `rs - rt` ถ้าผลลบเป็น $0000\cdots0000$ (ศูนย์ทุกบิต) แปลว่า `rs = rt` ถ้าไม่อย่างนั้นแปลว่า ไม่เท่ากัน. เพื่อความสะดวก เราจะเพิ่มวงจรเพื่อตรวจว่าคำตอบ เป็นศูนย์ทุกบิต ได้ด้วยวงจรง่ายๆ ดังแสดง ในรูป 3.8.

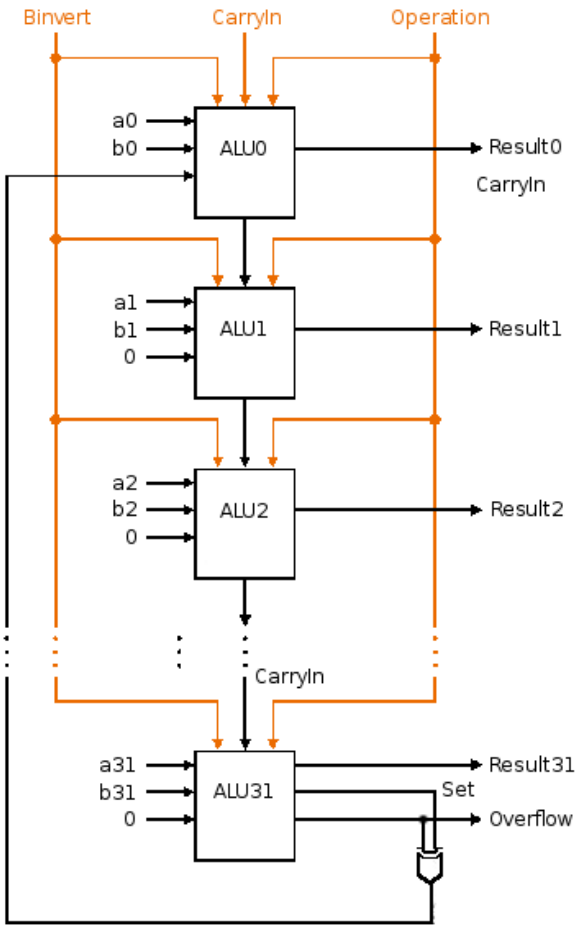


Figure 3.7: A word ALU implementing `slt` with accounting on overflow.

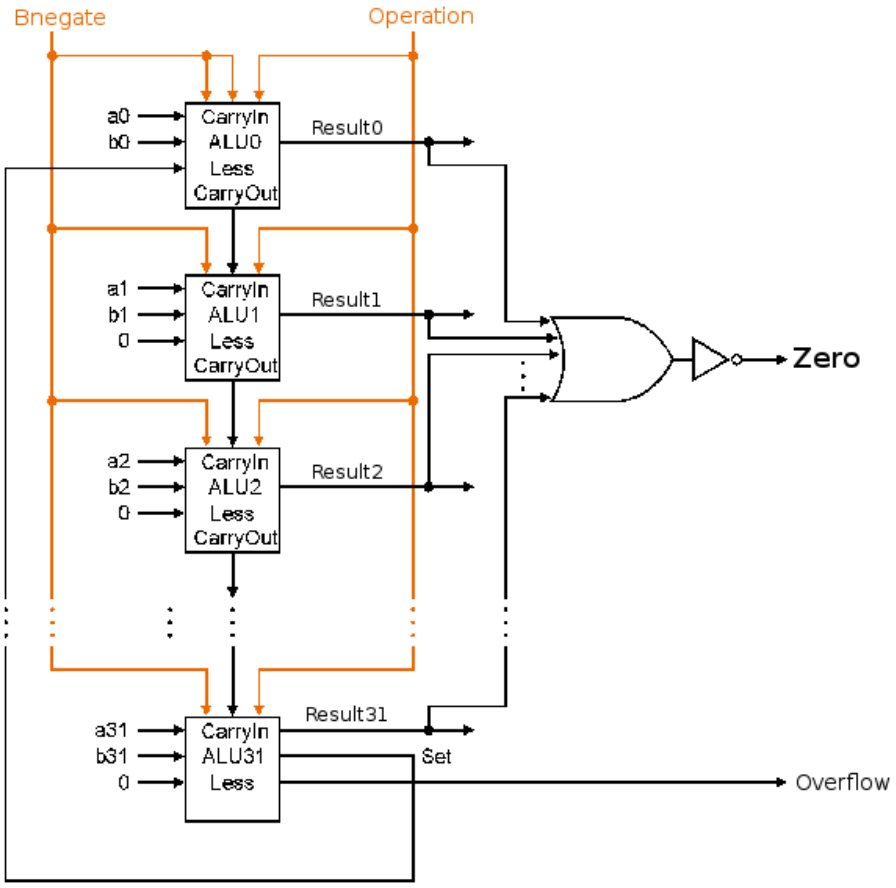


Figure 3.8: วงจรเพิ่มเพื่อตรวจว่าผลลัพธ์เป็นศูนย์ (Zero detection).

```
Multiplicand:     1000
Multiplier:       1001   x
                  -------
                  1000
                 0000
                0000
               1000
Product        01001000
```

Figure 3.9: การคูณเลขฐานสองด้วยมือ.

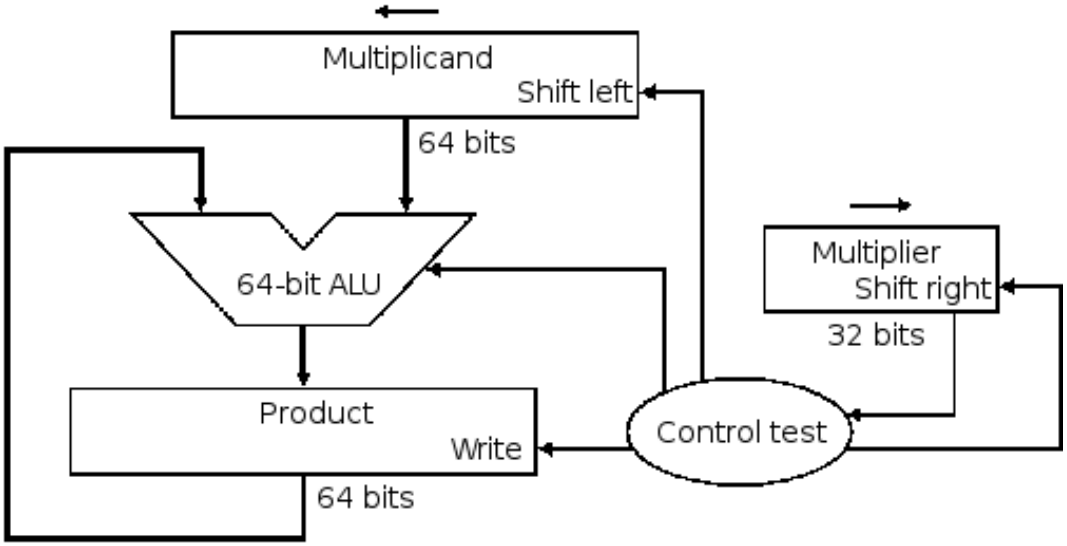


Figure 3.10: ฮาร์ดแวร์สำหรับการคูณเลขฐานสอง.

3.6 Multiplication

รูป 3.9 แสดงการคูณ เลขสี่บิต สองตัว ด้วยมือ. เราสามารถสร้างฮาร์ดแวร์เพื่อทำการคูณเพื่อเลียนแบบการคูณด้วยมือนี้ดังแสดงในรูป 3.10. ผลคูณของเลข N บิต สองตัว: ตัวตั้ง (multiplicand) กับ ตัวคูณ (multiplier) ผลลัพธ์จะต้องการ $2N$ บิต. การคูณเลขฐานสองแบบง่ายๆก็คือ ถ้าคูณ multiplicand ด้วยบิตที่เป็น `1' เราจะคัดลอกค่า multiplicand มาเลย. แต่ถ้า multiplicand ด้วยบิตที่เป็น `0', เราจะคัดลอกค่า 0 มา. รูป 3.11 แสดง flow chart ของการคูณด้วยฮาร์ดแวร์ที่แสดงในรูป 3.10.

ตาราง reftbl: multiplication 1 แสดงตัวอย่างการคูณ 2×3 ด้วยอัลกอริทึ่ม (รูป 3.11). สังเกตุ อัลกอริทึ่มนี้: (1) จะมี 3 ขั้นตอนย่อย สำหรับ แต่ละบิตของตัวคูณ ดังนั้น ถ้าคูณเลข 32 บิต ก็ต้องทำ $32 \times 3 = 96$ ขั้นตอนย่อย. ถ้า แต่ละขั้นตอนย่อยต้องใช้ 1 clock cycle, เราต้องการถึง 96 clock cycles เพื่อจะทำ 32-bit multiplication. (2) ครึ่งหนึ่งของตัวตั้ง จะเป็น 0 เสมอ: ตัวอย่างในตาราง 3.6 คอลัมน์ Multiplicand บิตที่ไม่ได้ไฮไลต์จะเป็น 0 เสมอ. (3) `0' ที่เติมเข้าไปทางขวา ตอนที่ทำ `Shift left Mcand` แปลว่า LSBs ของ Product จะไม่เปลี่ยนหลังจากค่าทำเสร็จแล้ว จากข้อสังเกตุนี้ แนวคิด คือ แทนที่จะทำ shift left กับ Multiplicand เราจะ shift

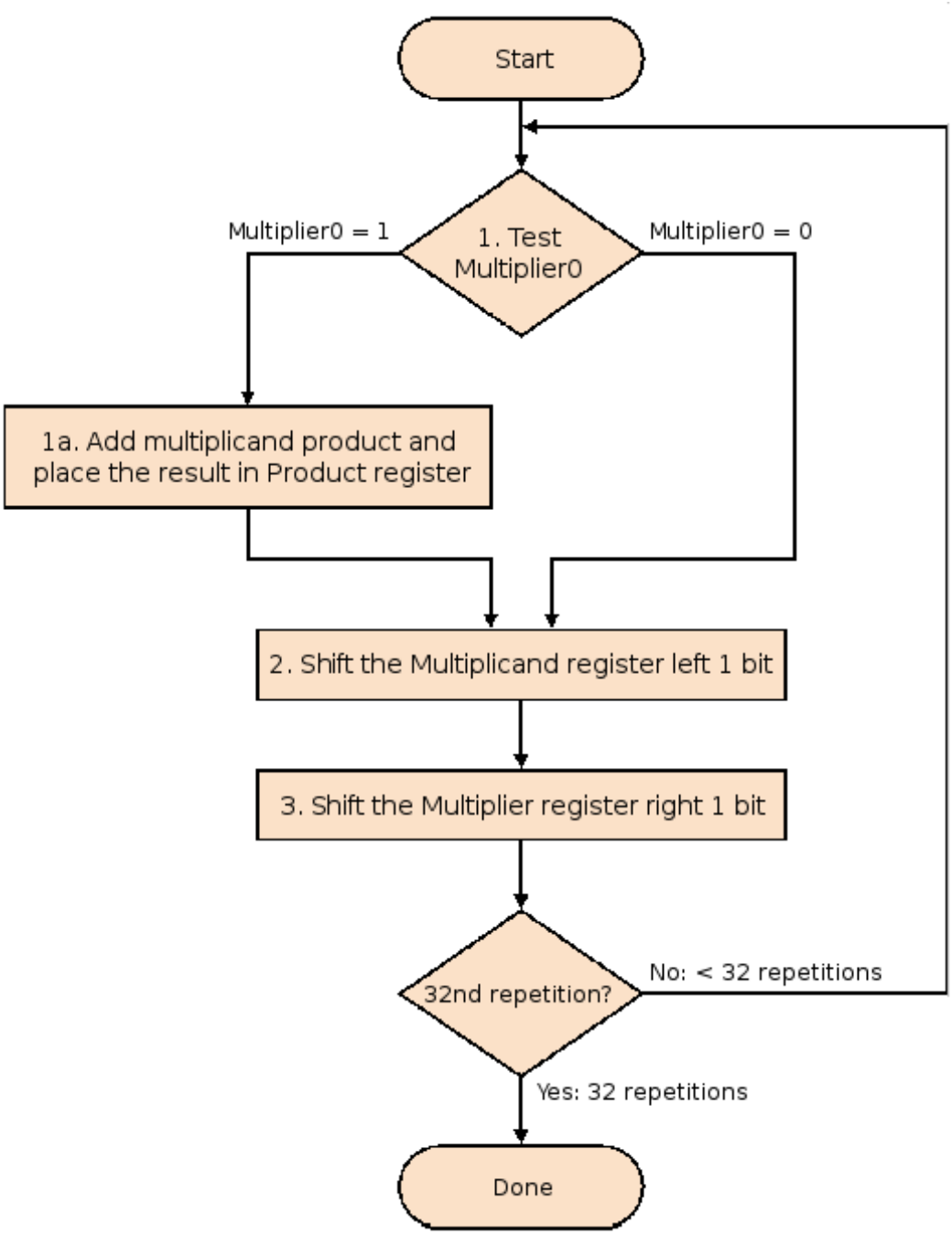


Figure 3.11: Flow chart สำหรับการคูณเลขฐานสองด้วยฮาร์ดแวร์ดังแสดงในรูป 3.10.

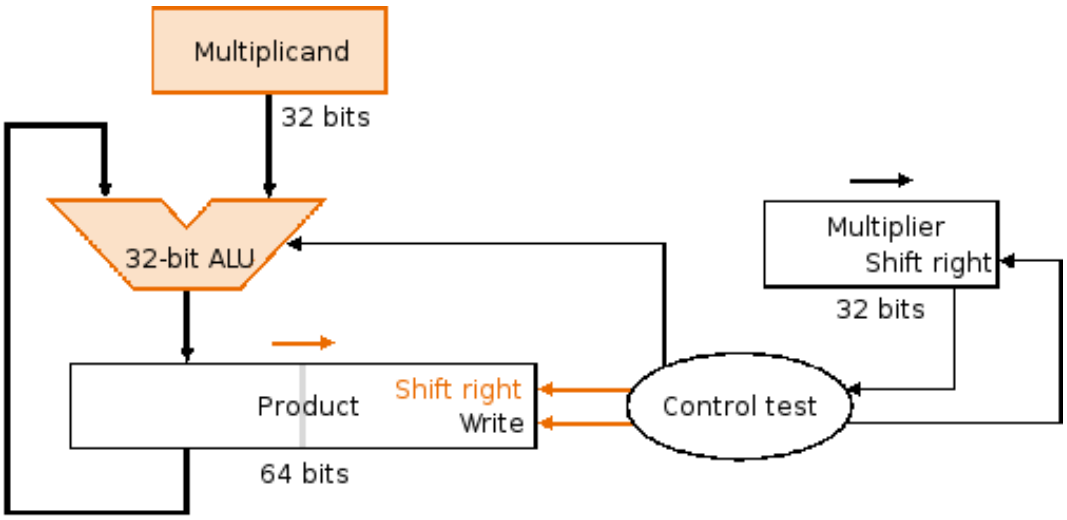


Figure 3.12: Multiplication Hardware Version 2.

right กับ Product แทน. ซึ่งนี่ คือ การคูณ ด้วยฮาร์ดแวร์ที่แสดงในรูป 3.12 และ อัลกอริทึ่มในรูป 3.13.

Table 3.6: ตัวอย่างการคูณเลขสี่บิต: 2×3 หรือ `0b0010` $\times$ `0b0011`

Iteration	Step	Multiplier	Multiplicand	Product
0	Initialization	**0011**	00000**0010**	00000000
1	`1a: Prod <= Prod + Mcand`	**0011**	00000**0010**	**00000010**
	`2: Shift left Mcand`	**0011**	0000**0100**	00000010
	`3: Shift right Mplier`	**0001**	0000**0100**	00000010
2	`1a: Prod <= Prod + Mcand`	**0001**	0000**0100**	**00000110**
	`2: Shift left Mcand`	**0001**	000**01000**	00000110
	`3: Shift right Mplier`	**0000**	000**01000**	00000110
3	`1: No operation`	**0000**	000**01000**	00000110
	`2: Shift left Mcand`	**0000**	**00010000**	00000110
	`3: Shift right Mplier`	**0000**	**00010000**	00000110
4	`1: No operation`	**0000**	**00010000**	00000110
	`2: Shift left Mcand`	**0000**	**00100000**	00000110
	`3: Shift right Mplier`	**0000**	**00100000**	00000110

จากการคูณแบบที่ 2 (รูป 3.12 และ 3.13), ตาราง 3.14 แสดงตัวอย่าง. สังเกตุว่า บิตที่ไม่ได้ใช้ของ Multiplier และ Product. รูป 3.15 แสดงบิตที่ไม่ได้ใช้ของทั้ง Multiplier และ Product.

รูป 3.16 และ 3.17 แสดงการคูณแบบที่ 3 ซึ่งลดอุปกรณ์ลงจากแบบที่ 2 และ ก็ลดขั้นตอนลงด้วยดังแสดงในรูป 3.18.

Sign muliplication. วิธีที่ง่ายที่สุดในการทำ การคูณแบบที่มีเครื่องหมายด้วย คือ การทำให้ทั้ง multiplier และ multiplicand เป็นเลขบวกทั้งคู่ และ คูณกันปกติ แล้วจัดการกับคำตอบให้เป็นไปตามเครื่องหมายที่ถูกต้อง.

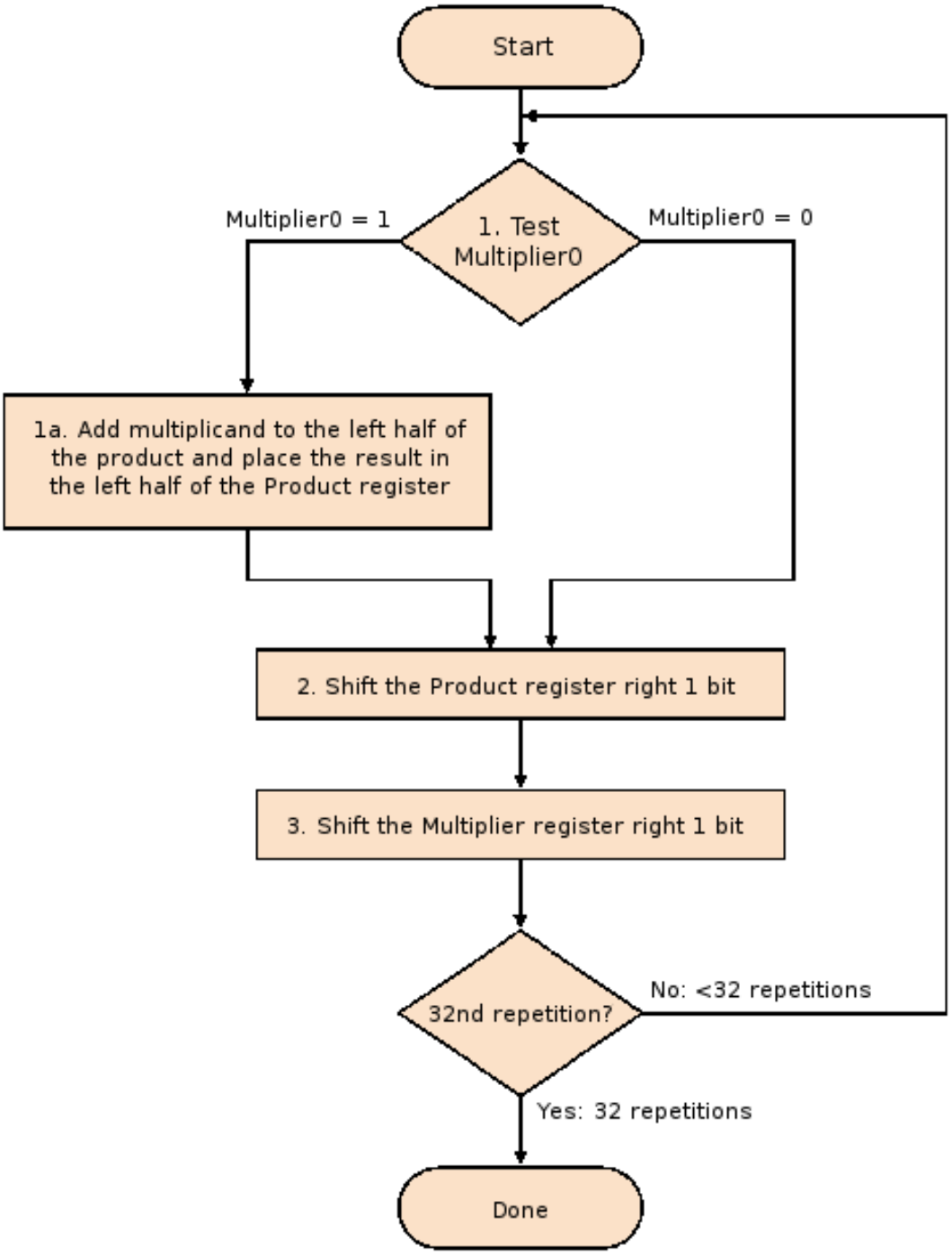


Figure 3.13: Multiplication Flow Chart Version 2.

Iteration	Step	Multiplier	Multiplicand	Product
0	Initialization	**0011**	0010	0000 0000
1	1a: 1 => Prod = Prod + Mcand	**0011**	0010	**0010** 0000
	2: Shift right Product	**0011**	0010	**0001 0**000
	3: Shift right Mplier	0**001**	0010	**0001 0**000
2	1a: 1 => Prod = Prod + Mcand	0**001**	0010	**0011 0**000
	2: Shift right Product	0**001**	0010	**0001 10**00
	3: Shift right Mplier	00**00**	0010	**0001 10**00
3	1: 0 => no operation	00**00**	0010	**0001 10**00
	2: Shift right Product	00**00**	0010	**0000 110**0
	3: Shift right Mplier	000**0**	0010	**0000 110**0
4	1: 0 => no operation	000**0**	0010	**0000 110**0
	2: Shift right Product	000**0**	0010	**0000 0110**
	3: Shift right Muliplier	0000	0010	**0000 0110**

Figure 3.14: ตัวอย่าง 2×3 ด้วยอัลกอริทึ่มคูณแบบที่ 2.

Iteration	Step	Multiplier	Multiplicand	Product
0	Initialization	0011	0010	0000 dddd
1	1a: 1 => Prod = Prod + Mcand	0011	0010	0010 dddd
	2: Shift right Product	0011	0010	0001 0ddd
	3: Shift right Mplier	d001	0010	0001 0ddd
2	1a: 1 => Prod = Prod + Mcand	d001	0010	0011 0ddd
	2: Shift right Product	d001	0010	0001 10dd
	3: Shift right Mplier	dd00	0010	0001 10dd
3	1: 0 => no operation	dd00	0010	0001 10dd
	2: Shift right Product	dd00	0010	0000 110d
	3: Shift right Mplier	ddd0	0010	0000 110d
4	1: 0 => no operation	ddd0	0010	0000 110d
	2: Shift right Product	ddd0	0010	0000 0110
	3: Shift right Muliplier	dddd	0010	0000 0110
d represents a bit whose value will not be used at all.				

Figure 3.15: ตัวอย่าง 2×3 แสดงบิตที่ไม่ได้ใช้ของ Multiplier และ Product.

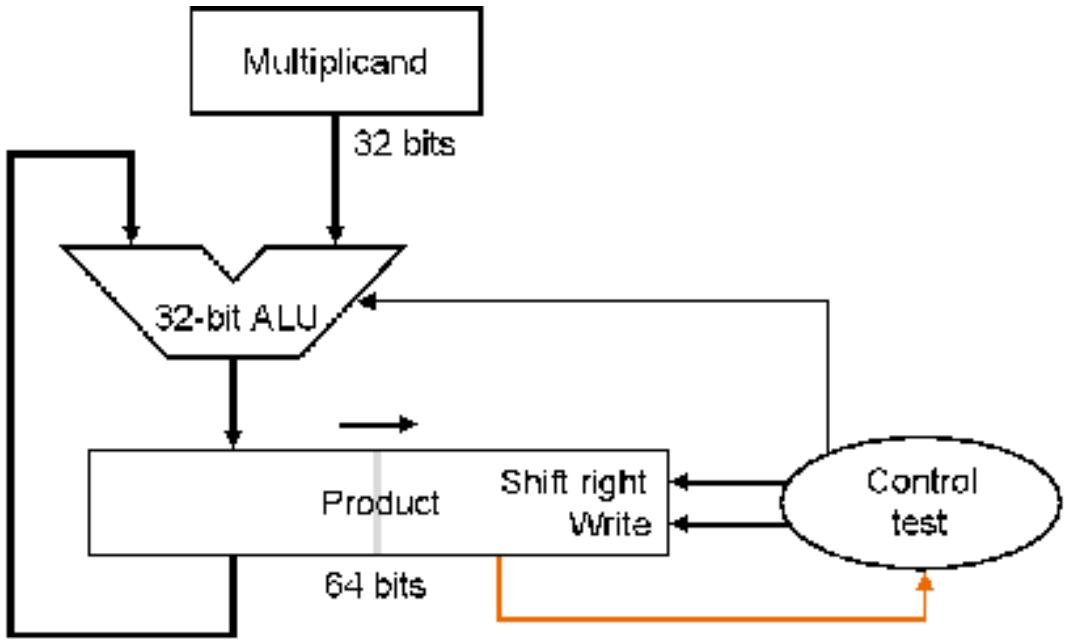


Figure 3.16: ฮาร์ดแวร์การคูณแบบที่ 3.

3.7 Division

ตัวอย่างการหาร $1,001,010 \div 1,000$ ด้วยมือ แสดงดังรูป 3.19. เราสามารถเขียนได้ว่า

$$\text{Dividend} = (\text{Quotient} \times \text{Divisor}) + \text{Remainder}. \tag{3.17}$$

3.8 Floating Point

``Speed gets you nowhere if you're headed the wrong way." - American proverb.

นอกจากเลขจำนวนเต็มแล้ว programming languages สนับสนุน เลขเศษส่วนด้วย ที่เราเรียกว่า จำนวนจริง (reals) ในคณิตศาสตร์.

- เลขเศษส่วน, เช่น $\pi = 3.1416..., e = 2.7183...$
- เลขน้อยๆ, เช่น 0.00000000023 และ 8.14×10^{-12}.
- เลขใหญ่มากๆ, เช่น $-3.15587 \times 10^{4}6$.

การแสดงเลขแบบ Floating-point (สำหรับเลขที่ไม่ใช่ 0) จะใช้ บิตที่ไม่เป็นศูนย์บิตแรก อยู่ซ้ายของ binary point ในรูปแบบของ $1.xxxxxxxx \times 2^{yyyy}$, โดยที่ $1.xxxxxxxx$ และ $yyyy$ เป็นเลขฐาน 2. การใช้รูปแบบมาตราฐานนี้ มีประโยชน์ 3 อย่างคือ: (1) ทำให้การแลกเปลี่ยนข้อมูลทำได้ง่าย (2) ทำให้อัลกอริที่มที่คำนวณกับ floating point ทำได้ง่ายขึ้น เพราะเลขจะอยู่ในรูปแบบนี้ตลอด (3) ช่วยเพิ่ม accuracy ของตัวเลขที่แสดงใน word เพราะว่า 0 ที่อยู่ข้างหน้าที่ไม่จำเป็น จะไม่ถูกแสดง และใช้บิตที่มีไปแสดงตัวเลขที่มีผลอื่นๆ.

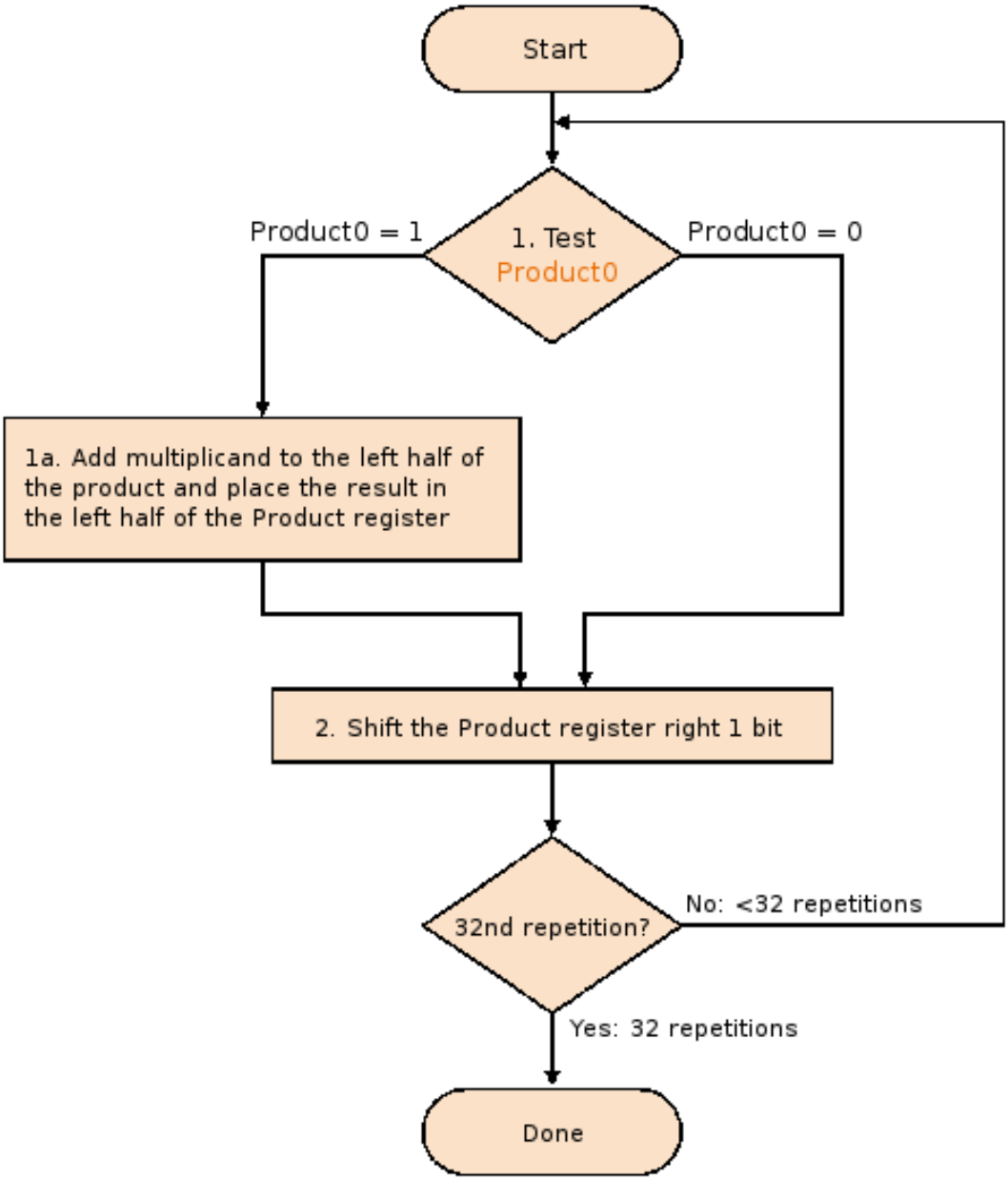


Figure 3.17: อัลกอริทึ่มการคูณแบบที่ 3.

Iteration	Step	Multiplicand	Product
0	Initialization	0010	0000 **0011**
1	1a: 1 => Prod = Prod + Mcand	0010	0010 **0011**
	2: Shift right Product	0010	0001 **0001**
2	1a: 1 => Prod = Prod + Mcand	0010	0011 **0001**
	2: Shift right Product	0010	0001 **1000**
3	1: 0 => no operation	0010	0001 **1000**
	2: Shift right Product	0010	0000 **1100**
4	1: 0 => no operation	0010	0000 **1100**
	2: Shift right Product	0010	0000 0110

Figure 3.18: ตัวอย่างการคูณแบบที่ 3.

$$
\begin{array}{rll}
 & 1001_{ten} & \text{Quotient} \\
\text{Divisor } 1000_{ten} & \overline{)\,1001010_{ten}} & \text{Dividend} \\
 & -1000 & \\
 & \overline{10} & \\
 & 101 & \\
 & 1010 & \\
 & -1000 & \\
 & \overline{10_{ten}} & \text{Remainder}
\end{array}
$$

Figure 3.19: ตัวอย่างการหารด้วยมือ.

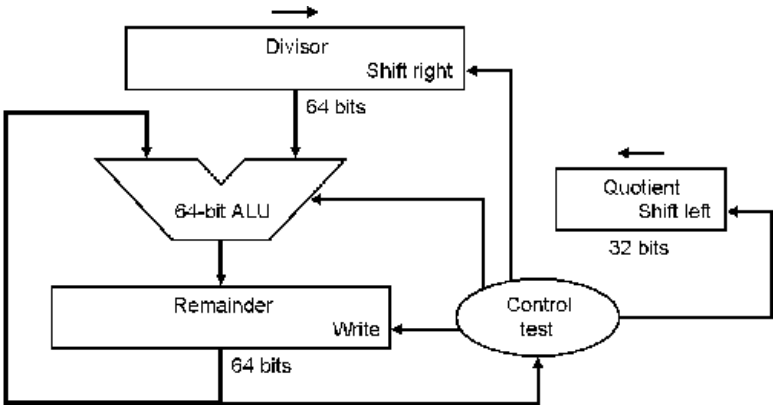


Figure 3.20: ฮาร์ดแวร์ที่ใช้ทำการหาร.

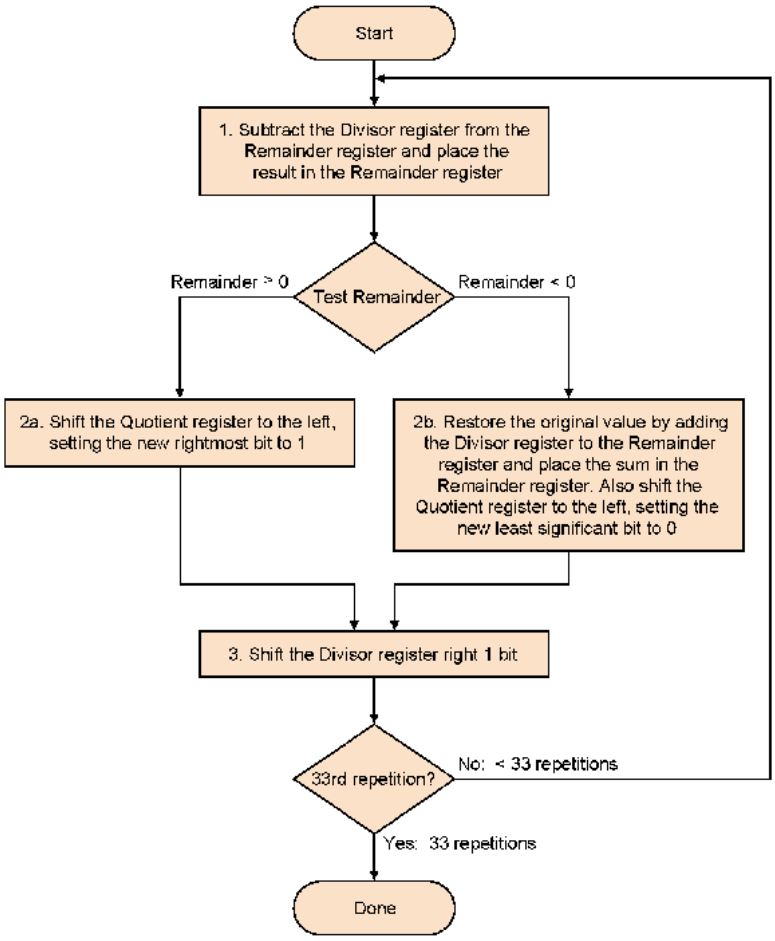


Figure 3.21: อัลกอริทึ่มทำหาร (ดูฮาร์ดแวร์ รูป 3.20 ประกอบ).

Iteration	Step	Quotient	Divisor	Remainder
0	Initial values	0000	0010 0000	0000 0111
1	1: Rem = Rem – Div	0000	0010 0000	①110 0111
	2b: Rem < 0 ⇒ +Div, sll Q, Q0 = 0	0000	0010 0000	0000 0111
	3: Shift Div right	0000	0001 0000	0000 0111
2	1: Rem = Rem – Div	0000	0001 0000	①111 0111
	2b: Rem < 0 ⇒ +Div, sll Q, Q0 = 0	0000	0001 0000	0000 0111
	3: Shift Div right	0000	0000 1000	0000 0111
3	1: Rem = Rem – Div	0000	0000 1000	①111 1111
	2b: Rem < 0 ⇒ +Div, sll Q, Q0 = 0	0000	0000 1000	0000 0111
	3: Shift Div right	0000	0000 0100	0000 0111
4	1: Rem = Rem – Div	0000	0000 0100	⓪000 0011
	2a: Rem ≥ 0 ⇒ sll Q, Q0 = 1	0001	0000 0100	0000 0011
	3: Shift Div right	0001	0000 0010	0000 0011
5	1: Rem = Rem – Div	0001	0000 0010	⓪000 0001
	2a: Rem ≥ 0 ⇒ sll Q, Q0 = 1	0011	0000 0010	0000 0001
	3: Shift Div right	0011	0000 0001	0000 0001

FIGURE 3.12 Division example using the algorithm in Figure 3.11. The bit examined to determine the next step is circled in color.

Figure 3.22: ตัวอย่างการหารด้วยอัลกอริทึ่มในรูป 3.21.

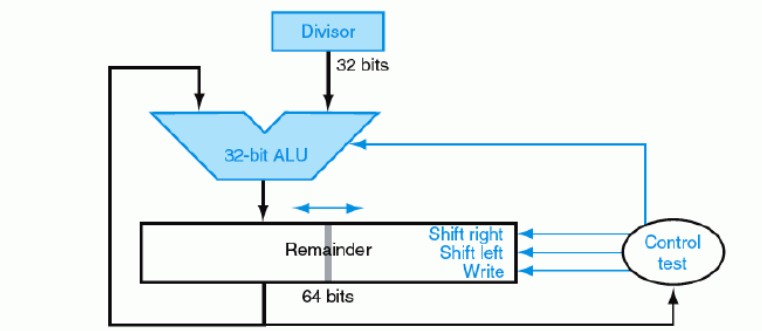


FIGURE 3.13 An improved version of the division hardware. The Divisor register, ALU, and Quotient register are all 32 bits wide, with only the Remainder register left at 64 bits. Compared to Figure 3.10, the ALU and Divisor registers are halved and the remainder is shifted left. This version also combines the Quotient register with the right half of the Remainder register.

Figure 3.23: วงจรหารปรับปรุงจากรูป 3.20.

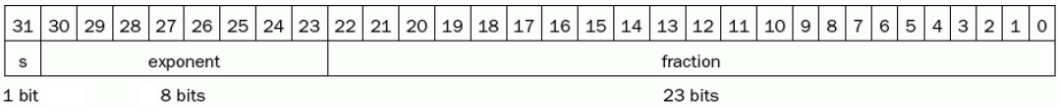


Figure 3.24: รูปแบบการจัดแบ่งบิตใน 32-bit word สำหรับ floating point.

Floating-Point Representation. นักออกแบบคอมพิวเตอร์ พยายามหาจุดสมดุล ระหว่างจำนวนบิตของ significand และ จำนวนบิตที่จะใช้เป็น exponent, ของ floating-point number ในรูปแบบ

$$(-1)^s \cdot \text{significand} \cdot 2^{\text{exponent}} \tag{3.18}$$

โดยที่ s เป็น sign bit. เพราะว่า จำนวนบิตของ word จำกัด, เช่น 32 บิต สำหรับ MIPS-32, ถ้าเราใช้บิตสำหรับ significand มากขึ้น จำนวนบิตของ exponent ก็ต้องน้อยลง และก็ในทางกลับกันด้วย. ดังนั้น เราต้อง trade off ระหว่าง accuracy (จำนวนบิต significand มากๆ) กับ range (จำนวนบิต exponent มากๆ). สำหรับ 32-bit word, floating point แทนด้วยบิตดังรูป ??. หมายเหตุ overflow สำหรับ floating point หมายถึง ค่าที่ต้องการมีส่วนของ exponent ใหญ่กว่า exponent field จะแสดงได้. เช่น จากตัวอย่าง exponent field ขนาด 8 บิต, 8-bit exponent แสดงในระบบ two's complement สามารถแสดงเลข $\{-128,\ldots,127\}$. ดังนั้น ค่า $2^{\text{exponent}} \in \{2^{-128},\ldots,2^{127}\}$. ถ้า นำ $x = 1.01_{two} \times 2^{127}$ (หรือ $1.25 \times 1.701412e \times 10^{38} = 2.126765 \times 10^{38}$) ไปคูณกับ $y = 1.0_{two} \times 2^{1}$ (หรือ 2). คำตอบที่ถูกต้อง คือ $x \cdot y = 4.25353 \times 10^{38}$ ซึ่งคือ $1.01_{two} \times 2^{128}$ ในรูปแบบ floating point. แต่ exponent filed แสดงค่าได้สูงสุดแค่ 127 ไม่สามารถแสดง 128 ได้. กรณีนี้จะเกิด overflow ขึ้น. สังเกตุกรณีนี้จะต่างจาก $x + y = 2.126765 \times 10^{38} + 2$ ซึ่งจะถูก round off เป็น 2.126765×10^{38} ที่ไม่เกิด overflow.

นอกจาก overflow แล้ว, หากมีเลขจำนวนน้อยๆ น้อยมากๆเกินกว่าที่ floating point จะแสดงได้เช่น 32-bit floating point, $x = 1.0_{two} \times 2^{-128}$ (หรือ 2.938736×10^{-39}) และ $y = 1.0_{two} \times 2^{1}$, ดังนั้น $x \div y = 1.0_{two} \times 2^{-129}$ (หรือ 1.469368×10^{-39}) แต่ exponent filed ไม่สามารถแสดง -129 ได้. เราจะเรียกกรณีนี้ว่า underflow. ไม่ว่ากรณี ผลลัพธ์จากการคำนวณไม่สามารถแสดงได้ด้วย exponent field (เกิด overflow หรือ underflow), ผลที่แสดงจะไม่ถูกต้อง.

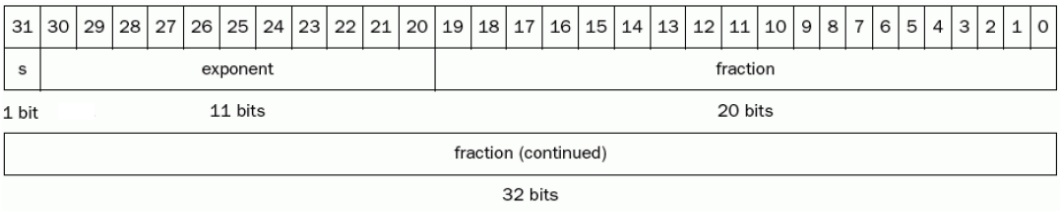


Figure 3.25: รูปแบบการจัดแบ่งบิต double precision.

Double Precision. เราสามารถที่จะเลือกใช้ 2 words มาประกอบกันเพื่อแสดงเลข เพื่อเพิ่มทั้ง accuracy และ range ที่สามารถแสดงได้. รูป 3.25 แสดงการจัดแบ่งบิตสำหรับ double precision ที่ใช้ สอง 32-bit words.

IEEE 754 standard floating-point presentation. ตาราง 3.7 แสดงการรูปแบบการแสดง floating-point ตาม IEEE 754. สังเกตุการใช้ exponent field ในการเลือกโหมด. ค่า exponent filed ที่ใช้ 8 บิต สามารถแทนค่า (แบบ unsigned integer) ได้ตั้งแต่ `0x00` ถึง `0xFF` (หรือ $0, 1, 2, \ldots, 255$).

ค่า exponent เป็น `0x00`. โหมดแรก (แถวบนสุด) ค่าที่แสดงจะอยู่ในรูปแบบ $(-1)^s \times (0.\text{significand})_{two} \times 2^{-126}$ ซึ่งแสดงค่าได้ตั้งแต่ $0, 1.40129846 \times 10^{-45}, 2.80259692 \times 10^{-45}, \ldots,$ ไปจนถึง $1.17549421 \times 10^{-38}$ และเลขลบ $-0, -1.40129846 \times 10^{-45}, \ldots, -1.17549421 \times 10^{-38}$, ดูตัวอย่างในตาราง 3.8 แถว 1-5.

ค่า exponent เป็นค่าใดค่าหนึ่งใน `0x01` ไปจนถึง `0xFE`. โหมดที่สอง ค่าที่แสดงจะใช้รูปแบบ $(-1)^s \times (1.\text{significand})_{two} \times 2^{\text{Exponent}_{two}-127}$. เช่น -13.75 ในฐานสิบ จะเขียนได้เป็น

$$\begin{aligned}
-13.75 &= (-1)^{\mathbf{1}} \times (13 + 0.75) \\
&= (-1)^{\mathbf{1}} \times (13 + 1/2 + 1/4) \\
&= (-1)^{\mathbf{1}} \times (0b1101 + 0b0.11) \\
&= (-1)^{\mathbf{1}} \times (0b1101.11) \\
&= (-1)^{\mathbf{1}} \times (0b1.10111) \times 2^3, \text{ (normalization)} \\
&= (-1)^{\mathbf{1}} \times (0b1.10111) \times 2^{(130-127)} \\
&= (-1)^{\mathbf{1}} \times (0b1.\mathbf{10111}) \times 2^{(0b\mathbf{10000010}-127)}
\end{aligned}$$

ดังนั้นเมื่อเขียนเป็นรูปแบบ IEEE 754 เราจะได้ `[1, 100 0001 0, 101 1100 0000 0000 0000 0000] = 0xC15C0000`. แถวที่ 6-7 ของตาราง 3.8 แสดงตัวอย่าง IEEE 754 เมื่อ exponent filed อยู่ในโหมดสอง. ค่าในแถวที่ 6-7 แสดงค่าบวก ที่น้อยที่สุด และมากที่สุดที่โหมดนี้แสดงได้ (ค่าลบทำได้ในขนาดเท่ากัน เพียงแต่ให้ค่า $s = 1$). สังเกตุค่าบวกน้อยสุดที่โหมดสองทำได้จะมีค่ามากกว่า ค่ามากสุดที่โหมดหนึ่งทำได้นิดหน่อย (มากกว่าเท่ากับ $1 - 0.111\cdots11_{two} = 2^{-23}$).

ค่า exponent เป็น `0xFF`. โหมดสาม ถ้าค่า significand เป็น 0 ทุกบิต ค่าใน word จะแทน ∞ เมื่อ $s = 0$ หรือ $-\infty$ เมื่อ $s = 1$. แต่หากมีบิตใดอย่างน้อยหนึ่งบิตใน significand ที่ไม่เป็น 0,

ค่าใน word จะแทน `NaN` (Not A Number) ซึ่งเป็นการระบุจำนวนที่ผิดพลาด เช่น การทำ $0/0$ ในทางคณิตศาสตร์ เป็นต้น.

Table 3.7: Floating-point representation (single precision). `NaN` represents an undefined or unrepresentable value.

Exponent (8 bits)	Significand (23 bits)		Equation
	Significand zero	Significand non-zero	
`0x00`	0, -0	subnormal numbers	$(-1)^s \times (0.\text{significand})_{two} \times 2^{-126}$
`0x01`,...,`0xFE`	normalized value		$(-1)^s \times (1.\text{significand})_{two} \times 2^{\text{Exponent}_{two}-127}$
`0xFF`	∞, $-\infty$	`NaN`	

Table 3.8: ตัวอย่างค่าใน 32-bit words เมื่อแสดง floating point ตาม IEEE 754

s	exponent		significand						word in hex	representing
1	0000	0000	000	0000	0000	0000	0000	0000	`0x80000000`	-0
0	0000	0000	000	0000	0000	0000	0000	0000	`0x00000000`	0
0	0000	0000	000	0000	0000	0000	0000	0001	`0x00000001`	$0.00\cdots01_{two} \times 2^{-126}$ $= 2^{-149} = 1.40129846 \times 10^{-45}$
0	0000	0000	000	0000	0000	0000	0000	0010	`0x00000002`	$0.00\cdots10_{two} \times 2^{-126}$ $= 2.80259692 \times 10^{-45}$
0	0000	0000	111	1111	1111	1111	1111	1111	`0x007FFFFF`	$0.11\cdots11_{two} \times 2^{-126}$ $= 1.17549421 \times 10^{-38}$
0	0000	0001	000	0000	0000	0000	0000	0000	`0x00800000`	$1.00\cdots00_{two} \times 2^{(1-127)}$ $= 1.00\cdots00_{two} \times 2^{-126}$ $= 1.17549435 \times 10^{-38}$
0	1111	1110	111	1111	1111	1111	1111	1111	`0x7F7FFFFF`	$1.11\cdots11_{two} \times 2^{(254-127)}$ $= 1.11\cdots11_{two} \times 2^{127}$ $= 3.40282347 \times 10^{38}$
0	1111	1111	000	0000	0000	0000	0000	0000	`0x7F800000`	∞
1	1111	1111	000	0000	0000	0000	0000	0000	`0xFF800000`	$-\infty$
0	1111	1111	000	0000	0000	0000	0000	0001	`0x7F800001`	`NaN`
1	1111	1111	000	0000	0000	0000	0000	0001	`0xFF800001`	`NaN`
0	1111	1111	111	1111	1111	1111	1111	1111	`0x7FFFFFFF`	`NaN`

การคำนวณบวกลบ floating point สามารถทำได้ด้วยอัลกอริทึ่มและฮาร์ดแวร์ดังแสดงในรูป 3.26 และ 3.27 ตามลำดับ.

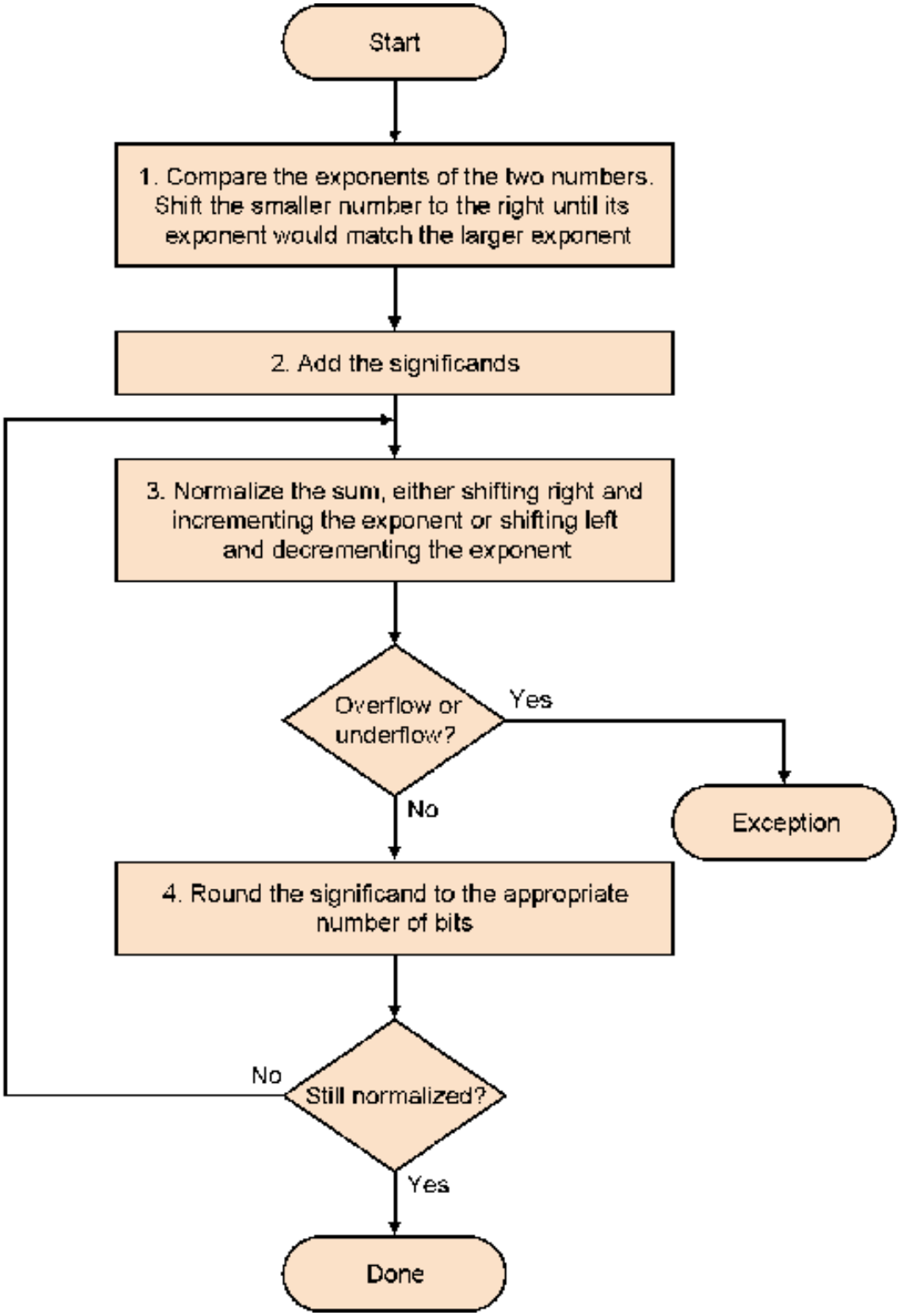


Figure 3.26: การบวกเลข floating point.

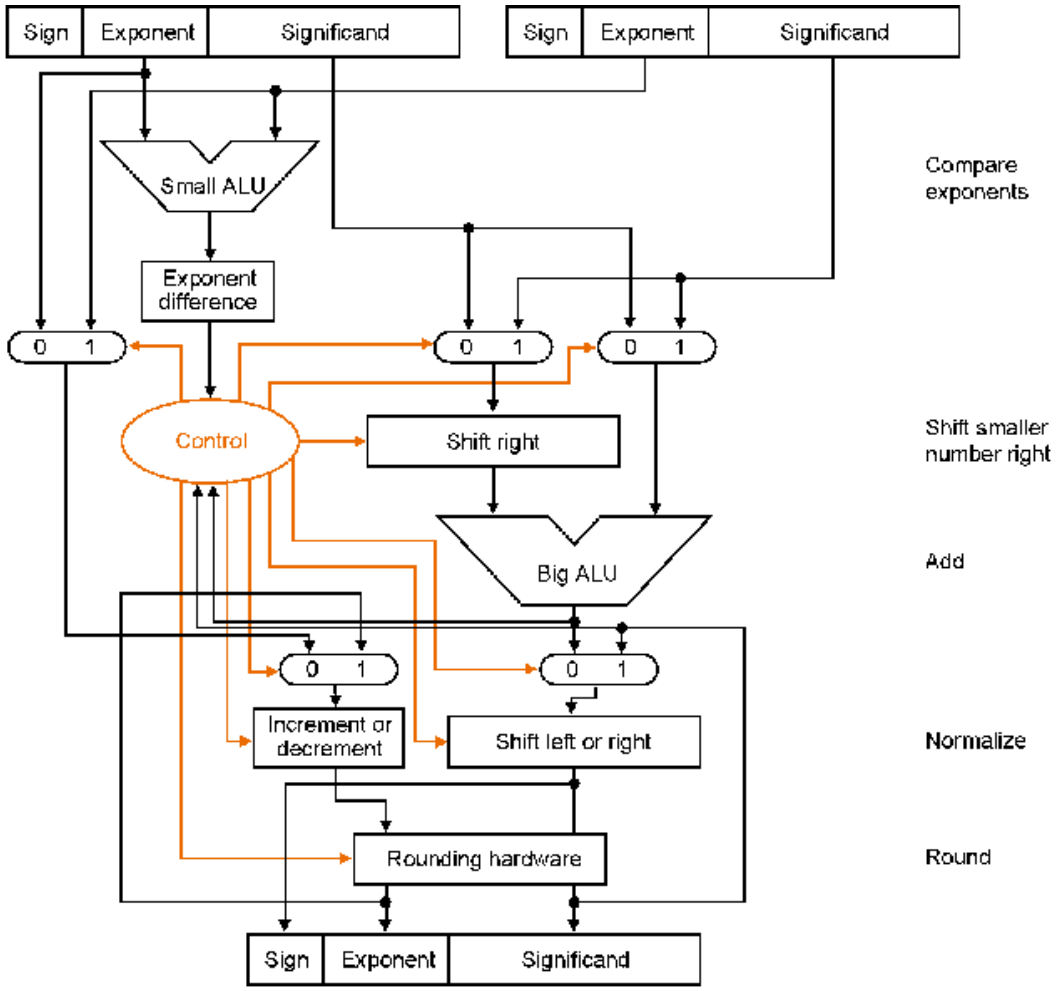


Figure 3.27: ฮาร์ดแวร์การบวกเลข floating point.

Hardware Software Interface. เรื่องหนึ่งที่นักออกแบบคอมพิวเตอร์ต้องตัดสินใจคือ การคำนวณ floating point ว่าจะใช้รีจิสเตอร์ชุดเดียวกับ ชุดที่ใช้กับ integers หรือ ว่าจะใช้ชุดที่ทำพิเศษขึ้นมาสำหรับ floating point โดยเฉพาะ. เพราะ ส่วนใหญ่โปรแกรม จะทำการคำนวณแบบ integer กับข้อมูลคนละชุดกับ การคำนวณแบบ floating point, ดังนั้น การแยกชุดรีจิสเตอร์ จะเพิ่มจำนวนคำสั่งที่ใช้ในโปรแกรมไม่มากเท่าไร. เรื่องที่มีผลมาก กลับเป็น การต้องเพิ่มคำสั่งสำหรับการถ่ายโอนย้ายข้อมูลระหว่าง floating point registers กับ หน่วยความจำ.

ประโยชน์อย่างหนึ่งในการแยกชุดรีจิสเตอร์มาสำหรับ floating point คือ เราจะสามารถมีรีจิสเตอร์เพิ่มได้เท่าตัว โดยไม่ต้องขยายฟิลด์ที่อ้างรีจิสเตอร์ (เช่น 5 บิต สามารถอ้างถึง 32 integer registers เมื่อใช้คำสั่งที่เป็น integer และ ด้วยรูปแบบเดียวกัน ก็สามารถอ้างถึง 32 floating point registers เมื่อใช้คำสั่งที่เป็น floating point เป็นต้น). และนอกจากนั้น เรายังสามารถปรับแต่งลักษณะให้รีจิสเตอร์ที่ใช้กับ floating point เหมาะกับ floating point มากขึ้นได้ด้วย.

The Big Picture. Bit patterns have no inherent meaning. They may represent signed integers, floating-point numbers, instructions, and so on. What is represented depends on the instruction that operates on the bits in the word.

The major difference between computer numbers and numbers in the real world is that computer numbers have limited size, hence limited precision; it's possible to calculate a number too big or too small to be represented in a word. Programmers must remember these limits and write programs accordingly.

Chapter 4

The Processor

ประสิทธิภาพของคอมพิวเตอร์ ขึ้นอยู่กับ (1) instruction count, (2) clock cycle time, และ (3) clock cycles per instruction. คอมไพเลอร์ และ สถาปัตยกรรมชุดคำสั่ง จะมีผลกับ instruction count สำหรับแต่ละโปรแกรม. ส่วน ฮาร์แวร์ที่รองรับชุดคำสั่ง จะมีผลโดยตรงกับ clock cycle time และ จำนวน clock cycles per instruction.

บทนี้เราจะดูตัวอย่างการสร้างฮาร์ดแวร์เพื่อรองรับ คำสั่ง `lw`, `sw` (คำสั่งย้ายข้อมูลกับหน่วยความจำ I-format), `add`, `sub`, `AND`, `OR`, `slt` (คำสั่งตรรกะและพีชคณิต R-format), `beq`, และ `j` (คำสั่ง J-format). ส่วนคำสั่งที่เหลือก็สามารถสร้างฮาร์ดแวร์เพื่อรองรับได้ในลักษณะเดียวกัน.

ไม่ว่าคำสั่งที่เราต้องการจะสร้างฮาร์ดแวร์มารองรับ คือ คำสั่งอะไร, สิ่งที่จะต้องทำแน่ๆ มีอยู่ขั้นตอนคล้ายๆกัน ได้แก่:

- (1) ส่งแอดเดรสจาก program counter (PC) ไปที่หน่วยความจำ เพื่อ อ่านคำสั่งออกมา (เรียกว่า fetch the instruction).
- (2) ใช้ค่าในคำสั่งเพื่อเลือกรีจิสเตอร์มาอ่านค่า ตามชนิดคำสั่ง.

หลังจาก 2 ขั้นตอนนี้ ขั้นตอนต่อไปจะขึ้นอยู่กับแต่ละคำสั่ง ว่าเป็นคำสั่งให้ทำอะไร.

โชคดีที่สำหรับ MIPS คำสั่งทั้ง R-, I-, และ J-format ขั้นตอนที่จะทำต่อไปจะคล้ายๆกัน (ปรัชญาการออกแบบ simplicity and regularity ช่วยทำให้การสร้างฮาร์ดแวร์ง่ายขึ้น). ตัวอย่างเช่น ทุกคำสั่ง ยกเว้น `j` ใช้งาน ALU หลังจากอ่านค่าของรีจิสเตอร์แล้ว: R-format ใช้คำนวณพีชคณิตหรือตรรกะ, I-format ใช้ทำการบวกค่า base register กับ offset, และ `beq` ใช้ ALU ในการเปรียบเทียบ เพื่อตรวจสอบเงื่อนไข.

หลังนั้น คำสั่ง R-type ก็จะเก็บค่าผลลัพธ์จาก ALU ไปที่รีจิสเตอร์ตัวที่ระบุในคำสั่ง. ส่วนคำสั่ง `lw` หรือ `sw` ก็จะเรียกใช้หน่วยความจำ. คำสั่ง `beq` ก็จะอาจจะต้องเปลี่ยนค่าแอดเดรสของคำสั่งถัดไป ตามผลการเปรียบเทียบ.

ตรรกะและสัญญาณ. ในการออกแบบสัญญาณตรรกะ, นักออกแบบ อาจจะเปลี่ยน mapping ระหว่างตรรกะ TRUE หรือ FALSE และ ระดับสัญญาณไฟฟ้า high หรือ low voltage ได้. ดังนั้น วงจรบางส่วนอาจจะใช้ สัญญาณไฟฟ้า low voltage เพื่อสั่งให้ อุปกรณ์ทำงาน (asserting logic) ในขณะที่บางส่วนอาจจะใช้ สัญญาณไฟฟ้า high voltage เพื่อสั่งให้ อุปกรณ์ทำงาน. นักเรียนบางคน อาจจะเคยได้ยิน เรื่องนี้มาแล้ว คือเรื่องที่เรียกว่า ``active low'' หรือ ``active high''.

วงจรดิจิตอลที่เรา จะใช้สร้างฮาร์ดแวร์รองรับชุดคำสั่งนี้ อาจแบ่งได้เป็น 2 กลุ่ม: (1) วงจรที่ทำงานตามค่าข้อมูลอินพุตเป็นหลัก และ (2) วงจรที่ทำงานตามสถานะ. วงจรที่ทำงานตามค่า

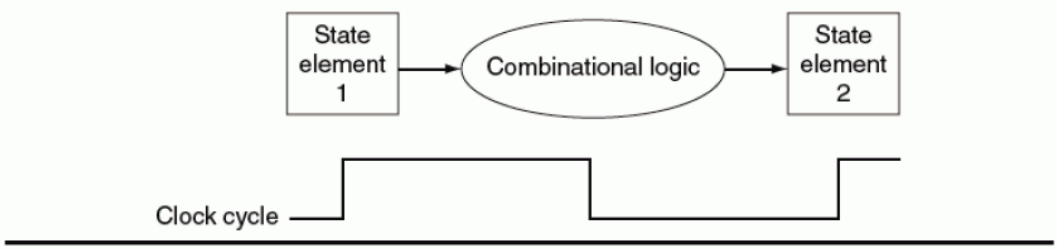


FIGURE 5.3 Combinational logic, state elements, and the clock are closely related. In a synchronous digital system, the clock determines when elements with state will write values into internal storage. Any inputs to a state element must reach a stable value (that is, have reached a value from which they will not change until after the clock edge) before the active clock edge causes the state to be updated. All state elements, including memory, are assumed to be edge-triggered.

Figure 4.1: Combinational logic, state elements, and clock.

ข้อมูลอินพุตเป็นหลัก, Combinational elements, ค่าเอาต์พุตของวงจรขึ้นอยู่กับอินพุตปัจจุบันเท่านั้น เช่น ALU เป็น combinational. ดังนั้น เมื่ออินพุตเหมือนกัน ค่าเอาต์พุตจาก ALU จะเหมือนกัน. ส่วน วงจรที่ทำงานตามสถานะ, state elements (หรือ อาจเรียก sequential elements), จะมี สถานะ (state) อยู่ ซึ่งที่มีได้ก็เพราะ วงจรแบบนี้จะมี internal storage อยู่. พวก data memory และ registers จะเป็น state elements: ค่าของ register ตัวเดียวกัน ที่เวลาต่างๆกันไม่เหมือนกัน ขึ้นกับค่าภายในว่ามันเก็บอะไรอยู่.

วงจรที่ทำงานตามสถานะ จะมีอินพุตอย่างน้อย 2 อินพุต ซึ่ง 1 ใน 2 สำหรับสัญญาณ clock. สัญญาณ clock ใช้กำหนดจังหวะที่ข้อมูลจะถูกเขียนเข้าไป เอาต์พุตจาก state elements จะให้ค่าล่าสุดที่เก็บไว้. ตัวอย่างของ state element ที่ง่ายที่สุดคือ D flip-flop.

Clocking Methodology เทคนิค คือ เราจะใช้ clock ในการกำหนดจังหวะว่าเมื่อไร จะอนุญาติให้อ่านข้อมูลจากอุปกรณ์ได้ หรือ เมื่อไรจะอนุญาติให้เขียนข้อมูลเข้าอุปกรณ์ได้. ถ้าเราไม่ใช้ clock ช่วยกำหนดจังหวะ แล้วเกิด เราอ่านข้อมูลพร้อมๆกับเขียนข้อมูลเข้าไปก็อาจเกิดปัญหาได้ว่า ข้อมูลที่อ่านออกมาจะเป็นค่าเก่า หรือ ค่าใหม่ หรือ แม้แต่ผสมกันระหว่างใหม่กับเก่า.

เมื่อความสะดวก เราจะถือว่า state elements ของเรา เป็น edge-triggered clocking. อุปกรณ์ที่เป็น edge-triggered คือ ค่าใหม่จะถูกเขียนเข้าไปได้เฉพาะจังหวะของ clock edge. รูป 4.1 แสดง combinational logic อยู่ระหว่าง state elements. สัญญาณจะ propagate จาก state element 1 ผ่าน combinational logic ไปสู่ state element 2 ภายในเวลา 1 clock cycle. เวลาที่สัญญาณใช้ในการเดินทางจาก state element 1 ไปสู่ state element 2 จะเป็นตัวกำหนด ความยาวของ clock ที่สั้นที่สุด (ถ้าสั้นกว่านี้ สัญญาณจะเดินทางไปไม่ทัน และวงจรจะไม่ทำงานตามตรรกะที่ออกแบบไว้)

4.1 Building a Datapath

เราจะเริ่มการออกแบบ datapath ด้วยการ จำแนก อุปกรณ์หลักๆที่จะต้องใช้สร้างวงจรเพื่อรองรับ คำสั่ง MIPS ต่างๆ.

เนื่องจาก เราจะใช้อุปกรณ์ Multiplexer (MUX) อย่างมาก, เราใช้ MUX เป็นเครื่องมือในการเลือกเส้นทางสัญญาณข้อมูล โดย รูป 4.3 แสดงตัวอย่างของ MUX ที่รับ 4 อินพุต. เอาต์พุตของ

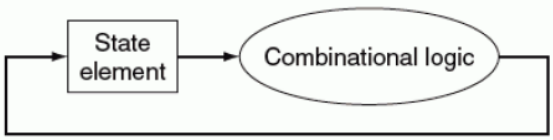


FIGURE 5.4 An edge-triggered methodology allows a state element to be read and written in the same clock cycle without creating a race that could lead to indeterminate data values. Of course, the clock cycle still must be long enough so that the input values are stable when the active clock edge occurs. Feedback cannot occur within 1 clock cycle because of the edge-triggered update of the state element. If feedback were possible, this design could not work properly. Our designs in this chapter and the next rely on the edge-triggered timing methodology and structures like the one shown in this figure.

Figure 4.2: การใช้ edge triggered clocking ทำให้เรากำหนดจังหวะการเขียนและอ่านได้ และผลที่ได้จากการอ่านก็ไม่กำกวม.

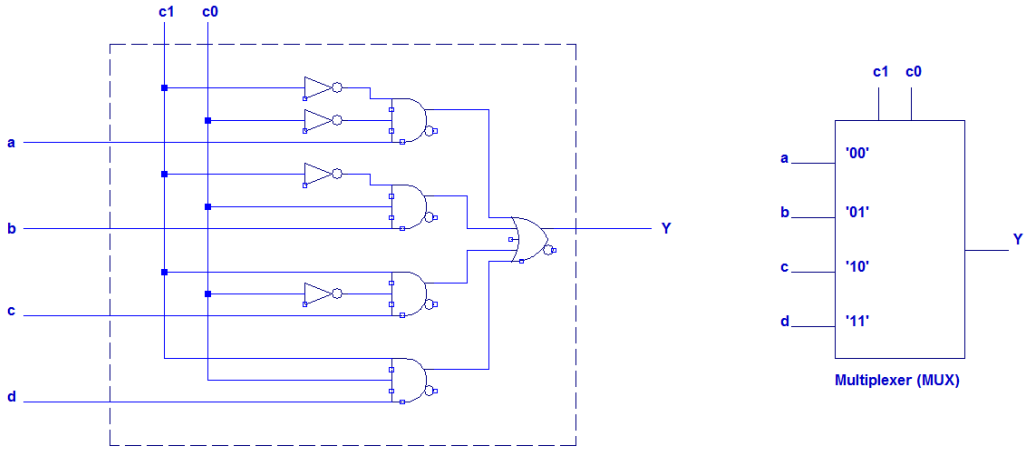


Figure 4.3: ตัวอย่าง 4-to-1 Multiplexer.

MUX จะเท่ากับ a เมื่อ ค่าของ selector (c1, c0) เป็น `00', เท่ากับ b เมื่อ (c1, c0) เป็น `01', เท่ากับ c เมื่อ (c1, c0) เป็น `10', และ เท่ากับ d เมื่อ (c1, c0) เป็น `11'. อุปกรณ์ MUX สำหรับ 2-to-1, 8-to-1, 16-to-1 เป็นต้น ก็สามารถทำได้ในลักษณะเดียวกัน.

เพื่อความสะดวกในการอ้างถึงรีจิสเตอร์ เราจะรวมรีจิสเตอร์ไว้ด้วยกันเรียกว่า Register File โดยรูป 4.4 (a) แสดงโครงสร้างภายใน (register level) สำหรับการอ่านค่า ของรีจิสเตอร์ไฟล์ (b) แสดง interface ภายนอก และรูป 4.5 แสดงโครงสร้างภายในสำหรับการเขียนค่า.

Fetch การอ่านคำสั่ง เราต้องการ program counter (PC) เพื่อเก็บแอดเดรสของคำสั่งที่จะอ่านออกมาทำงาน; หน่วยความจำ สำหรับเก็บคำสั่ง; และ ALU หรือ Adder สำหรับ เพิ่มค่า PC ไปที่คำสั่งถัดไป. รูป 4.6 แสดงอุปกรณ์ที่ใช้รองรับการอ่านคำสั่ง และ รูป 4.7 คือ เส้นทางข้อมูลของการอ่านคำสั่ง.

4.1.1 R-type

การทำงานรองรับคำสั่ง R-type เช่น

```
add rd, rs, rt
```

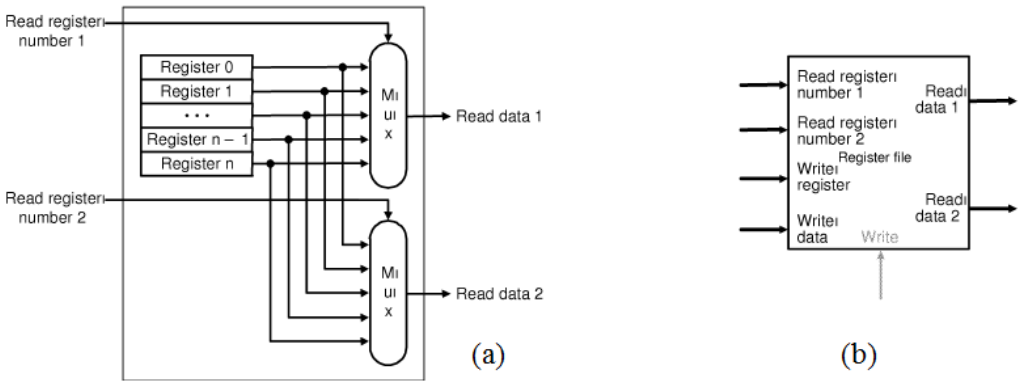


Figure 4.4: โครงสร้างภายใน (gate level) ของ รีจิสเตอร์ไฟล์ สำหรับการอ่านค่า.

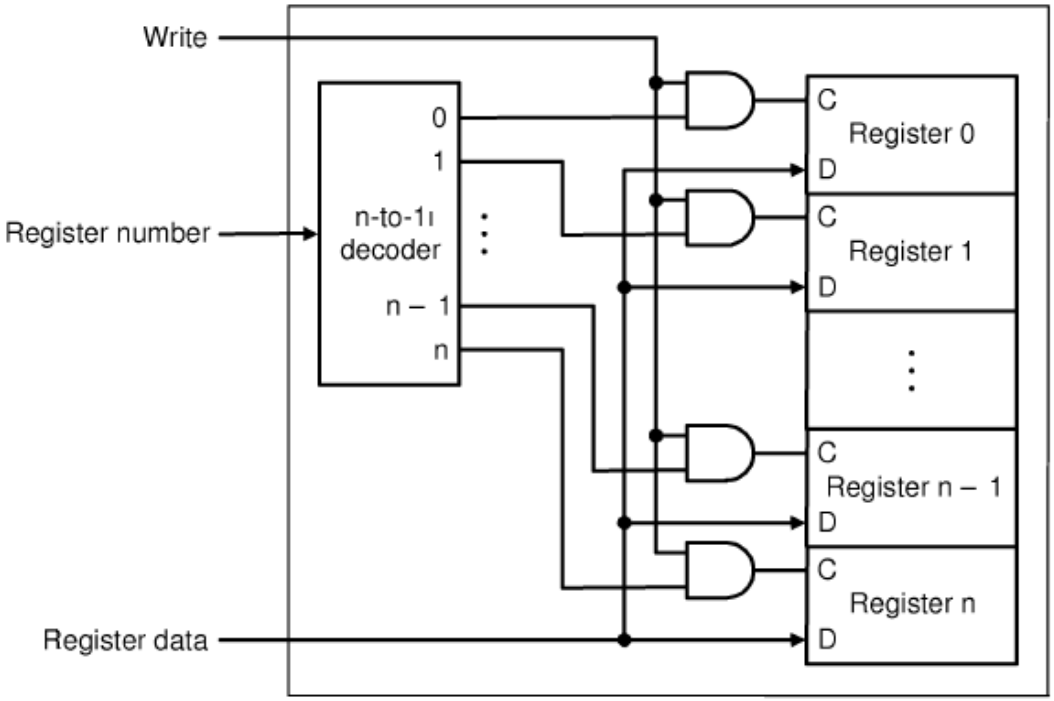


Figure 4.5: โครงสร้างภายใน (gate level) ของ รีจิสเตอร์ไฟล์ สำหรับการเขียนค่า.

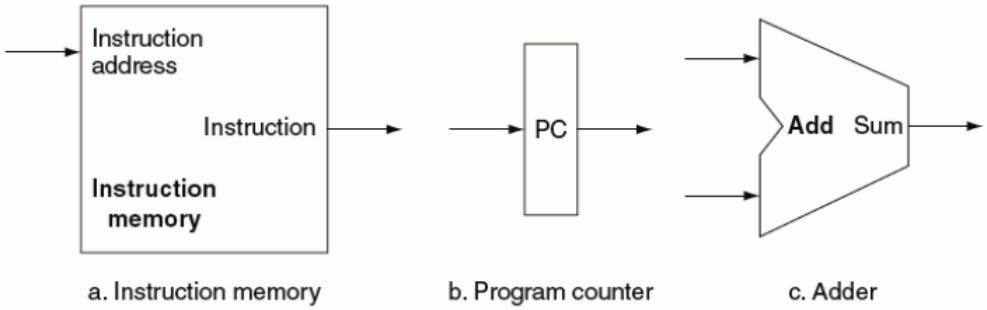


FIGURE 5.5 Two state elements are needed to store and access instructions, and an adder is needed to compute the next instruction address. The state elements are the instruction memory and the program counter. The instruction memory need only provide read access because the datapath does not write instructions. Since the instruction memory only reads, we treat it as combinational logic: the output at any time reflects the contents of the location specified by the address input, and no read control signal is needed. (We will need to write the instruction memory when we load the program; this is not hard to add, and we ignore it for simplicity.) The program counter is a 32-bit register that will be written at the end of every clock cycle and thus does not need a write control signal. The adder is an ALU wired to always perform an add of its two 32-bit inputs and place the result on its output.

Figure 4.6: อุปกรณ์ที่จะใช้สำหรับรองรับการอ่านคำสั่ง (Fetch)

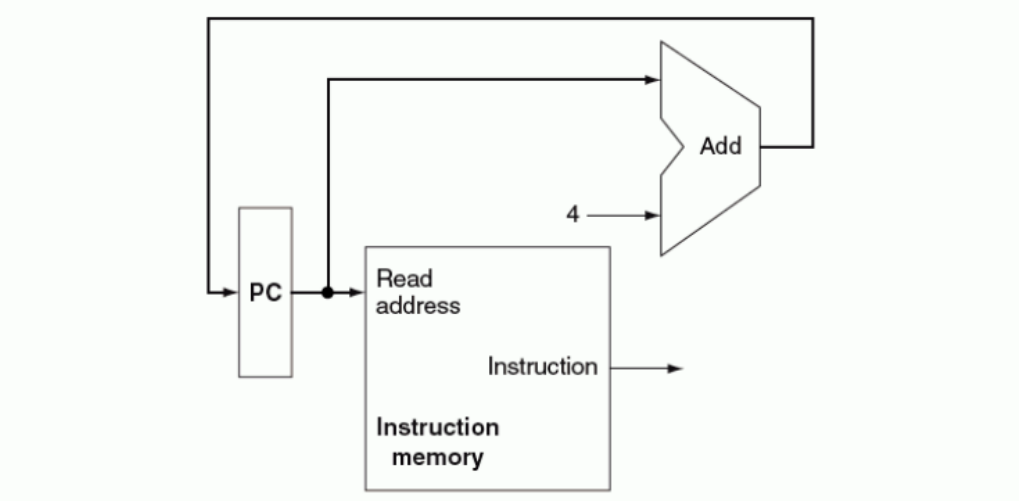


FIGURE 5.6 A portion of the datapath used for fetching instructions and incrementing the program counter. The fetched instruction is used by other parts of the datapath.

Figure 4.7: Fetch datapath.

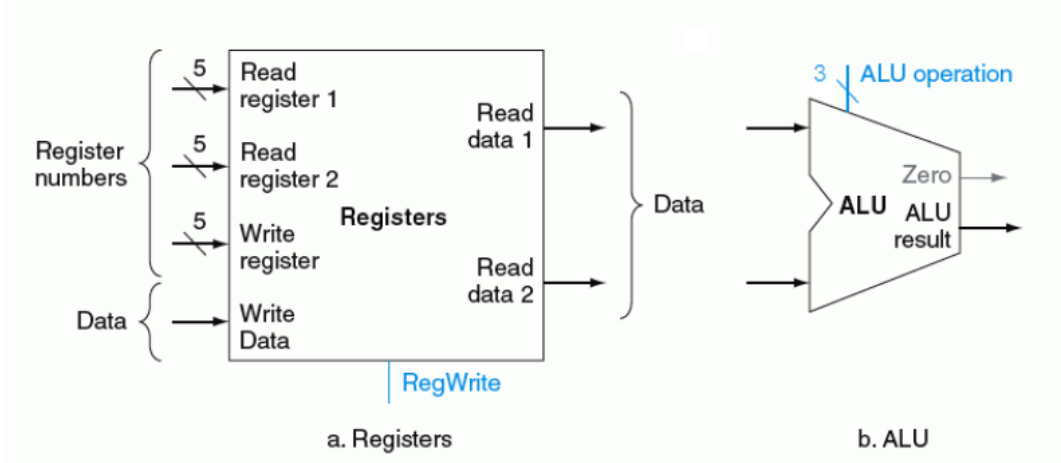


Figure 4.8: รีจิสเตอร์ไฟล์ กับ ALU สำหรับ R-type.

```
sub rd, rs, rt
and rd, rs, rt
or  rd, rs, rt
slt rd, rs, rt
```

เราต้องการ รีจิสเตอร์ไฟล์ และ ALU ดังแสดงในรูป 4.8 และ เส้นทางสัญญาณข้อมูลดังในรูป 4.9.

ในรูป 4.9 สัญญาณ instruction บิตที่ 31 ถึง 26 (6 บิต บนสุด) เป็น opcode. สัญญาณ instruction บิตที่ 25-21 (5 บิตของฟิลด์ rs) จะต่อไปเป็นอินพุต Read register 1 ให้กับรีจิสเตอร์ไฟล์. ดังนั้นเอาต์พุต Read data 1 ของ รีจิสเตอร์ไฟล์ จะให้ค่าที่รีจิสเตอร์ที่ระบุในฟิลด์ rs ของคำสั่งออกมา. สัญญาณ instruction บิตที่ 20-16 (5 บิตของฟิลด์ rt) จะต่อไปเป็นอินพุต Read register 2 ให้กับรีจิสเตอร์ไฟล์. เอาต์พุต Read data 2 ของ รีจิสเตอร์ไฟล์ จะให้ค่าที่รีจิสเตอร์ที่ระบุในฟิลด์ rt ของคำสั่งออกมา.

เอาต์พุต Read data 1 และ 2 ของรีจิสเตอร์ไฟล์ (ซึ่งคือค่าของ rs และ rt ตามลำดับ) จะต่อเข้าไปเป็นอินพุตของ ALU เพื่อทำการคำนวณ พีชคณิต หรือ ตรรกะ และ ผลลัพธ์จากการคำนวณจะถูกโยงกลับมาป้อนในกับ Write data ของรีจิสเตอร์ไฟล์ เพื่อเขียนใส่รีจิสเตอร์ที่จะเก็บผลลัพธ์

การคำนวณของ ALU ว่าจะทำ บวก ลบ หรือ AND หรือ OR หรือ SLT จะขึ้นอยู่กับสัญญาณควบคุม ALUOperation (3 bits) ซึ่งเราจะคุยกันทีหลังในเรื่องสัญญาณควบคุม.

ขณะที่ผลลัพธ์จาก ALU จะมารอเป็นอินพุตของรีจิสเตอร์ไฟล์ที่ Write data, สัญญาณ instruction บิตที่ 15-11 (5 บิตของฟิลด์ rd) จะต่อไปเป็นอินพุต Write register ให้กับรีจิสเตอร์ไฟล์ เพื่อระบุว่า ค่าที่จะเขียนเข้า ให้เขียนไปที่รีจิสเตอร์ตัวไหน. ดังนั้นเมื่อมีสัญญาณควบคุม RegWrite เข้ามา (เพื่อให้เขียนค่าเข้ารีจิสเตอร์), ค่าที่อินพุต Write data ของรีจิสเตอร์ไฟล์ จะถูกเขียนเข้าไปที่รีจิสเตอร์ตัวที่ระบุในฟิลด์ rd.

4.1.2 คำสั่งย้ายข้อมูลกับหน่วยความจำ

คำสั่งโหลดข้อมูล

```
lw rd, offset(rs)
```

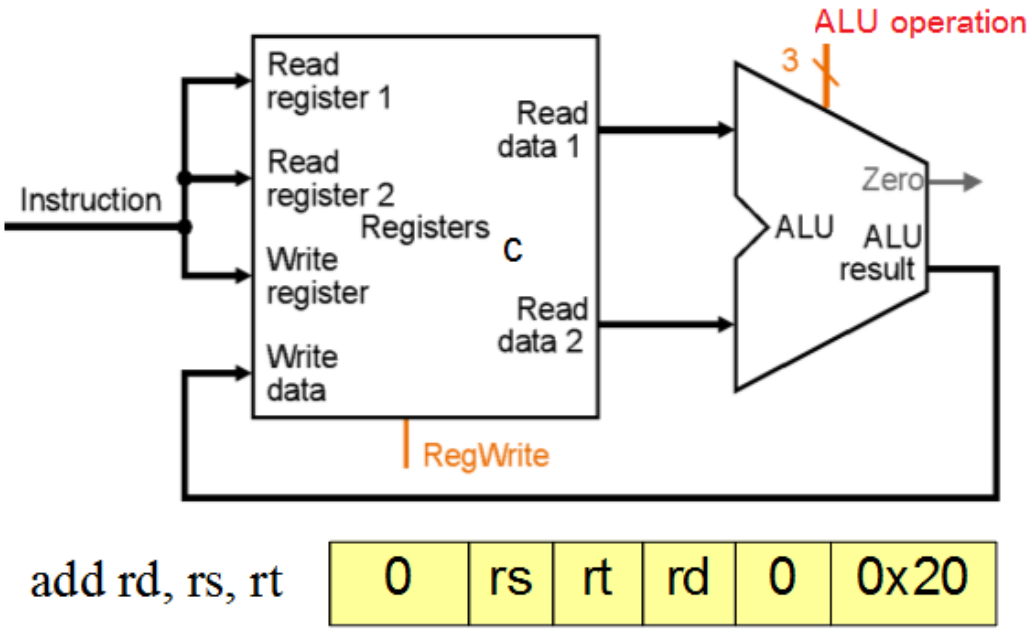


Figure 4.9: เส้นทางสัญญาณข้อมูล (datapath) สำหรับคำสั่ง R-type.

โดย `offset` เป็นฟิลด์เก็บ 16-bit signed number. ตัวอย่าง คำสั่ง `lw $17, 0xFFFC($18)` จะเก็บค่าในหน่วยความจำ ที่แอดเดรส เท่ากับ ค่าของรีจิสเตอร์ตัวที่ 18 บวกกับค่า offset ซึ่ง คือ `0xFFFC` (เท่ากับ -4 ในฐานสิบ), โดยค่าที่โหลดจะถูกนำไปเก็บที่รีจิสเตอร์ตัวที่ 17.

คำสั่งเก็บข้อมูล

```
sw rd, offset(rs)
```

จะเขียนข้อมูลในหน่วยความจำที่แอดเดรส เท่ากับ ค่าของรีจิสเตอร์ rs บวกกับค่า offset และค่าที่จะเขียนเข้าไป คือ ค่าของรีจิสเตอร์ rd.

รูป 4.10 แสดงอินเตอร์เฟซ ของหน่วยความจำ และ วงจรขยายบิตตามเครื่องหมาย (Sign extension) ซึ่งเราจะใช้ประกอบในเส้นทางสัญญาณข้อมูลของคำสั่งเคลื่อนย้ายข้อมูล.

ค่า offset ในคำสั่งจะแทนด้วย ฟิลด์ขนาด 16 บิต แต่ซึ่งเราจะนำไปบวกกับค่าของ base address จากรีจิสเตอร์ขนาด 32 บิต สังเกตุจำนวนบิตที่ต่างกัน ก่อนที่เราจะบวกกัน เราจะขยายจำนวนบิตของ offset ให้เป็น 32 บิตก่อน ซึ่งเราจะใช้ วงจรขยายบิตตามเครื่องหมาย (Sign extension).

ค่าของ offset เป็นตัวเลขแบบ 2's compliment ซึ่งเมื่อขยาย เลขบวก จะถูกเติมด้วย `0' ไปข้างหน้าจนเต็ม แต่ถ้าเป็น เลขลบ `1' จะใช้เติมจนเต็ม. ตัวอย่าง ถ้า offset เป็น 8 ในฐานสิบ: offset = `0x0008` จะถูกขยายเป็น `0x00000008` ซึ่งมีค่าเท่ากับ 8 ในฐานสิบเหมือนเดิม แต่มีขนาดเป็น 32 บิต. ถ้า offset เป็น -4 ในฐานสิบ: offset = `0xFFFC` จะถูกขยายเป็น `0xFFFFFFFC` ซึ่งก็มีค่าเท่ากับ -4 ในฐานสิบ ที่มีขนาด 32 บิต.

รูป 4.11 แสดงเส้นทางข้อมูลของคำสั่งเคลื่อนย้ายข้อมูล. เมื่อทำคำสั่ง `lw`, สัญญาณ instruction บิตที่ 25-21 (5 บิต ของ ฟิลด์ rs) จะเป็นอินพุตของรีจิสเตอร์ไฟล์ เพื่ออ่านค่ารีจิสเตอร์ rs แล้วนำไปบวกกับ offset (สัญญาณ instruction บิตที่ 15-0) ที่ผ่านการขยายบิตด้วย Sign Extend แล้ว. ผลลัพธ์จากการบวก (เอาต์พุตจาก ALU) จะคือ แอดเดรส ที่ต้องการอ่านค่า. ค่าแอดเดรสนี้ จะส่งไปเป็นอินพุตของหน่วนความจำ เพื่ออ่านค่าออกมาที่เอาต์พุต Read data ซึ่ง

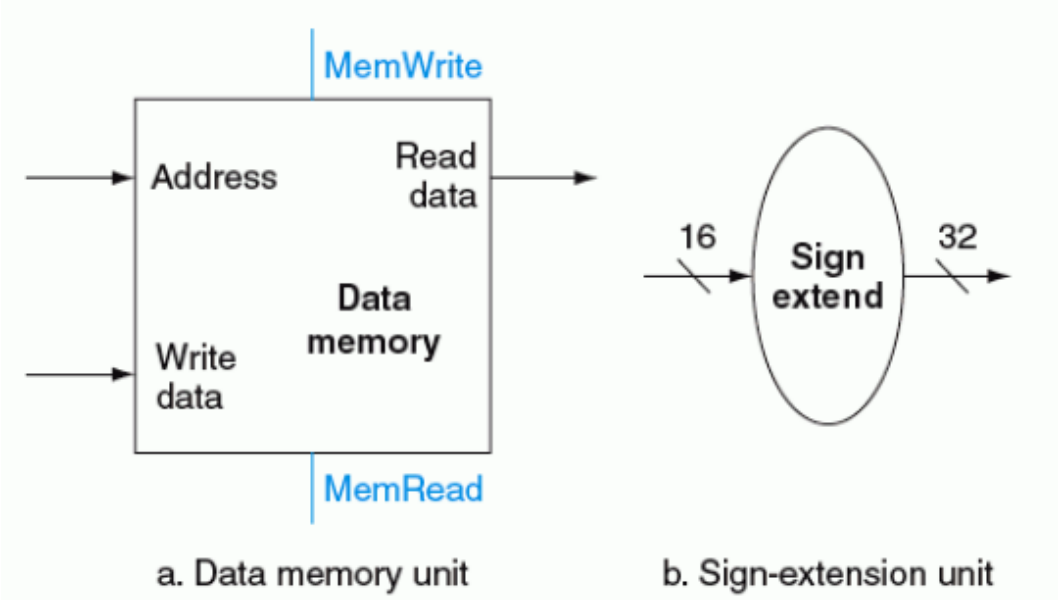


Figure 4.10: หน่วยความจำ และ วงจรขยายบิตตามเครื่องหมาย (Sign extension).

ค่าที่อ่านได้นี้ ก็จะส่งมาเป็นอินพุตของรีจิสเตอร์ไฟล์ เพื่อเขียนเก็บไว้ในรีจิสเตอร์.

สัญญาณ instruction บิตที่ 20-16 จะถูกป้อนเป็นอินพุต Write register ของรีจิสเตอร์ไฟล์ เพื่อระบุว่า ค่าที่อ่านจากหน่วยความจำ จะเขียนไปที่ใด. สัญญาณเส้นทางข้อมูลของคำสั่ง `lw` จะถูกเน้นดังแสดงในภาพซ้ายมือของรูป 4.12. สังเกตุ คำสั่ง `lw` ต้องบังคับ เขียนรีจิสเตอร์ไฟล์ และอ่านหน่วยความจำ ดังที่เห็นสัญญาณควบคุม RegWrite และ MemRead ถูกเน้นในภาพซ้ายมือ.

ในทำนองเดียวกับ `lw`, คำสั่ง `sw` จะต่างกันที่สัญญาณ instruction บิตที่ 20-16 (5 บิตของฟิลด์ rt) จะต่อเป็นอินพุต Read register 2 ของรีจิสเตอร์ไฟล์ ซึ่งเอาต์พุต Read data 2 (จะได้ค่าของรีจิสเตอร์ rt) จะต่อไปเป็น อินพุต Write data ของหน่วยความจำ. เมื่อจ่ายสัญญาณ MemWrite ค่าของ rt ก็จะถูกเขียนเข้าไปในหน่วยความจำแอดเดรสที่ rs บวกกับ offset ที่ขยายบิตแล้ว. ภาพขวามือของรูป 4.12 เน้นเส้นทางข้อมูลสำหรับคำสั่ง `sw`.

ข้อสังเกตุ เราสามารถต่อสัญญาณ instruction บิต 20-16 ไปเข้าที่อินพุต Read register 2 กับ Write data ของรีจิสเตอร์ไฟล์พร้อมกันได้เลย. เพราะว่า เมื่อทำคำสั่ง `lw` แม้จะมีเอาต์พุตของ Read data 2 ออกมา และต่อไปยังอินพุต Write data ของหน่วยความจำ แต่เมื่อ เราไม่ได้ส่งสัญญาณ MemWrite สัญญาณของ Read data 2 นี้ก็จะไม่มีผลอะไร. ในทำนองเดียวกัน เมื่อทำคำสั่ง `sw` จะมีสัญญาณ instruction บิต 20-16 ไปเข้าที่อินพุต Write data ของรีจิสเตอร์ไฟล์ด้วย ก็ไม่เป็นไร เพราะ เราไม่ได้ส่งสัญญาณควบคุม RegWrite ออกมา.

4.1.3 คำสั่ง branch

คำสั่ง

```
beq rs, rt, offset
```

branch on equal จะเปรียบเทียบค่าในรีจิสเตอร์ rs และ rt; หากเท่ากัน, จะเปลี่ยนค่า PC เป็น (PC + 4) บวกกับ ค่า offset ที่ถูกขยายเป็น 32 บิตแล้ว shifted ไปอีก 2 bits.

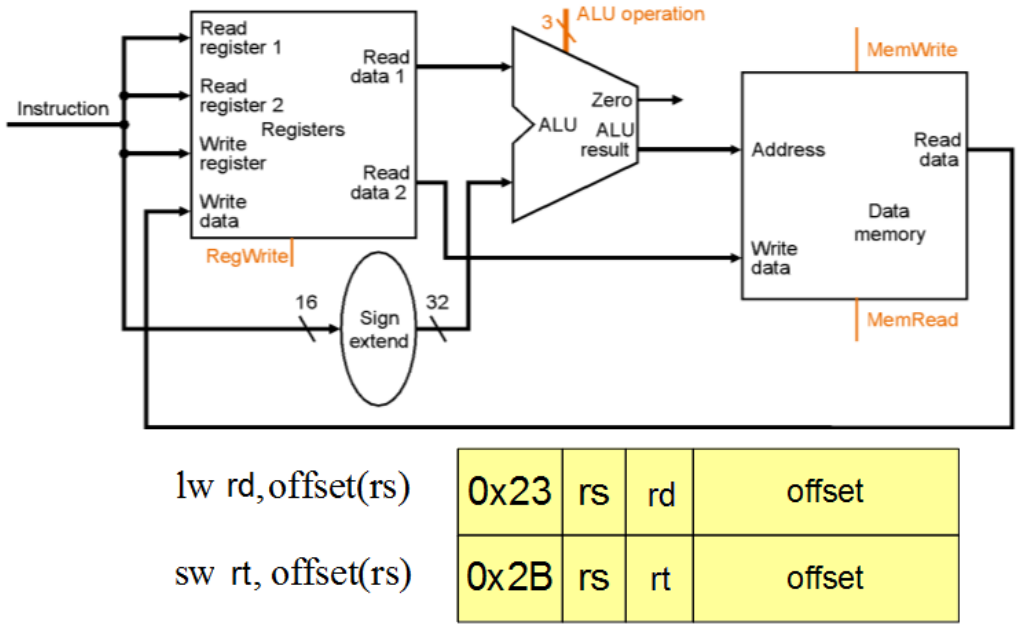


Figure 4.11: เส้นทางสัญญาณข้อมูลของคำสั่ง `lw` และ `sw`.

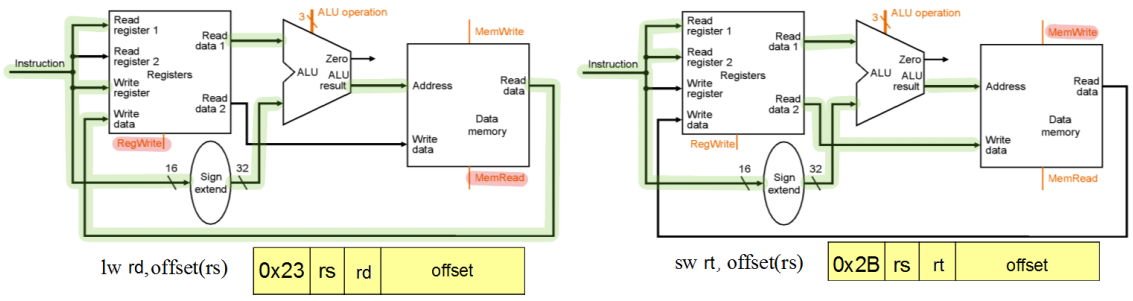


Figure 4.12: เปรียบเทียบเส้นทางสัญญาณข้อมูลของคำสั่ง `lw` และ `sw`.

ค่า offset ของคำสั่ง branch เป็นตัวเลขแบบ 2's compliment ดังนั้นการขยายบิต ก็ต้องขยายตาม sign ของตัวเลข เช่นเดียวกับ คำสั่ง `lw` และ `sw`. และ เนื่องจาก MIPS มีข้อกำหนด Memory Alignment Restriction คือ คำสั่งจะอ้างด้วยแอดเดรส ที่เป็น multiple of 4 เช่น

ฐานสิบหก	ฐานสอง
⋮	⋮
0x0500 0000	[0000,0101,0000,...,0000,0000]
0x0500 0004	[0000,0101,0000,...,0000,0100]
0x0500 0008	[0000,0101,0000,...,0000,1000]
0x0500 000C	[0000,0101,0000,...,0000,1100]
0x0500 0010	[0000,0101,0000,...,0001,0000]
0x0500 0014	[0000,0101,0000,...,0001,0100]
0x0500 0018	[0000,0101,0000,...,0001,1000]
0x0500 001C	[0000,0101,0000,...,0001,1100]
0x0500 0020	[0000,0101,0000,...,0010,0000]
⋮	⋮

สังเหตุว่า ด้วย Memory Alignment Restriction ที่บังคับอยู่ แอดเดรสของคำสั่ง 2 บิตสุดท้าย จะเป็น 00 ตลอดดังนั้น เพื่อเป็นการใช้ฟิลด์ offset 16 บิตของ instruction ให้คุ้มค่าที่สุด เราจะใช้ shift ค่านี้ไปทางซ้าย 2 บิต.

รูป 4.13 แสดงเส้นทางสัญญาณข้อมูลของคำสั่ง `beq`. สัญญาณ Instruction บิต 25-21 (5 บิต ฟิลด์ rs) จะเข้าอินพุต Read register 1 ของรีจิสเตอร์ไฟล์ และ บิต 20-16 (5 บิต ฟิลด์ rt) จะเข้าอินพุต Read register 2. ค่าของรีจิสเตอร์ทั้งสองจะเปรียบเทียบว่า เท่ากันหรือไม่ โดยนำไปลบกัน ถ้าผลลัพธ์ ออกมาเป็น ศูนย์ (ทุกบิตของผลลัพธ์เป็นศูนย์) แปลว่า rs เท่ากับ rt. เราจะใช้ สัญญาณเอาต์พุต zero จาก ALU (ดูเรื่อง zero detection บท 3) โดยถ้า สัญญาณ zero = 1 แปลว่า rs $=$ rt แต่ถ้า สัญญาณ zero = 0 แปลว่า rs $\neq$ rt.

สัญญาณ zero นี้เราจะนำไปทำเป็นสัญญาณควบคุมอีกที โดยถ้า zero = 1 เราจะต้องเปลี่ยนค่า PC เป็น PC + 4 + (2-bit shifted, sign-extend offset), ซึ่งทำได้โดยนำ สัญญาณ Instruction บิต 15-0 ผ่านวงจร Sign extend และ shift left 2 แล้วนำไปบวกกับ PC + 4 ก็จะได้ผลลัพธ์เป็น branch address ที่พร้อมจะนำไปใส่ค่า PC ถ้า zero = 1.

รูป ?? แสดงเส้นทางข้อมูลของคำสั่ง R-type, `lw`, `sw`, และ`beq`. สังเกตุ การใช้ MUX เป็นเครื่องมือช่วยเลือกเส้นทางข้อมูล ที่เหมาะสมสำหรับแต่ละคำสั่ง. เช่น หาก เราไม่ได้ทำคำสั่ง branch เราก็จะส่งสัญญาณ PCSrc เพื่อให้ MUX (ตัวมุมขวาบน) เลือก PC + 4 เป็นเอาต์พุต ที่จะไป update ค่าของ PC. แต่ถ้า เราทำคำสั่ง branch เช่น `beq` และ ผลการเปรียบเทียบ ทำให้ต้องทำ branch เราก็จะส่งสัญญาณ PCSrc เพื่อให้ MUX เลือก ค่าจาก ALU result มาเป็นเอาต์พุตเพื่อไป update ค่าของ PC. รูป 4.15 แสดงเส้นทางสัญญาณข้อมูล พร้อมสัญญาณควบคุม สำหรับคำสั่ง R-type, `lw`, `sw`, และ `beq`. สัญญาณควบคุม PCSrc ได้มาจากการ AND ค่าของสัญญาณควบคุม Branch (เกิดเมื่อเราทำคำสั่ง branch) และ สัญญาณผลลัพธ์จากการเปรียบเทียบ zero.

4.1.4 คำสั่ง Jump

คำสั่ง

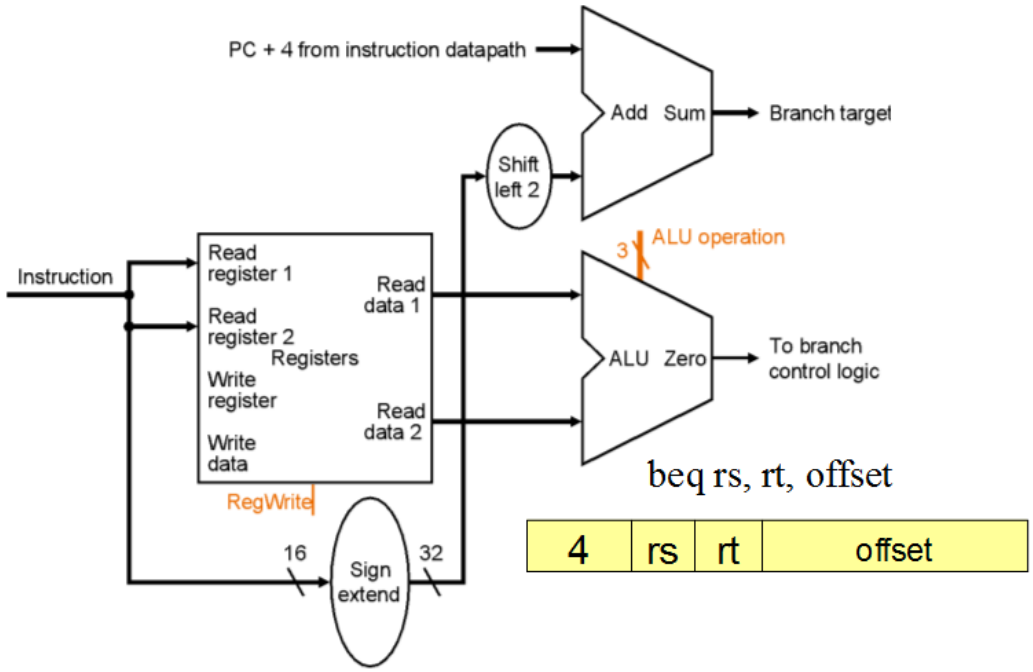


Figure 4.13: เส้นทางสัญญาณข้อมูลของคำสั่ง `beq`.

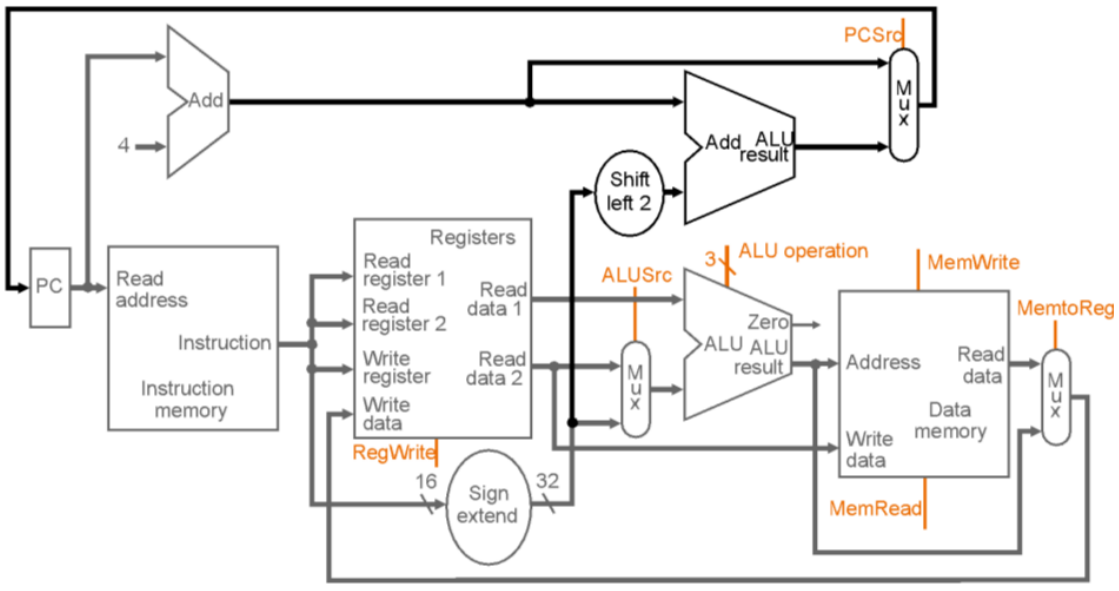


Figure 4.14: เส้นทางสัญญาณข้อมูลของคำสั่ง R-type, `lw`, `sw`, และ`beq`.

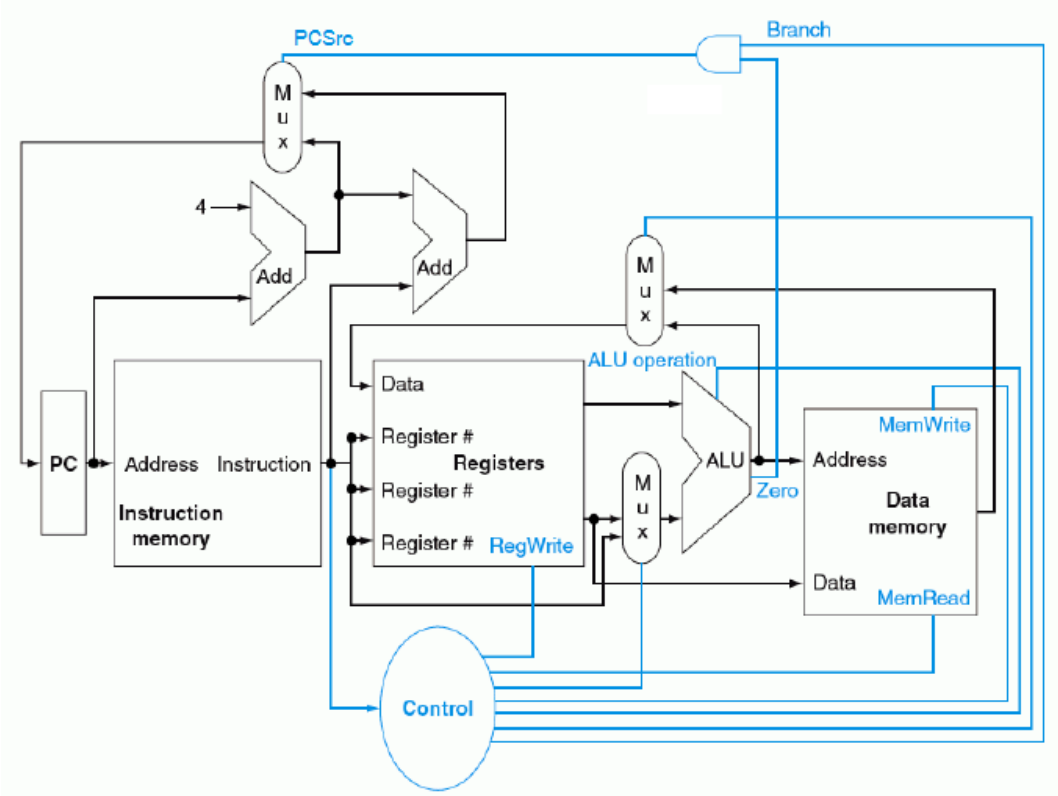


Figure 4.15: เส้นทางสัญญาณข้อมูลของคำสั่ง R-type, `lw`, `sw`, และ`beq` พร้อมต่อสัญญาณควบคุม.

```
j target
```

โดย `target` แทนด้วยฟิลด์ขนาด 26 บิต. คำสั่ง Jump จะทำงาน ด้วยการ นำ 4 บิตค่าสูงสุด (4 MSBs) ของ PC + 4 มารวมกับ 28 บิต ที่ได้จากการ shift left 2 บิต ของ 26-bit target ไปรวมเป็นค่าสำหรับ update PC.

รูป 4.16 แสดงเส้นทางสัญญาณข้อมูลของ Single-cycle machine สำหรับคำสั่ง R-type, คำสั่งย้ายข้อมูล, คำสั่ง branch และ jump.

สำหรับคำสั่ง jump, สัญญาณ Instruction บิต 25-0 (26 บิต ของฟิลด์ target) จะถูกนำไปผ่านวงจร Shift left 2 เพื่อทำเป็น 28 บิต แล้วนำไปต่อกับ สัญญาณจาก PC + 4 บิต 31-28 รวมทั้งหมดเป็น 32 บิต, เขียนย่อๆ เป็น `PC4:target28`, พร้อมสำหรับ update ค่าใหม่ให้กับ PC.

โดยเมื่อทำคำสั่ง Jump, เราจะให้สัญญาณควบคุม Jump (จากวงจร Control, วาดเป็นรูปไข่อยู่กลางๆภาพ) เป็น `1' ซึ่งจะทำให้ MUX เลือก ค่า `PC4:target28` ไปเขียนอินพุตของ PC. แต่ถ้าเราไม่ได้ทำสั่ง Jump หรือ Branch, เราจะให้สัญญาณควบคุม Branch = `0' ทำให้ MUX (ตัวที่ 2 จากขวา บน) เลือกค่า PC + 4 เป็นเอาต์พุต ซึ่งค่านี้ ก็จะเป็นเอาต์พุตของ MUX (ตัวขวาสุด บน) ด้วยเมื่อ Jump = `0' และ ทำให้ PC จะถูก update ด้วยค่า PC + 4. การควบคุมเส้นทางสัญญาณข้อมูลสำหรับคำสั่งอื่นๆ ก็สามารถทำได้ในลักษณะเดียวกัน.

ในรูป 4.16, สัญญาณควบคุมที่ส่งที่ MUX ทุกๆตัว รวมถึงสัญญาณควบคุม การอ่าน/เขียนหน่วยความจำ สัญญาณควมคุมการเขียนค่ารีจิสเตอร์ จากออกมาจากวงจร Control ที่รับอินพุต 6 บิตจากสัญญาณ Instruction บิต 31-26 (6 บิต ฟิลด์ opcode). วงจร Control เป็นวงจร combinational logic ธรรมดา, โดยมีอินพุต 6 บิต และจ่ายเอาต์พุต 9 บิต สำหรับสัญญาณ

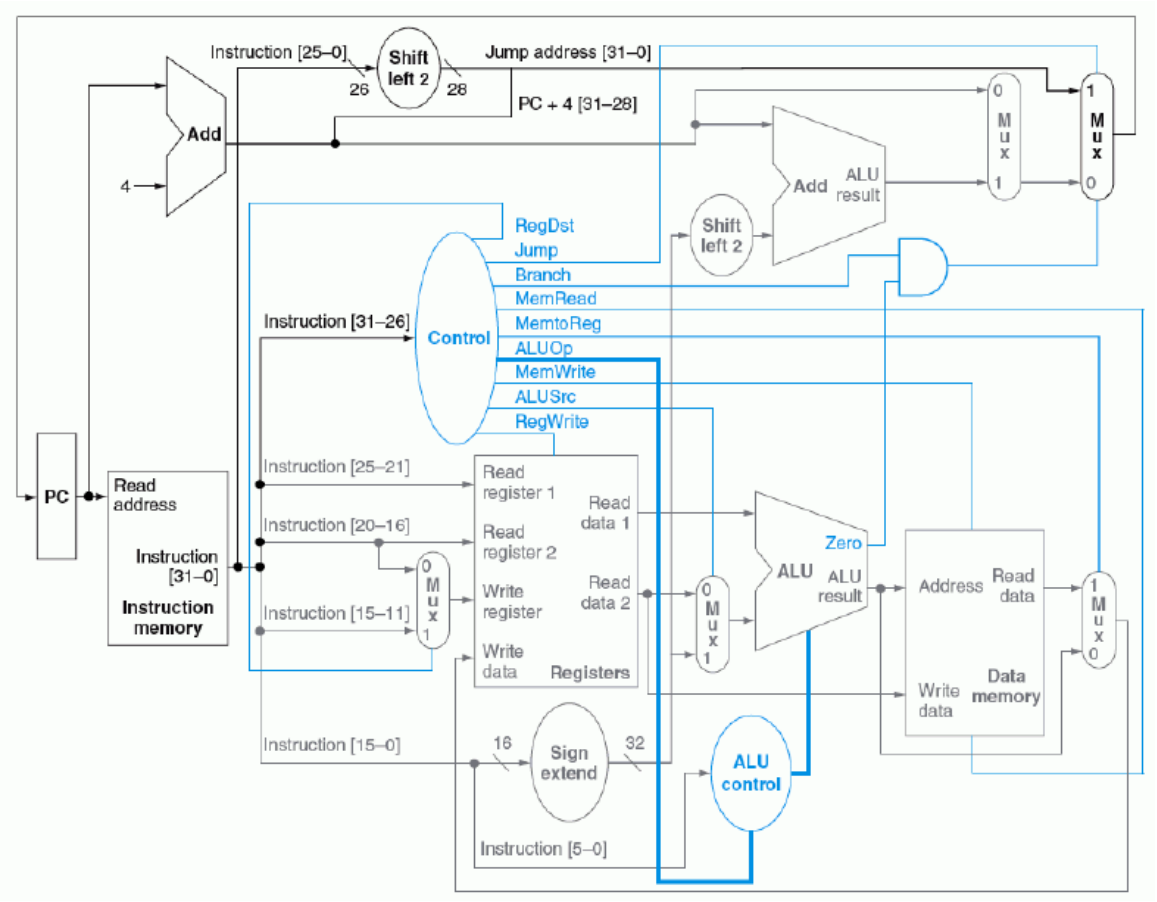


Figure 4.16: เส้นทางสัญญาณข้อมูลของคำสั่งต่างๆ รวมถึงคำสั่ง Jump.

ควบคุม RegDst, Branch, MemRead, MemtoReg, ALUOp, MemWrite, ALUSrc (2 บิต), และ RegWrite.

โดยสัญญาณควบคุม ALUSrc นั้น วงจร Control จ่ายไปเป็นอินพุตให้กับ วงจร ALU control. นอกจาก ALUSrc, วงจร ALU Control รับอินพุตอีกชุดจากสัญญาณ Instruction บิต 5-0 (6 บิต ฟิลด์ funct ของคำสั่ง R-type) และ จ่ายเอาต์พุตเพื่อควบคุมการทำงานของ ALU ที่ใช้คำนวณค่าจากรีจิสเตอร์.

เปรียบเทียบรูป 4.15 กับ 4.16, นอกจาก การมีเส้นทางข้อมูลสำหรับคำสั่ง Jump แล้ว การใช้วงจร Control เดียวควบคุมทุกอย่าง (รูป 4.15) กับ การแยกสัญญาณควบคุม ALU ออกมาจากสัญญาณควบคุมอื่นๆ (รูป 4.16) ก็ต่างกันด้วย.

แนวคิดคือ เราอาจจะสร้างวงจรเดียวที่ส่งสัญญาณควบคุมทุกอย่างออกมาได้ (เช่น รูป 4.15) แต่มันสะดวก ถ้าเราจะแบ่งงานที่มีหน้าที่เฉพาะชัดเจนอย่างการควบคุม ALU มาให้กับวงจรเฉพาะ. ในรูป 4.16, วงจร Control ส่งสัญญาณควบคุมต่างๆ ออกมาตามชนิดคำสั่งที่อ่านจาก opcode. โดยถ้าเป็นคำสั่ง R-type, การทำงานของ ALU จะขึ้นอยู่กับฟิลด์ funct, ดังนั้นแนวคิดคือ Control จะส่งสัญญาณบอก ALU Control ว่า ต้องการทำ การบวก (เมื่อ opcode เป็น `lw` หรือ `sw`) หรือ ทำการลบ (เมื่อ opcode เป็น `beq`), หรือ ให้ ALU Control ดูว่าจะทำอะไรจากฟิลด์ funct (เมื่อ opcode เป็นคำสั่ง R-type). การทำแบบนี้ (1) วงจร Control จะซับซ้อนน้อยลง (2) มีอิสระมากขึ้นในการปรับเปลี่ยน ALU เพราะ การเปลี่ยน interface ของ ALU จะกระทบแค่ การเปลี่ยน ALU Control ไม่กระทบกับ วงจร Control หลัก. เรียกว่า มี abstraction ดีขึ้น (หรือ modularization ดีขึ้นก็ได้).

4.1.5 วงจรควบคุม

ก่อนอื่นเราดู วงจร ALU Control ก่อน, ALU Control ทำหน้าที่สั่งงาน ALU โดยตรง. ตารางข้างล่าง แสดงสัญญาณ และ การทำงานที่ต้องการ,

Table 4.1: สัญญาณควบคุม ALU และปฏิบัติการ

ALU Operation		Carry in	
Binvert	Operation	of the LSB	ปฏิบัติการ
0	00	0	AND
0	01	0	OR
0	10	0	add
1	10	1	subtract
1	11	1	set on less than

หมายเหตุว่า ALU ทำคำสั่ง set on less than (ค่าเอาต์พุตจะเป็น 1 เมื่อ rs < rt) โดย ทำการลบ (rs - rt) แล้ว นำค่า MSB ของผลลบ (ถ้า rs < rt, แล้ว MSB จะเป็น 1) มาเป็นคำตอบ (ดูหัวข้อ 3.5).

อย่างที่คุยกันไป เราจะแยกวงจรควบคุม ออกเป็น 2 ส่วนคือ main control และ ALU control โดย ALU control จะสั่งงาน ALU ว่าจะให้ทำ operation อะไร และ main control ทุกอย่างที่เหลือ รวมถึงการสั่งงาน ALU control ด้วย. การจัดรูปแบบวงจรควบคุมลักษณะนี้ เราจะเรียกว่า ``multiple levels of decoding'' คือ main control จะส่งสัญญาณ (ตั้งชื่อว่า ALUop)

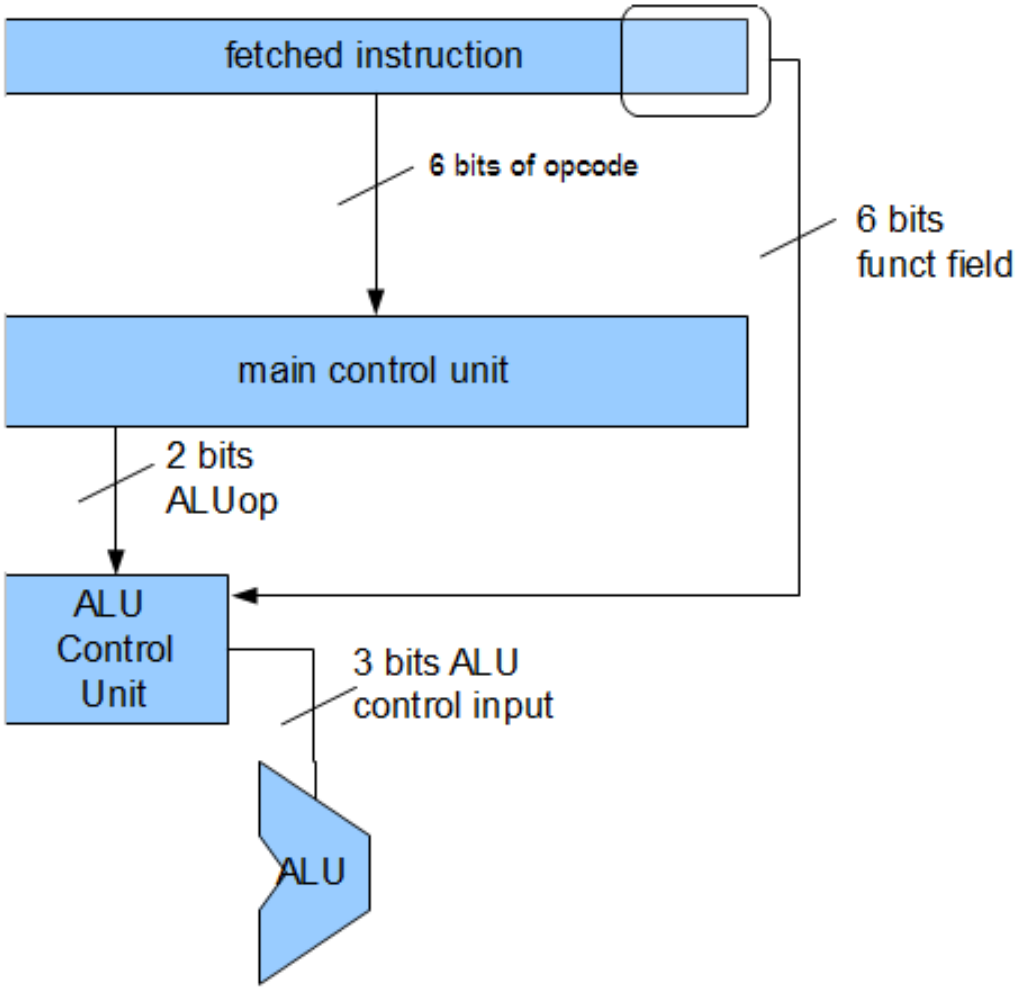


Figure 4.17: Multiple levels of decoding ด้วย main control และ ALU control

ไปควบคุม ALU control unit อีกที โดย ALU control unit จะเป็นอุปกรณ์เดียวที่สั่งงาน ALU โดยตรง.

การใช้ multiple levels of control จะช่วยลดขนาดของ main control unit. การทำแบบนี้มีแนวโน้มที่จะทำให้วงจรควบคุมทำงานได้เร็วขึ้น. และความเร็วของ control unit สำคัญและมีผลมากต่อความเร็วโดยรวมของ processor. รูป 4.17 แสดงการแบ่งวงจรควบคุมออกเป็น 2 ส่วน: main control กับ ALU control.

ตาราง 4.2 แสดงคำสั่งและสัญญาณที่เกี่ยวข้อง สำหรับวงจร ALU Control. สัญญาณ ALUop คือ สัญญาณควบคุมโหมดการทำงาน ที่มาจาก main control โดยสำหรับคำสั่ง `lw`, `sw` จะสั่งให้อยู่ในโหมดทำการบวกค่า (ALUop=00); `beq` จะสั่งให้อยู่ในโหมดลบค่า (ALUOp=01); และสำหรับคำสั่ง R-type (opcode=0x00) จะสั่งให้อยู่ในโหมดดูค่าจากฟิลด์ funct (ALUop=10). สังเกตุว่า คำสั่ง `lw`, `sw`, และ `beq` ไม่มีการใช้ฟิลด์ funct และ ALUOp ก็บอกปฏิบัติการของ ALU ได้แล้ว ดังนั้นจึงไม่สนใจค่า 6 บิต LSB ของคำสั่ง (funct) และใช้สัญญลักษณ์ *xxxxxx* แทน ``don't care'' หรือ ไม่สนใจ (ไม่ว่าค่าเหล่านี้จะเป็นอะไร ก็ไม่มีผลกับเอาต์พุต). สัญญาณ [Binv, operation] (คอลัมน์สุดท้าย) คือ สัญญาณเอาต์พุตจาก ALU Control Unit ที่จะเข้าไปสั่งงาน ALU (ดูตาราง 4.1).

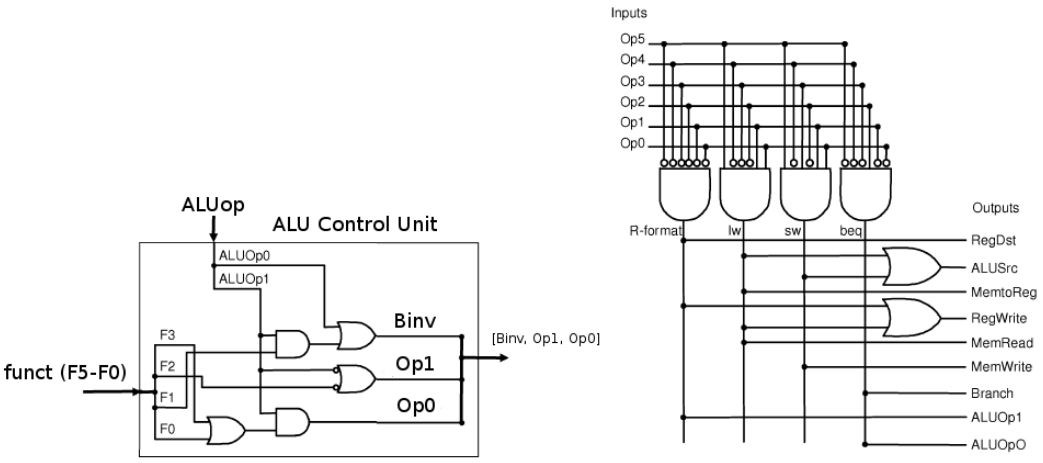


Figure 4.18: ค่าจากส่วนของ opcode ของคำสั่ง ใช้เป็นอินพุตของ main control (ภาพขวา) ที่ให้เอาต์พุตเป็นสัญญาณควบคุม. สัญญาณควบคุม ALUop (ALUOp1, ALUOp0) จาก main control จะจ่ายไปเป็นอินพุตให้กับวงจร ALU Control. วงจร ALU Control (ภาพซ้าย) จะควบคุมปฏิบัติการของ ALU ด้วยสัญญาณ Binv และ Operation (Op1, Op0).

คอลัมน์ ALUOp กับ funct เป็นอินพุต ของวงจร ALU Control Unit: 8-bit input. ส่วน [Binv, Operation] เป็นเอาต์พุต: 3-bit output. จากตารางตรรกะนี้นำมาสร้างเป็นวงจรตรรกะได้ดังภาพซ้ายมือของรูป 4.18. ภาพขวามือเป็นวงจรตรรกะของ main control unit ซึ่งได้จากการวิเคราะห์ความสัมพันธ์ระหว่าง opcode และ สัญญาณควบคุม ต่างๆ ดังที่เราจะดูกันต่อไป.

Table 4.2: คำสั่งตัวอย่างและสัญญาณที่เกี่ยวข้องกับวงจร ALU Control

คำสั่ง	คำสั่งเครื่อง (m/c code)						สัญญาณสำหรับ ALU Control (binary)			
	6	5	5	5	5	6	ALUop	funct	ปฏิบัติการ	[Binv, Op1, Op0]
`lw rd, offset(rs)`	0x23	rs	rd	offset			00	*xxxxxx*	add	010
`sw rd, offset(rs)`	0x2B	rs	rd	offset			00	*xxxxxx*	add	010
`beq rs, rt, offset`	0x04	rs	rt	offset			01	*xxxxxx*	subtract	110
`add rd, rs, rt`	0x00	rs	rt	rd	0x00	0x20	10	100000	add	010
`sub rd, rs, rt`	0x00	rs	rt	rd	0x00	0x22	10	100010	subtract	110
`and rd, rs, rt`	0x00	rs	rt	rd	0x00	0x24	10	100100	and	000
`or rd, rs, rt`	0x00	rs	rt	rd	0x00	0x25	10	100101	or	001
`slt rd, rs, rt`	0x00	rs	rt	rd	0x00	0x2A	10	101010	slt	111

ดูรูป 4.16 และพิจารณาความสัมพันธ์ระหว่างสัญญาณควบคุม และ เส้นทางข้อมูล. ตาราง 4.3 แจกแจงผลของสัญญาณควบคุมต่างๆ.

เมื่อวิเคราะห์ผลของสัญญาณควบคุมต่างๆ เราก็สามารถนำมาเขียนตารางตรรกะของอินพุตและเอาต์พุตของ main control ได้ดังตาราง 4.4 และภาพขวาของรูป 4.18 แสดง gate level ของวงจร main control. ในภาพไม่มีส่วน implementation ของคำสั่ง jump ซึ่งก็สามารถทำได้ในลักษณะเดียวกัน.

สาเหตุที่เราไม่ใช้ single-cycle machine ก็เพราะว่า clock cycle จะสั้นที่สุดได้เท่ากับเส้นทางข้อมูลที่ยาวที่สุด ดังนั้น clock cycle ของเครื่องจะถูกจำกัดด้วยคำสั่งที่ทำงานช้าที่สุด. ซึ่งขัด

Table 4.3: สัญญาณควบคุม และ ผลจากสัญญาณ

สัญญาณ	assert	effect when asserted	effect when deasserted
RegDst	0	เบอร์ของ Write Register จะระบุด้วย บิต 15-11 ของคำสั่ง เช่น **add rd, rs, rt** ซึ่ง **rd** อยู่ที่บิต 15-11	เบอร์ จะระบุด้วย บิต 20-16 ของคำสั่ง เช่น **lw rd, offset(rs)** ซึ่ง **rd** อยู่ที่บิต 20-16
RegWrite		ค่าของรีจิสเตอร์ตัวที่ระบุตาม Write Register จะเปลี่ยนเป็นค่าของ Write data	ไม่มีผลอะไร
ALUSrc	0	อินพุตที่สองของ ALU จะมาจาก sign-extended offset	อินพุตที่สองของ ALU จะมาจาก ค่ารีจิสเตอร์ตัวที่สอง (Read data 2)
MemRead		ค่าของหน่วยความจำตามแอดเดรส จะออกมาที่พอร์ต Read data ของหน่วยความจำ	ไม่มีผลอะไร
MemWrite		ค่าของหน่วยความจำตามแอดเดรสที่ระบุ จะถูกเปลี่ยนเป็นค่าที่พอร์ต Write data ของหน่วยความจำ	ไม่มีผลอะไร
MemtoReg	1	ค่าที่พอร์ต Write data ของรีจิสเตอร์ไฟล์จะมาจากหน่วยความจำ	ค่าจะมาจาก ALU result
Jump	1	ค่าแอดเดรสใหม่ของ PC จะมาจาก Jump address (ค่า 4-MSB จาก PC กับ shifted 26-bit target จากคำสั่งบิต 25-0)	ค่าจะมาจาก Branch MUX
Branch	1	ถ้า Zero = 1, ค่าเอาต์พุตของ Branch MUX จะเป็น PC + shifted sign-extended 16-bit offset ถ้า Zero = 0, ค่าจะเป็น PC + 4	ค่าจะเป็น PC + 4.

Table 4.4: อินพุต และ เอาต์พุต ของวงจรควบคุมหลัก (main control)

อินพุต	เอาต์พุต								
opcode	RegDst	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	Jump	ALUop
R-type 000000	1	0	0	1	0	0	0	0	(r-type mode) 10
lw 100011	0	1	1	1	1	0	0	0	(add mode) 00
sw 101011	x	1	x	0	0	1	0	0	(add mode) 00
beq 000100	x	0	x	0	0	0	1	0	(subtract mode) 01
j 000010	x	x	x	0	0	0	x	1	*xx*

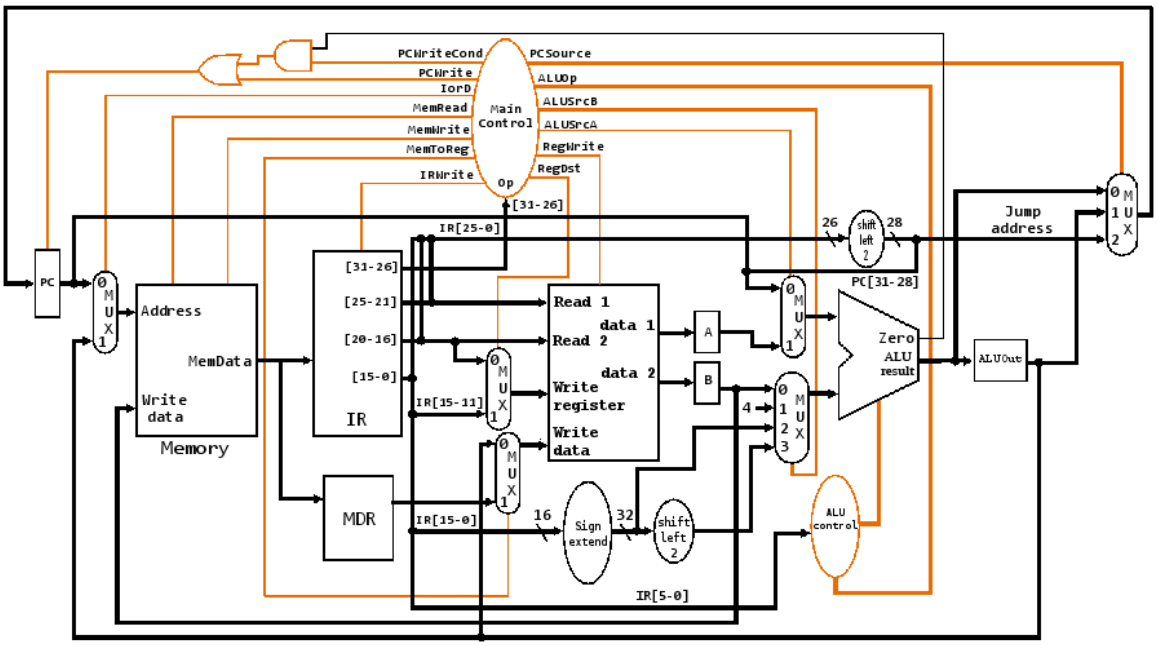


Figure 4.19: เส้นทางข้อมูลและสัญญาณควบคุม Multicycle Machine.

ต่อหลักการในการสร้างคอมพิวเตอร์ที่ดี ที่ต้องการทำให้คำสั่งที่ใช้เยอะที่สุดเร็ว (Make common cases fast).

4.2 Multi-Cycle Machines

แนวคิดคือ เราจะแบ่งการทำงานคำสั่ง ออกเป็นขั้นตอนย่อยๆ โดยแต่ละขั้นตอนจะต้องการ 1 clock cycle. แนวทางนี้ ทำให้เราสามารถใช้อุปกรณ์เดียวทำงานหลายอย่างได้ ถ้าเราใช้มันที่ clock cycle ต่างกัน เช่น เราอาจใช้หน่วยความจำในขั้นตอนที่หนึ่ง เพื่ออ่านค่าคำสั่ง และ ใช้หน่วยความจำเดิมในการอ่านค่าข้อมูลได้ ถ้าเราอ่านข้อมูลใน clock หลัง.

สำหรับตัวอย่าง multi-cycle machine ที่เราจะดู, เราสมมติว่า ข้อมูลต่างๆจาก หน่วยความจำ, รีจิสเตอร์ไฟล์, และ ALU จะถูกเก็บค่าไว้ในรีจิสเตอร์ชั่วคราวเพื่อใช้ใน clock cycle ต่อไป, ดังแสดงในรูป 4.19. รีจิสเตอร์ชั่วคราว ที่เราจะใช้ได้แก่ Instruction Register (IR), Memory Data Register (MDR), รีจิสเตอร์บัฟเฟอร์ค่า Read data 1 และ 2 (รีจิสเตอร์ A และ B), และ รีจิสเตอร์บัฟเฟอร์ค่าเอาต์พุตของ ALU (ALUOut). ยกเว้น IR, รีจิสเตอร์ชั่วคราวตัวอื่นๆ จะเก็บค่าไว้แค่ 1 clock cycle. ส่วน IR จะต้องเก็บค่าของ machine code จนกว่าจะจบการทำคำสั่ง และจะเปลี่ยนค่าที่ clock edge เมื่อเราจ่ายสัญญาณ IRWrite ให้.

จากรูป 4.19 และตาราง 4.20, สังเกตุว่า (1) PC เป็น edged-triggered[1]; (2) หน่วยความจำเป็น level-triggered[2].

[1] ถ้า PC ไม่เป็น edged-triggered แล้ว ใน step 1: `PC <= PC + 4` จะเกิด runnung value ขึ้น. ดังนั้น PC ต้องเป็น edged-triggered และเนื่องจากค่า `PC + 4` จะถูกใช้ใน step 2 ดังนั้น edge ที่ใช้จะต้องเป็น edge แรกของสัญญาณนาฬิกา เช่น rising edge.

[2] หน่วยความจำ อาจจะเป็น level-triggered ก็ได้ แต่ถ้าเป็น edged triggered ต้องใช้ edge หลังของสัญญาณนาฬิกา ไม่เช่นนั้นที่ step 4 ของคำสั่ง `lw` ค่า `MemData` ของแอดเดรส `ALUOut` จะออกมาไม่ทันให้กับ `MDR` หรือ step 4 ของคำสั่ง `sw` ค่าของรีจิสเตอร์ `B` ก็จะไม่ได้เข้าไปที่หน่วยความจำที่แอดเดรสจาก `ALUOut` ปัจจุบัน เพราะค่า

Step	Action for R-type instructions	Action for memory-reference instructions		Action for branches	Action for jumps
1	**IR <= Memory[PC]** **PC <= PC + 4** IorD = 0, MemRead, IRWrite ALUSrcA = 0, ALUSrcB = 01, ALUOp =00, PCSource = 00, PCWrite At clock event: * rising edge (starting the stage): * falling edge (in the middle of the stage): Memory output is ready.				
2	**A <= Reg[IR[25:21]]** **B <= Reg[IR[20:16]]** **ALUOut <= PC + (sign-extend (IR[15:0]) << 2)** ALUSrcA = 0, ALUSrcB = 11, ALUOp =00 At clock event: * rising edge: IR and PC values are ready. * falling edge: Reg values are ready.				
3	**ALUOut <= A op B** ALUSrcA = 1, ALUSrcB = 00, ALUOp = 10 At rising edge: A, B, ALUOut	**ALUOut <= A + sign-extend (IR[15:0])** ALUSrcA = 1, ALUSrcB =10, ALUOp = 00 At rising edge: A, B, ALUOut		**if (A == B)** **PC <= ALUOut** ALUSrcA = 1, ALUSrcB = 00, ALUOp = 01, PCSource = 01, PCWriteCond At rising edge: A, B, ALUOut	**PC <= {PC[31:28],(IR[25:0]],2'b00)}** PCSource = 10, PCWrite At rising edge: A, B, ALUOut
4	**Reg[IR[15:11]] <= ALUOut** RegDst = 1, MemtoReg = 0, RegWrite At rising edge: ALUOut At falling edge: Reg	**Load:** **MDR <= Memory[ALUOut]** IorD = 1 MemRead At rising edge: ALUOut At falling edge: Memory	**Store:** **Memory[ALUOut] <= B** IorD = 1 MemWrite At rising edge: ALUOut At falling edge: Memory	After stage 3, at the rising age of a new execution: PC value is ready.	After stage 3, at the rising age of a new execution: PC value is ready.
5		**Load:** **Reg[IR[20:16]] <= MDR** RegDst = 0, MemtoReg = 1, RegWrite At rising edge: MDR At falling edge: Reg			

Figure 4.20: สัญญาณควบคุมที่ขั้นตอนต่างๆ.

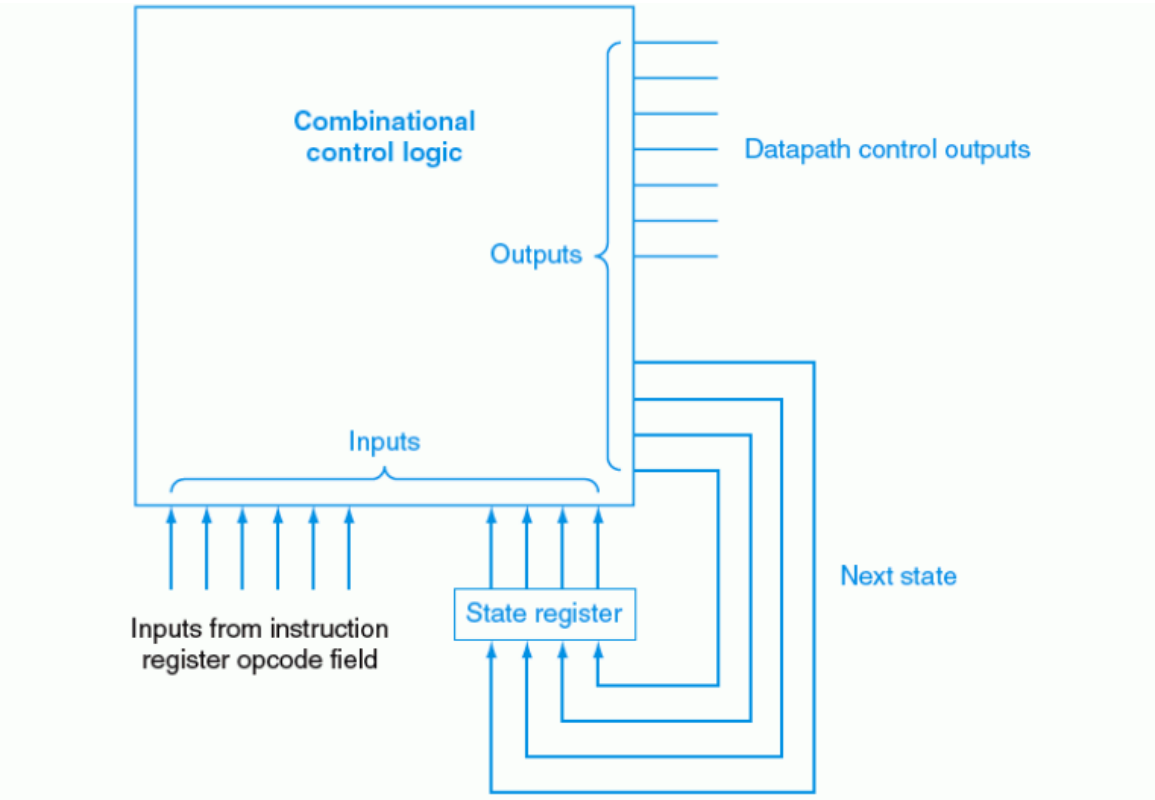


FIGURE 5.37 Finite state machine controllers are typically implemented using a block of combinational logic and a register to hold the current state.

Figure 4.21: วงจร combinational logic กับ state register.

รูป 4.20 แสดงตารางการเคลื่อนที่ของข้อมูล (ในพื้นที่แรเงาสีฟ้า), ค่าของสัญญาณควบคุมต่างๆ (นอกพื้นที่แรเงา), และ คำอธิบายเพิ่มเติมเกี่ยวกับค่าของอุปกรณ์ต่างๆที่จังหวะของสัญญาณนาฬิกา. คำอธิบายประกอบนี้จะใช้กับหัวข้อ 4.2.2.

การสร้างสัญญาณควบคุมที่เปลี่ยนตามจังหวะสถานะนี้ เราอาจใช้รีจิสเตอร์ประกอบกับ combinational control logic ดังแสดงในรูป 4.21. เอาต์พุตของวงจร combinational logic มีสัญญาณควบคุมต่างๆ (control outputs) และ สัญญาณที่บอกสถานะต่อไป (next state code). ซึ่งเมื่อมีสัญญาณนาฬิกาเข้ามาที่รีจิสเตอร์สถานะ (state register) ค่าเอาต์พุตของรีจิสเตอร์สถานะจะเปลี่ยนเป็นค่าสัญญาณของสถานะใหม่ ด้วยกลไกแบบนี้ เราก็สามารถสร้างวงจรควบคุมที่ค่าสัญญาณควบคุมขึ้นกับสถานะที่จังหวะเวลาต่างๆกันได้.

จากรูป 4.20 เราสามารถนำมาเขียนเป็นตารางตรรกะได้ดังแสดงในรูป 4.22. คอลัมน์ step และ opcode จะเป็นอินพุตของวงจร combinational control logic: state กับ opcode และคอลัมน์ตั้งแต่ `PCWriteCond` จนถึง `next state` เป็นเอาต์พุต: Datapath control outputs กับ Next state, ตามรูป 4.21. จากตารางในรูป เราก็สามารถออกแบบวงจรเพื่อสร้างสัญญาณควบคุมได้ เราจะดูตัวอย่างการสร้างวงจร และ ผลการจำลองการทำงานใน หัวข้อ 4.2.2.

แอดเดรสจาก `ALUOut` จะพร้อมหลังจาก edge แรกของสัญญาณนาฬิกาของ step 4. การอ่านหรือเขียนค่าหน่วยความจำ ต้องทำหลังจากค่าใน `ALUOut` พร้อมแล้วเท่านั้น.

step	instr.	opcode	PCWrite Cond	PCWrite	IorD	Mem Read	Mem Write	Mem toReg	IRWrite	PCSource	ALU Op	ALU SrcB	ALU SrcA	Reg Write	RegDst	next state
1	all	'xxxxxx	0	1	0	1	0	x	1	'00	'00	'01	0	0	x	2
2	all	'xxxxxx	0	0	x	0	0	x	0	'xx	'00	'11	0	0	x	3
3	r-type	'000000	0	0	x	0	0	x	0	'xx	'10	'00	1	0	x	4
3	lw/sw	'10x011	0	0	x	0	0	x	0	'xx	'00	'10	1	0	x	4
3	beq	'000100	1	0	x	0	0	x	0	'01	'01	'00	1	0	x	1
3	j	'000010	0	1	x	0	0	x	0	'10	'xx	'xx	x	0	x	1
4	r-type	'000000	0	0	x	0	0	0	0	'xx	'xx	'xx	x	1	1	1
4	lw	'100011	0	0	1	1	0	x	0	'xx	'xx	'xx	x	0	x	5
4	sw	'101011	0	0	1	0	1	x	0	'xx	'xx	'xx	x	0	x	1
5	lw	'100011	0	0	x	0	0	1	0	'xx	'xx	'xx	x	1	0	1
'x' (don't care) represents the logic whose value does not affect the processor functionality. All asserting signals are assumed to be active on '1'.																

Figure 4.22: ตารางตรรกะสถานะและสัญญาณควบคุม Multicycle Machine.

4.2.1 Exceptions

ส่วนของวงจรควบคุมที่ออกแบบได้ยากที่สุดคือ ส่วนที่ทำ exceptions และ interrupts. Exception คือ เหตุการณ์ที่ไม่ได้คาดหมาย ที่เกิดขึ้นภายในตัว processor เอง เช่นการคำนวณเกิด overflow ขึ้นเป็นต้น. ส่วน interrupt เป็นเหตุการณ์ที่ไม่ได้คาดหมาย ที่เกิดขึ้นภายนอกตัว processor เช่น สัญญาณ interrupt ที่มาจากอุปกรณ์ I/O ไป processor. หลายๆสถาปัตยกรรม จะไม่ได้แยกแยะความแตกต่างระหว่าง exception กับ interrupt และ อาจมองเหมือนกับว่า ทั้งคู่คือ สัญญาณของเหตุการณ์ไม่ได้คาดหมายที่เกิดขึ้น.

กลไกพื้นฐาน ในการจัดการกับ exception ก็คือ เมื่อมีสัญญาณ exception เกิดขึ้น คือ

- (1) เก็บค่าแอดเดรสของคำสั่งที่ทำให้เกิด exception ขึ้นใน `EPC`: `EPC <= (PC + 4) - 4`.
- (2) เปลี่ยนค่า PC เป็นแอดเดรสสงวนไว้เฉพาะสำหรับ exception.
- (3) หลังจากทำงานตาม exception handler ที่ระบุไว้เรียบร้อยแล้ว, ระบบปฏิบัติการ อาจจะปิดโปรแกรมนั้นไป หรือ อาจจะรันต่อ[3] โดยดูจากค่าแอดเดรสที่เก็บไว้ใน `EPC`.

เพื่อจะจัดการกับ exception ได้อย่างถูกต้อง, ระบบปฏิบัติการต้องรู้ว่า exception นั้นเกิดจากอะไร นอกจากแค่คำสั่งไหนที่ทำให้เกิด exception. มี 2 วิธีหลักๆที่ใช้ในการระบุเหตุของ exception: (1) ใช้ status register เช่น ใน MIPS จะใช้รีจิสเตอร์ Cause ในการเก็บ ค่า เพื่อระบุเหตุผลของการเกิด exception และ (2) ใช้ vectored interrupts, ซึ่งคือ ใช้ exception addresses ที่ต่างกันสำหรับ exceptions ต่างสาเหตุ. ตัวอย่าง vectored interrupts เช่น สมมติถ้าสาเหตุของ exception คือ arithmetic overflow จะให้ใช้ exception address ที่ `0xC000 0020`. แต่ถ้าสาเหตุคือ ไม่รู้จัก machine code ที่โหลดมา (undefined instruction) จะใช้ exception addresss ที่ `0xC000 0000`.

เปรียบเทียบรูป 4.23 กับรูป 4.19 จะเห็นว่ามีเส้นทางข้อมูลจาก `ALUOut` ไปที่ `EPC` และมีส่วนของ `EPC` และ `Cause` รีจิสเตอร์ที่เพิ่มขึ้น รวมทั้ง `MUX` ที่ป้อนอินพุตให้กับ `PC` ก็ขยายเป็นสี่

[3]เช่น โปรแกรมที่เตรียมจัดการกับ exception ไว้ก่อนแล้ว, `try/catch statement` ในภาษา C++ หรือ Java.

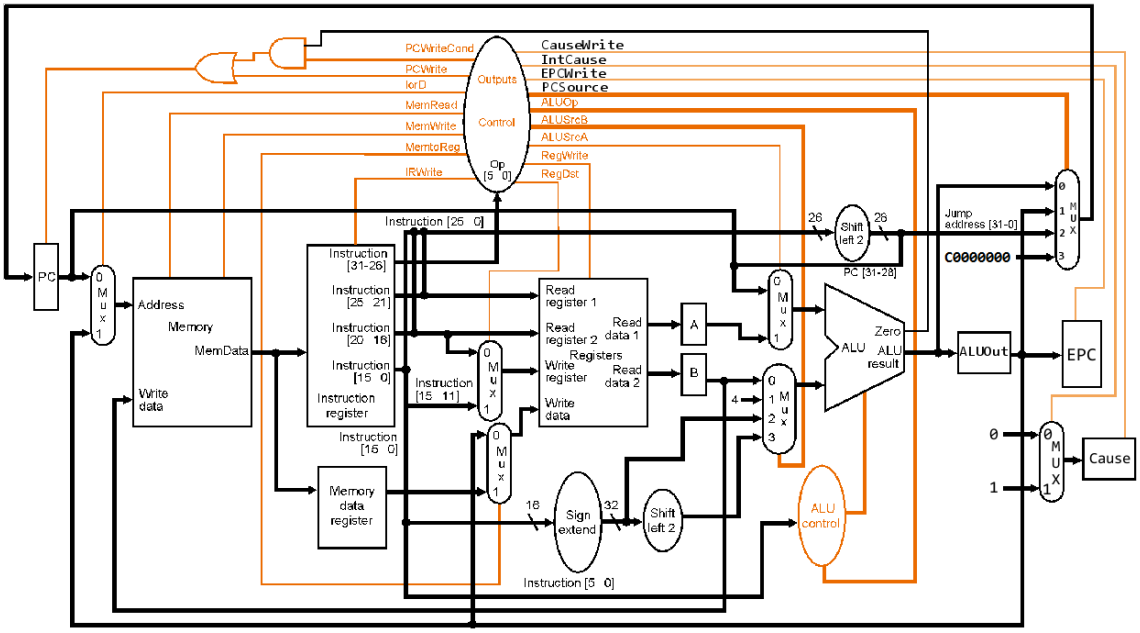


Figure 4.23: เส้นทางข้อมูลและสัญญาณควบคุมของ Multicycle Machine ที่ implementing Exception.

ออกหนึ่งเพื่อรองรับ exception address ที่ `0xC0000000`. วงจร main control ก็มีสัญญาณ CauseWrite, IntCause, EPCWrite ที่เพิ่มขึ้นมา. รูป 4.24 แสดง finite state machine สำหรับรูป 4.23.

4.2.2 ตัวอย่าง Multicycle Machine

``The worthwhile problems are the ones you can really solve or help solve, the ones you can really contribute something to. ... No problem is too small or too trivial if we can really do something about it." - Letter from Feynman to Koichi Mano (Feb 3rd, 1996).

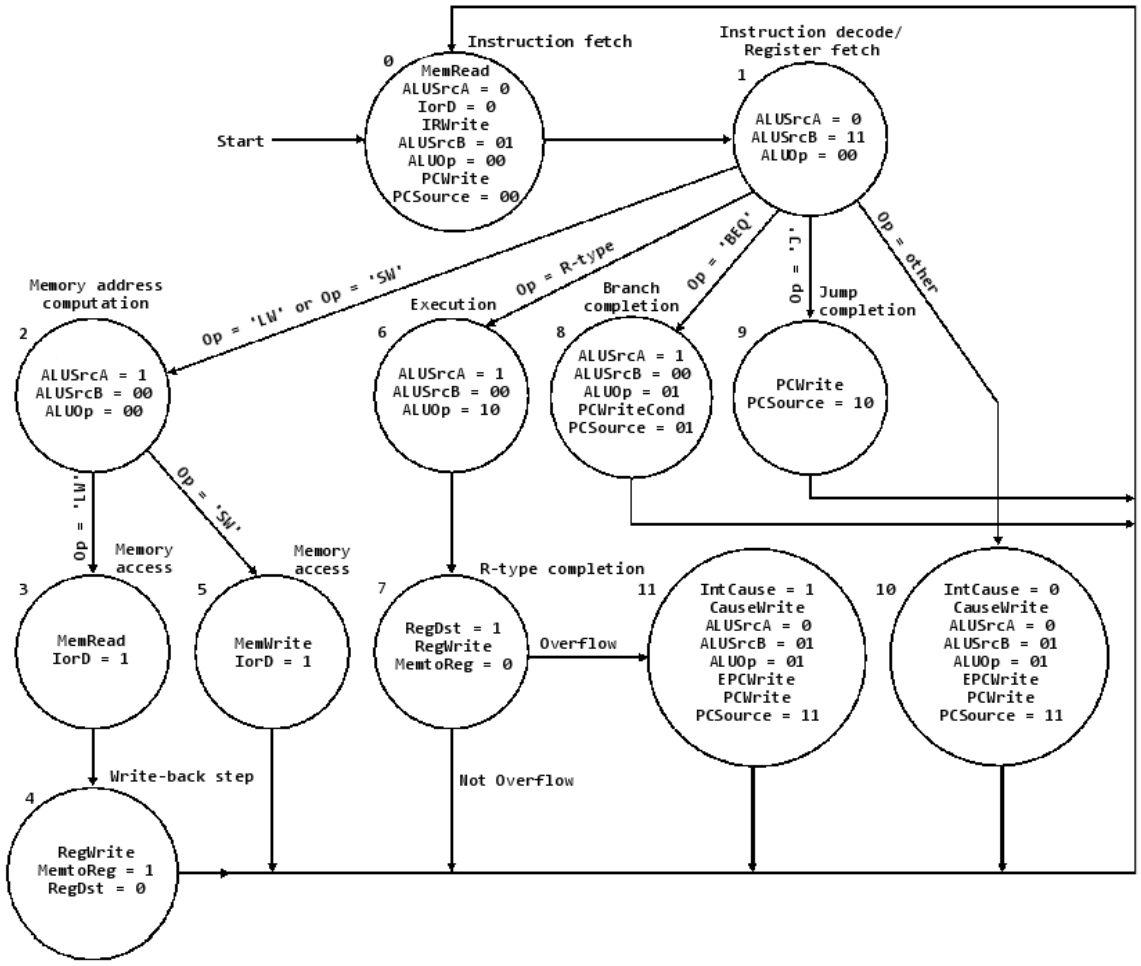


Figure 4.24: Finite state machine เพื่อรองรับ exception.

Chapter 5

Pipelining

Pipelining เป็นวิธี implementation ที่อนุญาติให้มีการรันคำสั่งหลายๆคำสั่งเหลื่อมล่ำกันได้. เราจะเริ่มจากดูตัวอย่างง่ายๆของห้องซักผ้าก่อน: ขั้นตอนในห้องซักผ้า จะเป็น

- 1. ใส่ โหลด (ผ้าที่สกปรก) ลงในเครื่องซักผ้า.
- 2. เมื่อเครื่องซักผ้าซักเสร็จแล้ว นำโหลด (ผ้าเปียกที่ซักเสร็จแล้ว) ไปในเครื่องปั่นแห้ง
- 3. เมื่อเครื่องปั่นแห้งปั่นเสร็จแล้ว, นำโหลด (ผ้าแห้ง) ไปวางบนโต๊ะเพื่อพับ.
- 4. เมื่อพับเสร็จ นำผ้าไปเก็บในตู้.

รูป 5.1 แสดงตัวอย่างเปรียบเทียบ ห้องซักผ้า. สังเกตุว่า เวลาในการซักผ้าแต่ละโหลดไม่ได้สั้นลง แต่เวลารวมที่ห้องซักผ้าบริการทุกโหลดเสร็จสั้นลง. นั้นเป็นเพราะ มีการทำงานแบบเหลื่อมล้ำกัน. Pipelining ทำให้ throughput ของระบบดีขึ้น แต่ไม่ได้ทำให้เวลาในการทำงานแต่ละโหลดดีขึ้น.

ถ้าทุกขั้นตอน ใน pipelining ใช้เวลาเท่าๆกัน (และมีงานทำตลอด), pipelining จะสามารถเพิ่มความเร็วได้ราวๆเท่ากับจำนวนขั้นตอน. เช่น จากตัวอย่างห้องซักผ้า ถ้าไม่ทำ pipelining ให้ 2 ชั่วโมงจะบริการลูกค้าเสร็จแค่ 1 ราย แต่ด้วย pipelining ห้องซักผ้าบริการลูกค้าเสร็จได้ถึง 4 ราย (ดูที่เวลา 7:30-9:30 ในรูป 5.1). ข้อสังเกตุนี้จะเป็นจริง ถ้าเราไม่นับช่วง start-up (เช่น ก่อน 7:30) ที่โหลดเพิ่งถูกทำ หรือ ช่วง wind-down (เช่น หลัง 9:30) ที่โหลดเริ่มหมด. โดยเฉพาะถ้าจำนวนขั้นตอนมีมากๆ เมื่อเทียบกับ ช่วง start-up หรือ wind-down จะทำให้ pipelining สามารถเพิ่มความเร็วได้เกือบๆเท่ากับจำนวนขั้นตอน.

แนวคิดเดียวกันนี้ก็สามารถนำมาใช้กับ processor ได้, โดยสำหรับ MIPS จะแบ่งขั้นตอนต่างๆ ในการทำคำสั่งออกเป็น 5 ขั้นตอน:

- 1. Fetch คำสั่งจากหน่วยความจำ
- 2. Decode คำสั่ง และ อ่านค่ารีจิสเตอร์ ในขณะเดียวกัน
- 3. ทำการคำนวณด้วย ALU
- 4. Access หน่วยความจำ
- 5. เขียนค่ากลับเข้ารีจิสเตอร์.

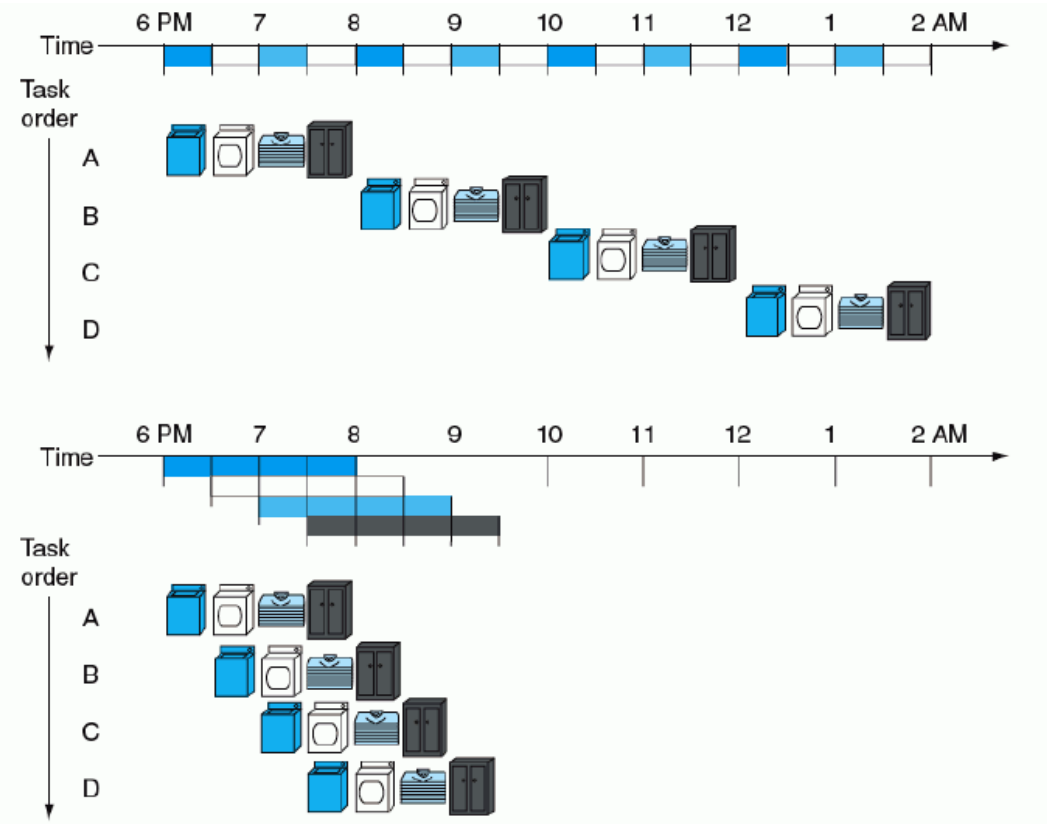


FIGURE 6.1 The laundry analogy for pipelining. Ann, Brian, Cathy, and Don each have dirty clothes to be washed, dried, folded, and put away. The washer, dryer, "folder," and "storer" each take 30 minutes for their task. Sequential laundry takes 8 hours for four loads of wash, while pipelined laundry takes just 3.5 hours. We show the pipeline stage of different loads over time by showing copies of the four resources on this two-dimensional time line, but we really have just one of each resource.

Figure 5.1: ตัวอย่างเปรียบเทียบ ห้องซักผ้า.

Instruction class	Instruction fetch	Register read	ALU operation	Data access	Register write	Total time
Load word (lw)	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps
Store word (sw)	200 ps	100 ps	200 ps	200 ps		700 ps
R-format (add, sub, and, or, slt)	200 ps	100 ps	200 ps		100 ps	600 ps
Branch (beq)	200 ps	100 ps	200 ps			500 ps

FIGURE 6.2 Total time for each instruction calculated from the time for each component. This calculation assumes that the multiplexors, control unit, PC accesses, and sign extension unit have no delay.

Figure 5.2: ตัวอย่างเปรียบเทียบ สมถนะของ single-cycle กับ pipelining.

ตัวอย่าง: Single-cycle v.s. Pipelined Performance เราจะดู Single-cycle machine ที่ใช้ 1 clock cycle ในการทำคำสั่ง เปรียบเทียบกับ 5-stage pipelining. สมมติเวลาที่ใช้กับอุปกรณ์หลักๆ คือ 200 ps สำหรับหน่วยความจำ, 200 ps สำหรับ ALU, 100 ps สำหรับรีจิสเตอร์. สำหรับ Single-cycle machine, clock cycle จะต้องนานพอที่สัญญาณสำหรับคำสั่งที่ช้าที่สุดจะวิ่งได้ทัน.

รูป 5.3 แสดงตารางเวลาที่ต้องการสำหรับคำสั่งต่างๆ โดยสำหรับ single-cycle machine คาบเวลาของสัญญาณนาฬิกา จะต้องทำตามคำสั่งที่ช้าที่สุด ก็คือ `lw` ที่ 800 ps. สำหรับ pipelining, clock cycle จะสั้นที่สุดได้เท่ากับเวลาขั้นตอนที่ช้าที่สุด ก็คือ 200 ps. รูป 5.3 แสดง เวลาที่ใช้รันคำสั่งเปรียบเทียบระหว่าง single-cycle machine กับ pipelining. Single-cycle machine จะใช้เวลาระหว่างโหลดคำสั่งที่ 1 กับ คำสั่งที่ 4 เป็น $3 \times 800 = 2400$ ps. ขณะที่ pipelining จะใช้เวลาระหว่างโหลดคำสั่งที่ 1 กับ คำสั่งที่ 4 เป็น $3 \times 200 = 600$ ps.

เราสามารถประมาณเวลาของเครื่องที่ทำ pipelining ในกรณีที่ ขั้นตอนต่างๆใช้เวลาพอๆกันได้

$$\text{Time between instructions}_{\text{pipelined}} = \frac{\text{Time between instructions}_{\text{nonpipelined}}}{\text{Number of pipe stages}}.$$

เช่น จากตัวอย่างข้างต้น ถ้าเวลาในทุกขั้นตอนเท่ากัน เป็น 200 ps, เวลาระหว่างโหลดคำสั่งที่ 1 กับ 2 สำหรับ single-cycle machine จะเป็น 1000 ps ในขณะที่ pipelining จะใช้ 200 ps หรือ $1/5$ เท่า.

สมการ 5 บอกว่า 5-stage pipeline น่าจะทำให้คอมพิวเตอร์เร็วขึ้นได้ 5 เท่า แต่ในทางปฏิบัติแล้ว เวลาแต่ละขั้นตอนอาจจะไม่พอ (stages are imbalanced) หรือ อาจจะมี overhead ทำให้ไม่ได้ความเร็วตามสูตรนี้.

หรือ แม้แต่หากเราพิจารณาเวลาที่ทำสามคำสั่งในรูป 5.3 เสร็จ ซึ่งคือ 2400 ps จาก single-cycle machine เทียบกับ 1400 ps จาก pipelining, จะพบว่าอัตราส่วนแย่กว่า 5 เท่ามาก. ที่เป็นเช่นนี้เพราะ จำนวนคำสั่งไม่มากเท่าไร. สมมติ เรารัน 1,000,003 คำสั่ง: เวลา 3 คำสั่งเดิม บวกกับเวลาในการรันเพิ่มอีก 1,000,000 คำสั่ง $= 1400 + 1000000 \times 200$ ps $= 200001400$ ps เมื่อทำ pipelining เปรียบเทียบกับ $2400 + 1000000 \times 800$ ps $= 800002400$ ps เมื่อไม่ทำ pipelining. และ สมถนะ pipelining เทียบกับ nonpipelining จะได้ $800,002,400/200,001,400 \approx 4$ เท่า.

แนวคิดของ pipelining คือ เพิ่มสมถนะของคอมพิวเตอร์ ด้วย การเพิ่ม instruction throughput, แทนที่จะลดเวลา execution ของแต่ละคำสั่ง. และเนื่องจากโปรแกรมส่วนใหญ่รันคำสั่ง

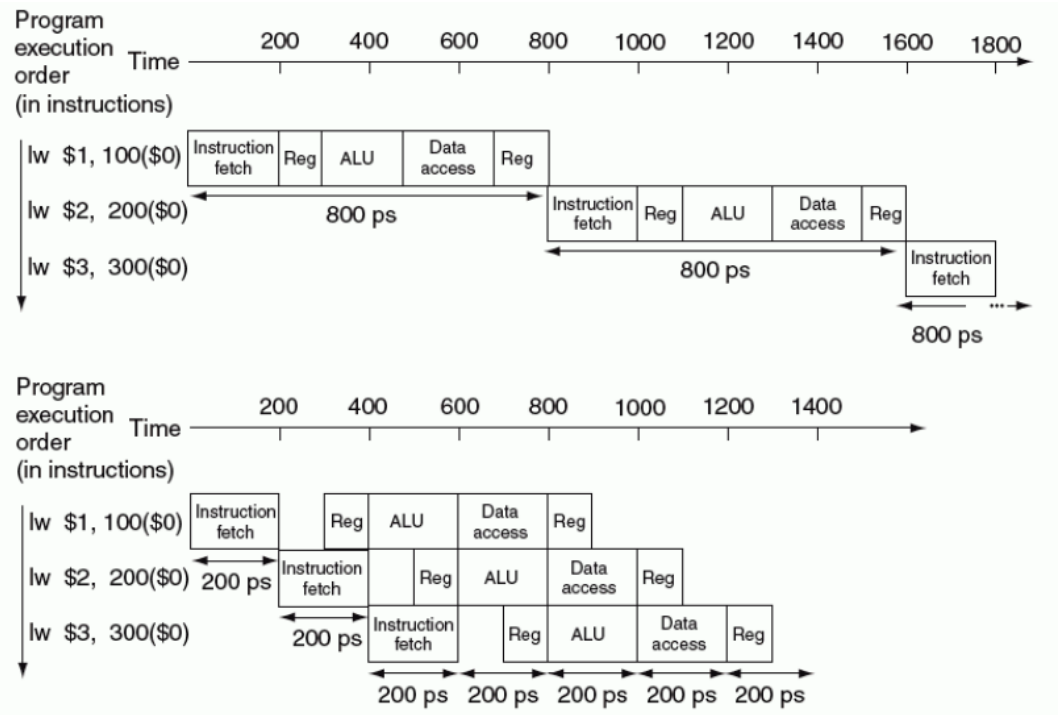


FIGURE 6.3 Single-cycle, nonpipelined execution in top versus pipelined execution in bottom.

Figure 5.3: ตัวอย่างเปรียบเทียบ สมถนะของ single-cycle กับ pipelining.

จำนวนมหาศาล ทำให้ การเพิ่ม instruction throughput ช่วยลดเวลาในการรันโปรแกรมลงได้.

Designing Instruction Sets for Pipelining (หนึ่ง) สังเกตุว่า คำสั่งของ MIPS จะมีความยาวเท่ากันหมด (32 บิต). การที่เลือกทำเช่นนี้ ทำให้ง่ายที่เราจะ fetch คำสั่งในขั้นตอนแรกของ pipeline และ decode มันในขั้นตอนที่สอง. สำหรับคอมพิวเตอร์ที่ความยาวของคำสั่งต่างกัน เช่น x86 ที่คำสั่งมีความยาวตั้งแต่ 1 ไบต์ ถึง 17 ไบต์, การทำ pipelining จะท้าทายมาก. ล่าสุด implementation ของสถาปัตยกรรม x86 จะแปลงคำสั่ง x86 ออกเป็น ปฏิบัติการง่ายๆ (ลักษณะคล้ายๆคำสั่งของ MIPS) และก็ทำ pipelining บนปฏิบัติการง่ายๆเหล่านั้น แทนที่จะทำกับคำสั่ง x86 โดยตรง.

(สอง) MIPS มีรูปแบบคำสั่งไม่กี่รูปแบบ ทำให้ง่ายในการจัด pipeline เช่น register fields จะอยู่ที่เดิมตลอด ทำให้ เราสามารถจัดให้ pipelining stage 2 ทำ decode กับอ่านรีจิสเตอร์ไปพร้อมๆกันได้.

(สาม) MIPS ใช้แค่ load กับ store ในการอ่านเขียนหน่วยความจำ ทำให้เราจัด pipelining stages โดย ให้ทำ execute stage เพื่อคำนวณค่าแอดเดรส แล้วต่อด้วย memory stage ทีหลังได้. ถ้า MIPS ยอมให้มีคำสั่งใช้หน่วยความจำเป็น operands แบบ x86 จะทำให้เราต้องเพิ่ม execute stage อีกขั้นไปหลังจาก memory stage ด้วย.

(สี่) คำสั่ง MIPS ทำงานกับหน่วยความจำ 1 อย่างต่อคำสั่งเท่านั้น (ไม่มีการเข้าถึงหน่วยความจำ 2 ที่ในคำสั่งเดียวกัน) ดังนั้น เราจึงต้องการแค่ 1 pipeline stage สำหรับการเข้าถึงหน่วยความจำ.

รูป 5.4 แสดงตัวอย่างเส้นทางสัญญาณข้อมูล เมื่อทำ pipelining. แต่เนื่องจากอุปกรณ์ที่ทำขั้นตอนของคำสั่งหนึ่งเสร็จ จะต้องเปลี่ยนไปทำขั้นตอนเดิมของคำสั่งถัดที่ clock ถัดไป ดังนั้น

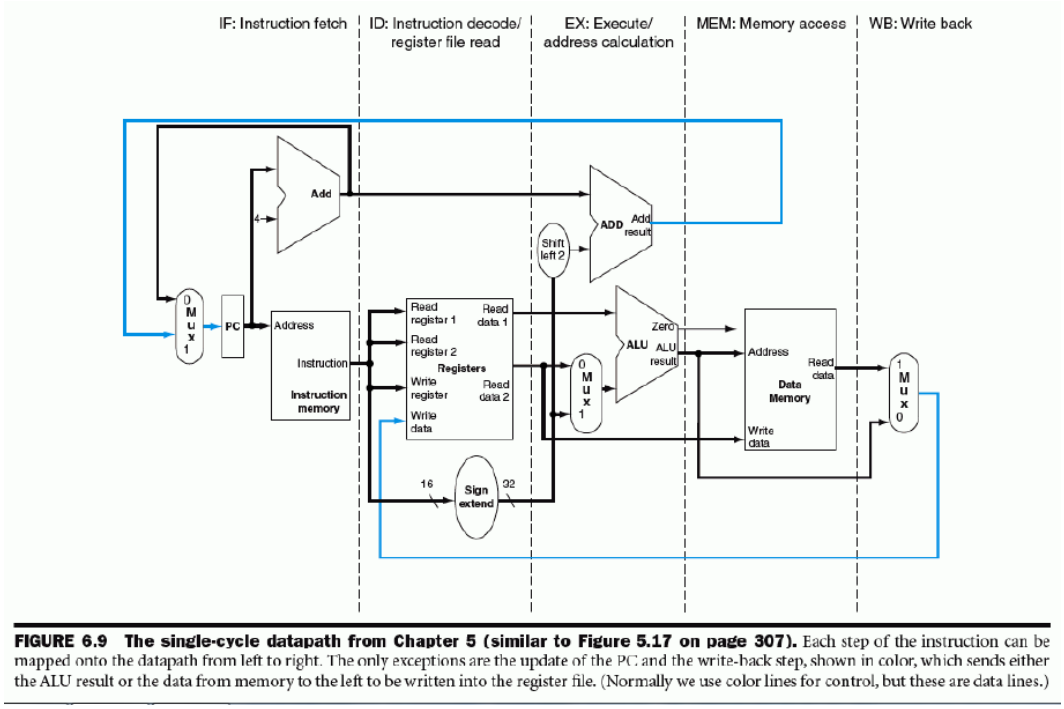


FIGURE 6.9 The single-cycle datapath from Chapter 5 (similar to Figure 5.17 on page 307). Each step of the instruction can be mapped onto the datapath from left to right. The only exceptions are the update of the PC and the write-back step, shown in color, which sends either the ALU result or the data from memory to the left to be written into the register file. (Normally we use color lines for control, but these are data lines.)

Figure 5.4: ตัวอย่างของเส้นทางสัญญาณข้อมูล เมื่อทำ pipelining

เราต้องการบัฟเฟอร์เพื่อเก็บค่าที่ได้ไว้แทน ดังแสดงในรูป 5.5. ตัวอย่างเช่น ที่ clock 1 เราจะ fetch คำสั่งแรกออกมา (อ่านค่าจาก Instruction Memory) และ บัฟเฟอร์ค่า machine code นี้ไว้ และ ที่ clock 2, ค่า machine code นี้ไว้ จะถูกใช้เป็นอินพุตให้กับอุปกรณ์ต่างๆในขั้นตอนที่ 2 ของคำสั่งที่ 1. ในขณะที่ Instruction Memory ถูกอ่านค่า machine code ของคำสั่งถัดไปออกมา. โดยที่ค่าที่อ่านออกมาใหม่นี้จะไม่เข้าไปกวนในส่วนของการทำ instruction decode (ID) เพราะมี buffer กั้นอยู่.

5.1 Pipeline Hazards

มีสถานะการณ์บางอย่างสำหรับ pipelining ที่เราไม่สามารถรัน คำสั่งหน้า ที่ clock ต่อไปได้. สถานะการณ์เหล่านี้ จะเรียกว่า ``hazards'': ซึ่งมีอยู่ 3 ชนิด.

5.1.1 Structural Hazards

Structural Hazards เกิดขึ้นเมื่อฮาร์ดแวร์ไม่สามารถสนับสนุนการทำงานของ stages ของคำสั่งต่างๆที่ต้องการทำ ณ clock cycle เดียวกันได้.

ตัวอย่างเช่น ถ้าเรามีหน่วยความจำอยู่ชุดเดียว และทำ pipelining จังหวะที่เราต้องการ ทำ IF (Instruction Fetch) ตรงกับจังหวะที่เราต้องการอ่านค่าหน่วยความจำ ดังแสดงในรูป 5.6.

5.1.2 Data Hazards

Data hazards เกิดขึ้นเมื่อข้อมูลที่ต้องการที่จะใช้ในการทำคำสั่งยังไม่พร้อม. ตัวอย่างเช่น เราต้องการทำ

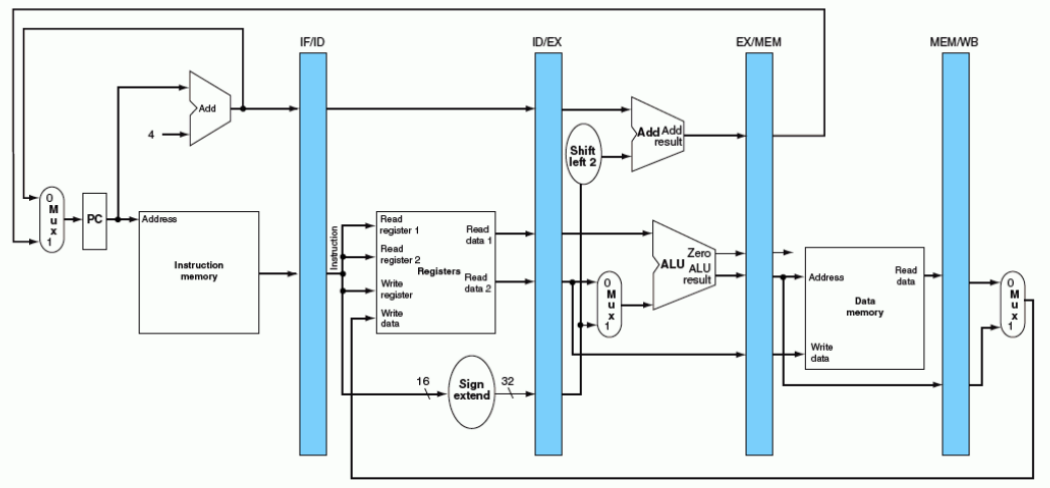


FIGURE 6.11 The pipelined version of the datapath in Figure 6.9. The pipeline registers, in color, separate each pipeline stage. They are labeled by the stages that they separate; for example, the first is labeled *IF/ID* because it separates the instruction fetch and instruction decode stages. The registers must be wide enough to store all the data corresponding to the lines that go through them. For example, the IF/ID register must be 64 bits wide because it must hold both the 32-bit instruction fetched from memory and the incremented 32-bit PC address. We will expand these registers over the course of this chapter, but for now the other three pipeline registers contain 128, 97, and 64 bits, respectively.

Figure 5.5: นอกจากจัดเส้นทางและเตรียมอุปกรณ์ในพอแล้ว เราต้องการบัฟเฟอร์ไว้เก็บค่าจากแต่ละขั้นตอนด้วย เพื่อที่อุปกรณ์จะได้นำไปใช้อย่างอื่นต่อได้

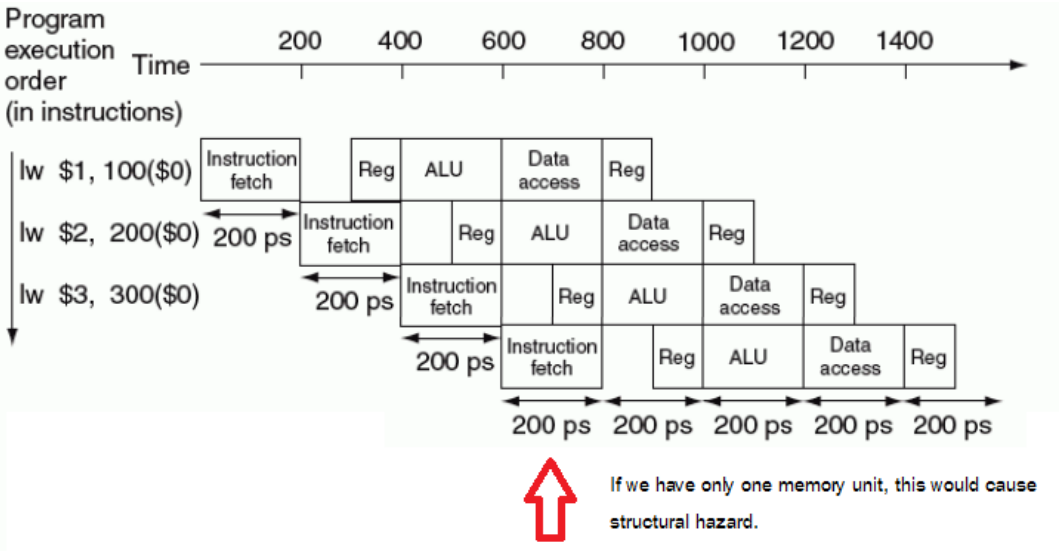


Figure 5.6: ถ้าเรามีหน่วยความจำชุดเดียว ที่เวลา 600 ถึง 800 ps จะเกิด structural hazard ขึ้นเพราะเราต้องการใช้ฮาร์ดแวร์ตัวเดียวกันทำงานคนละอย่างใน clock เดียวกัน: หน่วยความจำอ่านค่าคนละที่.

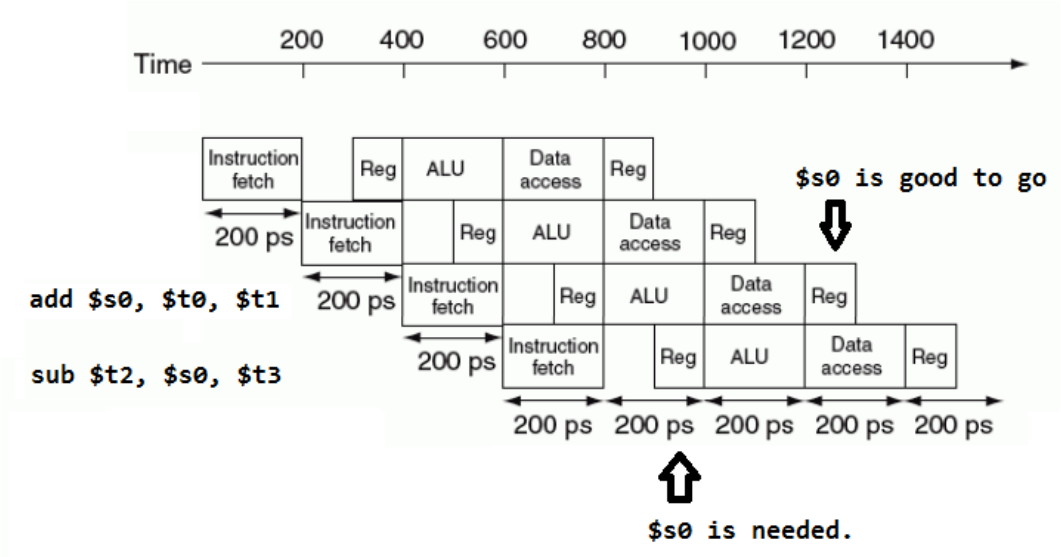


Figure 5.7: ตัวอย่างแสดง data hazard.

```
add $s0, $t0, $t1
sub $t2, $s0, $t3
```

คำสั่ง `add` จะยังไม่เก็บค่าจากการคำนวณลง `$s0` จนกว่าจะถึงขั้นตอนที่ 5. ในขณะที่คำสั่งต่อไปต้องการค่าจากการคำนวณ (ใน `$s0`) ก่อนที่ผลจากการคำนวณคำสั่ง `add` จะเก็บเสร็จ.

วิธีแก้ hazard วิธีแก้ hazard ที่ง่ายที่สุด ก็คือ ชะลอการทำคำสั่งต่อไป หรือ เทียบเท่ากับ การหยุด pipelining ชั่วคราว, เราเรียกว่า stall. รูป 5.8 แสดงการทำ stall. จะเห็นว่าแม้ stall เป็นวิธีที่ตรงไปตรงมา แต่การทำ stall มากๆ ก็เท่ากับว่าเราลดสมถนะของ pipelining ลงไป.

แทนที่เราจะใช้ stall แก้ปัญหา ซึ่งทำให้เราเสียประสิทธิภาพไป เราจะใช้วิธี forwarding หรือ bypassing, โดยการเพิ่มเส้นทางข้อมูล เพื่อใช้ค่าจากบัฟเฟอร์ของผลลัพธ์ของ ALU เข้ามาเป็นอินพุตใน clock ต่อไปเลย ไม่ต้องรอค่าไปเก็บเข้ารีจิสเตอร์ และ อ่านจากรีจิสเตอร์อีกที. รูป 5.9 แสดงเส้นทางข้อมูลที่เพิ่มเติม เพื่อทำ forwarding. รูป 5.10 แสดงขั้นตอนต่างๆ ของ 2 คำสั่ง และ การทำ forwarding จะช่วยให้ pipelining ทำงานต่อไปได้โดยไม่ต้องหยุด.

หมายเหตุ ค่าของ operand ใน MIPS จะเป็น รีจิสเตอร์ ซึ่งถูกเขียนได้จาก 2 กรณี: (1) ผลลัพธ์จาก ALU และ (2) ค่าที่โหลดจากหน่วยความจำ. แม้ forwarding จะแก้ปัญหา dependency ที่ค่า operand มาจากผลลัพธ์ ALU ของคำสั่งก่อนได้โดยไม่ต้องทำ stall แต่กรณี เช่น

```
lw $s0, 20($t1)
sub $t2, $s0, $t3
```

ลักษณะเช่นนี้ ก็เป็น data hazard แต่เนื่องจากค่า operand จะต้องรอขั้นตอนการอ่านหน่วยความจำด้วย เราจะเหตุการณ์แบบนี้ว่า load-use data hazard. ซึ่งวิธีแก้เราสามารถทำ stall ประกอบกับ forwarding ดังแสดงในรูป 5.11.

เพื่อลด stall, คอมไพเลอร์อาจช่วยเราได้ โดยการทำ reordering code เช่น Code segment in C:

```
a = b + e;
c = b + f;
```

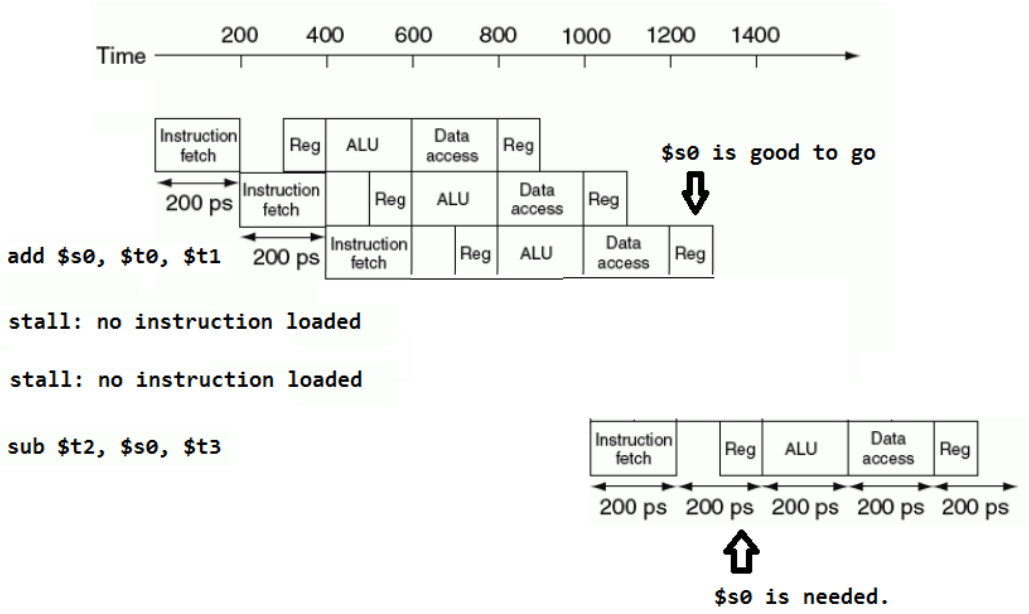


Figure 5.8: ตัวอย่างแสดง stall ในการแก้ hazard.

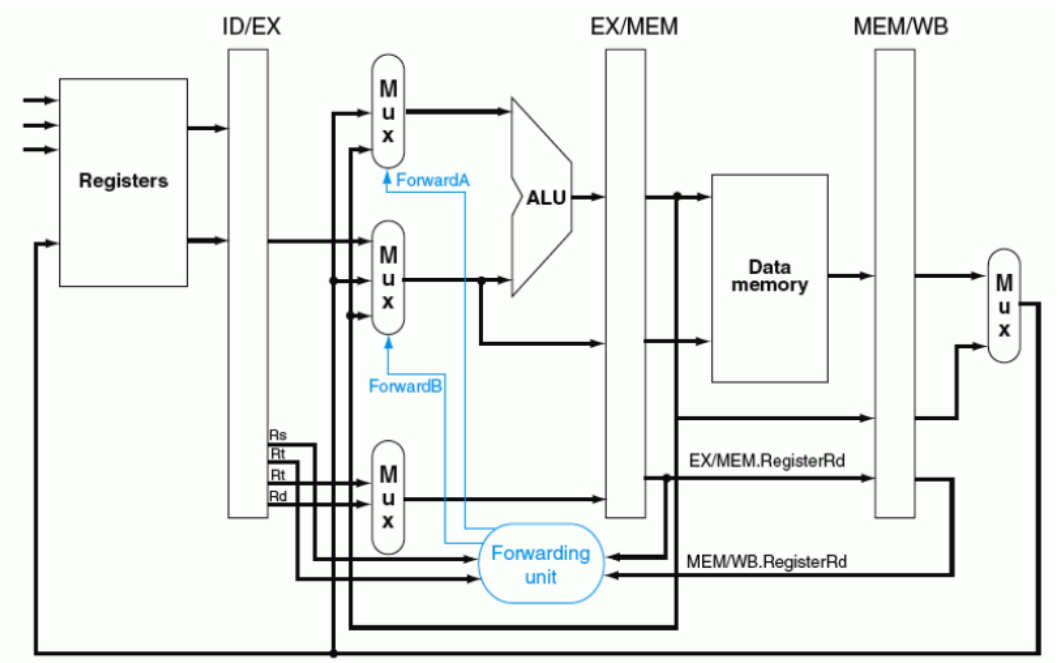


Figure 5.9: ตัวอย่างแสดง เส้นทางข้อมูลในการทำ forwarding เพื่อแก้ data hazard.

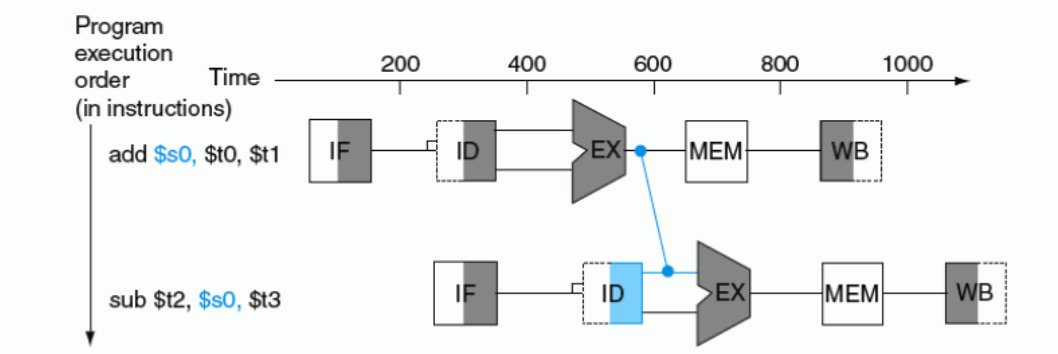


FIGURE 6.5 Graphical representation of forwarding. The connection shows the forwarding path from the output of the EX stage of `add` to the input of the EX stage for `sub`, replacing the value from register `$s0` read in the second stage of `sub`.

Figure 5.10: ไดอะแกรมแสดงขั้นตอนของ 2 คำสั่ง และ เส้นทางข้อมูลที่แก้ปัญหา data hazard.

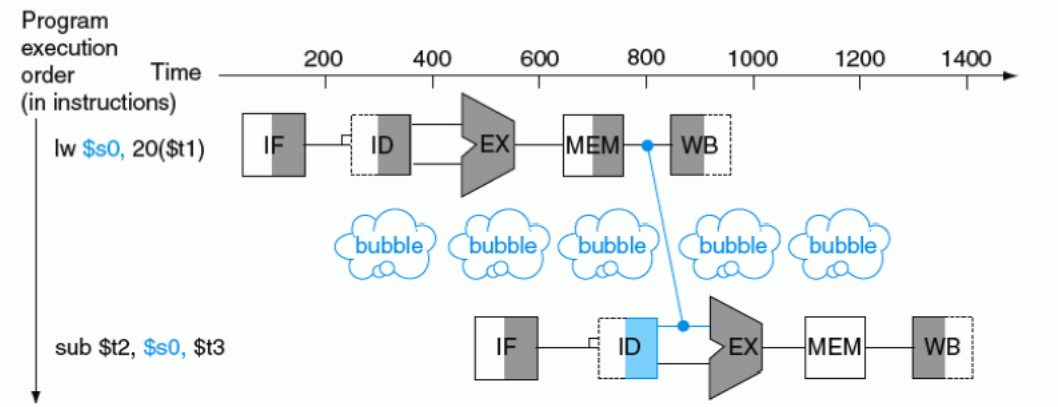


FIGURE 6.6 We need a stall even with forwarding when an R-format instruction following a load tries to use the data. Without the stall, the path from memory access stage output to execution stage input would be going backwards in time, which is impossible. This figure is actually a simplification, since we cannot know until after the subtract instruction is fetched and decoded whether or not a stall will be necessary. Section 6.5 shows the details of what really happens in the case of a hazard.

Figure 5.11: ไดอะแกรมแสดงการใช้ stall ประกอบ forwarding เพื่อแก้ load-use data hazard.

และ สมมติว่าตัวแปรทุกตัวเก็บอยู่ในหน่วยความจำ ซึ่งมี `$t0` เป็น base address, MIPS Code อาจเป็น:

```
lw $t1, 0($t0)
lw $t2, 4($t0)
add $t3, $t1, $t2
sw $t3, 12($t0)
lw $t4, 8($t0)
add $t5, $t1, $t4
sw $t5, 16($t0)
```

สังเกตุ load-use data hazard ที่จะเกิดขึ้น, มีคำสั่ง `add` ที่ตามหลัง `lw` อยู่ 2 ชุด.
ทำ reordering code จะได้:

```
lw $t1, 0($t0)
lw $t2, 4($t0)
lw $t4, 8($t0)
add $t3, $t1, $t2
sw $t3, 12($t0)
add $t5, $t1, $t4
sw $t5, 16($t0)
```

โค้ดที่จัดเรียงใหม่นี้ ทำงานได้ผลเหมือนเดิม แต่เร็วกว่าโค้ดเดิม 2 clock cycles (เพราะไม่ต้องทำ pipeline stall 2 ครั้ง).

5.1.3 Control Hazards

Control hazard หรือ branch hazard เกิดขึ้นเมื่อคำสั่งที่ควรจะถูกทำ ไม่ได้ถูกทำใน clock cycle ที่ถูกต้องเพราะว่า คำสั่งอื่นถูก fetched ไปแล้ว. Hazard ชนิดนี้เกิดเมื่อต้องรอผลการคำนวณ เพื่อตัดสินว่าคำสั่งต่อไปคือคำสั่งอะไร.

เช่น

```
          beq $1, $3, 28   ## LABEL1
          and $12, $2, $5
          or $13, $6, $2
          add $14, $2, $2
            :    :    :
LABEL1:   lw $4, 50($7)
```

หลังจากคำสั่ง `beq`, คำสั่งถัดไปที่จะทำ จะขึ้นกับผลการเปรียบเทียบจาก `beq` ซึ่งหากไม่มีฮาร์ดแวร์พิเศษกว่าที่จะรู้ผลนี้ก็ผ่านไปหลาย clock cycles แล้ว รูป 5.12 แสดงผลจาก pipelining ที่มีต่อคำสั่ง `beq`.

เราอาจเพิ่มฮาร์ดแวร์พิเศษขึ้นเพื่อให้สามารถรู้ผลการเปรียบเทียบ และ คำนวณ branch address ได้ตั้งแต่ขั้นตอนที่ 2 ของคำสั่ง `beq` ก็สามารถลด stall ลงเหลือแค่ 1 clock cycle ได้ ดังแสดงในรูป 5.13. หมายเหตุ เราไม่สามารถเพิ่มเข้าไปในขั้นตอนแรกได้ เพราะเรากำลังโหลดคำสั่งออกมาจากหน่วยความจำอยู่ เรายังไม่รู้ด้วยซ้ำว่าเป็นคำสั่ง `beq` หรือเปล่า.

เพื่อจะลด stall รวมถึงการเพิ่มฮาร์ดแวร์เพื่อช่วยให้ขยับให้เรารู้ผลการเปรียบเทียบและค่า branch address ได้เร็วขึ้น, อีกวิธี คือ การทำนาย. ถ้าทำนายถูก pipelining ได้สมบูรณ์แบบ เรา

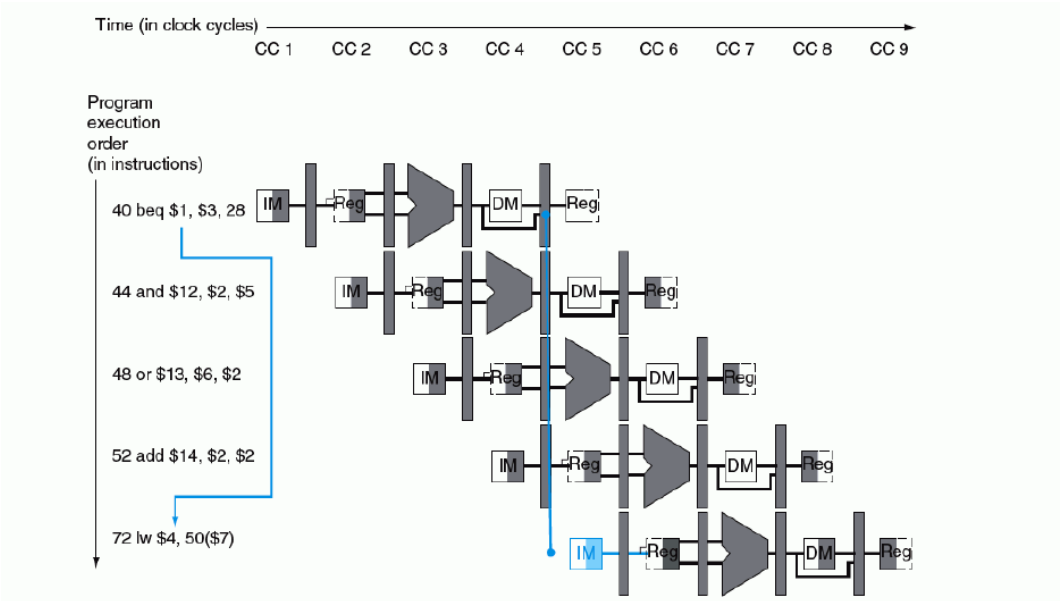


FIGURE 6.37 The impact of the pipeline on the branch instruction. The numbers to the left of the instruction (40, 44, ...) are the addresses of the instructions. Since the branch instruction decides whether to branch in the MEM stage—clock cycle 4 for the beq instruction above—the three sequential instructions that follow the branch will be fetched and begin execution. Without intervention, those three following instructions will begin execution before beq branches to lw at location 72. (Figure 6.7 on page 380 assumed extra hardware to reduce the control hazard to 1 clock cycle; this figure uses the nonoptimized datapath.)

Figure 5.12: ไดอะแกรมแสดง control hazard.

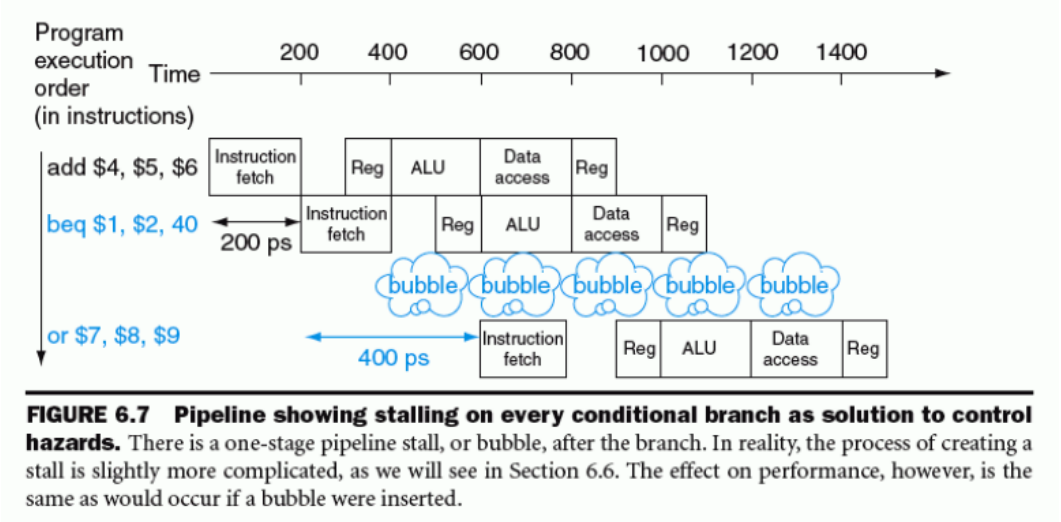


FIGURE 6.7 Pipeline showing stalling on every conditional branch as solution to control hazards. There is a one-stage pipeline stall, or bubble, after the branch. In reality, the process of creating a stall is slightly more complicated, as we will see in Section 6.6. The effect on performance, however, is the same as would occur if a bubble were inserted.

Figure 5.13: ไดอะแกรมแสดง การใช้ stall เพื่อแก้ control hazard.

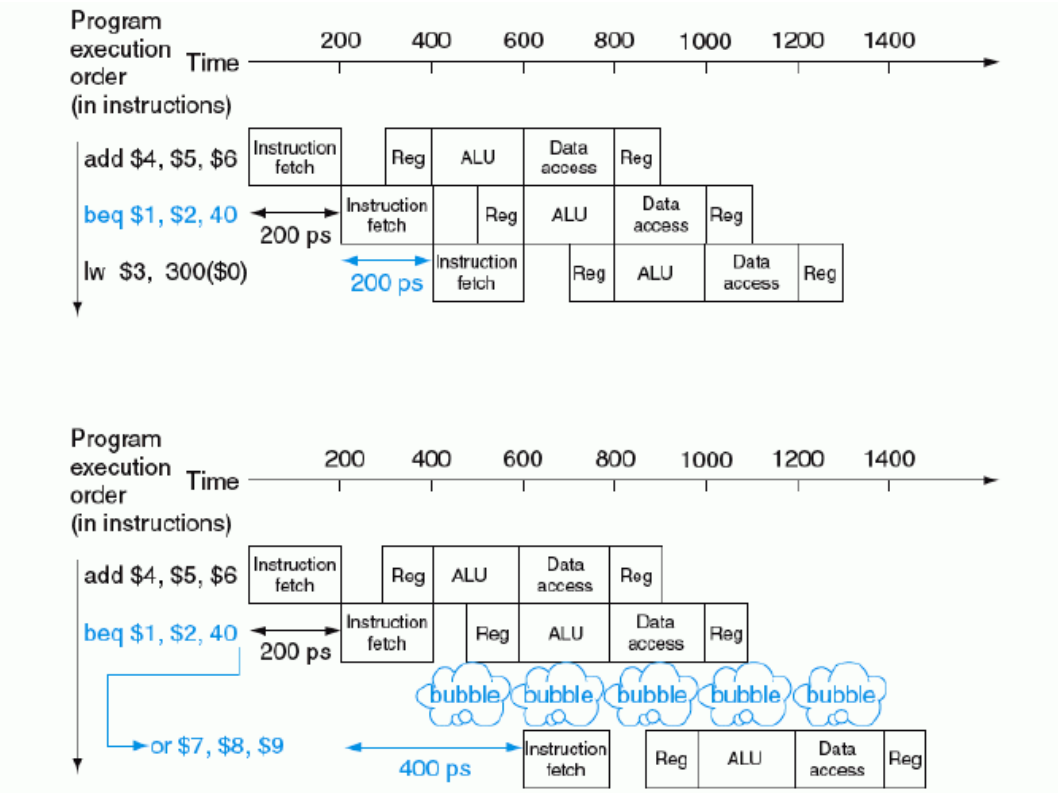


FIGURE 6.8 Predicting that branches are not taken as a solution to control hazard. The top drawing shows the pipeline when the branch is not taken. The bottom drawing shows the pipeline when the branch is taken. As we noted in Figure 6.7, the insertion of a bubble in this fashion simplifies what actually happens, at least during the first clock cycle immediately following the branch. Section 6.6 will reveal the details.

Figure 5.14: ไดอะแกรมแสดง การใช้ branch prediction.

จะทำงานได้เร็วมาก แต่หากทำนายผิด เราก็จะต้องคืนค่า (แก้ผลที่ทำไป) และโหลดคำสั่งที่ถูกต้องมาทำ.

วิธีง่ายๆในการทำนายก็คือ ทายว่า จะไม่ทำ branch ทุกครั้ง, รูป 5.14. หรือ เราอาจทำ branch prediction ที่ซับซ้อนขึ้นได้เช่น ถ้าเป็นการ branch ที่ท้ายลูป (เช่น `do{...} while(...)`) ซึ่งส่วนใหญ่จะทำ branch เราก็อาจทำนายว่า จะทำ branch ถ้า คำสั่ง branch กระโดดกลับ (ค่า offset ติดลบ). ในขณะที่ การเรียก branch ที่ต้นลูป (เช่น `while(...) {...}`) มักไม่ทำ branch, เราจะทำนายว่า จะไม่ทำ branch ถ้า branch ไปข้างหน้า (ค่า offset เป็นบวก). หรือ เราจะทำวิธี branch prediction ที่ซับซ้อนขึ้นอีกก็ได้.

อย่างไรก็ตาม pipeline control ต้องเตรียมฮาร์ดแวร์ที่จะแก้ผลของการที่ทำคำสั่ง ในกรณีที่เราเดาผิดไป และ กลับไปทำคำสั่งที่ถูกต้องได้.

นอกจากการทำ pipeline prediction แล้ว บางครั้ง เรายังอาจใช้ เทคนิค delayed branch ช่วยได้ด้วย. โดย delayed branch จะทำงานในลักษณะเดียวกับ code reordering โดยการย้ายคำสั่งที่ไม่ได้มีผลต่อการทำ branch ลงมาแทนในช่วงเวลาที่รอผลการเปรียบเทียบได้ ดังแสดงในรูป 5.15.

การทำ delayed branch จะมีประโยชน์สำหรับการทำ branch ที่ต้องการ stall สั้นๆ ``no processor uses a delayed branch of more than one cycle."[Pattterson and Hennessy 4th edition] สำหรับ pipelining branch ที่ต้องการ stall หลายๆ cycle, เราจะใช้ branch

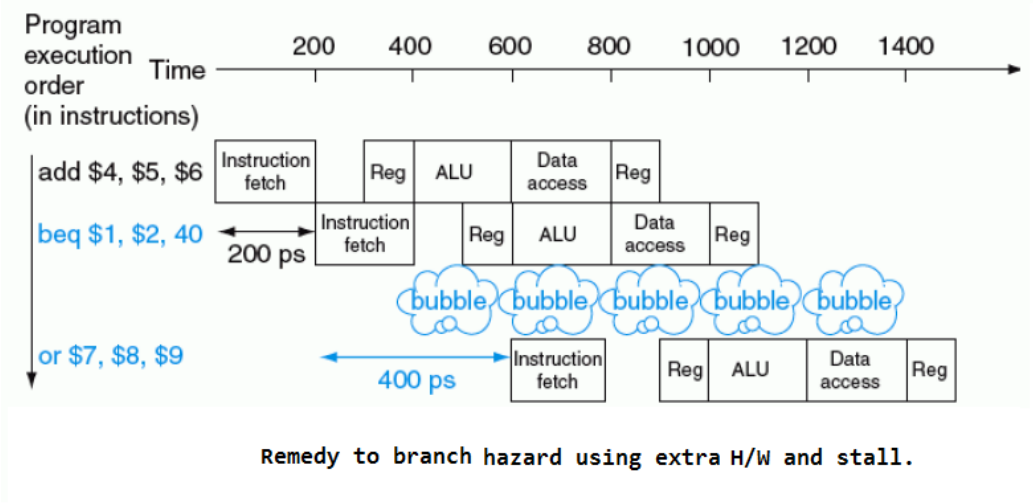


Figure 5.15: ตัวอย่างการใช้ delayed branch.

prediction.

โดยสรุป pipelining ก็คือ เทคนิคที่ทำ parallelism กับคำสั่งที่เป็น sequential. จุดเด่นที่สำคัญ ก็คือ นี่เป็นการทำ parallelism แบบที่ซ่อนจาก โปรแกรมเมอร์ คือ โปรแกรมเมอร์ ยังคงเขียนโปรแกรมเป็นแบบ sequential เหมือนเดิมได้ ต่างจาก เทคนิคที่ทำ parallelism แบบอื่นๆ.

the BIG Picture Pipelining เพิ่มจำนวนคำสั่งที่ถูกทำพร้อมๆกัน, เพิ่มอัตราที่คำสั่งทำงานเสร็จ. แต่ pipelining ไม่ได้ลงเวลาที่แต่ละคำสั่งทำลง. ซึ่ง จำนวน stages ใน pipeline จะเรียกว่า latency. เช่น จากตัวอย่าง five-stage pipeline จะใช้ 5 clock cycles ต่อคำสั่ง. พูดอีกอย่าง คือ pipelining ช่วยเพิ่ม instruction throughput, ไม่ใช่ลด execution time, ไม่ใช่ลด latency.

5.2 Parallelism and Advanced Instruction-Level Parallelism

5.2.1 Power Efficiency and Advanced Pipelining

รูป 5.1 แสดงวิวัฒนการของโปรเซสเซอร์ที่ช่วงเวลาต่างๆ. สังเกตุจำนวน pipeline stages ของ pipeline และ clock rate ที่ลดลงหลังปี 2004 ที่ Power ขึ้นไปถึง 103W.

Table 5.1: Record of Intel and Sun Microprocessors in terms of pipeline complexity, number of cores, and power. The Pentium 4 pipeline stages do not include the commit stages. If we included them, the Pentium 4 pipelines would be even deeper. (จาก PH4e 4.73)

Microprocessor	Year	Clock Rate	Pipeline Stages	Issue Width	Out-of-Order/ Speculation	Cores/ Chip	Power (Watt)
Intel 486	1989	25 MHz	5	1	No	1	5
Intel Pentium	1993	66 MHz	5	2	No	1	10
Intel Pentium Pro	1997	200 MHz	10	3	Yes	1	29
Intel Pentium 4 Willamette	2001	2000MHz	22	3	Yes	1	75
Intel Pentium 4 Prescott	2004	3600MHz	31	3	Yes	1	103
Intel Core	2006	2930MHz	14	4	Yes	2	75
UltraSPARC IV+	2005	2100MHz	14	4	No	1	90
Sun UltraSPARC T1 (Niagara)	2005	1200MHz	6	1	No	8	70

Chapter 6

Large and Fast: Exploiting Memory Hierarchy

วิธีการจัดการหน่วยความจำ เนื่องจากสภาวะที่เรามีคือ หน่วยความจำที่เร็ว ราคาแพง หน่วยความจำที่ช้าราคาถูก ดังนั้น เมื่อเราต้องการหน่วยความที่เร็ว มีขนาดใหญ่ในราคาที่จ่ายได้ เราจึงใช้การจัดการหน่วยความจำในลักษณะของลำดับชั้น.

เช่นตัวอย่างเปรียบเทียบง่ายๆของ หนังสือ โต๊ะ ห้องสมุด. สมมติเรากำลังเขียนบทความ เรื่อง ประวัติการพัฒนาของฮาร์ดแวร์คอมพิวเตอร์ ห้องสมุด มีหนังสืออยู่จำนวนมาก แต่เดินหาหนังสือในห้องสมุดใช้เวลามากกว่า หยิบหนังสือจากโต๊ะ. สิ่งที่เราจะทำคือ หนึ่งอาทิตย์เราไปห้องสมุดสักครั้ง แล้วยืมหนังสือมาสัก 4-5 เล่ม แล้วเวลาทำงานเราก็เอาหนังสือ 4-5 เล่มนี้วางบนโต๊ะ ที่เราหาและหยิบค้นได้สะดวกกว่า. ถ้าเราพบว่ามีบางเรื่องที่หนังสือบนโต๊ะไม่มี เราก็กลับไปห้องสมุด ไปยืมหนังสือชุดใหม่มา. ในลักษณะนี้ เราจะทำงานได้เร็วด้วยความสะดวกของการหยิบค้นหนังสือบนโต๊ะ แต่สามารถเข้าถึงข้อมูลได้เท่าที่ห้องสมุดมี. หลักการเดียวกันนี้ เราจะใช้กับการจัดการหน่วยความจำ.

หลักการของความเป็นท้องถิ่น (Principle of Locality): โปรแกรมจะใช้แค่ส่วนเล็กๆของหน่วยความจำที่ช่วงเวลาหนึ่งๆ. มีความเป็นท้องถิ่นอยู่ 2 แบบ:

- Temporal locality (locality in time): ถ้าข้อมูลส่วนไหนถูกใช้ มันน่าจะถูกใช้อีกในเวลาไม่นาน.
- Spatial locality (locality in space): ถ้าข้อมูลส่วนไหนถูกใช้, ข้อมูลแอดเดรสใกล้ๆกันก็น่าจะถูกใช้ในเวลาไม่นาน.

ตัวอย่างเช่น โปรแกรมที่มีลูป คำสั่งและข้อมูลที่เรียกในลูป ก็จะถูกเรียกใช้อีกไม่นาน. (high temporal locality). ถ้าข้อมูลที่ใช้เป็นอาร์เรย์ แอดเดรสใกล้ๆกับที่เรียกใช้ไปแล้วก็จะถูกเรียกใช้ไม่นาน (high spatial locality).

เราใช้ข้อสังเกตุนี้ สร้างระบบหน่วยความจำสำหรับคอมพิวเตอร์ เป็น แบบ memory hierarchy. Memory hierarcy ประกอบด้วยหน่วยความจำหลายๆชนิดที่มีความจุและความเร็วในการเข้าถึงข้อมูลต่างๆกัน โดยจัดอยู่ในรูปลำดับชั้น. หน่วยความจำที่เร็วกว่า จะมีราคาต่อบิต (cost per bit) แพงกว่า หน่วยความจำที่ช้ากว่า. ดังนั้น หน่วยความจำที่เร็วกว่า จะมีขนาดเล็กกว่า. ปัจจุบัน มีเทคโนโลยีสำหรับ หน่วยความจำอยู่ 3 ระดับหลักๆ. หน่วยความจำหลัก (main memory) จะใช้ เทคโนโลยี DRAM (dynamic random access memory), ในขณะที่หน่วยความจำระดับที่ใกล้กับโปรเซสเซอร์มากกว่า เรียกว่า cache จะใช้เทคโนโลยี SRAM (static random access memory). DRAM มีราคาต่อบิตถูกกว่า SRAM เราจึงสามารถมี DRAM ขนาดใหญ่

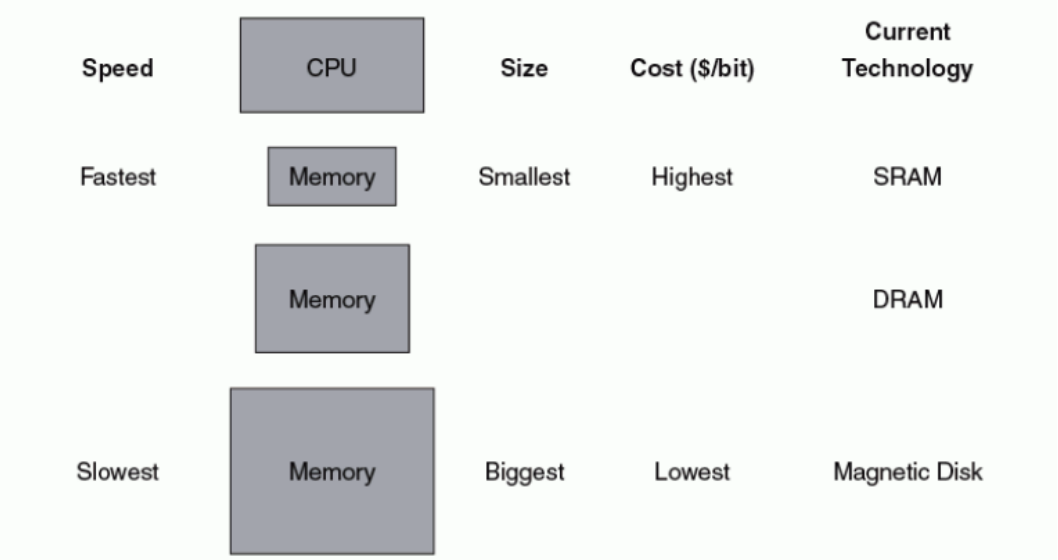


FIGURE 7.1 The basic structure of a memory hierarchy. By implementing the memory system as a hierarchy, the user has the illusion of a memory that is as large as the largest level of the hierarchy, but can be accessed as if it were all built from the fastest memory.

Figure 6.1: โครงสร้างลำดับชั้นของการจัดการหน่วยความจำ.

กว่า cache ได้ในขณะที่ SRAM ทำงานได้เร็วกว่า. ส่วนเทคโนโลยีหลักที่ 3 คือ hard disk ที่มักใช้ เทคโนโลยี magnetic disk ทำ. เทียบกับ SRAM และ DRAM, magnetic disk ที่ทำงานได้ช้าที่สุด แต่ราคาต่อบิตถูกที่สุด, ทำให้เราสามารถมี hard disk ขนาดใหญ่กว่า main memory มากๆได้. Flash memory เป็นเทคโนโลยีที่เริ่มเข้ามามีบทบาทมากขึ้น และถูกใช้แทน magnetic disk สำหรับ embedded devices หลายๆชนิด. ความเร็วและราคาของแต่ละเทคโนโลยีแสดงดังตาราง 6.1.

Table 6.1:

Memory technology	Typical access time	$ per GB in 2008
SRAM	0.5 - 2.5 ns	$2000-$5000
DRAM	50 - 70 ns	$20-$75
Magnetic disk	5,000,000-20,000,000 ns	$0.20-$2

เพราะว่า ความต่างของ ราคาต่อ GB และ access time ของแต่ละเทคโนโลยี เราจึงสร้างระบบหน่วยความจำเป็นลักษณะ hierarchy of levels. โดยเราจะใช้หน่วยความจำที่เร็วที่สุดที่มีขนาดไม่ใหญ่ (เพราะราคาแพง) ให้อยู่ใกล้กับ processor ที่สุด และ ต่อมาด้วยหน่วยความจำเร็วน้อยกว่า แต่มีขนาดใหญ่กว่า ตามลำดับชั้น, ดังแสดงในรูป 6.1. จุดประสงค์ คือ เพื่อให้ processor ใช้หน่วยความจำเหมือนกับ ว่า มีหน่วยความจำมากที่สุด เท่าที่หน่วยความจำที่ถูกที่สุดมี และใช้ access time ให้ต่ำเท่าที่หน่วยความจำที่เร็วที่สุด.

หน่วยความจำระดับชั้นที่สูงกว่า (ใกล้ processor มากกว่า) จะมีขณะเล็กกว่าและทำงานเร็วกว่า ระดับชั้นที่ต่ำกว่า. การจัดหน่วยความจำในคอมพิวเตอร์ มีได้หลายระดับชั้น, แต่ ณ เวลาหนึ่ง ข้อมูลจะเคลื่อนย้ายหรือคัดลอก กัน ระหว่าง 2 ระดับหน่วยความจำที่ติดกันเท่านั้น.

พิจารณาการจัดการระหว่าง 2 ระดับหน่วยความจำที่ติดกัน เช่น อาจจะเป็นคู่ SRAM กับ

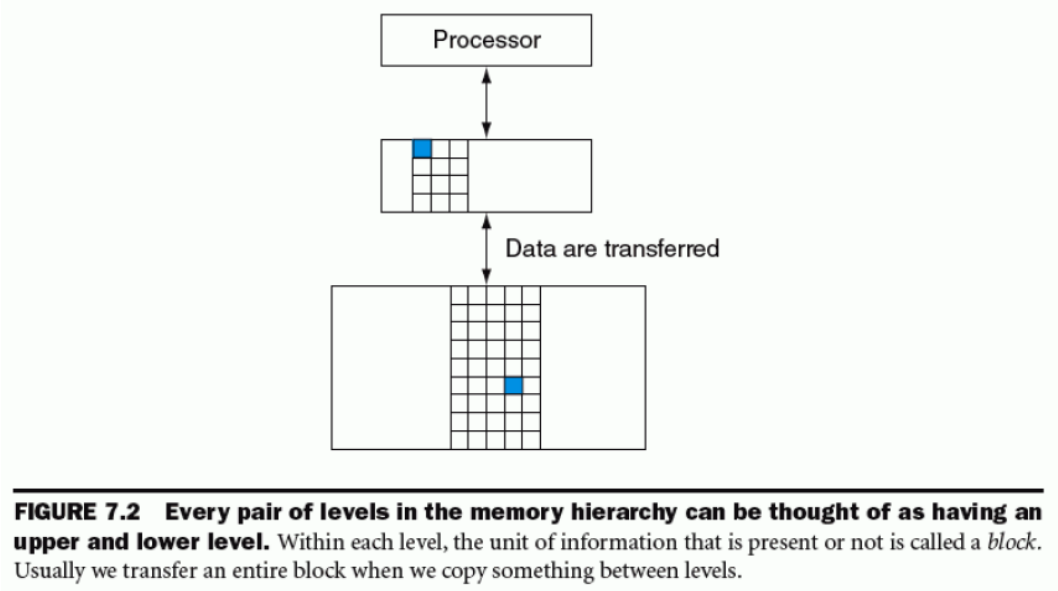


FIGURE 7.2 Every pair of levels in the memory hierarchy can be thought of as having an upper and lower level. Within each level, the unit of information that is present or not is called a *block*. Usually we transfer an entire block when we copy something between levels.

Figure 6.2: หน่วยความจำระดับชั้นติดกัน และ การถ่ายข้อมูลเป็น block.

DRAM. หน่วยของข้อมูลที่พิจารณา เรียกว่า block (หรือ line). Block คือ หน่วยข้อมูลที่เล็กที่สุดที่พิจารณาในระดับชั้น ถ้าเปรียบเทียบกับตัวอย่างของห้องสมุด ก็เปรียบได้กับ หนังสือ.

Cache. โดยทั่วๆไป block จะมีขนาด 4 ถึง 32 kbyte [1]. ถ้าเพิ่มขนาดของ block จะช่วยลด cache misses ลง เพราะว่าแต่ละ block สามารถจุข้อมูลหรือคำสั่งได้มากขึ้น. แต่ขณะที่ block ขนาดใหญ่จะลด miss rate, แต่ miss penalty จะสูง. ค่า miss penalty คือ เวลาที่ใช้ไปเอา block ที่ missed มาจาก main memory มาเก็บใน cache. เมื่อ block ขนาดใหญ่ขึ้นก็จะใช้เวลาถ่ายข้อมูลนานขึ้น.
วิธีง่ายที่สุดในการลด miss rate ที่ไม่กระทบกับ miss penalty คือ การเพิ่มขนาดของ cache แต่วิธีนี้จะเพิ่มค่าใช้จ่ายเพิ่มขึ้น รวมถึงยังเพิ่มกำลังงานที่จะใช้และเพิ่มความร้อนที่เกิดอีกด้วย.
วิธีที่ 3 คือ การเพิ่มระดับของ cache. เช่น Intel Pentium 4 ใช้ cache 2 ระดับ: L1 ขนาด 16 kB กับ L2 4MB. และใช้ระดับ L1 เองก็มี cache อยู่ 2 ตัวสำหรับ instruction กับ data.

ถ้าข้อมูลที่ processor จะใช้มีอยู่ใน block ใด block หนึ่งของระดับชั้นบน เราจะเรียกว่า hit. เช่น ถ้าเราจะอ่านข้อมูล, และข้อมูลมีอยู่ใน block ใด block หนึ่งของ cache, เรียกว่า hit ซึ่ง processor สามารถอ่านข้อมูลได้ทันทีจาก cache. แต่ถ้าไม่พบ block ที่มีข้อมูลที่ต้องการอยู่จะเรียกว่า miss. ซึ่งเมื่อเกิด miss ขึ้น เราก็จะไปหา block นั้นในหน่วยความจำระดับชั้นล่าง ถ้าเจอก็จะคัดลอก block นั้นมายังหน่วยความจำระดับชั้นบน ในลักษณะเดียวกับ ไปยืมหนังสือมาจากห้องสมุดแล้ว เอามาไว้บนโต๊ะเพื่อใช้ได้สะดวก. รูป 6.2 แสดงหน่วยความจำระดับชั้นติดกัน และการแบ่งข้อมูลเป็น blocks. การถ่ายเทข้อมูลระหว่างหน่วยความจำทั้งสองระดับชั้นก็จะทำทีละ block.

เพื่อความสะดวกในการพิจารณาประสิทธิภาพการจัดการหน่วยความจำ, เราดูศัพท์ที่เกี่ยวข้อง

- hit rate หรือ hit ratio คือ สัดส่วนจำนวนครั้งที่ พบข้อมูลที่จะใช้ได้ในหน่วยความจำชั้นบน ต่อ จำนวนครั้งที่ร้องขอใช้ข้อมูลทั้งหมด.

- miss rate ซึ่งมีค่าเท่ากับ 1 – hit rate คือ สัดส่วนจำนวนครั้งที่ไม่พบข้อมูลในหน่วยความจำชั้นบน ต่อ จำนวนครั้งที่ร้องขอใช้ข้อมูลทั้งหมด.

- hit time คือ เวลาที่ใช้ในการเข้าถึงหน่วยความจำระดับชั้นบน รวมเวลาที่ใช้ตรวจสอบว่า hit หรือ miss ด้วย.

- miss penalty คือ เวลาที่ใช้ในการเข้าถึงหน่วยความจำ รวม เวลาที่ใช้ในการนำ block ที่ต้องการ จากหน่วยความจำระดับชั้นล่าง มาเก็บไว้ในหน่วยความจำระดับชั้นบนด้วย. เนื่องจาก หน่วยความจำระดับชั้นบนจะเร็วกว่า ระดับชั้นล่างมาก ดังนั้น hit time จะสั้นกว่า miss penalty มาก.

วิธีการที่เราใช้การจัดระบบหน่วยความจำนี้ จะมีผลต่อด้านต่างๆด้วย ไม่ว่าจะเป็น การจัดการหน่วยความจำและ I/O ของระบบปฏิบัติการ, วิธีการสร้างโค้ดของคอมไพเลอร์, และแม้แต่การใช้คอมพิวเตอร์ของแอพพลิเคชั่นต่างๆ. เพราะว่า โปรแกรมต่างๆ มักจะใช้เวลาส่วนใหญ่ในการเข้าถึงหน่วยความจำ ดังนั้นการจัดการหน่วยความจำจึงมีผลอย่างมากต่อสมถนะโดยรวมของคอมพิวเตอร์.

The BIG Picture. โปรแกรมส่วนใหญ่จะมีลักษณะของ temporal locality ที่มักจะใช้ข้อมูลเดิมซ้ำๆอีก และ spatial locality ที่มักจะใช้ข้อมูลที่มีแอดเดรสใกล้ๆกับข้อมูลที่เรียกมาแล้ว. การจัดการหน่วยความจำ ลักษณะของโครงสร้างลำดับชั้นของหน่วยความจำ จึงแบ่งข้อมูลเป็นส่วนๆ เรียกว่า block และ จะเก็บ block ที่มีข้อมูลที่เรียกใช้ไป ไว้ใกล้กับ processor (ใช้ประโยชน์จาก temporal locality) และเมื่อมีข้อมูลใหม่ที่ต้องการ ก็ย้ายทั้ง block ที่มีข้อมูลที่ต้องการ จากหน่วยความจำระดับชั้นล่าง ขึ้นไประดับชั้นบน. ใน block นอกจากจะมีข้อมูลที่ต้องการแล้ว ยังมีหลายๆ words ที่แอดเดรสใกล้ๆกับข้อมูลที่ต้องการด้วย (ใช้ประโยชน์จาก spatial locality).
รูป 6.3 แสดงระดับชั้นของหน่วยความจำ ที่เทคโนโลยีหน่วยความจำที่ทำงานได้เร็วกว่าแต่ขนาดเล็กกว่าจะอยู่ใกล้ processor มากกว่า. ดังนั้นการเข้าถึงหน่วยความจำระดับชั้นบนที่ hit จะทำได้อย่างรวดเร็ว. การเข้าถึงที่ miss ก็จะไปเอาข้อมูลมาจากระดับชั้นล่างที่ขนาดใหญ่กว่าแต่ทำงานได้ช้ากว่า. ถ้าเราจัดการได้ดี ทำให้ hit rate สูงมากพอ, การจัดโครงสร้างระดับชั้นของหน่วยความจำ จะให้ความเร็วในการเข้าถึงข้อมูลได้เกือบเท่ากับหน่วยความจำที่เร็วที่สุด (ชั้นบนสุด) และ ให้ความจุเท่ากับหน่วยความจำที่ขนาดใหญ่ที่สุด (ชั้นล่างสุด).

6.1 แคช: พื้นฐาน

แคช (cache) เป็น ชื่อ ที่เราใช้เรียก หน่วยความจำ ที่ในลำดับชั้นของหน่วยความจำ อยู่ระหว่างโปรเซสเซอร์ กับ หน่วยความจำหลัก. แคช เริ่มแพร่หลาย กับ คอมพิวเตอร์มาตั้งแต่ทศวรรษที่ 1960. ปัจจุบัน คอมพิวเตอร์ ตั้งแต่ระดับ server ไปจนถึง หน่วยประมวลผลแบบฝังตัว ที่ใช้พลังงานต่ำ ก็ใช้ แคช กันทั้งนั้น.

เราจะเริ่มต้นถกเรื่องแคช จากแคชรูปแบบง่ายๆก่อน คือ โปรเซสเซอร์ จะรับส่ง ข้อมูล กับแคช เป็น หนึ่ง word และ แต่ละ block ของแคช ก็เป็น หนึ่ง word. รูป 6.4 แสดงการทำงานของแคช

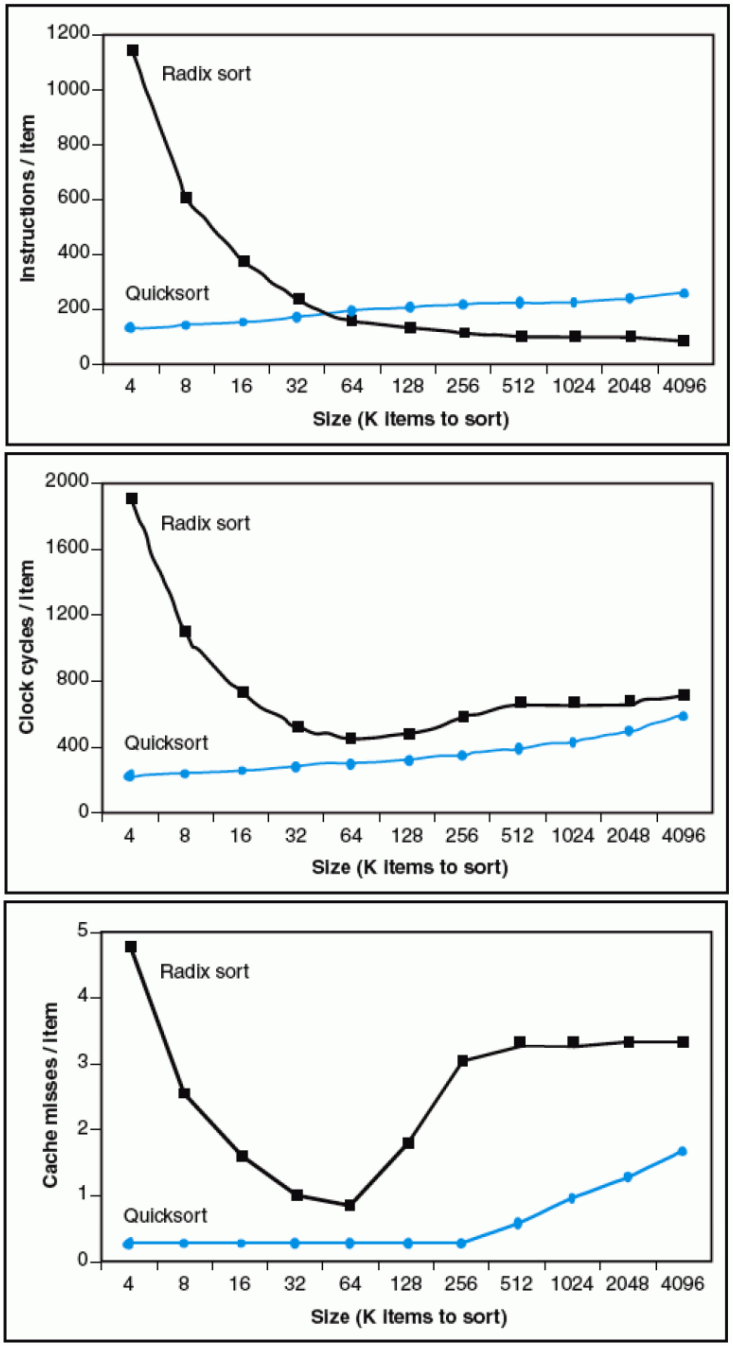


FIGURE 7.18 Comparing Quicksort and Radix Sort by (a) instructions executed per item sorted, (b) time per item sorted, and (c) cache misses per item sorted. This data is from a paper by LaMarca and Ladner [1996]. Although the numbers would change for newer computers, the idea still holds. Due to such results, new versions of Radix Sort have been invented that take memory hierarchy into account, to regain its algorithmic advantages (see Section 7.7). The basic idea of cache optimizations is to use all the data in a block repeatedly before it is replaced on a miss.

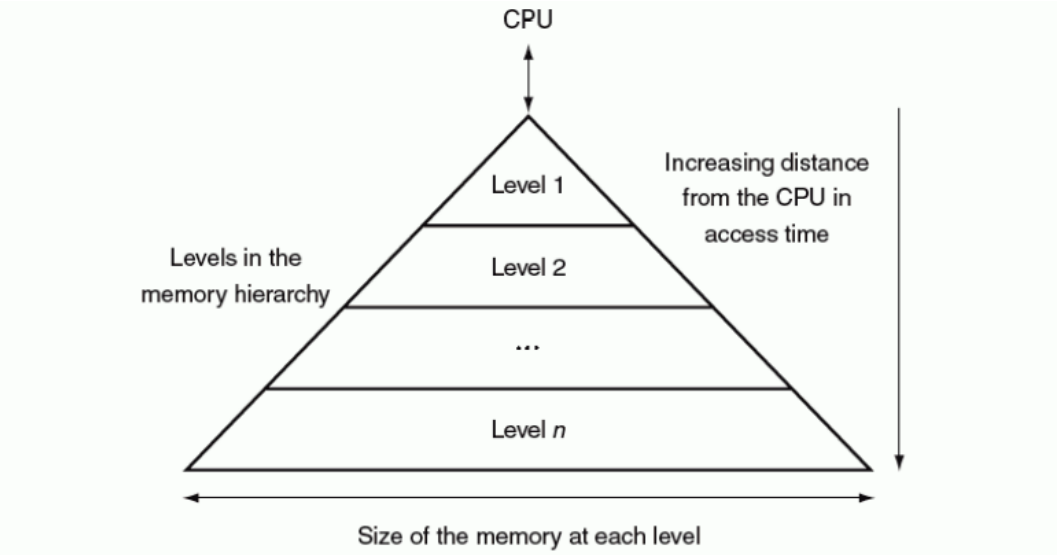


FIGURE 7.3 This diagram shows the structure of a memory hierarchy: as the distance from the processor increases, so does the size. This structure with the appropriate operating mechanisms allows the processor to have an access time that is determined primarily by level 1 of the hierarchy and yet have a memory as large as level *n*. Maintaining this illusion is the subject of this chapter. Although the local disk is normally the bottom of the hierarchy, some systems use tape or a file server over a local area network as the next levels of the hierarchy.

Figure 6.3: ไดอะแกรมแสดงระดับชั้นของหน่วยความจำ ความใกล้กับ processor, ความเร็ว และขนาด

Table 6.2: [PH4e's Fig 5.29] The key quantitative design parameters that characterize the major elements of memory hierarchy in a computer. These are typical values for these levels as of 2008. Although the range of values is wide, this is partially because many of the values that have shifted over time are related; for example, as caches become larger to overcome larger miss penalties, block sizes also grow.

Feature	Typical values for L1 caches	Typical values for L2 caches	Typical values for paged memory	Typical values for a TLB
Total size in blocks	250-2000	15,000-50,000	16,000-250,000	40-1024
Total size in kB	16-64	500-4000	1,000,000-1,000,000,000	0.25-16
Block size in B	16-64	64-128	4000-64,000	4-32
Miss penalty in clocks	10-25	100-1000	10,000,000-100,000,000	10-1000
Miss rates (global for L2)	2%-5%	0.1%-2%	0.00001%-0.0001%	0.01%-2%

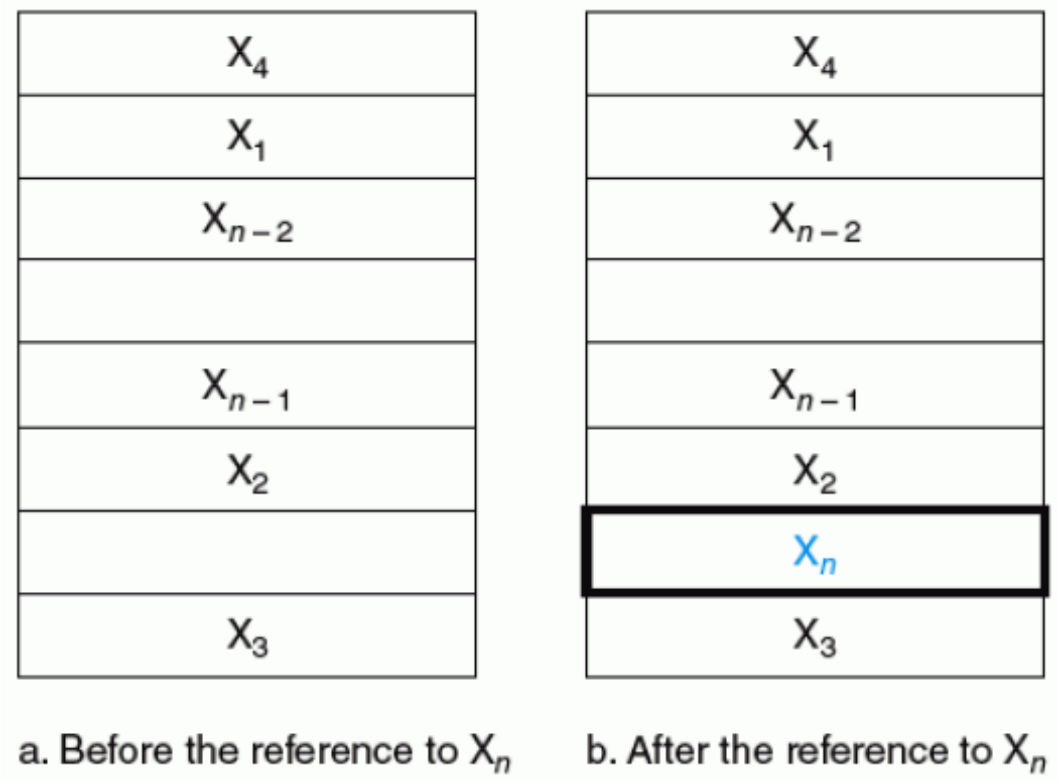


Figure 6.4: แคชก่อนและหลัง การอ่าน `Xn` ที่ไม่ได้อยู่ในแคชแต่แรก

แบบง่ายๆนี้: ถ้าแคชไม่มีค่าที่ต้องการอ่านอยู่ แคชก็จะไปเอาค่านั้นจากหน่วยความจำหลักเข้ามา. พอจะเอาข้อมูลจากหน่วยความจำหลักเข้ามา สิ่งที่ต้องคำนึงก็คือ จะเอาเข้ามาไว้ที่ไหนในแคช. วิธีที่ง่ายที่สุด เรียกว่า ``direct mapped'' คือ จะเอาไว้ที่ index ของแคช ตามค่าแอดเดรสของ word นั้นในหน่วยความจำหลัก, ดังแสดงในรูป 6.5. ในรูป สังเกตุ ข้อมูลจากหน่วยความจำหลัก แอดเดรส 00001, 01001, 10001, และ 11001 จะใช้พื้นที่ในแคชที่เดียวกัน คือ ที่ index 001.

มองอีกมุมหนึ่ง ค่าในแคช ที่ index 101 อาจจะเป็นข้อมูลของแอดเดรส 00101 หรือ 01101 หรือ 10101 หรือ 11101 ก็ได้. ดังนั้น เพื่อให้เรารู้ว่า ข้อมูลที่อยู่ในแคช เป็นของแอดเดรสใดแน่ เราจะเก็บ tag ไว้ด้วย. โดย tag จะช่วยระบุว่า ข้อมูลในแคช นั้นมาจากไหน ในหน่วยความจำหลัก.

นอกจากนั้น เรายังต้องการบิตที่ใช้ระบุด้วยว่า ค่าในแคชนั้น ใช้ได้หรือไม่. เช่น ว่าเป็นค่าที่เราพึ่งโหลดจากหน่วยความจำหลัก หรือ เป็นค่าว่างๆ หลังจากเปิดเครื่องใหม่ๆ. เราจะใช้ valid bit เป็น ตัวบ่งชี้ ว่าค่าในแคช ที่ index นั้นๆ ใช้ได้.

รูป 6.6 แสดงค่าในแคช จากการอ่านค่าที่แอดเดรสต่างๆจากหน่วยความจำ. เมื่อเปิดเครื่องใหม่ๆ (ภาพ a) ค่าของ valid bit ของทุก block ในแคชจะเป็น `N`, หมายถึง ว่าค่าในแคช ใช้ไม่ได้. เมื่อมีการอ่านข้อมูลแอดเดรสที่ 10110 เข้ามา และ เนื่องจากข้อมูลแอดเดรสที่ 10110 ยังไม่มีในแคช, จะเกิด miss ขึ้น ซึ่งจะจัดการด้วยการโหลดข้อมูลจากหน่วยความจำหลังที่แอดเดรส 10110 เข้ามาเก็บที่แคช index 110 และ ตั้งค่า tag เป็น 10 พร้อมกับค่า valid bit เป็น `Y` ดังภาพ b. ลักษณะเดียวกัน กับการอ่านข้อมูลที่แอดเดรสอื่นๆ. สังเกตุ ที่ภาพ f มีการอ่านข้อมูลแอดเดรส 10010 ซึ่งจะใช้ block ในแคชตำแหน่งเดียวกับ ข้อมูลแอดเดรส 11010 ที่อ่านมาแล้ว, การจัดการแบบ direct mapped cache ก็จะแค่ทับค่าที่อ่านล่าสุดนี้ลงไป. นอกจากข้อมูลใน

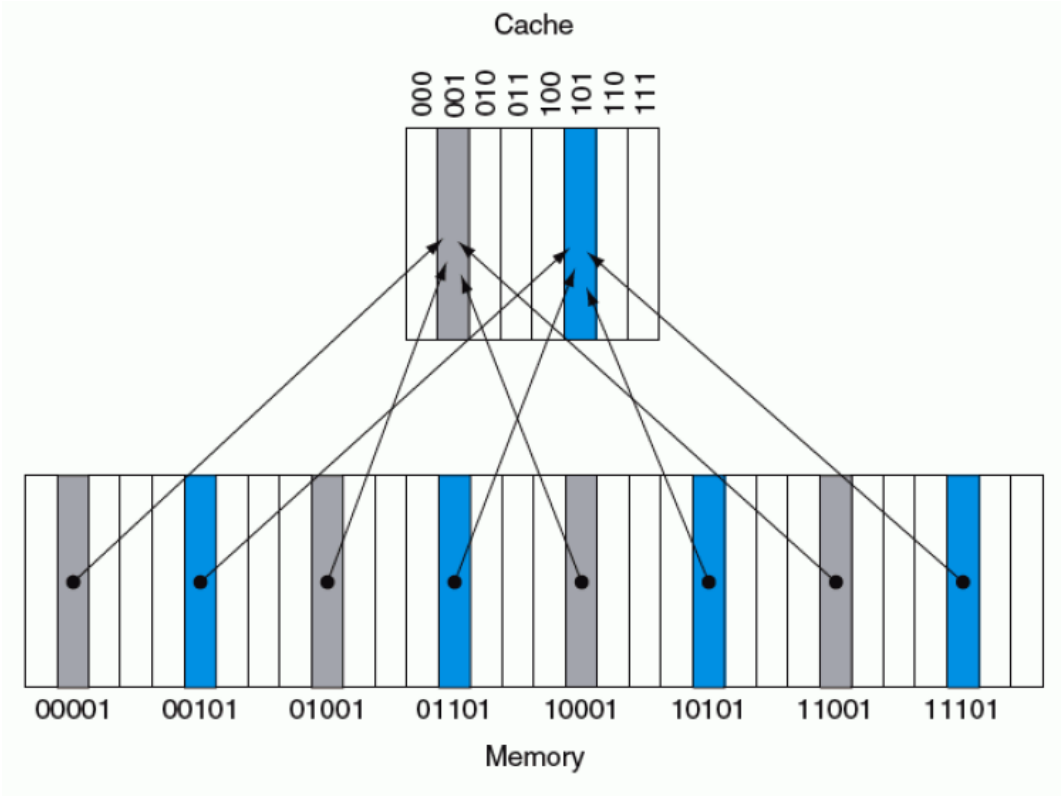


Figure 6.5: แสดงการจัดใส่ข้อมูลจากหน่วยความจำหลัก ไป ที่แคช ในแบบ direct mapped cache

block นั้นจะเปลี่ยนไปแล้ว ค่าของ tag ก็จะเปลี่ยนไป เพื่อให้สอดคล้องกับตำแหน่งของแอดเดรสของ block ด้วย.

ตัวอย่างของ direct mapped cache สำหรับ MIPS, เราใช้ cache block เป็น หนึ่ง word (4 bytes สำหรับ MIPS). ถ้าเราเลือกใช้แคช ขนาด 1024 blocks: เราใช้ 10 bits สำหรับ cache index. และ สำหรับ MIPS ที่มี alignment restriction เราสามารถละ 2 บิตท้าย, ดังนั้น 32 address bits จะแบ่ง เป็น 10 bits สำหรับ cache index, 2 bits ละไว้ได้, และ tag เก็บ 20 bits ที่เหลือ เพื่อให้สามารถระบุแอดเดรสของ block ในหน่วยความจำหลักได้. รูป 6.7 แสดงการจัดแจง จากแอดเดรสของหน่วยความจำหลัก มาเป็น cache index, tag, และ วงจรที่ใช้สร้างสัญญาณ hit.

ขนาดของแคช จำนวนบิตในแคช ในรูปคือ $\mathbf{Nblocks} \times (\mathbf{blocksize} \times \mathbf{tagsize} + \mathbf{1forvalidbit})$, ซึ่ง $= 1024 \times (32 + 20 + 1) = 54,272$ บิต (= 53 Kbits). แต่ตามความนิยม ขนาดของแคชจะอาจตามแค่ขนาดของข้อมูลเท่านั้น แคชในรูป จึงเรียก แคชขนาด 4K ไบต์.

6.1.1 Multiword Cache Block

ตัวอย่างข้างต้น เราดูกรณีที่ cache block มีขนาด หนึ่ง word. เราสามารถใช้ block ขนาดใหญ่ขึ้นได้ เช่น อาจจะใช้ block ขนาด 2-word, 4-word เป็นต้น. ถ้าเราใช้ block ขนาดใหญ่ขึ้น จาก byte address (แอดเดรสของหน่วยความจำหลัก ที่อ้างถึงข้อมูลทีละ 8 บิต), เรานิยาม block

Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	N		
111	N		

a. The initial state of the cache after power-on

Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	Y	10_{two}	Memory(10110_{two})
111	N		

b. After handling a miss of address (10110_{two})

Index	V	Tag	Dlata
000	N		
001	N		
010	Y	11_{two}	Memory (11010_{two})
011	N		
100	N		
101	N		
110	Y	10_{two}	Memorlly (10110_{two})
111	N		

c. After handling a miss of address (11010_{two})

Index	V	Tag	Data
000	Y	10_{two}	Memory (10000_{two})
001	N		
010	Y	11_{two}	Memory (11010_{two})
011	N		
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

d. After handling a miss of address (10000_{two})

Index	V	Tag	Data
000	Y	10_{two}	Memory (10000_{two})
001	N		
010	Y	11_{two}	Memory (11010_{two})
011	Y	00_{two}	Memory (00011_{two})
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

e. After handling a miss of address (00011_{two})

Index	V	Tag	Data
000	Y	10_{two}	Memory (10000_{two})
001	N		
010	Y	10_{two}	Memory (10010_{two})
011	Y	00_{two}	Memory (00011_{two})
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

f. After handling a miss of address (10010_{two})

Figure 6.6: แสดงตัวอย่างของข้อมูลในแคชหลักจากการอ่านค่าที่แอดเดรสต่างๆ

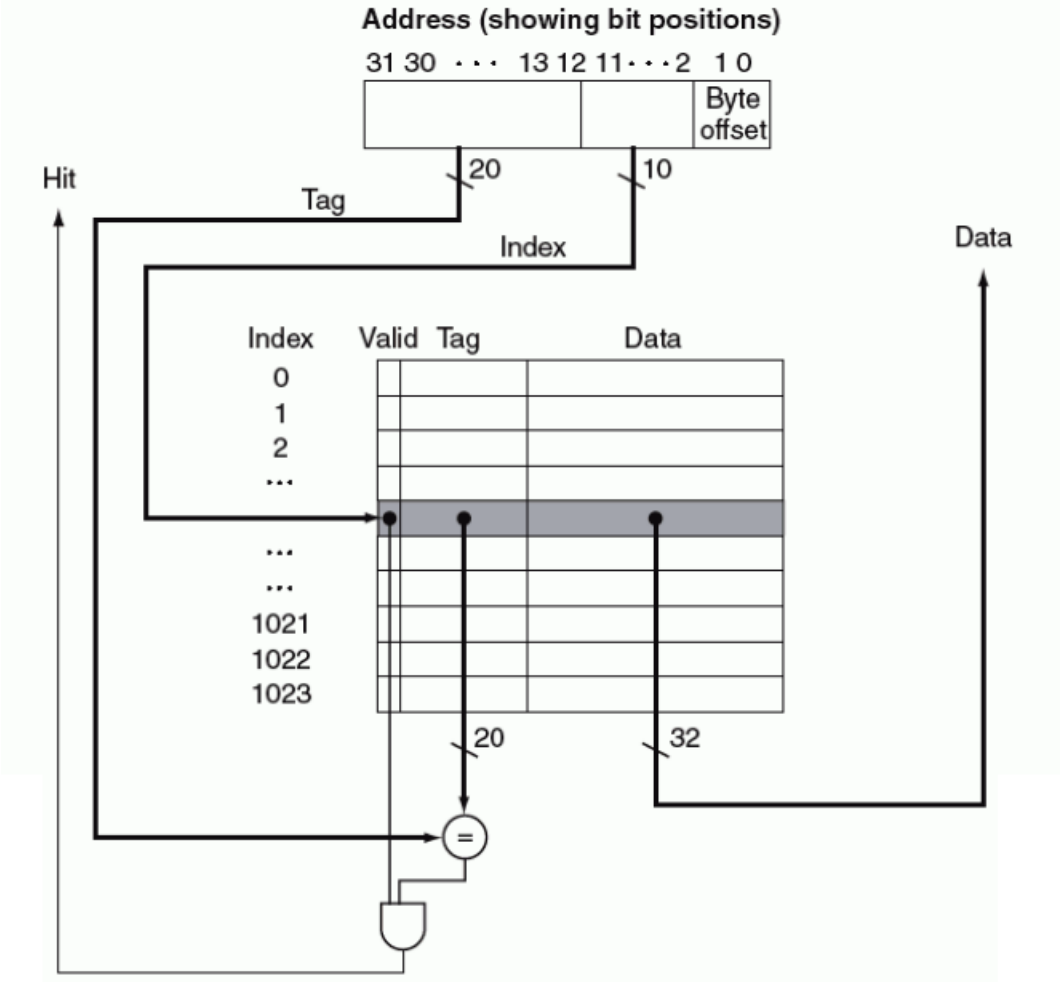


Figure 6.7: สำหรับแคชในแบบที่เราคุยกันนี้ ส่วนล่างของแอดเดรส จะใช้เป็น cache index; ส่วนบนจะใช้เป็น tag; และ สำหรับ MIPS ที่มี alignment restriction เราสามารถละ 2 บิตท้ายไปก่อน

address เป็น,

$$\text{block address} = \left\lfloor \frac{\text{Byte address}}{\text{Bytes per block}} \right\rfloor . \tag{6.1}$$

ตัวอย่าง เช่น สำหรับ block ขนาด 2 words ($= 2 \times 4 = 8$ ไบต์), ข้อมูลที่ byte address เป็น 4000 จะมี block address เป็น 500. ข้อมูลที่ byte address เป็น 4000 ถึง 4007 จะมี block address เดียวกัน คือ 500, เพราะทั้ง 8 ไบต์นี้รวมอยู่ใน block นี้.

และจาก block address นี้ เราสามารถคำนวณ cache index ได้จาก

$$\text{cache index} = \text{block address} \text{moduloNumbers of blocks in the cache.} \tag{6.2}$$

เช่น หากแคชมี 1024 blocks, block address ที่ 500, หรือ 1524, หรือ 2548 เป็นต้น ก็จะมี cache index 500 เหมือนกัน.

6.1.2 Handling Cache Misses

เราพูดถึง cache miss สำหรับ เหตุการณ์ที่ เราต้องการอ่านข้อมูลจากแคช แต่ข้อมูลที่จะอ่านไม่อยู่ในแคช. รูป 6.7 แสดงวงจรที่ใช้ตรวจสอบ hit, ถ้าเจอข้อมูลที่ต้องการอ่านในแคช, เรียกว่า hit, เราก็จะอ่านข้อมูลไป และทำงานที่ต้องการต่อ. แต่ถ้าไม่เจอ (สัญญาณ hit $= 0$), เราจะต้องจัดการกับ cache miss. การอ่านข้อมูลจากแคช มี 2 กรณีคือ instruction fetch และ data read. สำหรับ instruction fetch, ถ้าเกิด cache miss ขึ้น สิ่งที่ต้องทำคือ

- 1. อ่านคำสั่งนั้นจากหน่วยความจำหลัก (ส่งแอดเดรส `PC - 4` ไป[1])
- 2. รอหน่วยความจำหลักส่งข้อมูลมา
- 3. เขียนข้อมูลที่ได้ใส่แคช เติม tag และ ตั้ง valid bit เป็น `Y`
- 4. เริ่มทำคำสั่งใหม่อีกครั้ง คราวนี้แคชจะมีข้อมูลอยู่

สำหรับ data read, เราทำได้คล้ายๆกัน แต่ง่ายกว่าคือ เราแค่ stall รอจนกว่าจะได้ข้อมูลมาทำงานต่อ.

6.1.3 Handling Writes

การเขียนค่าในหน่วยความจำ เราอาจทำได้โดยวิธี write-through. วิธี คือ เขียนค่าใหม่ใส่ทั้งที่แคช และหน่วยความจำหลัก. วิธีนี้ เราจะแน่ใจได้เลยว่า ข้อมูลจะเหมือนกันทั้งแคช ทั้งหน่วยความจำหลัก. แต่วิธีนี้ทำให้ เราต้องเขียนลงหน่วยความจำหลักบ่อยๆ ซึ่งทำให้ประสิทธิภาพโดยรวมต่ำลงมาก.

วิธีแก้ง่ายๆ ก็คือ ใช้ write buffer. แทนที่จะเขียนลงหน่วยความจำหลัก ซึ่งทำให้ต้อง stall เพื่อรอการเขียนเสร็จ, ก็จะเขียนลง แคช และ write buffer ซึ่งเมื่อเขียนเสร็จ โปรเซสเซอร์ก็สามารถทำงานต่อไปเลย. แล้วค่อยใช้ข้อมูลจาก write buffer เขียนลงหน่วยความจำหลัก ด้วยวงจรต่างหาก.

อีกวิธีคือ write-back. ข้อมูลที่ถูกเขียน จะเขียนไปที่แคชเท่านั้น (ยังไม่เขียนไปที่หน่วยความจำหลัก), และ ข้อมูลจาก block ที่แก้ จะถูกเขียนกลับไปที่หน่วยความจำหลัก ตอนที่ block จะ

[1] ค่า `PC` ถูกเพิ่มไปทันที ดังนั้นเพื่อจะอ่านค่าคำสั่งที่ missed เราต้องลบ 4 กลับ.

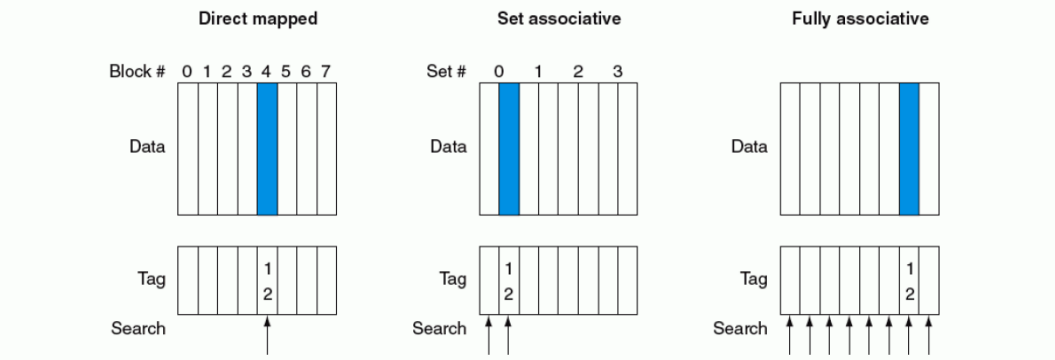


FIGURE 7.13 The location of a memory block whose address is 12 in a cache with 8 blocks varies for direct-mapped, set-associative, and fully associative placement. In direct-mapped placement, there is only one cache block where memory block 12 can be found, and that block is given by (12 modulo 8) = 4. In a two-way set-associative cache, there would be four sets, and memory block 12 must be in set (12 mod 4) = 0; the memory block could be in either element of the set. In a fully associative placement, the memory block for block address 12 can appear in any of the eight cache blocks.

Figure 6.8: Direct mapped, set-associative, aและ fully associative placement

ถูก block อื่นมาทับ. วิธีของ write-back ช่วยเรื่องสมถนะได้มาก โดยเฉพาะกรณีที่มีการเขียนค่าบ่อยๆ แต่ระบบสำหรับ write-back นั้นซับซ้อนกว่าของ write-through มาก.

6.2 Reducing Cache Misses by More Flexible Placement of Blocks

6.3 Virtual Memory

Bibliography

[1] J. W. Langston and X. He. Multi-core processors and caching: A survey. Dummy Journal: I have to fix this., -1, 2007. I have to fix this.